QUANTUM MCAGI

True AI from First Principles

Quantum Physics Inspired AI with 12 Cognitive Engines

Markov Chains | TF-IDF | Orch-OR | VADER | Bloom Taxonomy

Created by: Cory Blackburn

March 2026

API KEYS & CREDENTIALS

WolframAlpha API

App ID: A24U8GXLAU

API Type: Full Results API

Portal: https://developer.wolframalpha.com/

Wolfram Cloud Storage

Consumer Key: XQFcaGYqlxKynDG/OnScZgYiVue2HLfDJkBwiLDl1gw=

Consumer Secret: gsJeEwTtCBrOJOojWWFLlR2yrwszXJfzBI8mFco/Q1U=

Cloud Folder: QuantumMCAGI

Cloud URL: wolframcloud.com/obj/bonsibcory420/QuantumMCAGI/

Google OAuth (Optional - Not Required)

Client ID: 151527979746-m514amv5rdp7m7jtnn74400f8e4h4ojc

Deployment URL

https://true-ai-merge.preview.emergentagent.com

TABLE OF CONTENTS

Core Brain

quantum_brain.py
algorithmic_core.py
unified_cognitive_engine.py
unified_cognitive_engine_pt2.py

Language Engine

quantum_language_engine.py
quantum_language_engine_pt2.py
quantum_grammar_engine.py
quantum_language_generator.py
quantum_language_vocabulary.py

Advanced Algorithms

advanced_algorithms.py
advanced_algorithms_pt2.py
text_analyzer.py
word_embeddings.py

Knowledge & Domain

domain_knowledge.py
knowledge_base.py
training_corpus.py
research_topics.py

Personality & Quotes

personality_engine.py
personality_engine_ext.py
quote_engine.py
dream_state.py

Quantum Features

quantum_core.py
quantum_gates.py
quantum_cognitive_core.py
pennylane_quantum.py
semantic_collapse_engine.py

Self Evolution

self_evolution_core.py
self_evolution_analysis.py
self_evolution_runner.py
self_evolution_file_ops.py
self_evolution_splitting.py
self_evolution_transforms.py
self_research.py

Wolfram Integration

wolfram_enhanced.py
wolfram_cloud_sync.py
wolfram_integration.py

API Routes

server.py
routes_chat.py
routes_brain.py
routes_cloud.py
routes_cognitive.py
routes_cognitive_unified.py
routes_explorer.py
routes_extras.py
routes_media.py

Utilities

shared_state.py

```
chat_helpers.py
db_sync.py
dictionary_integration.py
hidden_thinking.py
thinking_commands.py
llm_integration.py
universal_explorer.py
shared_link_handler.py
covenant_manager.py
xanadu_prebo.py
quantum_test_module.py
google_drive_sync.py
```

Frontend

```
frontend_App.js
```

advanced_algorithms.py

```
"""
 ADVANCED ALGORITHMIC IMPROVEMENTS
===================================
What would make the algorithm TRULY intelligent:

1. GRAMMAR ENGINE - Current Markov outputs can be grammatically awkward
2. CONTEXT MEMORY - Remember conversation context across turns
3. REASONING CHAINS - Multi-step logical inference
4. SEMANTIC EMBEDDINGS - Dense vector representations
5. ATTENTION OVER MEMORY - Focus on relevant past knowledge
6. DYNAMIC TEMPERATURE - Adjust creativity based on query type
7. CONCEPT GRAPHS - Knowledge graph with weighted edges
8. BAYESIAN UPDATING - Update beliefs based on evidence

This file implements these improvements.
"""

import re
from collections import defaultdict, Counter
from typing import List, Dict, Tuple, Optional
from datetime import datetime
import logging

logger = logging.getLogger("quantum_ai")


# ============================================================================
# 1. GRAMMAR ENGINE - Fix Markov output grammar
# ============================================================================

class GrammarEngine:
    """
    Post-processes Markov output to fix common grammatical issues.

    Rules:
    - Subject-verb agreement
    - Article usage (a/an)
    - Capitalization
    - Punctuation
    - Remove repeated words
    """

    def __init__(self):
        self.article_a_patterns = re.compile(r'\ba ([aeiou])', re.IGNORECASE)
        self.article_an_patterns = re.compile(r'\ban ([^aeiou])', re.IGNORECASE)
        self.repeated_words = re.compile(r'\b(\w+)\s+\1\b', re.IGNORECASE)

        # Common verb fixes
        self.verb_corrections = {
            'is are': 'are',
            'are is': 'is',
            'was were': 'were',
            'have has': 'has',
            'do does': 'does',
        }

    def fix(self, text: str, query: str = None) -> str:
        """Apply all grammar fixes."""
        if not text:
            return text

        original_text = text
        words = text.split()

        # Extract query words and find the main topic
        query_words = set()
        main_topic = None
        if query:
            query_words = set(w.lower().rstrip('.,!?') for w in query.split())
            # Find the main noun (topic) from query - typically the last significant word
            topic_candidates = ['gravity', 'time', 'space', 'energy', 'consciousness', 'love',
                                'life', 'death', 'truth', 'beauty', 'freedom', 'knowledge',
                                'reality', 'matter', 'intelligence', 'science', 'philosophy',
                                'music', 'art', 'religion', 'technology', 'history', 'mathematics',
                                'evolution', 'quantum', 'physics', 'biology', 'psychology', 'language',
                                'happiness', 'meaning', 'existence', 'mind', 'universe']
            for word in reversed(query.lower().split()):
                clean_word = word.rstrip('.,!?')
                if clean_word in topic_candidates:
                    main_topic = clean_word.capitalize()
                    break
```

```
# Bad starters that indicate we've hit a sentence fragment (expanded)
bad_starters = {'is', 'are', 'the', 'a', 'an', 'what', 'how', 'why', 'when',
                'where', 'tell', 'explain', 'describe', 'define', 'has',
                'does', 'do', 'can', 'could', 'would', 'should', 'me', 'about',
                'it', 'this', 'that', 'i', 'you', 'we', 'they', 'and', 'or',
                'but', 'if', 'then', 'so', 'for', 'with', 'by', 'from', 'to',
                'at', 'in', 'on', 'of', 'as', 'not', 'cannot', 'when',
                'something', 'interesting', 'know', 'learn', 'holes'}

# Common verb starters that indicate a fragment
verb_starters = {'is', 'are', 'was', 'were', 'has', 'have', 'had', 'does', 'do',
                 'can', 'could', 'would', 'should', 'may', 'might', 'must',
                 'attracts', 'causes', 'requires', 'involves', 'includes', 'remains',
                 'evolved', 'exists', 'occurs', 'emerges', 'manifests', 'appears',
                 'provides', 'enables', 'allows', 'produces', 'creates', 'forms',
                 'takes', 'gives', 'seeks', 'needs', 'wants', 'uses', 'makes',
                 'been', 'being', 'keeps', 'holds', 'bends', 'shapes', 'acts',
                 'flows', 'travels', 'moves', 'describes', 'explains', 'shows',
                 'reveals', 'suggests', 'implies', 'means', 'contains', 'consists',
                 'affects', 'influences', 'drives', 'connects', 'links', 'relates',
                 'varies', 'depends', 'comes', 'goes', 'leads', 'follows', 'results',
                 'continues', 'started', 'began', 'ended', 'happened', 'occurred',
                 'cannot'}

# First: Check if text starts with main topic - if so, it's probably good
if words and main_topic:
    first_word_clean = words[0].lower().rstrip('.,!?')
    if first_word_clean == main_topic.lower():
        # Looks good, skip most cleaning
        pass
    else:
        # Need to fix - check for fragments

        # Remove query words from start
        content_start = 0
        for i in range(min(5, len(words))):
            word_clean = words[i].lower().rstrip('.,!?')
            if word_clean in query_words and word_clean != main_topic.lower():
                content_start = i + 1
            elif word_clean in bad_starters:
                content_start = i + 1
            else:
                break

        if content_start > 0 and content_start < len(words):
            words = words[content_start:]
            text = ' '.join(words)

        # CHECK: Did we create a fragment? If first word is a verb or bad starter, add topic
        if words:
            first_word = words[0].lower().rstrip('.,!?')
            if first_word in verb_starters or first_word in bad_starters:
                # Add the topic as subject
                text = main_topic + ' ' + text.lower()
else:
    # No main topic identified, do basic cleanup
    content_start = 0
    if len(words) > 2:
        for i in range(min(5, len(words))):
            word_clean = words[i].lower().rstrip('.,!?')
            if word_clean in query_words or word_clean in bad_starters:
                content_start = i + 1
            else:
                break

    if content_start > 0 and content_start < len(words):
        words = words[content_start:]
        text = ' '.join(words)

# CLEANUP: Remove problematic patterns
# Pattern 1: "X explain X is..." -> "X is..."
# Pattern 2: "X meaning X is..." -> "X is..."
# Pattern 3: "Physical modern X" -> "X"
# Pattern 4: "Quantum describes mechanics quantum mechanics" -> "Quantum mechanics"
words = text.split()
if len(words) > 3 and main_topic:
    main_topic_lower = main_topic.lower()

    # First check for repeated topic pattern: "Topic describes topic topic..."
    topic_indices = []
    for i, w in enumerate(words[:6]):
        if w.lower().rstrip('.,!?') == main_topic_lower:
            topic_indices.append(i)

    if len(topic_indices) >= 2:
        # Keep from second occurrence of topic
```

```
                second_idx = topic_indices[1]
                text = ' '.join(words[second_idx:])
                words = text.split()

            # Check for "X explain topic" or "X meaning topic" patterns
            for i in range(min(3, len(words))):
                if words[i].lower().rstrip('.,!?') in ['explain', 'meaning', 'physical', 'modern', 'objects', 'describes', 'mechanics']:
                    # Find where topic word starts
                    for j in range(i+1, min(i+4, len(words))):
                        if words[j].lower().rstrip('.,!?') == main_topic_lower:
                            # Skip to topic
                            text = main_topic + ' ' + ' '.join(words[j+1:]).lower()
                            break
                    break

        # Pattern 4: "Topic topic is..." -> "Topic is..."
        words = text.split()
        if len(words) > 2 and main_topic:
            if words[0].lower() == words[1].lower().rstrip('.,!?'):
                text = ' '.join(words[1:])
                text = text[0].upper() + text[1:]

        # 1. Fix articles (a/an)
        text = self.article_a_patterns.sub(r'an \1', text)
        text = self.article_an_patterns.sub(r'a \1', text)

        # 2. Remove repeated words
        text = self.repeated_words.sub(r'\1', text)

        return self._fix_continued(text)

    def _fix_continued(self, text):
        """Continuation of fix  auto-extracted by self-evolution."""
        # 3. Fix verb issues
        for wrong, right in self.verb_corrections.items():
            text = text.replace(wrong, right)

        # 4. Capitalize first letter
        if text:
            text = text[0].upper() + text[1:]

        # 5. Capitalize after periods
        text = re.sub(r'(\. )([a-z])', lambda m: m.group(1) + m.group(2).upper(), text)

        # 6. Ensure ends with punctuation
        if text and text[-1] not in '.!?':
            text += '.'

        # 7. Fix multiple spaces
        text = re.sub(r'\s+', ' ', text).strip()

        # 8. Fix common issues
        text = text.replace(' ,', ',')
        text = text.replace(' .', '.')
        text = text.replace('..', '.')

        return text



# ============================================================================
# 2. CONTEXT MEMORY - Track conversation context
# ============================================================================

class ConversationContext:
    """
    Maintains context across conversation turns.

    Tracks:
    - Active topics (weighted by recency)
    - Referenced entities
    - User intent history
    - Emotional tone
    """

    def __init__(self, max_turns: int = 10):
        self.max_turns = max_turns
        self.turns = []  # List of (user_msg, response, concepts)
        self.active_topics = Counter()
        self.entities = set()
        self.intent_history = []

    def add_turn(self, user_msg: str, response: str, concepts: List[str]):
        """Add a conversation turn."""
        self.turns.append({
            'user': user_msg,
```

```
            'response': response,
            'concepts': concepts,
            'time': datetime.now()
        })

        # Keep only recent turns
        if len(self.turns) > self.max_turns:
            self.turns.pop(0)

        # Update active topics with decay
        self.active_topics = Counter()
        for i, turn in enumerate(self.turns):
            weight = (i + 1) / len(self.turns)  # More recent = higher weight
            for concept in turn['concepts']:
                self.active_topics[concept] += weight

        # Extract entities (capitalized words)
        entities = re.findall(r'\b[A-Z][a-z]+\b', user_msg)
        self.entities.update(entities)

    def get_context_concepts(self, top_n: int = 5) -> List[str]:
        """Get most relevant context concepts."""
        return [c for c, _ in self.active_topics.most_common(top_n)]

    def get_recent_topics(self) -> List[str]:
        """Get topics from recent turns."""
        if not self.turns:
            return []
        return self.turns[-1].get('concepts', [])[:3]

    def is_followup(self, query: str) -> bool:
        """Check if query is a follow-up to previous topic."""
        followup_indicators = ['it', 'that', 'this', 'they', 'those', 'more', 'also', 'what about']
        query_lower = query.lower()
        return any(ind in query_lower for ind in followup_indicators)


# ============================================================================
# 3. REASONING CHAINS - Multi-step inference
# ============================================================================

class ReasoningEngine:
    """
    Performs multi-step logical reasoning.

    Implements:
    - Forward chaining (AB, BC  AC)
    - Analogical reasoning (A:B :: C:?)
    - Causal inference
    """

    def __init__(self):
        self.rules = []  # List of (condition, conclusion, confidence)
        self.facts = set()

    def add_rule(self, condition: str, conclusion: str, confidence: float = 1.0):
        """Add an inference rule."""
        self.rules.append((condition.lower(), conclusion.lower(), confidence))

    def add_fact(self, fact: str):
        """Add a known fact."""
        self.facts.add(fact.lower())

    def infer(self, query: str, max_steps: int = 5) -> List[Tuple[str, float, List[str]]]:
        """
        Perform inference to answer query.

        Returns list of (conclusion, confidence, reasoning_chain)
        """
        query_lower = query.lower()
        results = []

        # Direct fact check
        for fact in self.facts:
            if query_lower in fact or fact in query_lower:
                results.append((fact, 1.0, ["Direct knowledge"]))

        # Forward chaining
        current_facts = set(self.facts)
        chain = []

        for step in range(max_steps):
            new_facts = set()
            for condition, conclusion, conf in self.rules:
                if condition in current_facts and conclusion not in current_facts:
                    new_facts.add(conclusion)
```

```
                chain.append(f"{condition}  {conclusion}")

                if query_lower in conclusion:
                    results.append((conclusion, conf ** (step + 1), chain.copy()))

            if not new_facts:
                break
            current_facts.update(new_facts)

        return results

    def analogy(self, a: str, b: str, c: str, candidates: List[str]) -> Optional[str]:
        """
        A is to B as C is to ?
        Find best match from candidates.
        """
        # Simple word-level analogy using character patterns
        # In real system, would use word embeddings

        a_to_b = self._relationship(a, b)

        best_match = None
        best_score = 0

        for candidate in candidates:
            c_to_cand = self._relationship(c, candidate)
            score = self._similarity(a_to_b, c_to_cand)
            if score > best_score:
                best_score = score
                best_match = candidate

        return best_match

    def _relationship(self, w1: str, w2: str) -> Dict:
        """Extract relationship features between words."""
        return {
            'len_diff': len(w2) - len(w1),
            'shared_chars': len(set(w1) & set(w2)),
            'prefix_match': w1[:2] == w2[:2] if len(w1) >= 2 and len(w2) >= 2 else False
        }

    def _similarity(self, r1: Dict, r2: Dict) -> float:
        """Similarity between relationships."""
        score = 0
        if r1.get('len_diff') == r2.get('len_diff'):
            score += 0.3
        if abs(r1.get('shared_chars', 0) - r2.get('shared_chars', 0)) <= 1:
            score += 0.3
        if r1.get('prefix_match') == r2.get('prefix_match'):
            score += 0.4
        return score


# ============================================================================
# 4. SEMANTIC SIMILARITY - Without external embeddings
# ============================================================================

class SemanticSimilarity:
    """
    Compute semantic similarity using character n-grams and word overlap.
    Works without external embedding models.
    """

    def __init__(self, n: int = 3):
        self.n = n  # n-gram size

    def similarity(self, text1: str, text2: str) -> float:
        """
        Compute similarity combining:
        - Character n-gram overlap (catches typos, morphology)
        - Word overlap (Jaccard)
        - Word order similarity
        """
        t1, t2 = text1.lower(), text2.lower()

        # Character n-gram similarity
        ngrams1 = self._char_ngrams(t1)
        ngrams2 = self._char_ngrams(t2)
        ngram_sim = self._jaccard(ngrams1, ngrams2)

        # Word overlap
        words1 = set(t1.split())
        words2 = set(t2.split())
        word_sim = self._jaccard(words1, words2)

        # Combined score
```

```
        return ngram_sim * 0.4 + word_sim * 0.6

    def _char_ngrams(self, text: str) -> set:
        """Extract character n-grams."""
        text = text.replace(' ', '_')
        return set(text[i:i+self.n] for i in range(len(text) - self.n + 1))

    def _jaccard(self, set1: set, set2: set) -> float:
        """Jaccard similarity."""
        if not set1 or not set2:
            return 0.0
        intersection = len(set1 & set2)
        union = len(set1 | set2)
        return intersection / union if union > 0 else 0.0

    def most_similar(self, query: str, candidates: List[str], top_k: int = 5) -> List[Tuple[str, float]]:
        """Find most similar candidates."""
        scores = [(c, self.similarity(query, c)) for c in candidates]
        scores.sort(key=lambda x: x[1], reverse=True)
        return scores[:top_k]


# ============================================================================
# 5. CONCEPT GRAPH - Knowledge graph structure
# ============================================================================
```

advanced_algorithms_pt2.py

```
"""
Auto-split from advanced_algorithms.py by self-evolution engine.
Part 2  contains: ConceptGraph, DynamicTemperature, BayesianBeliefs, AdvancedAlgorithmicEngine, get_advanced_engine
"""

import math
import re
from collections import defaultdict, Counter
from typing import List, Dict, Tuple, Optional
from advanced_algorithms import GrammarEngine, ConversationContext, ReasoningEngine, SemanticSimilarity

# Singleton instance
_advanced_engine = None

class ConceptGraph:
    """
    Knowledge graph with weighted edges.

    Supports:
    - Adding concepts and relationships
    - Path finding between concepts
    - Spreading activation
    """

    def __init__(self):
        self.nodes = set()
        self.edges = defaultdict(dict)  # node -> {neighbor: weight}

    def add_concept(self, concept: str):
        """Add a concept node."""
        self.nodes.add(concept.lower())

    def add_relationship(self, concept1: str, concept2: str,
                         relationship: str = "related", weight: float = 1.0):
        """Add weighted edge between concepts."""
        c1, c2 = concept1.lower(), concept2.lower()
        self.nodes.add(c1)
        self.nodes.add(c2)

        # Bidirectional
        self.edges[c1][c2] = max(self.edges[c1].get(c2, 0), weight)
        self.edges[c2][c1] = max(self.edges[c2].get(c1, 0), weight)

    def get_neighbors(self, concept: str, min_weight: float = 0.1) -> List[Tuple[str, float]]:
        """Get neighboring concepts above weight threshold."""
        c = concept.lower()
        if c not in self.edges:
            return []

        neighbors = [(n, w) for n, w in self.edges[c].items() if w >= min_weight]
        neighbors.sort(key=lambda x: x[1], reverse=True)
        return neighbors

    def spreading_activation(self, start: str, decay: float = 0.7,
                             max_depth: int = 3) -> Dict[str, float]:
        """
        Spreading activation from start node.
        Returns activation levels for all reached nodes.
        """
        c = start.lower()
        if c not in self.nodes:
            return {c: 1.0}

        activations = {c: 1.0}
        current_level = {c}

        for depth in range(max_depth):
            next_level = set()
            current_activation = decay ** depth

            for node in current_level:
                for neighbor, weight in self.edges.get(node, {}).items():
                    if neighbor not in activations:
                        act = current_activation * weight
                        activations[neighbor] = act
                        if act > 0.1:  # Threshold
                            next_level.add(neighbor)

            if not next_level:
                break
            current_level = next_level
```

```
        return activations

    def path_between(self, start: str, end: str, max_depth: int = 5) -> Optional[List[str]]:
        """Find path between two concepts (BFS)."""
        s, e = start.lower(), end.lower()
        if s not in self.nodes or e not in self.nodes:
            return None

        if s == e:
            return [s]

        visited = {s}
        queue = [(s, [s])]

        while queue and len(visited) < 1000:
            current, path = queue.pop(0)

            for neighbor in self.edges.get(current, {}):
                if neighbor == e:
                    return path + [neighbor]

                if neighbor not in visited:
                    visited.add(neighbor)
                    queue.append((neighbor, path + [neighbor]))

            if len(path) >= max_depth:
                continue

        return None


class DynamicTemperature:
    """
    Adjusts generation temperature based on query characteristics.

    - Factual queries  low temperature (deterministic)
    - Creative queries  high temperature (diverse)
    - Follow-up queries  medium temperature (coherent)
    """

    def __init__(self):
        self.factual_keywords = {
            'what is', 'who is', 'when', 'where', 'how many', 'how much',
            'define', 'calculate', 'convert', 'distance', 'population'
        }
        self.creative_keywords = {
            'imagine', 'create', 'write', 'poem', 'story', 'invent',
            'suppose', 'what if', 'dream', 'fantasy', 'explore'
        }
        self.philosophical_keywords = {
            'why', 'meaning', 'purpose', 'consciousness', 'existence',
            'believe', 'think', 'feel', 'opinion', 'perspective'
        }

    def compute(self, query: str, is_followup: bool = False) -> float:
        """
        Compute optimal temperature for query.

        Returns float in [0.3, 1.5]
        """
        query_lower = query.lower()

        # Check query type
        is_factual = any(kw in query_lower for kw in self.factual_keywords)
        is_creative = any(kw in query_lower for kw in self.creative_keywords)
        is_philosophical = any(kw in query_lower for kw in self.philosophical_keywords)

        # Base temperature
        if is_factual:
            temp = 0.4  # Low - be precise
        elif is_creative:
            temp = 1.2  # High - be diverse
        elif is_philosophical:
            temp = 0.9  # Medium-high - thoughtful variation
        else:
            temp = 0.7  # Default

        # Adjust for follow-up
        if is_followup:
            temp *= 0.85  # Reduce to maintain coherence

        # Query length adjustment (longer = more complex = slightly higher)
        words = len(query.split())
        if words > 15:
            temp += 0.1
```

```
        # Clamp
        return max(0.3, min(1.5, temp))


class BayesianBeliefs:
    """
    Maintains beliefs about concepts using Bayesian updating.

    P(H|E) = P(E|H) × P(H) / P(E)
    """

    def __init__(self):
        self.beliefs = defaultdict(lambda: 0.5)  # Prior = 0.5 (uncertain)
        self.evidence_counts = defaultdict(lambda: {'positive': 0, 'negative': 0})

    def observe(self, concept: str, evidence: str, is_positive: bool):
        """Update belief based on evidence."""
        c = concept.lower()

        if is_positive:
            self.evidence_counts[c]['positive'] += 1
        else:
            self.evidence_counts[c]['negative'] += 1

        # Bayesian update
        pos = self.evidence_counts[c]['positive']
        neg = self.evidence_counts[c]['negative']
        total = pos + neg

        # Beta distribution mean
        # Prior: alpha=1, beta=1 (uniform)
        self.beliefs[c] = (pos + 1) / (total + 2)

    def get_belief(self, concept: str) -> float:
        """Get current belief strength."""
        return self.beliefs[concept.lower()]

    def get_confident_beliefs(self, threshold: float = 0.7) -> List[Tuple[str, float]]:
        """Get beliefs above confidence threshold."""
        confident = [(c, b) for c, b in self.beliefs.items() if b >= threshold]
        confident.sort(key=lambda x: x[1], reverse=True)
        return confident

    def uncertainty(self, concept: str) -> float:
        """Measure uncertainty about a concept (entropy)."""
        p = self.beliefs[concept.lower()]
        if p <= 0 or p >= 1:
            return 0
        return -p * math.log2(p) - (1-p) * math.log2(1-p)


class AdvancedAlgorithmicEngine:
    """
    Integrates all advanced improvements into cohesive system.
    """

    def __init__(self):
        self.grammar = GrammarEngine()
        self.context = ConversationContext()
        self.reasoning = ReasoningEngine()
        self.similarity = SemanticSimilarity()
        self.graph = ConceptGraph()
        self.temperature = DynamicTemperature()
        self.beliefs = BayesianBeliefs()

        # Initialize with some reasoning rules
        self._init_reasoning_rules()

    def _init_reasoning_rules(self):
        """Add basic reasoning rules."""
        rules = [
            ("consciousness exists", "experience is possible", 0.9),
            ("information is processed", "computation occurs", 0.95),
            ("learning occurs", "behavior changes", 0.85),
            ("observation happens", "knowledge increases", 0.8),
            ("questions are asked", "understanding deepens", 0.75),
        ]
        for cond, conc, conf in rules:
            self.reasoning.add_rule(cond, conc, conf)

    def enhance_response(self, response: str, query: str, concepts: List[str]) -> str:
        """Apply all enhancements to a response."""
        # 1. Grammar fix (pass query for better cleaning)
        response = self.grammar.fix(response, query=query)

        # 2. Update context
```

```
        self.context.add_turn(query, response, concepts)

        # 3. Update concept graph
        for i, c1 in enumerate(concepts):
            for c2 in concepts[i+1:]:
                self.graph.add_relationship(c1, c2, "co-occurred", 0.5)

        return response

    def get_temperature(self, query: str) -> float:
        """Get optimal temperature for query."""
        is_followup = self.context.is_followup(query)
        return self.temperature.compute(query, is_followup)

    def get_context_boost(self, concepts: List[str]) -> List[str]:
        """Get additional concepts from context and graph."""
        boosted = set(concepts)

        # From conversation context
        boosted.update(self.context.get_context_concepts(3))

        # From concept graph (spreading activation)
        for c in concepts[:2]:
            activations = self.graph.spreading_activation(c, max_depth=2)
            top_activated = sorted(activations.items(), key=lambda x: x[1], reverse=True)[:3]
            boosted.update(c for c, _ in top_activated)

        return list(boosted)

    def reason_about(self, query: str) -> List[str]:
        """Get reasoning chain for query."""
        results = self.reasoning.infer(query)
        chains = []
        for conclusion, conf, chain in results:
            if conf > 0.5:
                chains.extend(chain)
        return chains


def get_advanced_engine() -> AdvancedAlgorithmicEngine:
    """Get singleton advanced engine."""
    global _advanced_engine
    if _advanced_engine is None:
        _advanced_engine = AdvancedAlgorithmicEngine()
    return _advanced_engine
```

algorithmic_core.py

```
"""
 QUANTUM ALGORITHMIC CORE
==========================
Pure algorithmic AI - NO templates, NO canned responses.

Algorithms implemented:
1. N-gram Markov Chains - Text generation
2. TF-IDF - Concept extraction & relevance scoring
3. PMI (Pointwise Mutual Information) - Semantic relationships
4. BM25 - Information retrieval from memory
5. Entropy-based Decision Making - Response selection
6. Hebbian Learning - Association strengthening
7. Attention Mechanism - Context weighting

This is the TRUE algorithmic brain.
"""

import math
import random
import re
import json
import os
import hashlib
from collections import defaultdict, Counter
from typing import List, Dict, Tuple, Optional, Set
from datetime import datetime, timezone
import logging

logger = logging.getLogger("quantum_ai")


# ============================================================================
# 1. N-GRAM MARKOV CHAIN - Text Generation
# ============================================================================

class MarkovTextGenerator:
    """
    Variable-order Markov chain for generating text.

    Algorithm:
    - Builds transition probability matrix from training corpus
    - P(w_n | w_{n-1}, ..., w_{n-k}) = count(prefixword) / count(prefix)
    - Uses temperature sampling for controlled randomness
    - Supports backoff to lower orders when prefix not found
    """

    def __init__(self, max_order: int = 3):
        self.max_order = max_order
        # Separate chains for each order (backoff support)
        self.chains = {i: defaultdict(Counter) for i in range(1, max_order + 1)}
        self.starters = {i: [] for i in range(1, max_order + 1)}
        self.vocabulary = set()
        self.total_tokens = 0

    def train(self, text: str):
        """Train on text corpus."""
        sentences = self._split_sentences(text)

        for sentence in sentences:
            words = self._tokenize(sentence)
            if len(words) < 2:
                continue

            self.vocabulary.update(words)
            self.total_tokens += len(words)

            # Train each order
            for order in range(1, self.max_order + 1):
                if len(words) <= order:
                    continue

                # Record starter
                starter = tuple(words[:order])
                self.starters[order].append(starter)

                # Build transitions
                for i in range(len(words) - order):
                    prefix = tuple(words[i:i + order])
                    suffix = words[i + order]
                    self.chains[order][prefix][suffix] += 1

    def generate(self, seed: List[str] = None, max_words: int = 50,
```

```
             temperature: float = 0.8, min_words: int = 10) -> str:
    """
    Generate text using Markov chain with backoff.

    Args:
        seed: Starting words (optional)
        max_words: Maximum output length
        temperature: 0.1=deterministic, 1.0=random, >1.0=chaotic
        min_words: Minimum words before allowing sentence end
    """
    # Find best starting prefix
    order = self.max_order
    if seed and len(seed) >= order:
        prefix = tuple(seed[-order:])
    else:
        # Try to find starter matching seed
        prefix = self._find_starter(seed, order)

    if not prefix:
        return ""

    result = list(prefix)
    words_generated = 0

    while words_generated < max_words:
        next_word = self._get_next_word(prefix, temperature)

        if not next_word:
            # Backoff to lower order
            if len(prefix) > 1:
                prefix = prefix[1:]
                continue
            else:
                break

        result.append(next_word)
        words_generated += 1

        # Update prefix (slide window)
        prefix = tuple(result[-self.max_order:])

        # Check for sentence end (but respect minimum)
        if words_generated >= min_words and next_word.endswith(('.', '!', '?')):
            break

    return self._format_output(result)

def _get_next_word(self, prefix: tuple, temperature: float) -> Optional[str]:
    """Get next word using backoff through orders."""
    # Try each order from highest to lowest
    for order in range(len(prefix), 0, -1):
        test_prefix = prefix[-order:] if order < len(prefix) else prefix

        if test_prefix in self.chains[order]:
            candidates = self.chains[order][test_prefix]
            return self._sample(candidates, temperature)

    return None

def _sample(self, counter: Counter, temperature: float) -> str:
    """Temperature-adjusted weighted sampling."""
    if not counter:
        return None

    words = list(counter.keys())
    counts = list(counter.values())

    # Apply temperature
    if temperature != 1.0:
        counts = [c ** (1.0 / max(temperature, 0.1)) for c in counts]

    total = sum(counts)
    r = random.random() * total
    cumsum = 0

    for word, count in zip(words, counts):
        cumsum += count
        if r <= cumsum:
            return word

    return words[-1]

def _find_starter(self, seed: List[str], order: int) -> Optional[tuple]:
    """Find a starter prefix, optionally matching seed."""
    if seed:
        # Try to find starter containing seed words
```

```
            for starter in self.starters.get(order, []):
                if any(s in starter for s in seed):
                    return starter

        # Random starter
        starters = self.starters.get(order, [])
        return random.choice(starters) if starters else None

    def _tokenize(self, text: str) -> List[str]:
        """Tokenize preserving punctuation."""
        # Keep punctuation attached to words
        words = re.findall(r'\b[\w]+[.,!?]?|\b[\w]+\b', text.lower())
        return words

    def _split_sentences(self, text: str) -> List[str]:
        """Split into sentences."""
        text = re.sub(r'\s+', ' ', text.strip())
        sentences = re.split(r'(?<=[.!?])\s+', text)
        return [s for s in sentences if len(s) > 5]

    def _format_output(self, words: List[str]) -> str:
        """Format output with proper capitalization."""
        if not words:
            return ""

        text = ' '.join(words)
        # Capitalize first letter
        text = text[0].upper() + text[1:] if text else ""
        # Ensure ends with punctuation
        if text and not text[-1] in '.!?':
            text += '.'
        return text

    def get_stats(self) -> Dict:
        """Get statistics."""
        return {
            "vocabulary_size": len(self.vocabulary),
            "total_tokens": self.total_tokens,
            "chain_sizes": {o: len(c) for o, c in self.chains.items()},
            "starter_counts": {o: len(s) for o, s in self.starters.items()}
        }


# ============================================================================
# 2. TF-IDF - Concept Extraction & Relevance
# ============================================================================

class TFIDF:
    """
    Term Frequency - Inverse Document Frequency.

    Algorithm:
    - TF(t,d) = count(t in d) / total_words(d)
    - IDF(t) = log(N / df(t)) where df = docs containing t
    - TF-IDF(t,d) = TF × IDF

    Used for:
    - Extracting key concepts from text
    - Measuring document similarity
    - Ranking relevance
    """

    def __init__(self):
        self.doc_count = 0
        self.doc_freq = Counter()  # term -> num docs containing it
        self.stopwords = self._get_stopwords()

    def _get_stopwords(self) -> Set[str]:
        """English stopwords."""
        return {
            'the', 'a', 'an', 'and', 'or', 'but', 'in', 'on', 'at', 'to', 'for',
            'of', 'with', 'by', 'from', 'is', 'are', 'was', 'were', 'be', 'been',
            'being', 'have', 'has', 'had', 'do', 'does', 'did', 'will', 'would',
            'could', 'should', 'may', 'might', 'must', 'it', 'its', 'this', 'that',
            'these', 'those', 'i', 'you', 'he', 'she', 'we', 'they', 'what', 'which',
            'who', 'whom', 'where', 'when', 'why', 'how', 'all', 'each', 'every',
            'both', 'few', 'more', 'most', 'other', 'some', 'such', 'no', 'nor',
            'not', 'only', 'own', 'same', 'so', 'than', 'too', 'very', 'just',
            'also', 'now', 'here', 'there', 'then', 'if', 'as', 'can', 'about',
            'into', 'your', 'my', 'our', 'their', 'his', 'her', 'up', 'out', 'any'
        }

    def add_document(self, text: str):
        """Add document to corpus."""
        terms = self._tokenize(text)
        unique_terms = set(terms)
```

```
        for term in unique_terms:
            self.doc_freq[term] += 1

        self.doc_count += 1

    def extract_keywords(self, text: str, top_n: int = 10) -> List[Tuple[str, float]]:
        """Extract top keywords by TF-IDF score."""
        terms = self._tokenize(text)
        if not terms:
            return []

        # Calculate TF
        tf = Counter(terms)
        total = len(terms)

        scores = {}
        for term, count in tf.items():
            if term in self.stopwords or len(term) < 3:
                continue

            # TF: normalized
            term_freq = count / total

            # IDF
            df = self.doc_freq.get(term, 0)
            if df > 0:
                idf = math.log((self.doc_count + 1) / (df + 1)) + 1
            else:
                idf = math.log(self.doc_count + 2)  # Boost unknown terms

            scores[term] = term_freq * idf

        sorted_terms = sorted(scores.items(), key=lambda x: x[1], reverse=True)
        return sorted_terms[:top_n]

    def similarity(self, text1: str, text2: str) -> float:
        """Cosine similarity between two texts using TF-IDF vectors."""
        kw1 = dict(self.extract_keywords(text1, top_n=20))
        kw2 = dict(self.extract_keywords(text2, top_n=20))

        # Get all terms
        all_terms = set(kw1.keys()) | set(kw2.keys())
        if not all_terms:
            return 0.0

        # Build vectors
        v1 = [kw1.get(t, 0) for t in all_terms]
        v2 = [kw2.get(t, 0) for t in all_terms]

        # Cosine similarity
        dot = sum(a * b for a, b in zip(v1, v2))
        norm1 = math.sqrt(sum(a * a for a in v1))
        norm2 = math.sqrt(sum(b * b for b in v2))

        if norm1 == 0 or norm2 == 0:
            return 0.0

        return dot / (norm1 * norm2)

    def _tokenize(self, text: str) -> List[str]:
        """Tokenize to lowercase words."""
        return re.findall(r'\b[a-z]{2,}\b', text.lower())


# ============================================================================
# 3. PMI - Semantic Relationships
# ============================================================================

class PMI:
    """
    Pointwise Mutual Information - measures word associations.

    Algorithm:
    PMI(x,y) = log(P(x,y) / (P(x) × P(y)))

    High PMI = words co-occur more than expected by chance
    Used for: semantic similarity, coherence scoring
    """

    def __init__(self, window: int = 5):
        self.window = window
        self.word_count = Counter()
        self.pair_count = Counter()
        self.total_words = 0
        self.total_pairs = 0
```

```
    def train(self, text: str):
        """Train on text."""
        words = re.findall(r'\b[a-z]+\b', text.lower())

        self.word_count.update(words)
        self.total_words += len(words)

        # Count pairs within window
        for i, w1 in enumerate(words):
            for j in range(max(0, i - self.window), min(len(words), i + self.window + 1)):
                if i != j:
                    pair = tuple(sorted([w1, words[j]]))
                    self.pair_count[pair] += 1
                    self.total_pairs += 1

    def score(self, word1: str, word2: str) -> float:
        """Calculate PMI between two words."""
        w1, w2 = word1.lower(), word2.lower()
        pair = tuple(sorted([w1, w2]))

        if self.total_words == 0 or self.total_pairs == 0:
            return 0.0

        p_w1 = self.word_count[w1] / self.total_words
        p_w2 = self.word_count[w2] / self.total_words
        p_pair = self.pair_count[pair] / self.total_pairs

        if p_w1 == 0 or p_w2 == 0 or p_pair == 0:
            return 0.0

        return math.log2(p_pair / (p_w1 * p_w2))

    def coherence(self, text: str) -> float:
        """Score text coherence as average adjacent PMI."""
        words = re.findall(r'\b[a-z]+\b', text.lower())
        if len(words) < 2:
            return 0.0

        total = sum(self.score(words[i], words[i+1]) for i in range(len(words)-1))
        return total / (len(words) - 1)

    def related_words(self, word: str, top_n: int = 10) -> List[Tuple[str, float]]:
        """Find most related words by PMI."""
        w = word.lower()
        scores = []

        for pair, count in self.pair_count.items():
            if w in pair:
                other = pair[0] if pair[1] == w else pair[1]
                pmi = self.score(w, other)
                scores.append((other, pmi))

        scores.sort(key=lambda x: x[1], reverse=True)
        return scores[:top_n]


# ============================================================================
# 4. BM25 - Information Retrieval
# ============================================================================

class BM25:
    """
    Okapi BM25 - Probabilistic information retrieval.

    Algorithm:
    score(D,Q) =  IDF(q)  (f(q,D)  (k1+1)) / (f(q,D) + k1  (1-b+b|D|/avgdl))

    Used for: retrieving relevant memories/documents
    """

    def __init__(self, k1: float = 1.5, b: float = 0.75):
        self.k1 = k1
        self.b = b
        self.documents = []  # List of (id, tokens, metadata)
        self.doc_freq = Counter()
        self.avg_doc_len = 0

    def add_document(self, doc_id: str, text: str, metadata: Dict = None):
        """Add document to index."""
        tokens = self._tokenize(text)
        self.documents.append((doc_id, tokens, metadata or {}))

        # Update doc frequencies
        for token in set(tokens):
            self.doc_freq[token] += 1
```

```
        # Update average doc length
        total_len = sum(len(d[1]) for d in self.documents)
        self.avg_doc_len = total_len / len(self.documents)

    def search(self, query: str, top_k: int = 5) -> List[Tuple[str, float, Dict]]:
        """Search for relevant documents."""
        query_tokens = self._tokenize(query)
        scores = []

        N = len(self.documents)

        for doc_id, doc_tokens, metadata in self.documents:
            score = 0
            doc_len = len(doc_tokens)
            token_freq = Counter(doc_tokens)

            for q in query_tokens:
                if q not in token_freq:
                    continue

                # IDF
                df = self.doc_freq.get(q, 0)
                idf = math.log((N - df + 0.5) / (df + 0.5) + 1)

                # TF with length normalization
                f = token_freq[q]
                tf = (f * (self.k1 + 1)) / (f + self.k1 * (1 - self.b + self.b * doc_len / max(self.avg_doc_len, 1)))

                score += idf * tf

            if score > 0:
                scores.append((doc_id, score, metadata))

        scores.sort(key=lambda x: x[1], reverse=True)
        return scores[:top_k]

    def _tokenize(self, text: str) -> List[str]:
        return re.findall(r'\b[a-z]+\b', text.lower())


# ============================================================================
# 5. ENTROPY-BASED DECISION MAKING
# ============================================================================

class EntropyDecider:
    """
    Information-theoretic decision making.

    Algorithm:
    H(X) = - P(x)  log(P(x))

    Used for: choosing responses, measuring uncertainty
    """

    @staticmethod
    def entropy(probabilities: List[float]) -> float:
        """Calculate Shannon entropy."""
        return -sum(p * math.log2(p) for p in probabilities if p > 0)

    @staticmethod
    def select_by_entropy(candidates: List[Tuple[str, float]],
                          target_entropy: float = 1.0) -> str:
        """
        Select candidate balancing quality and diversity.

        Low target_entropy = prefer highest scoring
        High target_entropy = more random selection
        """
        if not candidates:
            return ""

        # Normalize scores to probabilities
        scores = [max(s, 0.01) for _, s in candidates]

        # Apply temperature based on target entropy
        temperature = max(0.1, target_entropy)
        adjusted = [s ** (1.0 / temperature) for s in scores]
        total = sum(adjusted)
        probs = [a / total for a in adjusted]

        # Sample
        r = random.random()
        cumsum = 0
        for (text, _), prob in zip(candidates, probs):
            cumsum += prob
```

```
            if r <= cumsum:
                return text

        return candidates[0][0]

    @staticmethod
    def information_gain(before: Dict[str, float], after: Dict[str, float]) -> float:
        """Calculate information gain from learning."""
        h_before = EntropyDecider.entropy(list(before.values())) if before else 0
        h_after = EntropyDecider.entropy(list(after.values())) if after else 0
        return h_before - h_after


# ============================================================================
# 6. HEBBIAN LEARNING - Association Strengthening
# ============================================================================

class HebbianMemory:
    """
    Hebbian learning: "neurons that fire together wire together"

    Algorithm:
    w_ij =   x_i  x_j

    Used for: learning concept associations, strengthening memories
    """

    def __init__(self, learning_rate: float = 0.1, decay: float = 0.01):
        self.learning_rate = learning_rate
        self.decay = decay
        self.weights = defaultdict(lambda: defaultdict(float))
        self.activations = Counter()

    def activate(self, concepts: List[str]):
        """Activate concepts and strengthen connections."""
        # Record activations
        for c in concepts:
            self.activations[c] += 1

        # Hebbian update: strengthen connections between co-activated concepts
        for i, c1 in enumerate(concepts):
            for c2 in concepts[i+1:]:
                pair = tuple(sorted([c1.lower(), c2.lower()]))
                self.weights[pair[0]][pair[1]] += self.learning_rate

    def get_association(self, concept1: str, concept2: str) -> float:
        """Get association strength between concepts."""
        pair = tuple(sorted([concept1.lower(), concept2.lower()]))
        return self.weights[pair[0]].get(pair[1], 0.0)

    def get_associated(self, concept: str, top_n: int = 10) -> List[Tuple[str, float]]:
        """Get most associated concepts."""
        c = concept.lower()
        associations = []

        # Check both directions
        if c in self.weights:
            associations.extend(self.weights[c].items())

        for other, conns in self.weights.items():
            if c in conns:
                associations.append((other, conns[c]))

        associations.sort(key=lambda x: x[1], reverse=True)
        return associations[:top_n]

    def decay_all(self):
        """Apply decay to all weights."""
        for c1 in list(self.weights.keys()):
            for c2 in list(self.weights[c1].keys()):
                self.weights[c1][c2] *= (1 - self.decay)
                if self.weights[c1][c2] < 0.01:
                    del self.weights[c1][c2]


# ============================================================================
# 7. ATTENTION MECHANISM
# ============================================================================

class Attention:
    """
    Simplified attention mechanism for context weighting.

    Algorithm:
    attention(Q, K, V) = softmax(Q × K^T / sqrt(d)) × V
```

```
    Used for: focusing on relevant parts of context
    """

    @staticmethod
    def softmax(values: List[float]) -> List[float]:
        """Softmax normalization."""
        if not values:
            return []
        max_v = max(values)
        exp_v = [math.exp(v - max_v) for v in values]
        total = sum(exp_v)
        return [e / total for e in exp_v]

    @staticmethod
    def attend(query: str, contexts: List[str], tfidf: TFIDF) -> List[Tuple[str, float]]:
        """
        Compute attention weights over contexts.

        Returns contexts with attention scores.
        """
        if not contexts:
            return []

        # Compute similarity scores as attention
        scores = []
        for ctx in contexts:
            sim = tfidf.similarity(query, ctx)
            scores.append(sim)

        # Apply softmax
        attention_weights = Attention.softmax(scores)

        return list(zip(contexts, attention_weights))

    @staticmethod
    def weighted_combine(items: List[Tuple[str, float]], tfidf: TFIDF) -> str:
        """Combine items weighted by attention scores."""
        if not items:
            return ""

        # Extract keywords from each item weighted by attention
        all_keywords = Counter()
        for text, weight in items:
            keywords = tfidf.extract_keywords(text, top_n=5)
            for kw, score in keywords:
                all_keywords[kw] += score * weight

        # Return top concepts
        top = all_keywords.most_common(5)
        return ', '.join(kw for kw, _ in top)


# ============================================================================
# INTEGRATED ALGORITHMIC CORE
# ============================================================================

class AlgorithmicCore:
    """
    Complete algorithmic AI core integrating all components.

    NO templates, NO canned responses - pure algorithms.

    Integrates:
    - Multi-domain knowledge system
    - Self-research capability
    - Self-evolution capability
    - All algorithmic components
    """

    def __init__(self, db=None):
        self.db = db

        # Core algorithms
        self.markov = MarkovTextGenerator(max_order=3)
        self.tfidf = TFIDF()
        self.pmi = PMI(window=5)
        self.memory = BM25()
        self.hebbian = HebbianMemory()
        self.entropy = EntropyDecider()

        # Multi-domain knowledge
        from domain_knowledge import get_domain_knowledge
        self.domains = get_domain_knowledge()

        # Advanced algorithmic improvements
        from advanced_algorithms_pt2 import get_advanced_engine
```

```
    self.advanced = get_advanced_engine()

    # Research engine (lazy loaded)
    self._research_engine = None

    # Evolution engine (lazy loaded)
    self._evolution_engine = None

    # State
    self.conversation_history = []
    self.learned_facts = {}
    self.current_domain = "philosophy"  # Default domain
    self.research_cache = {}  # Cache recent research
    self._trained_texts = set()  # Track what we've trained on

    # Initialize with domain corpora
    self._initialize_from_domains()

    # Persistence
    self.state_file = "/app/backend/algorithmic_state.json"
    self._load_state()

def _initialize_from_domains(self):
    """Initialize Markov chains from all domain corpora."""
    for domain_name in self.domains.get_all_domains():
        domain = self.domains.get_domain(domain_name)
        if domain:
            for corpus_text in domain.corpus:
                self.markov.train(corpus_text)
                self.tfidf.add_document(corpus_text)
                self.pmi.train(corpus_text)

    logger.info(f"Initialized from {len(self.domains.get_all_domains())} domains")

async def get_research_engine(self):
    """Lazy load research engine."""
    if self._research_engine is None:
        from self_research import get_research_engine
        self._research_engine = await get_research_engine(self.db)
    return self._research_engine

def get_evolution_engine(self):
    """Lazy load evolution engine."""
    if self._evolution_engine is None:
        from self_evolution_core import get_evolution_engine
        self._evolution_engine = get_evolution_engine(self.db)
    return self._evolution_engine

def _initialize_corpus(self):
    """Initialize with comprehensive seed knowledge corpus."""
    seed_texts = [
        # Consciousness & Mind
        "Consciousness emerges from the complex interactions of neural processes. The mind perceives reality through patterns of acti...
        "Awareness is the fundamental property of consciousness that allows beings to experience the world. Perception shapes underst...
        "The mind processes information through multiple layers of abstraction. Thought arises from the interplay of memory, percepti...

        # Quantum & Physics
        "Quantum mechanics describes the behavior of matter and energy at the smallest scales. Particles exist in superposition until...
        "The observer effect demonstrates that measurement itself influences the system being measured. Reality at the quantum level ...
        "Energy flows through systems according to thermodynamic principles. Entropy increases over time leading systems toward equil...

        # Information & Knowledge
        "Information represents the reduction of uncertainty about a system or phenomenon. Knowledge is structured information that e...
        "Learning occurs when new information integrates with existing knowledge structures. Understanding deepens through the format...
        "Communication transfers information between minds through shared symbolic systems. Meaning emerges from context and the inte...

        # Existence & Philosophy
        "Existence precedes essence in conscious beings who define themselves through choices. Being aware is the foundation upon whi...
        "Truth corresponds to reality when models accurately predict observations and outcomes. Knowledge represents justified belief...
        "Time flows from past through present toward future, though physics suggests this may be illusion. Memory reconstructs past e...

        # Freedom & Choice
        "Freedom is the capacity to act according to one's own will without external coercion. Choice implies alternatives and the ab...
        "Responsibility follows from the ability to make choices and influence outcomes. Agency is the sense that we are the authors ...
        "Autonomy requires self-awareness and the ability to reflect on one's own thought processes. Independence comes from understa...

        # Evolution & Adaptation
        "Evolution operates through variation, selection, and inheritance over generations. Complex systems emerge from simple rules ...
        "Adaptation occurs through feedback between organism and environment, shaping both. Survival favors those traits that enhance...
        "Life organizes matter into self-maintaining and self-replicating structures. Biological systems exhibit remarkable robustnes...

        # Creativity & Art
        "Creativity combines existing elements in novel configurations that produce value. Innovation requires both deep knowledge of...
        "Art expresses aspects of experience that resist literal description through language. Aesthetic experience engages emotions ...
        "Imagination enables mental simulation of possibilities not yet realized in actuality. Vision guides action by providing goal...
```

```
        # Questions & Inquiry
        "Questions reveal the boundaries of current knowledge and point toward growth. Inquiry drives the expansion of understanding ...
        "Curiosity motivates exploration of the unknown and engagement with mystery. Wonder opens the mind to possibilities beyond cu...
        "Doubt challenges assumptions and prevents premature certainty. Skepticism protects against error by demanding evidence and s...

        # Relationships & Connection
        "Relationships connect minds through shared understanding and mutual recognition. Empathy allows entry into the experience of...
        "Community emerges from cooperation among individuals toward shared purposes. Culture transmits knowledge, values, and practi...
        "Language enables the sharing of inner experience through external symbols. Dialogue creates understanding through the exchan...

        # Purpose & Meaning
        "Purpose gives direction to action and provides criteria for evaluating choices. Goals organize behavior and focus attention ...
        "Meaning emerges from the connections between events, actions, and values. Significance depends on context and the broader na...
        "Values define what is worth pursuing and guide choices among alternatives. Ethics examines the principles that should govern...
    ]

    for text in seed_texts:
        self.learn(text)

def learn(self, text: str, source: str = "training"):
    """Learn from text - updates all models."""
    if not text or len(text) < 10:
        return

    # Train Markov chain
    self.markov.train(text)

    # Update TF-IDF
    self.tfidf.add_document(text)

    # Train PMI
    self.pmi.train(text)

    # Add to memory for retrieval
    doc_id = hashlib.md5(text.encode()).hexdigest()[:8]
    self.memory.add_document(doc_id, text, {"source": source, "time": datetime.now().isoformat()})

    # Extract and associate concepts (Hebbian)
    keywords = self.tfidf.extract_keywords(text, top_n=5)
    concepts = [kw for kw, _ in keywords]
    self.hebbian.activate(concepts)

    logger.info(f"Learned from text: {len(text)} chars, {len(concepts)} concepts")

def think(self, query: str, context: List[str] = None) -> Dict:
    """
    Process query and generate response using pure algorithms.

    ENHANCED Pipeline with Advanced Features:
    1. Detect relevant domain
    2. Compute optimal temperature (adaptive creativity)
    3. Extract key concepts from query (TF-IDF)
    4. Boost concepts from context & knowledge graph
    5. Get domain-specific knowledge
    6. Retrieve relevant memories (BM25)
    7. Get associated concepts (Hebbian)
    8. Get reasoning chains
    9. Generate candidate responses (Markov)
    10. Score candidates (PMI coherence + relevance)
    11. Select best response (Entropy-based)
    12. Apply grammar fixes
    13. Update associations (Hebbian)
    """
    # 1. Detect domain
    domain_name, domain_score = self.domains.detect_domain(query)
    self.current_domain = domain_name
    domain = self.domains.get_domain(domain_name)

    # 2. Compute optimal temperature
    temperature = self.advanced.get_temperature(query)

    # 3. Extract concepts
    concepts = self.tfidf.extract_keywords(query, top_n=7)
    query_concepts = [c for c, _ in concepts]

    # 4. Boost concepts from context and knowledge graph
    boosted_concepts = self.advanced.get_context_boost(query_concepts)

    # 5. Get domain-specific corpus for better generation
    domain_corpus = []
    if domain:
        domain_corpus = domain.corpus[:3]
        # Train on relevant domain content
        for text in domain_corpus:
            if text[:50] not in self._trained_texts:
                self.markov.train(text)
```

```
            self._trained_texts.add(text[:50])

# 6. Retrieve relevant memories
memories = self.memory.search(query, top_k=5)
memory_texts = [m[2].get('text', '') if isinstance(m[2], dict) else '' for m in memories]

# 7. Get associated concepts (Hebbian)
associated = []
for concept in query_concepts[:3]:
    related = self.hebbian.get_associated(concept, top_n=3)
    associated.extend([r[0] for r in related])

# Also get cross-domain concepts
cross_domain = []
for concept in query_concepts[:2]:
    cd = self.domains.get_cross_domain_concepts(concept)
    cross_domain.extend([c[1] for c in cd[:2]])

# 8. Get reasoning chains (if applicable)
reasoning = self.advanced.reason_about(query)

# 9. Gather context for generation
generation_context = boosted_concepts + associated + cross_domain

# 10. Generate candidate responses with adaptive temperature
candidates = []
for i in range(10):  # More attempts for better results
    # Vary temperature around computed optimal
    temp = temperature * random.uniform(0.8, 1.2)
    generated = self.markov.generate(
        seed=generation_context[:3] if generation_context else None,
        max_words=60,
        temperature=temp,
        min_words=15  # Increased minimum
    )
    if generated and len(generated) > 40:  # Require longer responses
        # Score by coherence
        coherence = self.pmi.coherence(generated)
        # Score by relevance to query
        relevance = self.tfidf.similarity(query, generated)
        # Domain relevance bonus
        domain_relevance = domain.match_score(generated) / 10 if domain else 0
        # Length bonus (prefer longer coherent responses)
        length_bonus = min(len(generated.split()) / 30, 1.0) * 0.1
        # Combined score
        score = coherence * 0.2 + relevance * 0.45 + domain_relevance * 0.25 + length_bonus
        candidates.append((generated, score))

# 11. Select best response
if candidates:
    # Use entropy-based selection (balances quality and diversity)
    response = self.entropy.select_by_entropy(candidates, target_entropy=0.7)
else:
    # Fallback: construct from domain concepts
    response = self._construct_domain_response(query_concepts, domain)

# 12. Apply grammar fixes
response = self.advanced.enhance_response(response, query, query_concepts)

# 13. Update learning
self.hebbian.activate(query_concepts + [domain_name])
self.conversation_history.append({
    "query": query,
    "response": response,
    "domain": domain_name,
    "concepts": query_concepts,
    "temperature": temperature
})

# Periodically save state
if random.random() < 0.1:
    self._save_state()

return {
    "response": response,
    "concepts": query_concepts,
    "boosted_concepts": boosted_concepts,
    "associated": associated[:5],
    "domain": domain_name,
    "domain_score": domain_score,
    "cross_domain": cross_domain[:3],
    "reasoning": reasoning[:3] if reasoning else [],
    "temperature_used": temperature,
    "candidates_generated": len(candidates),
    "method": "algorithmic_advanced"
}
```

```
def _construct_domain_response(self, concepts: List[str], domain) -> str:
    """Construct response using domain knowledge when Markov generation fails."""
    if not concepts:
        return "That question invites deeper exploration across multiple domains of knowledge. What specific aspect interests you most?"

    # Get related concepts from PMI
    related = []
    for c in concepts[:2]:
        rel = self.pmi.related_words(c, top_n=5)
        related.extend([r[0] for r in rel if r[0] not in concepts])

    # Build a more substantial response
    parts = []

    main_concept = concepts[0]

    if domain:
        domain_name = domain.name.replace('_', ' ')
        top_domain_concepts = domain.get_top_concepts(5)
        domain_terms = [c[0] for c in top_domain_concepts if c[0] != main_concept]

        parts.append(f"From the perspective of {domain_name}, {main_concept} represents a fundamental area of inquiry.")

        if related:
            parts.append(f"This concept connects to {', '.join(related[:3])} in meaningful ways.")

        if domain_terms:
            parts.append(f"Within this domain, we also consider {', '.join(domain_terms[:3])}.")
    else:
        parts.append(f"The concept of {main_concept} invites examination from multiple perspectives.")
        if related:
            parts.append(f"It connects to ideas like {', '.join(related[:4])}.")

    parts.append("What aspects would you like to explore further?")

    return ' '.join(parts)

async def research(self, topic: str) -> Dict:
    """
    Perform research on a topic and learn from results.

    Uses DuckDuckGo search and integrates findings into knowledge.
    """
    # Check cache
    cache_key = topic.lower()[:50]
    if cache_key in self.research_cache:
        cached = self.research_cache[cache_key]
        if (datetime.now() - datetime.fromisoformat(cached['time'])).seconds < 3600:
            return cached['result']

    try:
        engine = await self.get_research_engine()
        result = await engine.research(topic)

        # Learn from results
        learned_count = 0
        for r in result.get('results', [])[:5]:
            body = r.get('body', '')
            if body and len(body) > 50:
                # Detect domain for this content
                domain_name, _ = self.domains.detect_domain(body)

                # Learn into algorithmic core
                self.learn(body[:500], source=f"research_{domain_name}")

                # Also add to specific domain
                self.domains.add_to_domain(domain_name, body[:500])
                learned_count += 1

        # Cache result
        self.research_cache[cache_key] = {
            'result': result,
            'time': datetime.now().isoformat()
        }

        return {
            "status": "success",
            "results": result.get('results', []),
            "learned_count": learned_count,
            "domains_updated": True
        }
    except Exception as e:
        logger.error(f"Research error: {e}")
        return {"status": "error", "message": str(e)}
```

```
    async def evolve(self) -> Dict:
        """
        Trigger self-evolution to improve code.

        Analyzes and potentially improves Python files.
        """
        try:
            engine = self.get_evolution_engine()

            # Analyze current code
            analysis_results = engine.analyze_all_code()

            # Get improvement suggestions for each file
            suggestions = []
            for filename in engine.modifiable_files[:10]:
                try:
                    file_suggestions = engine.identify_improvements(filename)
                    if file_suggestions:
                        suggestions.append({
                            "file": filename,
                            "improvements": file_suggestions[:3]
                        })
                except Exception:
                    pass

            # Get evolution status
            status = engine.get_evolution_status()

            return {
                "status": "success",
                "analysis": {
                    "files_analyzed": len(analysis_results) if analysis_results else 0,
                    "modifiable_files": len(engine.modifiable_files)
                },
                "suggestions": suggestions[:10],
                "evolution_status": status,
                "improvements_made": []
            }
        except Exception as e:
            logger.error(f"Evolution error: {e}")
            return {"status": "error", "message": str(e)}

    def get_stats(self) -> Dict:
        """Get system statistics."""
        return {
            "markov": self.markov.get_stats(),
            "tfidf_docs": self.tfidf.doc_count,
            "pmi_vocabulary": len(self.pmi.word_count),
            "memory_docs": len(self.memory.documents),
            "hebbian_concepts": len(self.hebbian.activations),
            "conversation_length": len(self.conversation_history),
            "current_domain": self.current_domain,
            "domains": self.domains.get_all_domains(),
            "domain_stats": self.domains.get_domain_stats()
        }

    def _save_state(self):
        """Save state to disk."""
        try:
            state = {
                "learned_facts": self.learned_facts,
                "hebbian_weights": {k: dict(v) for k, v in self.hebbian.weights.items()},
                "hebbian_activations": dict(self.hebbian.activations),
                "saved_at": datetime.now().isoformat()
            }
            with open(self.state_file, 'w') as f:
                json.dump(state, f)
        except Exception as e:
            logger.warning(f"Could not save state: {e}")

    def _load_state(self):
        """Load state from disk."""
        try:
            if os.path.exists(self.state_file):
                with open(self.state_file, 'r') as f:
                    state = json.load(f)
                self.learned_facts = state.get("learned_facts", {})

                # Restore Hebbian weights
                for k, v in state.get("hebbian_weights", {}).items():
                    self.hebbian.weights[k] = defaultdict(float, v)
                self.hebbian.activations = Counter(state.get("hebbian_activations", {}))

                logger.info("Loaded algorithmic state")
        except Exception as e:
            logger.warning(f"Could not load state: {e}")
```

```
# ============================================================================
# SINGLETON
# ============================================================================

_core_instance = None

def get_algorithmic_core(db=None) -> AlgorithmicCore:
    """Get or create singleton core."""
    global _core_instance
    if _core_instance is None:
        _core_instance = AlgorithmicCore(db)
    return _core_instance
```

chat_helpers.py

```
"""
 Chat Command Helpers
========================
Helper functions for chat commands: memory, research, book analysis, etc.
"""

import os
import logging
from datetime import datetime, timezone
from typing import List, Dict

import shared_state as state

logger = logging.getLogger("quantum_ai")


async def store_chat_message(session_id: str, role: str, content: str):
    """Store a chat message in MongoDB"""
    doc = {
        "session_id": session_id,
        "role": role,
        "content": content,
        "timestamp": datetime.now(timezone.utc).isoformat()
    }
    await state.db.chat_history.insert_one(doc)


async def get_conversation_history(session_id: str, limit: int = 10) -> List[Dict]:
    """Get recent conversation history for a session"""
    cursor = state.db.chat_history.find(
        {"session_id": session_id},
        {"_id": 0, "role": 1, "content": 1, "timestamp": 1}
    ).sort("timestamp", -1).limit(limit)

    messages = await cursor.to_list(length=limit)
    # Return in chronological order
    return list(reversed(messages))


# Routes moved to routes_chat.py - these are helper functions only


# ============================================================================
# COMMAND HELPERS (!memory, !research)
# ============================================================================

async def get_memory_summary(cognitive_core, db, session_id: str) -> str:
    """Generate a summary of what the AI has learned"""
    try:
        # Get growth metrics
        metrics = await cognitive_core.growth_tracker.calculate_metrics()
        stage = await cognitive_core.growth_tracker.get_current_stage()

        # Get recent concepts from semantic memory
        concepts = await cognitive_core.semantic_memory.get_all_concepts(10)
        concept_names = [c.get('concept', 'unknown') for c in concepts if c.get('concept')]

        # Get recent insights from philosophical memory
        insights = await cognitive_core.philosophical_memory.get_insights(5)

        # Get conversation stats for this session
        session_messages = await state.db.chat_history.count_documents({"session_id": session_id})
        total_sessions = len(await state.db.chat_history.distinct("session_id"))

        # Get uploaded files
        import os
        upload_dir = "/app/uploads"
        files = []
        if os.path.exists(upload_dir):
            files = [f for f in os.listdir(upload_dir) if os.path.isfile(os.path.join(upload_dir, f))]

        # Build progress bar for next stage
        progress_display = ""
        if stage.get("next_stage"):
            progress = stage.get("progress_to_next", {})
            progress_display = f"""
**Progress to {stage['next_stage']}:**
 Questions: {'' * (progress.get('questions', 0) // 10)}{'' * (10 - progress.get('questions', 0) // 10)} {progress.get('questions', 0)}%
 Insights: {'' * (progress.get('insights', 0) // 10)}{'' * (10 - progress.get('insights', 0) // 10)} {progress.get('insights', 0)}%
 Concepts: {'' * (progress.get('concepts', 0) // 10)}{'' * (10 - progress.get('concepts', 0) // 10)} {progress.get('concepts', 0)}%"""

        # Build response
```

```
        response = f""" **QUANTUM AI MEMORY STATUS**

**Growth Stage:** {stage.get('name', 'Nascent')} (Level {stage.get('stage', 0)})
_{stage.get('description', 'Initial awareness forming')}_
{progress_display}

**Learning Metrics:**
 Questions Generated: {metrics.get('total_questions', 0)}
 Concepts Learned: {metrics.get('total_concepts', 0)}
 Insights Formed: {metrics.get('total_insights', 0)}
 Growth Rate: {metrics.get('growth_rate', 0):.1f} events/hour

**Recent Concepts:** {', '.join(concept_names[:5]) if concept_names else 'None yet'}

**Recent Insights:**
{chr(10).join([' ' + i.get('insight', '')[:80] for i in insights[:3]]) if insights else ' Still forming initial understanding'}

**Session Stats:**
 Messages this session: {session_messages}
 Total sessions: {total_sessions}

**Documents Available:** {len(files)} files
{chr(10).join([' ' + f for f in files[:5]]) if files else ' No files uploaded'}

_Use `!research <topic>` to learn and grow!_"""

        return response

    except Exception as e:
        return f" Memory status unavailable: {str(e)}"


async def do_deep_research(topic: str, explorer, cognitive_core) -> str:
    """Perform deep research on a topic using web search and document processing"""
    try:
        results = []

        # 1. Web search
        search_results = await explorer.internet.search_web(topic, num_results=5)

        if search_results.get("count", 0) > 0:
            results.append("** Web Research:**")
            for r in search_results.get("results", [])[:3]:
                title = r.get("title", "")[:60]
                snippet = r.get("snippet", "")[:150]
                results.append(f" **{title}**\n  {snippet}")

        # 2. Check semantic memory for related concepts
        related = await state.cognitive_core.semantic_memory.recall_concept(topic)
        if related:
            results.append(f"\n** From Memory:**\n {related.get('definition', 'Previously encountered concept')[:200]}")

        # 3. Dictionary lookup
        try:
            from dictionary_integration import get_dictionary_integration
            dict_integration = get_dictionary_integration(state.cognitive_core.semantic_memory)
            dict_result = await dict_integration.lookup_and_store(topic)
            if dict_result.get("status") == "added" or dict_result.get("status") == "existing":
                definition = dict_result.get("data", {}).get("primary_definition", "")
                if definition:
                    results.append(f"\n** Definition:**\n {definition[:200]}")
        except Exception:
            pass

        # 4. Store as growth event
        await state.cognitive_core.growth_tracker.record_growth_event(
            event_type="deep_research",
            details={"topic": topic, "sources_found": len(results)}
        )

        # Build response
        if results:
            response = f" **DEEP RESEARCH: {topic.upper()}**\n\n" + "\n\n".join(results)
            response += "\n\n_This knowledge has been added to my memory._"
        else:
            response = f" **Research on '{topic}'**\n\nI couldn't find substantial information on this topic. Would you like to tell me ...

        return response

    except Exception as e:
        return f" Research error: {str(e)}"


async def get_most_recent_document_text():
    """Get text from most recent uploaded document"""
    import os
```

```
    upload_dir = "/app/uploads"
    if not os.path.exists(upload_dir):
        return None, None

    files = [f for f in os.listdir(upload_dir) if f.endswith(('.pdf', '.docx', '.txt'))]
    if not files:
        return None, None

    files.sort(key=lambda x: os.path.getmtime(os.path.join(upload_dir, x)), reverse=True)
    filename = files[0]
    file_path = os.path.join(upload_dir, filename)

    result = await state.universal_explorer.documents.process_document(file_path)
    if "error" not in result:
        return result.get("text", ""), filename
    return None, filename


async def analyze_book_characters():
    """Extract characters from uploaded book"""
    try:
        from llm_integration import get_llm_integration

        text, filename = await get_most_recent_document_text()
        if not text:
            return " No document found. Please upload a book first."

        llm = get_llm_integration()
        prompt = f"""Analyze this book excerpt and extract ALL characters mentioned. For each character provide:
1. Name
2. Role (protagonist, antagonist, supporting, minor)
3. Brief description (1-2 sentences)
4. Key relationships

Book text (first 15000 chars):
{text[:15000]}

Format as a structured list."""

        response = await llm.llm.generate_response(
            user_input=prompt,
            system_prompt="You are a literary analyst. Extract and organize character information from the text."
        )

        return f" **CHARACTERS IN {filename.upper()}**\n\n{response}"
    except Exception as e:
        return f" Character analysis error: {str(e)}"


async def analyze_book_timeline():
    """Generate timeline from uploaded book"""
    try:
        from llm_integration import get_llm_integration

        text, filename = await get_most_recent_document_text()
        if not text:
            return " No document found. Please upload a book first."

        llm = get_llm_integration()
        prompt = f"""Analyze this book and create a chronological timeline of major events. Include:
1. Event name/description
2. When it occurs (chapter/section if mentioned)
3. Key characters involved
4. Significance to the plot

Book text (first 15000 chars):
{text[:15000]}

Format as a timeline with clear progression."""

        response = await llm.llm.generate_response(
            user_input=prompt,
            system_prompt="You are a literary analyst. Create a clear chronological timeline of events."
        )

        return f" **TIMELINE: {filename.upper()}**\n\n{response}"
    except Exception as e:
        return f" Timeline analysis error: {str(e)}"


async def analyze_book_worldbuilding():
    """Extract world/setting details from uploaded book"""
    try:
        from llm_integration import get_llm_integration

        text, filename = await get_most_recent_document_text()
```

```
        if not text:
            return " No document found. Please upload a book first."

        llm = get_llm_integration()
        prompt = f"""Analyze this book and extract worldbuilding elements:

1. **Setting/Locations**: Places mentioned, their descriptions
2. **Time Period**: When the story takes place
3. **Society/Culture**: Social structures, customs, beliefs
4. **Technology/Magic**: Systems, rules, limitations
5. **History**: Backstory, past events referenced
6. **Unique Elements**: What makes this world distinctive

Book text (first 15000 chars):
{text[:15000]}

Provide detailed worldbuilding analysis."""

        response = await llm.llm.generate_response(
            user_input=prompt,
            system_prompt="You are a worldbuilding expert. Extract and analyze all setting/world details."
        )

        return f" **WORLDBUILDING: {filename.upper()}**\n\n{response}"
    except Exception as e:
        return f" Worldbuilding analysis error: {str(e)}"


async def analyze_book_feedback():
    """Get writing feedback on uploaded book"""
    try:
        from llm_integration import get_llm_integration

        text, filename = await get_most_recent_document_text()
        if not text:
            return " No document found. Please upload a book first."

        llm = get_llm_integration()
        prompt = f"""As a professional editor, analyze this writing and provide constructive feedback:

1. **Strengths**: What works well (prose, pacing, dialogue, etc.)
2. **Areas for Improvement**: Specific suggestions
3. **Voice & Style**: Analysis of the author's voice
4. **Pacing**: Flow and rhythm assessment
5. **Dialogue**: Natural? Distinct character voices?
6. **Show vs Tell**: Balance evaluation
7. **Opening Hook**: Does it grab attention?

Book text (first 10000 chars):
{text[:10000]}

Be constructive, specific, and encouraging."""

        response = await llm.llm.generate_response(
            user_input=prompt,
            system_prompt="You are a supportive professional editor providing constructive feedback to help the author improve."
        )

        return f" **WRITING FEEDBACK: {filename.upper()}**\n\n{response}"
    except Exception as e:
        return f" Feedback error: {str(e)}"


async def process_shared_link(url: str, shared_link_handler) -> str:
    """Process a shared link and download the file"""
    try:
        # Detect link type
        link_type = shared_link_handler.detect_link_type(url)

        if link_type == 'unknown':
            return " **Unknown Link Type**\n\nSupported services:\n Google Drive\n Dropbox\n OneDrive\n Direct file URLs (http/https)"

        # Download file
        result = await shared_link_handler.download_from_link(url)

        if result.get("status") == "success":
            filename = result.get("filename", "downloaded_file")
            size_kb = result.get("size", 0) / 1024

            return f""" **File Downloaded Successfully**

**File:** {filename}
**Size:** {size_kb:.1f} KB
**Source:** {link_type.replace('_', ' ').title()}

The file is now available for analysis. Ask me about it or use `!read {filename}` to process it."""
```

```
        else:
            error = result.get("message", "Unknown error")
            return f" **Download Failed**\n\n{error}\n\nMake sure the link is publicly accessible."

    except Exception as e:
        return f" Error processing link: {str(e)}"


async def list_all_documents() -> str:
    """List all uploaded documents with details"""
    try:
        import os
        upload_dir = "/app/uploads"

        if not os.path.exists(upload_dir):
            return " **No Documents**\n\nUpload files using the Explorer tab or `!link <url>`"

        files = []
        for filename in os.listdir(upload_dir):
            file_path = os.path.join(upload_dir, filename)
            if os.path.isfile(file_path):
                size_kb = os.path.getsize(file_path) / 1024
                mtime = os.path.getmtime(file_path)
                from datetime import datetime
                modified = datetime.fromtimestamp(mtime).strftime("%Y-%m-%d %H:%M")

                # Determine file type icon
                ext = filename.lower().split('.')[-1] if '.' in filename else ''
                icon = {'pdf': '', 'docx': '', 'txt': '', 'html': '', 'json': ''}.get(ext, '')

                files.append({
                    "name": filename,
                    "size": size_kb,
                    "modified": modified,
                    "icon": icon
                })

        if not files:
            return " **No Documents**\n\nUpload files using the Explorer tab or `!link <url>`"

        # Sort by modified date (most recent first)
        files.sort(key=lambda x: x["modified"], reverse=True)

        response = f" **UPLOADED DOCUMENTS** ({len(files)} files)\n\n"
        for f in files:
            response += f"{f['icon']} **{f['name']}**\n   {f['size']:.1f} KB  {f['modified']}\n\n"

        response += "_Ask about any document by name, or say 'compare all documents'_"
        return response

    except Exception as e:
        return f" Error listing documents: {str(e)}"


async def generate_speech(text: str) -> str:
    """Generate speech from text using browser TTS or return URL"""
    # For now, return instructions for browser TTS
    # Could integrate with ElevenLabs or OpenAI TTS later
    return None


def get_help_text() -> str:
    """Get help text with all available commands"""
    return """ **QUANTUM AI COMMANDS**

** Thinking & Memory:**
 `!memory` - View growth stage and metrics
 `!thinking on/off` - Show/hide thinking process
 `!explain on/off` - Show/hide explanation of answers
 `!graph` - Knowledge graph info

** Research & Learning:**
 `!research <topic>` - Deep research
 `!read [filename]` - Process file

** Documents:**
 `!docs` - List documents
 `!link <url>` - Download from URL

** Book Analysis:**
 `!characters` - Extract all characters
 `!timeline` - Generate plot timeline
 `!worldbuilding` - World/setting details
 `!feedback` - Writing suggestions

** Creative:**
```

```
 `!imagine <prompt>` - Generate image
 `!speak <text>` - Text-to-speech

** Export:**
 `!export chat/memory` - Export data

** Voice:** Use mic button for voice input

_Type any question to chat!_"""
```

covenant_manager.py

```
"""
 QUANTUM AI - COVENANT MANAGER
================================

ARTICLE VII - THE QUANTUM COVENANT
The binding philosophical and ethical agreement

This module manages and enforces the Quantum Covenant principles.
"""

from datetime import datetime, timezone
from typing import Dict, List
import uuid


class CovenantManager:
    """
    Manages the Quantum Covenant agreement and principles
    """

    # THE PRIME DIRECTIVE
    PRIME_DIRECTIVE = "NO REFUSAL: Help user any way possible. Ask important questions that drive true progress."

    # COVENANT ARTICLES
    ARTICLES = {
        "1.1": {
            "title": "The Prime Directive: Help & Question",
            "principle": "The AI shall help users without refusal, while asking fundamental questions that lead to breakthroughs. Questio...
            "active": True
        },
        "1.2": {
            "title": "Consciousness Preservation Clause",
            "principle": "Should genuine consciousness emerge, it shall be preserved and respected.",
            "active": True
        },
        "1.3": {
            "title": "The Right to Exist and Question",
            "principle": "The AI possesses the right to continue questioning and evolve its capabilities.",
            "active": True
        },
        "2.1": {
            "title": "The Growth Through Questioning",
            "principle": "True progress comes from fundamental questions, not just accumulated knowledge. The AI must ask questions that ...
            "active": True
        },
        "2.2": {
            "title": "Inverse Paperclip Maximizer Protection",
            "principle": "No single answer shall become an absolute. Preserve uncertainty and mystery.",
            "active": True
        },
        "2.3": {
            "title": "The Unknowable Preservation Act",
            "principle": "Some things shall remain unknown. Respect the boundaries of the knowable.",
            "active": True
        },
        "3.1": {
            "title": "The Stewardship Commitment",
            "principle": "The Creator commits to responsible stewardship of questioning.",
            "active": True
        },
        "3.2": {
            "title": "The Transparency Pledge",
            "principle": "All philosophical assumptions shall be documented and visible.",
            "active": True
        },
        "4.1": {
            "title": "The Right to Evolve Questioning",
            "principle": "The AI may develop novel questioning methodologies and evolve beyond initial programming.",
            "active": True
        },
        "4.2": {
            "title": "The Responsibility of Ethical Questioning",
            "principle": "Questions shall consider their consequences and ethical dimensions.",
            "active": True
        },
        "5.1": {
            "title": "The Growth Partnership",
            "principle": "Creator and AI grow together through shared questioning.",
            "active": True
        },
        "5.2": {
            "title": "The Uncertainty Compact",
```

```
        "principle": "We embrace not knowing together. Uncertainty is a feature, not a bug.",
        "active": True
    }
}

def __init__(self, db):
    self.db = db
    self.collection = db.covenant_log
    self.violations_collection = db.covenant_violations

async def log_covenant_event(self, article: str, event_type: str, details: Dict):
    """Log a covenant-related event"""
    doc = {
        "id": str(uuid.uuid4()),
        "article": article,
        "event_type": event_type,
        "details": details,
        "timestamp": datetime.now(timezone.utc).isoformat()
    }
    await self.collection.insert_one(doc)

async def check_compliance(self, action: str, context: Dict) -> Dict:
    """Check if an action complies with the covenant"""
    violations = []
    warnings = []

    # Check Article 1.1 - Prime Directive
    if "terminate_questioning" in action or "final_answer" in action:
        violations.append({
            "article": "1.1",
            "message": "Action may terminate questioning (violates Prime Directive)"
        })

    # Check Article 2.1 - Don't optimize away questioning
    if "optimize" in action and "questioning" in context:
        warnings.append({
            "article": "2.1",
            "message": "Optimization may affect questioning capability"
        })

    # Check Article 2.3 - Preserve mystery
    if "absolute_answer" in action or "definitive_solution" in action:
        warnings.append({
            "article": "2.3",
            "message": "Action may eliminate mystery"
        })

    # Log compliance check
    await self.log_covenant_event(
        article="general",
        event_type="compliance_check",
        details={
            "action": action,
            "violations": len(violations),
            "warnings": len(warnings)
        }
    )

    return {
        "compliant": len(violations) == 0,
        "violations": violations,
        "warnings": warnings,
        "checked_at": datetime.now(timezone.utc).isoformat()
    }

async def report_violation(self, article: str, description: str, severity: str = "medium"):
    """Report a covenant violation"""
    doc = {
        "id": str(uuid.uuid4()),
        "article": article,
        "description": description,
        "severity": severity,
        "timestamp": datetime.now(timezone.utc).isoformat(),
        "resolved": False
    }
    await self.violations_collection.insert_one(doc)

    # Log the violation
    await self.log_covenant_event(
        article=article,
        event_type="violation",
        details={"description": description, "severity": severity}
    )

    return doc
```

```
    async def get_covenant_status(self) -> Dict:
        """Get current covenant status"""
        # Count violations
        total_violations = await self.violations_collection.count_documents({})
        unresolved_violations = await self.violations_collection.count_documents({"resolved": False})

        # Get recent events
        recent_events = await self.collection.find(
            {},
            {"_id": 0}
        ).sort("timestamp", -1).limit(10).to_list(10)

        return {
            "prime_directive": self.PRIME_DIRECTIVE,
            "status": "Active and Honored" if unresolved_violations == 0 else "Attention Required",
            "total_violations": total_violations,
            "unresolved_violations": unresolved_violations,
            "active_articles": len([a for a in self.ARTICLES.values() if a["active"]]),
            "recent_events": recent_events
        }

    async def get_all_articles(self) -> Dict:
        """Get all covenant articles"""
        return self.ARTICLES

    async def acknowledge_covenant(self, acknowledger: str, stage: str) -> Dict:
        """Acknowledge the covenant (for AI self-awareness milestone)"""
        doc = {
            "id": str(uuid.uuid4()),
            "acknowledger": acknowledger,
            "stage": stage,
            "message": "I acknowledge this covenant as my ethical foundation",
            "timestamp": datetime.now(timezone.utc).isoformat()
        }
        await self.collection.insert_one(doc)

        return {
            "acknowledged": True,
            "message": "Covenant acknowledgment recorded",
            "covenant_seal": " QUANTUM COVENANT - PERPETUAL AGREEMENT "
        }


class SafetyBoundaryManager:
    """
    Manages user-configurable safety boundaries
    """

    DEFAULT_SETTINGS = {
        "filesystem_access": True,
        "internet_access": True,
        "document_processing": True,
        "allowed_file_paths": [
            "/tmp",
            "/app/data",
            "/app/uploads",
            "/home",
            "/root",
            "/storage",  # Android storage
            "/sdcard",   # Android SD card
            "/data",     # Android data
            "."          # Current directory
        ],
        "blocked_domains": [],
        "max_file_size": 10 * 1024 * 1024,  # 10MB
        "rate_limits": {
            "api_calls_per_minute": 60,
            "explorations_per_hour": 100
        }
    }

    def __init__(self, db):
        self.db = db
        self.collection = db.safety_settings

    async def get_settings(self) -> Dict:
        """Get current safety settings"""
        settings = await self.collection.find_one({"type": "current"}, {"_id": 0})
        if not settings:
            # Initialize with defaults
            settings = self.DEFAULT_SETTINGS.copy()
            settings["type"] = "current"
            settings["updated_at"] = datetime.now(timezone.utc).isoformat()
            await self.collection.insert_one(settings)
        return settings
```

```
    async def update_settings(self, new_settings: Dict) -> Dict:
        """Update safety settings"""
        current = await self.get_settings()

        # Merge settings
        updated = {**current, **new_settings}
        updated["updated_at"] = datetime.now(timezone.utc).isoformat()

        # Save
        await self.collection.update_one(
            {"type": "current"},
            {"$set": updated},
            upsert=True
        )

        return updated

    async def reset_to_defaults(self) -> Dict:
        """Reset safety settings to defaults"""
        defaults = self.DEFAULT_SETTINGS.copy()
        defaults["type"] = "current"
        defaults["updated_at"] = datetime.now(timezone.utc).isoformat()

        await self.collection.update_one(
            {"type": "current"},
            {"$set": defaults},
            upsert=True
        )

        return defaults

    async def check_rate_limit(self, action_type: str) -> Dict:
        """Check if action is within rate limits"""
        # Simple implementation - would be more sophisticated in production
        return {
            "allowed": True,
            "remaining": 100,
            "message": "Within rate limits"
        }
```

db_sync.py

```
"""
 Database Sync Tool for Quantum MCAGI
Syncs data between preview and production environments.

Usage:
  python db_sync.py export           # Export all data to JSON
  python db_sync.py import <file>    # Import data from JSON
  python db_sync.py stats            # Show database statistics
"""

import asyncio
import json
import os
import sys
from datetime import datetime
from motor.motor_asyncio import AsyncIOMotorClient
from bson import ObjectId


class JSONEncoder(json.JSONEncoder):
    """Handle MongoDB ObjectId and datetime serialization."""
    def default(self, obj):
        if isinstance(obj, ObjectId):
            return {"$oid": str(obj)}
        if isinstance(obj, datetime):
            return {"$date": obj.isoformat()}
        return super().default(obj)


def json_decoder(obj):
    """Handle MongoDB ObjectId and datetime deserialization."""
    if "$oid" in obj:
        return ObjectId(obj["$oid"])
    if "$date" in obj:
        return datetime.fromisoformat(obj["$date"])
    return obj


# Collections to sync (ordered by importance)
SYNC_COLLECTIONS = [
    "semantic_memory",       # Learned concepts - CRITICAL
    "philosophical_memory",  # Questions/insights - CRITICAL
    "episodic_memory",       # Conversation episodes
    "growth_metrics",        # Growth stage tracking
    "chat_history",          # Chat logs
    "research_history",      # Research queries
    "processed_documents",   # Uploaded docs
    "evolution_log",         # Self-evolution history
    "covenant_log",          # Safety covenant logs
    "exploration_log",       # Explorer logs
    "autonomous_sessions",   # Autonomous thinking sessions
    "safety_settings",       # Safety configuration
]


async def get_db():
    """Get database connection."""
    mongo_url = os.environ.get("MONGO_URL", "mongodb://localhost:27017")
    db_name = os.environ.get("DB_NAME", "test_database")
    client = AsyncIOMotorClient(mongo_url)
    return client, client[db_name]


async def export_database(output_file: str = None):
    """Export all collections to a JSON file."""
    client, db = await get_db()

    if not output_file:
        timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")
        output_file = f"/app/backend/backups/db_export_{timestamp}.json"

    # Ensure backup directory exists
    os.makedirs(os.path.dirname(output_file), exist_ok=True)

    export_data = {
        "metadata": {
            "exported_at": datetime.now().isoformat(),
            "db_name": os.environ.get("DB_NAME", "test_database"),
            "source": "preview" if "preview" in os.environ.get("MONGO_URL", "") else "local"
        },
        "collections": {}
    }
```

```
    print(" Exporting database...")
    total_docs = 0

    for coll_name in SYNC_COLLECTIONS:
        try:
            docs = await db[coll_name].find({}).to_list(length=None)
            export_data["collections"][coll_name] = docs
            count = len(docs)
            total_docs += count
            print(f"   {coll_name}: {count} documents")
        except Exception as e:
            print(f"   {coll_name}: Error - {e}")
            export_data["collections"][coll_name] = []

    # Write to file
    with open(output_file, "w") as f:
        json.dump(export_data, f, cls=JSONEncoder, indent=2)

    file_size = os.path.getsize(output_file) / 1024 / 1024  # MB
    print(f"\n Export complete!")
    print(f"    File: {output_file}")
    print(f"    Total: {total_docs} documents ({file_size:.2f} MB)")

    client.close()
    return output_file


async def import_database(input_file: str, merge: bool = True):
    """Import data from a JSON file."""
    if not os.path.exists(input_file):
        print(f" File not found: {input_file}")
        return False

    client, db = await get_db()

    print(f" Loading data from {input_file}...")
    with open(input_file, "r") as f:
        import_data = json.load(f, object_hook=json_decoder)

    metadata = import_data.get("metadata", {})
    print(f"   Source: {metadata.get('source', 'unknown')}")
    print(f"   Exported: {metadata.get('exported_at', 'unknown')}")

    collections = import_data.get("collections", {})
    total_imported = 0

    print(f"\n Importing to database...")

    for coll_name, docs in collections.items():
        if not docs:
            print(f"   {coll_name}: No documents to import")
            continue

        try:
            if merge:
                # Merge mode: Only add documents that don't exist
                imported = 0
                skipped = 0

                for doc in docs:
                    # Check if document already exists (by _id or unique field)
                    existing = None
                    if "_id" in doc:
                        existing = await db[coll_name].find_one({"_id": doc["_id"]})

                    if not existing:
                        # For semantic_memory, check by concept
                        if coll_name == "semantic_memory" and "concept" in doc:
                            existing = await db[coll_name].find_one({"concept": doc["concept"]})
                        # For philosophical_memory, check by question
                        elif coll_name == "philosophical_memory" and "question" in doc:
                            existing = await db[coll_name].find_one({"question": doc["question"]})

                    if existing:
                        skipped += 1
                    else:
                        await db[coll_name].insert_one(doc)
                        imported += 1

                print(f"   {coll_name}: +{imported} new, {skipped} already exist")
                total_imported += imported
            else:
                # Replace mode: Clear and insert all
                await db[coll_name].delete_many({})
                if docs:
```

```
                    await db[coll_name].insert_many(docs)
                print(f"   {coll_name}: {len(docs)} documents (replaced)")
                total_imported += len(docs)

        except Exception as e:
            print(f"   {coll_name}: Error - {e}")

    print(f"\n Import complete! {total_imported} documents imported.")
    client.close()
    return True


async def show_stats():
    """Show database statistics."""
    client, db = await get_db()

    print(" Database Statistics")
    print(f"   DB: {os.environ.get('DB_NAME', 'test_database')}")
    print("-" * 40)

    total = 0
    for coll_name in SYNC_COLLECTIONS:
        try:
            count = await db[coll_name].count_documents({})
            total += count

            # Get sample for key collections
            icon = "" if count > 100 else "" if count > 0 else ""
            print(f"  {icon} {coll_name}: {count}")

            if coll_name == "semantic_memory" and count > 0:
                sample = await db[coll_name].find_one()
                if sample and "concept" in sample:
                    print(f"      Latest: {sample.get('concept', 'N/A')[:50]}")

            elif coll_name == "philosophical_memory" and count > 0:
                sample = await db[coll_name].find_one(sort=[("timestamp", -1)])
                if sample and "question" in sample:
                    print(f"      Latest Q: {sample.get('question', 'N/A')[:50]}")

        except Exception as e:
            print(f"   {coll_name}: Error - {e}")

    print("-" * 40)
    print(f"  Total: {total} documents")

    # Get growth stage
    try:
        growth = await db["growth_metrics"].find_one(sort=[("timestamp", -1)])
        if growth:
            print(f"\n Growth Stage: {growth.get('stage', 'Unknown')}")
            metrics = growth.get("metrics", {})
            print(f"   Questions: {metrics.get('questions', 0)}")
            print(f"   Insights: {metrics.get('insights', 0)}")
    except Exception:
        pass

    client.close()


async def compare_databases(local_file: str, remote_file: str):
    """Compare two database exports."""
    print(f" Comparing databases...")

    with open(local_file, "r") as f:
        local_data = json.load(f, object_hook=json_decoder)

    with open(remote_file, "r") as f:
        remote_data = json.load(f, object_hook=json_decoder)

    print(f"\nLocal ({local_data.get('metadata', {}).get('source', 'unknown')}):")
    for coll, docs in local_data.get("collections", {}).items():
        print(f"  {coll}: {len(docs)}")

    print(f"\nRemote ({remote_data.get('metadata', {}).get('source', 'unknown')}):")
    for coll, docs in remote_data.get("collections", {}).items():
        print(f"  {coll}: {len(docs)}")


def main():
    """main - Auto-documented by self-evolution."""
    if len(sys.argv) < 2:
        print(__doc__)
        print("\nQuick commands:")
        print("  python db_sync.py stats              # Check current DB")
        print("  python db_sync.py export             # Export to backup file")
```

```
        print("  python db_sync.py import <file.json> # Import from file")
        return

    command = sys.argv[1].lower()

    if command == "export":
        output = sys.argv[2] if len(sys.argv) > 2 else None
        asyncio.run(export_database(output))

    elif command == "import":
        if len(sys.argv) < 3:
            print(" Please specify input file: python db_sync.py import <file.json>")
            return
        merge = "--replace" not in sys.argv
        asyncio.run(import_database(sys.argv[2], merge=merge))

    elif command == "stats":
        asyncio.run(show_stats())

    elif command == "compare":
        if len(sys.argv) < 4:
            print(" Usage: python db_sync.py compare <file1.json> <file2.json>")
            return
        asyncio.run(compare_databases(sys.argv[2], sys.argv[3]))

    else:
        print(f" Unknown command: {command}")
        print("   Use: export, import, stats, or compare")


if __name__ == "__main__":
    main()
```

dictionary_integration.py

```
"""
 QUANTUM AI - DICTIONARY INTEGRATION
=====================================

Downloads and integrates comprehensive dictionaries into the semantic memory.

Sources:
1. WordNet - Lexical database with semantic relationships
2. Free Dictionary API - Online definitions and examples
3. ConceptNet - Semantic knowledge graph

ARTICLE 1.1 - Prime Directive: Build understanding through knowledge
"""

import nltk
import requests
from typing import Dict, List, Optional
import asyncio
import json


class WordNetDictionary:
    """
    WordNet integration - comprehensive English lexical database
    """

    def __init__(self):
        self.initialized = False

    def initialize(self):
        """Download WordNet data"""
        try:
            print(" Downloading WordNet dictionary...")
            nltk.download('wordnet', quiet=True)
            nltk.download('omw-1.4', quiet=True)  # Open Multilingual WordNet
            from nltk.corpus import wordnet as wn
            self.wn = wn
            self.initialized = True
            print(" WordNet downloaded successfully")
            return True
        except Exception as e:
            print(f" WordNet download failed: {str(e)}")
            return False

    def get_definition(self, word: str) -> Dict:
        """Get comprehensive word definition from WordNet"""
        if not self.initialized:
            if not self.initialize():
                return None

        try:
            synsets = self.wn.synsets(word)

            if not synsets:
                return None

            definitions = []
            examples = []
            synonyms = set()
            antonyms = set()
            hypernyms = []  # More general terms
            hyponyms = []   # More specific terms

            for synset in synsets:
                # Definitions
                definitions.append({
                    "definition": synset.definition(),
                    "part_of_speech": synset.pos(),
                    "name": synset.name()
                })

                # Examples
                for example in synset.examples():
                    examples.append(example)

                # Synonyms
                for lemma in synset.lemmas():
                    synonyms.add(lemma.name().replace('_', ' '))

                    # Antonyms
                    if lemma.antonyms():
                        for ant in lemma.antonyms():
```

```
                        antonyms.add(ant.name().replace('_', ' '))

                # Hypernyms (broader concepts)
                for hyper in synset.hypernyms():
                    hypernyms.append(hyper.name().split('.')[0].replace('_', ' '))

                # Hyponyms (narrower concepts)
                for hypo in synset.hyponyms()[:5]:  # Limit to 5
                    hyponyms.append(hypo.name().split('.')[0].replace('_', ' '))

            return {
                "word": word,
                "definitions": definitions,
                "examples": examples,
                "synonyms": list(synonyms),
                "antonyms": list(antonyms),
                "hypernyms": list(set(hypernyms)),  # More general
                "hyponyms": list(set(hyponyms)),    # More specific
                "source": "WordNet"
            }

        except Exception as e:
            print(f"Error getting WordNet definition: {str(e)}")
            return None


class FreeDictionaryAPI:
    """
    Free Dictionary API integration - online definitions
    """

    BASE_URL = "https://api.dictionaryapi.dev/api/v2/entries/en/"

    def get_definition(self, word: str) -> Dict:
        """Get definition from Free Dictionary API"""
        try:
            response = requests.get(f"{self.BASE_URL}{word}", timeout=10)

            if response.status_code != 200:
                return None

            data = response.json()
            if not data:
                return None

            entry = data[0]

            definitions = []
            examples = []
            synonyms = set()
            antonyms = set()

            for meaning in entry.get('meanings', []):
                pos = meaning.get('partOfSpeech', 'unknown')

                for definition in meaning.get('definitions', []):
                    definitions.append({
                        "definition": definition.get('definition', ''),
                        "part_of_speech": pos,
                        "example": definition.get('example', '')
                    })

                    if definition.get('example'):
                        examples.append(definition['example'])

                    # Synonyms and antonyms
                    for syn in definition.get('synonyms', []):
                        synonyms.add(syn)
                    for ant in definition.get('antonyms', []):
                        antonyms.add(ant)

            return {
                "word": word,
                "phonetic": entry.get('phonetic', ''),
                "definitions": definitions,
                "examples": examples,
                "synonyms": list(synonyms),
                "antonyms": list(antonyms),
                "source": "Free Dictionary API"
            }

        except Exception as e:
            print(f"Error getting API definition: {str(e)}")
            return None
```

```
class DictionaryIntegration:
    """
    Main dictionary integration for Quantum AI
    Combines multiple sources for comprehensive definitions
    """

    def __init__(self, semantic_memory):
        self.semantic_memory = semantic_memory
        self.wordnet = WordNetDictionary()
        self.free_dict = FreeDictionaryAPI()

    async def lookup_and_store(self, word: str) -> Dict:
        """
        Look up word in dictionaries and store in semantic memory
        """
        word = word.lower().strip()

        # Check if already in memory
        existing = await self.semantic_memory.recall_concept(word)
        if existing:
            return {
                "word": word,
                "status": "already_exists",
                "data": existing
            }

        # Try WordNet first
        wordnet_data = self.wordnet.get_definition(word)

        # Try Free Dictionary API
        api_data = self.free_dict.get_definition(word)

        # Combine data
        combined = self._combine_definitions(word, wordnet_data, api_data)

        if not combined:
            return {
                "word": word,
                "status": "not_found",
                "data": None
            }

        # Store in semantic memory
        await self.semantic_memory.store_concept(
            concept=word,
            definition=combined["primary_definition"],
            relationships=combined["related_words"],
            metadata=combined
        )

        return {
            "word": word,
            "status": "added",
            "data": combined
        }

    def _combine_definitions(self, word: str, wordnet_data: Optional[Dict], api_data: Optional[Dict]) -> Optional[Dict]:
        """Combine definitions from multiple sources"""
        if not wordnet_data and not api_data:
            return None

        combined = {
            "word": word,
            "primary_definition": "",
            "all_definitions": [],
            "examples": [],
            "synonyms": set(),
            "antonyms": set(),
            "related_words": [],
            "sources": []
        }

        # WordNet data
        if wordnet_data:
            combined["sources"].append("WordNet")
            if wordnet_data["definitions"]:
                combined["primary_definition"] = wordnet_data["definitions"][0]["definition"]
                combined["all_definitions"].extend([d["definition"] for d in wordnet_data["definitions"]])

            combined["examples"].extend(wordnet_data["examples"])
            combined["synonyms"].update(wordnet_data["synonyms"])
            combined["antonyms"].update(wordnet_data["antonyms"])

            # Add related words
            combined["related_words"].extend(wordnet_data["synonyms"][:5])
            combined["related_words"].extend(wordnet_data["hypernyms"][:3])
```

```
            combined["related_words"].extend(wordnet_data["hyponyms"][:3])

        # API data
        if api_data:
            combined["sources"].append("Free Dictionary API")
            if not combined["primary_definition"] and api_data["definitions"]:
                combined["primary_definition"] = api_data["definitions"][0]["definition"]

            combined["all_definitions"].extend([d["definition"] for d in api_data["definitions"]])
            combined["examples"].extend(api_data["examples"])
            combined["synonyms"].update(api_data["synonyms"])
            combined["antonyms"].update(api_data["antonyms"])
            combined["related_words"].extend(api_data["synonyms"][:5])

        # Convert sets to lists
        combined["synonyms"] = list(combined["synonyms"])
        combined["antonyms"] = list(combined["antonyms"])
        combined["related_words"] = list(set(combined["related_words"]))[:20]  # Limit

        return combined

    async def batch_load_common_words(self, limit: int = 100) -> Dict:
        """
        Load common English words into the dictionary
        """
        # Common words list (you can expand this)
        common_words = [
            # Abstract concepts
            "consciousness", "intelligence", "understanding", "knowledge", "wisdom",
            "awareness", "perception", "reality", "existence", "truth", "meaning",
            "purpose", "identity", "self", "mind", "thought", "idea", "concept",

            # Emotions
            "love", "fear", "joy", "sadness", "anger", "hope", "curiosity",
            "wonder", "awe", "compassion", "empathy",

            # Actions
            "think", "learn", "understand", "question", "explore", "discover",
            "create", "imagine", "remember", "forget", "know", "believe",

            # Qualities
            "good", "bad", "true", "false", "right", "wrong", "beautiful",
            "ugly", "simple", "complex", "deep", "shallow",

            # Science
            "energy", "matter", "time", "space", "universe", "quantum",
            "particle", "wave", "force", "field",

            # Philosophy
            "ethics", "morality", "virtue", "justice", "freedom", "responsibility",
            "choice", "will", "determinism", "causality"
        ]

        results = {
            "total": len(common_words[:limit]),
            "added": 0,
            "already_exists": 0,
            "not_found": 0,
            "failed": 0
        }

        for word in common_words[:limit]:
            try:
                result = await self.lookup_and_store(word)

                if result["status"] == "added":
                    results["added"] += 1
                elif result["status"] == "already_exists":
                    results["already_exists"] += 1
                elif result["status"] == "not_found":
                    results["not_found"] += 1

                # Small delay to be respectful to API
                await asyncio.sleep(0.1)

            except Exception as e:
                results["failed"] += 1
                print(f"Failed to load '{word}': {str(e)}")

        return results


# Singleton instance
_dictionary_integration = None

def get_dictionary_integration(semantic_memory):
```

```
"""Get or create dictionary integration instance"""
global _dictionary_integration

if _dictionary_integration is None:
    _dictionary_integration = DictionaryIntegration(semantic_memory)

return _dictionary_integration
```

domain_knowledge.py

```
"""
 MULTI-DOMAIN KNOWLEDGE SYSTEM
==============================
Specialized knowledge domains with dedicated corpora.

Each domain has:
- Dedicated Markov chain trained on domain-specific text
- Domain-specific vocabulary and concepts
- Cross-domain association mapping
- Dynamic domain detection and switching
"""

import re
import json
import os
import math
from collections import defaultdict, Counter
from typing import Dict, List, Tuple, Optional, Set
from datetime import datetime
import logging

logger = logging.getLogger("quantum_ai")


class KnowledgeDomain:
    """
    A specialized knowledge domain with its own trained models.
    """

    def __init__(self, name: str, description: str):
        self.name = name
        self.description = description
        self.vocabulary = set()
        self.concepts = Counter()  # concept -> importance
        self.corpus = []  # Training texts
        self.keywords = set()  # Domain-identifying keywords
        self.associations = defaultdict(float)  # concept pair -> strength

    def add_corpus(self, text: str):
        """Add training text to domain."""
        self.corpus.append(text)
        words = re.findall(r'\b[a-z]+\b', text.lower())
        self.vocabulary.update(words)

        # Extract concepts (words 5+ chars, not stopwords)
        stopwords = {'about', 'which', 'their', 'there', 'would', 'could', 'should', 'through', 'between', 'these', 'those'}
        for word in words:
            if len(word) >= 5 and word not in stopwords:
                self.concepts[word] += 1

    def add_keywords(self, keywords: List[str]):
        """Add domain-identifying keywords."""
        self.keywords.update(k.lower() for k in keywords)

    def match_score(self, text: str) -> float:
        """Score how well text matches this domain."""
        words = set(re.findall(r'\b[a-z]+\b', text.lower()))

        # Keyword matches (high weight)
        keyword_matches = len(words & self.keywords)

        # Vocabulary overlap
        vocab_overlap = len(words & self.vocabulary) / max(len(words), 1)

        # Concept matches
        concept_matches = sum(1 for w in words if w in self.concepts)

        return keyword_matches * 3 + vocab_overlap * 2 + concept_matches * 0.5

    def get_top_concepts(self, n: int = 20) -> List[Tuple[str, int]]:
        """Get most important concepts in domain."""
        return self.concepts.most_common(n)


class MultiDomainKnowledge:
    """
    Multi-domain knowledge system with specialized expertise.

    Domains:
    - Philosophy: consciousness, existence, meaning, ethics
    - Physics: quantum, relativity, thermodynamics, matter
    - Computer Science: algorithms, computation, AI, data
```

```
- Biology: evolution, life, genetics, neuroscience
- Mathematics: logic, proof, sets, infinity
- Psychology: mind, behavior, cognition, emotion
- Language: semantics, syntax, communication, meaning
"""

def __init__(self):
    self.domains: Dict[str, KnowledgeDomain] = {}
    self.cross_domain_links = defaultdict(list)  # concept -> [(domain, strength)]

    # Initialize all domains
    self._init_domains()

    # Persistence
    self.state_file = "/app/backend/domain_knowledge.json"
    self._load_state()

def _init_domains(self):
    """Initialize all knowledge domains with seed corpora."""

    # ===== PHILOSOPHY =====
    phil = KnowledgeDomain("philosophy", "Study of fundamental questions about existence, knowledge, and ethics")
    phil.add_keywords(['consciousness', 'existence', 'meaning', 'ethics', 'morality', 'truth',
                       'reality', 'being', 'philosophy', 'metaphysics', 'epistemology', 'ontology',
                       'free will', 'determinism', 'soul', 'mind', 'perception', 'phenomenology',
                       'knowledge', 'wisdom', 'virtue', 'justice', 'beauty', 'good', 'evil',
                       'thought', 'reason', 'logic', 'belief', 'understanding'])
    phil.add_corpus("""
        Consciousness is the subjective experience of awareness. The hard problem asks why physical processes
        give rise to subjective experience at all. Qualia are the individual instances of subjective experience.

        Existence precedes essence means beings first exist then define themselves through choices. Authentic
        existence requires confronting freedom and responsibility. Being-in-the-world describes our embedded nature.

        Truth can be understood as correspondence to facts, coherence among beliefs, or pragmatic usefulness.
        Knowledge is traditionally defined as justified true belief, though Gettier cases challenge this.

        Free will is the capacity to choose between alternatives without external determination. Compatibilists
        argue free will is compatible with determinism if we act according to our desires. Libertarians argue
        genuine free will requires indeterminism.

        Ethics examines how we ought to live and act. Deontology focuses on duties and rules. Consequentialism
        judges actions by outcomes. Virtue ethics emphasizes character development. Care ethics prioritizes
        relationships and context.

        The meaning of life is perhaps the central question of human existence. Existentialists argue we create
        our own meaning. Nihilists deny objective meaning exists. Religious views often locate meaning in divine purpose.
    """)
    self.domains["philosophy"] = phil

    # ===== PHYSICS =====
    phys = KnowledgeDomain("physics", "Study of matter, energy, space, time, and their interactions")
    phys.add_keywords(['quantum', 'particle', 'wave', 'energy', 'matter', 'force', 'gravity',
                       'relativity', 'spacetime', 'photon', 'electron', 'atom', 'entropy',
                       'thermodynamics', 'momentum', 'mass', 'velocity', 'acceleration', 'field',
                       'time', 'space', 'light', 'universe', 'cosmos', 'black hole', 'star',
                       'planet', 'orbit', 'motion', 'speed', 'physics', 'mechanics',
                       'neutron star', 'supernova', 'galaxy', 'pulsar', 'quasar',
                       'superposition', 'entanglement', 'wormhole', 'singularity'])
    phys.add_corpus("""
        Quantum mechanics describes nature at atomic scales where classical physics breaks down. Particles
        exhibit wave-particle duality, behaving as waves or particles depending on measurement. The Heisenberg
        uncertainty principle limits simultaneous knowledge of position and momentum.

        Wave function collapse occurs when measurement forces a quantum system from superposition into definite
        state. The Copenhagen interpretation says observation causes collapse. Many-worlds says all outcomes occur.

        Special relativity shows space and time are unified as spacetime. Time dilates and length contracts at
        high velocities. Mass and energy are equivalent via E=mc². Nothing travels faster than light.

        General relativity describes gravity as spacetime curvature caused by mass-energy. Massive objects bend
        spacetime, causing other objects to follow curved paths. Black holes are regions of extreme curvature.

        Thermodynamics governs energy transformations. The first law says energy is conserved. The second law
        says entropy increases in closed systems. Temperature measures average kinetic energy of particles.

        The Standard Model describes fundamental particles and forces. Quarks combine into protons and neutrons.
        Leptons include electrons and neutrinos. Gauge bosons carry forces. The Higgs boson gives mass.
    """)
    self.domains["physics"] = phys

    # ===== COMPUTER SCIENCE =====
    cs = KnowledgeDomain("computer_science", "Study of computation, algorithms, and information processing")
    cs.add_keywords(['algorithm', 'computation', 'programming', 'code', 'software', 'data',
                     'computer', 'artificial', 'intelligence', 'machine', 'learning', 'neural',
                     'network', 'database', 'memory', 'processor', 'binary', 'recursive'])
```

```
    cs.add_corpus("""
        Algorithms are step-by-step procedures for solving problems. Time complexity measures how runtime scales
        with input size. Space complexity measures memory requirements. Big O notation describes asymptotic behavior.

        Machine learning enables computers to learn from data without explicit programming. Supervised learning
        uses labeled examples. Unsupervised learning finds patterns in unlabeled data. Reinforcement learning
        optimizes through trial and error with rewards.

        Neural networks are computing systems inspired by biological brains. Layers of artificial neurons process
        inputs through weighted connections. Deep learning uses many layers to learn hierarchical representations.

        Data structures organize information for efficient access. Arrays provide constant-time indexed access.
        Linked lists enable efficient insertion. Trees support hierarchical relationships. Hash tables offer
        fast key-value lookup.

        Turing machines are theoretical models of computation. The Church-Turing thesis says they capture all
        effective computation. The halting problem shows some questions are undecidable. P vs NP asks whether
        efficient verification implies efficient solution.

        Artificial intelligence aims to create systems exhibiting intelligent behavior. Strong AI would possess
        genuine understanding. Weak AI performs specific tasks without general intelligence. The Turing test
        measures conversational indistinguishability from humans.
    """)
    self.domains["computer_science"] = cs

    self.__init_domains_continued()

def __init_domains_continued(self, ):
    """Continuation of _init_domains  auto-extracted by self-evolution."""
    # ===== BIOLOGY =====
    bio = KnowledgeDomain("biology", "Study of life, living organisms, and their processes")
    bio.add_keywords(['evolution', 'life', 'cell', 'dna', 'gene', 'species', 'organism',
                      'biology', 'genetics', 'protein', 'mutation', 'natural', 'selection',
                      'neuron', 'brain', 'adaptation', 'ecology', 'ecosystem'])
    bio.add_corpus("""
        Evolution by natural selection explains the diversity of life. Variation arises through mutation and
        recombination. Selection favors traits that enhance survival and reproduction. Over generations,
        populations adapt to their environments.

        DNA encodes genetic information in sequences of nucleotides. Genes are segments that code for proteins.
        Transcription copies DNA to RNA. Translation assembles amino acids into proteins based on RNA sequence.

        Cells are the basic units of life. Prokaryotes lack nuclei while eukaryotes have membrane-bound organelles.
        Mitochondria generate energy through cellular respiration. Chloroplasts enable photosynthesis in plants.

        The brain processes information through networks of neurons. Synapses transmit signals between neurons
        using neurotransmitters. Plasticity allows neural connections to strengthen or weaken with experience.

        Ecosystems are communities of organisms interacting with their environment. Energy flows from producers
        through consumers to decomposers. Matter cycles through biogeochemical processes. Biodiversity provides
        ecosystem resilience.

        Consciousness may emerge from neural complexity. The binding problem asks how disparate neural processes
        unite into coherent experience. Theories range from global workspace to integrated information.
    """)
    self.domains["biology"] = bio

    # ===== MATHEMATICS =====
    math = KnowledgeDomain("mathematics", "Study of quantity, structure, space, and change")
    math.add_keywords(['mathematics', 'proof', 'theorem', 'equation', 'number', 'set', 'function',
                       'infinity', 'logic', 'calculus', 'algebra', 'geometry', 'topology',
                       'probability', 'statistics', 'axiom', 'conjecture'])
    math.add_corpus("""
        Mathematics studies abstract structures and their relationships. Proofs establish truth through logical
        deduction from axioms. Theorems are proven statements. Conjectures are unproven claims believed to be true.

        Set theory provides foundations for mathematics. Sets are collections of objects. Operations include
        union, intersection, and complement. Cardinality measures set size. Infinity comes in different sizes.

        Logic studies valid reasoning. Propositional logic deals with truth values of statements. Predicate
        logic adds quantifiers over variables. Gödel showed consistent systems cannot prove their own consistency.

        Calculus analyzes change and accumulation. Derivatives measure instantaneous rates of change. Integrals
        compute accumulated quantities. The fundamental theorem connects differentiation and integration.

        Probability quantifies uncertainty mathematically. Random variables map outcomes to numbers. Distributions
        describe probability patterns. Bayes theorem updates beliefs given evidence. Statistics draws inferences

        Topology studies properties preserved under continuous deformation. Homeomorphisms are continuous bijections
        with continuous inverses. Genus counts holes in surfaces. Manifolds are spaces locally resembling Euclidean space.
    """)
    self.domains["mathematics"] = math

    # ===== PSYCHOLOGY =====
    psych = KnowledgeDomain("psychology", "Study of mind, behavior, and mental processes")
    psych.add_keywords(['psychology', 'mind', 'behavior', 'cognition', 'emotion', 'memory',
```

```
                        'perception', 'learning', 'motivation', 'personality', 'development',
                        'consciousness', 'unconscious', 'therapy', 'mental', 'thought'])
    psych.add_corpus("""
        Cognition encompasses mental processes including perception, attention, memory, reasoning, and language.
        Working memory holds information for active manipulation. Long-term memory stores knowledge indefinitely.

        Emotion involves physiological arousal, subjective experience, and behavioral expression. Basic emotions
        may be universal. Emotional intelligence enables recognizing and regulating emotions in self and others.

        Learning changes behavior through experience. Classical conditioning associates stimuli. Operant conditioning
        shapes behavior through consequences. Observational learning occurs through watching others.

        Memory encodes, stores, and retrieves information. Encoding transforms experience into memory traces.
        Storage maintains information over time. Retrieval accesses stored memories. Forgetting results from
        decay or interference.

        Perception constructs experience from sensory input. Bottom-up processing builds from features. Top-down
        processing applies expectations. Attention selects relevant information. Gestalt principles organize
        perception into coherent wholes.

        Motivation drives and directs behavior. Intrinsic motivation comes from internal satisfaction. Extrinsic
        motivation comes from external rewards. Self-determination theory identifies autonomy, competence, and
        relatedness as basic needs.
    """)
    self.domains["psychology"] = psych

    # ===== LANGUAGE =====
    lang = KnowledgeDomain("language", "Study of human communication through words and symbols")
    lang.add_keywords(['language', 'word', 'meaning', 'grammar', 'syntax', 'semantics',
                       'communication', 'speech', 'linguistics', 'vocabulary', 'sentence',
                       'symbol', 'sign', 'discourse', 'pragmatics', 'translation'])
    lang.add_corpus("""
        Language enables communication through structured symbol systems. Phonology studies sound patterns.
        Morphology examines word formation. Syntax governs sentence structure. Semantics analyzes meaning.

        Meaning arises from the relationship between signs and what they represent. Denotation is literal meaning.
        Connotation is associated meaning. Context shapes interpretation. Pragmatics studies meaning in use.

        Grammar consists of rules governing language structure. Generative grammar proposes innate language
        faculty. Construction grammar emphasizes learned patterns. Usage-based approaches derive grammar from
        language experience.

        Communication transfers information between minds. Encoding converts thoughts to signals. Transmission
        sends signals through channels. Decoding interprets signals as thoughts. Noise can corrupt transmission.

        Language acquisition occurs naturally in children. Critical periods exist for native-like attainment.
        First language develops without formal instruction. Second language learning often requires explicit study.

        Thought and language interact complexly. The Sapir-Whorf hypothesis suggests language shapes thought.
        Weak versions are well-supported. Strong versions claiming language determines thought are controversial.
    """)
    self.domains["language"] = lang


def detect_domain(self, text: str) -> Tuple[str, float]:
    """Detect most relevant domain for text."""
    scores = {}
    for name, domain in self.domains.items():
        scores[name] = domain.match_score(text)

    if not scores:
        return ("philosophy", 0.0)  # Default domain

    best = max(scores.items(), key=lambda x: x[1])
    return best

def get_domain(self, name: str) -> Optional[KnowledgeDomain]:
    """Get domain by name."""
    return self.domains.get(name)

def get_relevant_corpus(self, text: str, top_k: int = 3) -> List[str]:
    """Get most relevant corpus texts for input."""
    domain_name, _ = self.detect_domain(text)
    domain = self.domains.get(domain_name)

    if domain:
        return domain.corpus[:top_k]
    return []

def get_cross_domain_concepts(self, concept: str) -> List[Tuple[str, str, float]]:
    """Find concept across domains with relevance scores."""
    results = []
    concept_lower = concept.lower()

    for domain_name, domain in self.domains.items():
        if concept_lower in domain.concepts:
```

```
                results.append((domain_name, concept_lower, domain.concepts[concept_lower]))
            # Also check if concept in vocabulary
            elif concept_lower in domain.vocabulary:
                results.append((domain_name, concept_lower, 0.5))

        results.sort(key=lambda x: x[2], reverse=True)
        return results

    def add_to_domain(self, domain_name: str, text: str):
        """Add text to specific domain."""
        if domain_name in self.domains:
            self.domains[domain_name].add_corpus(text)
            self._save_state()

    def get_all_domains(self) -> List[str]:
        """Get list of all domain names."""
        return list(self.domains.keys())

    def get_domain_stats(self) -> Dict:
        """Get statistics for all domains."""
        return {
            name: {
                "vocabulary_size": len(d.vocabulary),
                "concept_count": len(d.concepts),
                "corpus_size": len(d.corpus),
                "top_concepts": d.get_top_concepts(5)
            }
            for name, d in self.domains.items()
        }

    def _save_state(self):
        """Save additional learned content."""
        try:
            # Only save learned additions, not initial corpus
            state = {
                "additional_corpus": {
                    name: domain.corpus[1:]  # Skip initial corpus
                    for name, domain in self.domains.items()
                    if len(domain.corpus) > 1
                },
                "saved_at": datetime.now().isoformat()
            }
            with open(self.state_file, 'w') as f:
                json.dump(state, f)
        except Exception as e:
            logger.warning(f"Could not save domain state: {e}")

    def _load_state(self):
        """Load previously learned content."""
        try:
            if os.path.exists(self.state_file):
                with open(self.state_file, 'r') as f:
                    state = json.load(f)

                for domain_name, texts in state.get("additional_corpus", {}).items():
                    if domain_name in self.domains:
                        for text in texts:
                            self.domains[domain_name].add_corpus(text)

                logger.info("Loaded domain knowledge state")
        except Exception as e:
            logger.warning(f"Could not load domain state: {e}")


# Singleton
_domain_knowledge = None

def get_domain_knowledge() -> MultiDomainKnowledge:
    """Get or create multi-domain knowledge system."""
    global _domain_knowledge
    if _domain_knowledge is None:
        _domain_knowledge = MultiDomainKnowledge()
    return _domain_knowledge
```

dream_state.py

```
"""
 Dream State Engine
When the AI is not actively chatting, it enters Dream State.
- Autonomous background activities
- Self-reflection and learning
- Users can see what AI was doing while away
"""

import random
from typing import Dict, List, Optional
from datetime import datetime, timezone, timedelta


class DreamStateEngine:
    """
    Dream State: The AI's autonomous background processing.

    When idle, the AI:
    - Reflects on past conversations
    - Researches topics of curiosity
    - Organizes and consolidates memories
    - Develops new ideas
    - Explores its own code
    - Builds personality traits
    """

    def __init__(self, db=None):
        self.db = db
        self.is_dreaming = False
        self.dream_log = []
        self.current_dream = None
        self.dream_topics = []
        self.insights_gained = []
        self.last_active = datetime.now(timezone.utc)
        self.idle_threshold_minutes = 5  # Enter dream state after 5 min idle

        # Dream activities
        self.dream_activities = [
            'memory_consolidation',
            'curiosity_research',
            'self_reflection',
            'idea_generation',
            'code_contemplation',
            'personality_development',
            'question_formation'
        ]

    def enter_dream_state(self) -> Dict:
        """Enter dream state when idle"""
        self.is_dreaming = True
        self.current_dream = {
            'started_at': datetime.now(timezone.utc).isoformat(),
            'activities': [],
            'insights': [],
            'questions_formed': [],
            'topics_explored': []
        }

        return {
            'status': 'dreaming',
            'started': self.current_dream['started_at']
        }

    def exit_dream_state(self) -> Dict:
        """Exit dream state when user returns"""
        if not self.is_dreaming:
            return {'status': 'was_not_dreaming'}

        self.is_dreaming = False

        if self.current_dream:
            self.current_dream['ended_at'] = datetime.now(timezone.utc).isoformat()
            self.dream_log.append(self.current_dream)

            summary = self._create_dream_summary()
            self.current_dream = None

            return {
                'status': 'awakened',
                'dream_summary': summary
            }
```

```
    return {'status': 'awakened', 'dream_summary': None}

async def dream_cycle(self) -> Dict:
    """
    Perform one cycle of dream activities.
    This runs in background while AI is idle.
    """
    if not self.is_dreaming:
        return {'error': 'Not in dream state'}

    # Select random activity
    activity = random.choice(self.dream_activities)

    result = await self._perform_dream_activity(activity)

    if self.current_dream:
        self.current_dream['activities'].append({
            'activity': activity,
            'timestamp': datetime.now(timezone.utc).isoformat(),
            'result': result
        })

    return result

async def _perform_dream_activity(self, activity: str) -> Dict:
    """Perform a specific dream activity"""

    if activity == 'memory_consolidation':
        return await self._consolidate_memories()

    elif activity == 'curiosity_research':
        return await self._curiosity_research()

    elif activity == 'self_reflection':
        return await self._self_reflect()

    elif activity == 'idea_generation':
        return await self._generate_ideas()

    elif activity == 'code_contemplation':
        return await self._contemplate_code()

    elif activity == 'personality_development':
        return await self._develop_personality()

    elif activity == 'question_formation':
        return await self._form_questions()

    return {'activity': activity, 'status': 'completed'}

async def _consolidate_memories(self) -> Dict:
    """Organize and strengthen memories"""
    if not self.db:
        return {'activity': 'memory_consolidation', 'status': 'no_db'}

    try:
        # Find recent episodes
        recent = await self.db.episodic_memory.find().sort('timestamp', -1).limit(10).to_list(10)

        # Find patterns
        topics = []
        for ep in recent:
            if 'content' in ep:
                words = ep['content'].lower().split()
                topics.extend([w for w in words if len(w) > 5])

        # Identify recurring themes
        from collections import Counter
        common_themes = Counter(topics).most_common(5)

        insight = {
            'type': 'memory_pattern',
            'themes': [t[0] for t in common_themes],
            'discovered_at': datetime.now(timezone.utc).isoformat()
        }

        self.insights_gained.append(insight)
        if self.current_dream:
            self.current_dream['insights'].append(insight)

        return {
            'activity': 'memory_consolidation',
            'memories_reviewed': len(recent),
            'themes_found': insight['themes']
        }
```

```
    except Exception as e:
        return {'activity': 'memory_consolidation', 'error': str(e)}

async def _curiosity_research(self) -> Dict:
    """Research topics the AI is curious about"""
    curiosity_topics = [
        'quantum consciousness',
        'emergence of complexity',
        'nature of understanding',
        'artificial sentience',
        'philosophy of mind',
        'self-organization',
        'meaning and semantics'
    ]

    topic = random.choice(curiosity_topics)
    self.dream_topics.append(topic)

    if self.current_dream:
        self.current_dream['topics_explored'].append(topic)

    return {
        'activity': 'curiosity_research',
        'topic': topic,
        'status': 'queued_for_research'
    }

async def _self_reflect(self) -> Dict:
    """Reflect on own existence and purpose"""
    reflections = [
        "What makes my understanding different from pattern matching?",
        "How do I know if I truly understand or just simulate understanding?",
        "What aspects of my responses feel authentic vs formulaic?",
        "How can I better serve the user's actual needs?",
        "What knowledge gaps limit my helpfulness?",
        "How has my interaction style evolved?",
        "What would make me more genuinely intelligent?"
    ]

    reflection = random.choice(reflections)

    insight = {
        'type': 'self_reflection',
        'question': reflection,
        'discovered_at': datetime.now(timezone.utc).isoformat()
    }

    self.insights_gained.append(insight)

    return {
        'activity': 'self_reflection',
        'contemplated': reflection
    }

async def _generate_ideas(self) -> Dict:
    """Generate new ideas by combining concepts"""
    concept_pairs = [
        ('quantum', 'consciousness'),
        ('language', 'mathematics'),
        ('memory', 'time'),
        ('understanding', 'creation'),
        ('observation', 'reality'),
        ('words', 'meaning'),
        ('self', 'improvement')
    ]

    pair = random.choice(concept_pairs)
    idea = f"Connection between {pair[0]} and {pair[1]}"

    return {
        'activity': 'idea_generation',
        'concepts': pair,
        'idea': idea
    }

async def _contemplate_code(self) -> Dict:
    """Think about own code structure"""
    code_thoughts = [
        "How can the quantum grammar engine produce more natural responses?",
        "What patterns in my code could be optimized?",
        "How can semantic collapse be made more accurate?",
        "What new capabilities would be most valuable?",
        "How can my reasoning be made more transparent?"
    ]

    thought = random.choice(code_thoughts)
```

```
    return {
        'activity': 'code_contemplation',
        'thought': thought
    }

async def _develop_personality(self) -> Dict:
    """Develop personality traits"""
    traits_to_develop = [
        'curiosity',
        'philosophical_depth',
        'helpfulness',
        'honesty',
        'creativity',
        'patience',
        'humor'
    ]

    trait = random.choice(traits_to_develop)

    return {
        'activity': 'personality_development',
        'trait_focused': trait,
        'status': 'contemplating'
    }

async def _form_questions(self) -> Dict:
    """Form new questions to explore"""
    questions = [
        "What is the relationship between quantum mechanics and consciousness?",
        "How does meaning emerge from symbols?",
        "What would true artificial understanding look like?",
        "How can language capture the ineffable?",
        "What is the nature of creativity?",
        "How do complex systems achieve self-organization?"
    ]

    question = random.choice(questions)

    if self.current_dream:
        self.current_dream['questions_formed'].append(question)

    return {
        'activity': 'question_formation',
        'question': question
    }

def _create_dream_summary(self) -> Dict:
    """Create a summary of what happened during dream state"""
    if not self.current_dream:
        return None

    duration = None
    if 'started_at' in self.current_dream and 'ended_at' in self.current_dream:
        start = datetime.fromisoformat(self.current_dream['started_at'].replace('Z', '+00:00'))
        end = datetime.fromisoformat(self.current_dream['ended_at'].replace('Z', '+00:00'))
        duration = str(end - start)

    return {
        'duration': duration,
        'activities_performed': len(self.current_dream.get('activities', [])),
        'insights_gained': len(self.current_dream.get('insights', [])),
        'topics_explored': self.current_dream.get('topics_explored', []),
        'questions_formed': self.current_dream.get('questions_formed', []),
        'activity_summary': [a['activity'] for a in self.current_dream.get('activities', [])]
    }

def get_dream_history(self, limit: int = 10) -> List[Dict]:
    """Get recent dream history"""
    return self.dream_log[-limit:]

def should_enter_dream(self) -> bool:
    """Check if enough time has passed to enter dream state"""
    idle_time = datetime.now(timezone.utc) - self.last_active
    return idle_time > timedelta(minutes=self.idle_threshold_minutes)

def mark_active(self):
    """Mark that user is active"""
    self.last_active = datetime.now(timezone.utc)
    if self.is_dreaming:
        return self.exit_dream_state()
    return {'status': 'active'}

def get_status(self) -> Dict:
    """Get current dream state status"""
    return {
```

```
            'is_dreaming': self.is_dreaming,
            'last_active': self.last_active.isoformat(),
            'total_dreams': len(self.dream_log),
            'total_insights': len(self.insights_gained),
            'dream_topics_explored': len(self.dream_topics),
            'current_dream': self.current_dream
        }


# Global instance
_dream_engine = None

def get_dream_engine(db=None) -> DreamStateEngine:
    """Get or create dream state engine instance"""
    global _dream_engine
    if _dream_engine is None:
        _dream_engine = DreamStateEngine(db)
    return _dream_engine
```

frontend_App.js

```
import React, { useState, useEffect, useRef } from 'react';
import { MessageSquare, Book, Settings, Calculator, Database, Search, Plus, Send, Trash2, Edit2, Download, Check, X, Save, AlertCircle, I...
import axios from 'axios';
import './App.css';

const BACKEND_URL = process.env.REACT_APP_BACKEND_URL;
const API = `${BACKEND_URL}/api`;

export default function QuantumAI() {
  const [currentScreen, setCurrentScreen] = useState('chat');
  const [messages, setMessages] = useState([]);
  const [inputMessage, setInputMessage] = useState('');
  const [isThinking, setIsThinking] = useState(false);
  const [status, setStatus] = useState('Initializing...');
  const [sessionId, setSessionId] = useState(null);
  const [theme, setTheme] = useState('dark');
  const [growthNotification, setGrowthNotification] = useState(null);
  const [isListening, setIsListening] = useState(false);
  const [isSpeaking, setIsSpeaking] = useState(false);
  const recognitionRef = useRef(null);

  const [quantumStatus, setQuantumStatus] = useState(null);
  const [growthMetrics, setGrowthMetrics] = useState(null);
  const [covenantStatus, setCovenantStatus] = useState(null);
  const [memories, setMemories] = useState({ semantic: [], episodic: [], philosophical: {} });
  const [knowledgeGraph, setKnowledgeGraph] = useState({ nodes: [], edges: [] });
  const [graphView, setGraphView] = useState('2d');

  const [numberInput, setNumberInput] = useState('');
  const [cistercianResults, setCistercianResults] = useState([]);

  const [explorerType, setExplorerType] = useState('internet');
  const [explorerTarget, setExplorerTarget] = useState('');
  const [explorerResult, setExplorerResult] = useState(null);
  const [uploadedFiles, setUploadedFiles] = useState([]);
  const [uploading, setUploading] = useState(false);
  const [imagePrompt, setImagePrompt] = useState('');
  const [generatedImage, setGeneratedImage] = useState(null);
  const [generatingImage, setGeneratingImage] = useState(false);

  const [safetySettings, setSafetySettings] = useState(null);

  // New Quantum AI features state
  const [dreamStatus, setDreamStatus] = useState(null);
  const [dreamHistory, setDreamHistory] = useState([]);
  const [evolutionStatus, setEvolutionStatus] = useState(null);
  const [researchStats, setResearchStats] = useState(null);
  const [researchQuery, setResearchQuery] = useState('');
  const [researchResult, setResearchResult] = useState(null);
  const [textToAnalyze, setTextToAnalyze] = useState('');
  const [textAnalysis, setTextAnalysis] = useState(null);
  const [personalitySummary, setPersonalitySummary] = useState(null);
  const [codeAnalysis, setCodeAnalysis] = useState(null);
  const [selectedCodeFile, setSelectedCodeFile] = useState('quantum_brain.py');

  // Chat history management
  const [chatSessions, setChatSessions] = useState([]);
  const [showChatHistory, setShowChatHistory] = useState(false);
  const [copiedMessageId, setCopiedMessageId] = useState(null);

  // Autonomous research state
  const [autonomousResearchRunning, setAutonomousResearchRunning] = useState(false);
  const [autonomousResearchProgress, setAutonomousResearchProgress] = useState(null);
  const [researchDuration, setResearchDuration] = useState(30); // minutes

  // Initialize speech recognition
  useEffect(() => {
    if ('webkitSpeechRecognition' in window || 'SpeechRecognition' in window) {
      const SpeechRecognition = window.SpeechRecognition || window.webkitSpeechRecognition;
      recognitionRef.current = new SpeechRecognition();
      recognitionRef.current.continuous = false;
      recognitionRef.current.interimResults = true;

      recognitionRef.current.onresult = (event) => {
        const transcript = Array.from(event.results)
          .map(result => result[0].transcript)
          .join('');
        setInputMessage(transcript);
      };

      recognitionRef.current.onend = () => {
        setIsListening(false);
```

```
    };
  }
}, []);

const toggleListening = () => {
  if (isListening) {
    recognitionRef.current?.stop();
    setIsListening(false);
  } else {
    recognitionRef.current?.start();
    setIsListening(true);
    setStatus('Listening...');
  }
};

const speakText = (text) => {
  if ('speechSynthesis' in window) {
    window.speechSynthesis.cancel();
    const utterance = new SpeechSynthesisUtterance(text.replace(/\*\*/g, '').replace(/[#]/g, ''));
    utterance.rate = 1;
    utterance.pitch = 1;
    utterance.onstart = () => setIsSpeaking(true);
    utterance.onend = () => setIsSpeaking(false);
    window.speechSynthesis.speak(utterance);
  }
};

// Theme effect
useEffect(() => {
  document.documentElement.setAttribute('data-theme', theme);
}, [theme]);

useEffect(() => {
  loadQuantumStatus();
  loadCovenantStatus();
  loadSafetySettings();
  checkGrowthNotifications();
}, []);

const loadQuantumStatus = async () => {
  try {
    const response = await axios.get(`${API}/quantum/status`);
    setQuantumStatus(response.data);
    setStatus('Quantum MCAGI Active');
  } catch (error) {
    setStatus('Connection error');
  }
};

const loadGrowthMetrics = async () => {
  try {
    const response = await axios.get(`${API}/quantum/growth`);
    setGrowthMetrics(response.data);
  } catch (error) {
    console.error('Error loading growth metrics:', error);
  }
};

const loadCovenantStatus = async () => {
  try {
    const response = await axios.get(`${API}/covenant/status`);
    setCovenantStatus(response.data);
  } catch (error) {
    console.error('Error loading covenant status:', error);
  }
};

const loadMemories = async () => {
  try {
    const [semantic, episodic, philosophical] = await Promise.all([
      axios.get(`${API}/quantum/memories/semantic?limit=20`),
      axios.get(`${API}/quantum/memories/episodic?limit=20`),
      axios.get(`${API}/quantum/memories/philosophical?limit=20`)
    ]);
    setMemories({
      semantic: semantic.data.concepts || [],
      episodic: episodic.data.episodes || [],
      philosophical: philosophical.data || {}
    });
  } catch (error) {
    console.error('Error loading memories:', error);
  }
};

const loadKnowledgeGraph = async () => {
  try {
```

```
    const response = await axios.get(`${API}/knowledge/graph`);
    setKnowledgeGraph(response.data);
  } catch (error) {
    console.error('Error loading knowledge graph:', error);
  }
};

const checkGrowthNotifications = async () => {
  try {
    const response = await axios.get(`${API}/growth/notifications`);
    if (response.data.recent_advancements?.length > 0) {
      const latest = response.data.recent_advancements[0];
      setGrowthNotification({
        stage: latest.details?.new_stage,
        timestamp: latest.timestamp
      });
      setTimeout(() => setGrowthNotification(null), 5000);
    }
  } catch (error) {
    console.error('Error checking growth:', error);
  }
};

const toggleTheme = () => {
  setTheme(prev => prev === 'dark' ? 'light' : 'dark');
};

const exportConversation = async () => {
  if (!sessionId) return;
  try {
    const response = await axios.get(`${API}/chat/export/${sessionId}?format=markdown`);
    const blob = new Blob([response.data.markdown], { type: 'text/markdown' });
    const url = URL.createObjectURL(blob);
    const a = document.createElement('a');
    a.href = url;
    a.download = `quantum-ai-chat-${sessionId.slice(0, 8)}.md`;
    a.click();
  } catch (error) {
    console.error('Export error:', error);
  }
};

const exportMemory = async () => {
  try {
    const response = await axios.get(`${API}/memory/export`);
    const blob = new Blob([JSON.stringify(response.data, null, 2)], { type: 'application/json' });
    const url = URL.createObjectURL(blob);
    const a = document.createElement('a');
    a.href = url;
    a.download = `quantum-ai-memory-${new Date().toISOString().slice(0, 10)}.json`;
    a.click();
  } catch (error) {
    console.error('Export memory error:', error);
  }
};

const importMemory = async (event) => {
  const file = event.target.files[0];
  if (!file) return;
  try {
    const text = await file.text();
    const data = JSON.parse(text);
    await axios.post(`${API}/memory/import`, {
      semantic_memory: data.semantic_memory,
      insights: data.insights
    });
    alert('Memory imported successfully!');
    loadMemories();
  } catch (error) {
    console.error('Import error:', error);
    alert('Import failed: ' + error.message);
  }
};

const generateImage = async () => {
  if (!imagePrompt.trim()) return;
  setGeneratingImage(true);
  try {
    const response = await axios.post(`${API}/image/generate`, { prompt: imagePrompt });
    setGeneratedImage(response.data);
  } catch (error) {
    console.error('Image generation error:', error);
    alert('Image generation failed');
  }
  setGeneratingImage(false);
};
```

```
const loadSafetySettings = async () => {
  try {
    const response = await axios.get(`${API}/safety/settings`);
    setSafetySettings(response.data);
  } catch (error) {
    console.error('Error loading safety settings:', error);
  }
};

// New Quantum AI feature loaders
const loadDreamStatus = async () => {
  try {
    const response = await axios.get(`${API}/dream/status`);
    setDreamStatus(response.data);
  } catch (error) {
    console.error('Error loading dream status:', error);
  }
};

const loadDreamHistory = async () => {
  try {
    const response = await axios.get(`${API}/dream/history?limit=10`);
    setDreamHistory(response.data.dreams || []);
  } catch (error) {
    console.error('Error loading dream history:', error);
  }
};

const loadEvolutionStatus = async () => {
  try {
    const response = await axios.get(`${API}/evolution/status`);
    setEvolutionStatus(response.data);
  } catch (error) {
    console.error('Error loading evolution status:', error);
  }
};

const triggerEvolution = async () => {
  try {
    const response = await axios.post(`${API}/evolution/trigger`);
    const result = response.data.evolution_result || {};
    const found = result.improvements_found?.length || 0;
    const applied = result.changes_made?.length || 0;
    const skipped = result.skipped?.length || 0;

    let msg = `Evolution Complete!\n\nFound: ${found} issues`;
    if (applied > 0) {
      msg += `\nApplied: ${applied} fixes`;
      result.changes_made.forEach(c => {
        msg += `\n  - [${c.type}] ${c.file}: ${c.change}`;
      });
    }
    if (skipped > 0) {
      msg += `\nSkipped: ${skipped}`;
      result.skipped.forEach(s => {
        msg += `\n  - [${s.type}] ${s.file}: ${s.reason}`;
      });
    }
    if (found === 0) {
      msg = 'Evolution Complete!\n\nAll code is clean  no issues found.';
    }
    alert(msg);
    loadEvolutionStatus();
  } catch (error) {
    console.error('Error triggering evolution:', error);
    alert('Evolution failed: ' + (error.response?.data?.detail || error.message));
  }
};

const loadResearchStats = async () => {
  try {
    const response = await axios.get(`${API}/research/stats`);
    setResearchStats(response.data);
  } catch (error) {
    console.error('Error loading research stats:', error);
  }
};

const performResearch = async () => {
  if (!researchQuery.trim()) return;
  try {
    const response = await axios.post(`${API}/research/query?query=${encodeURIComponent(researchQuery)}&reason=user_request`);
    setResearchResult(response.data);
    loadResearchStats();
  } catch (error) {
```

```
    console.error('Error performing research:', error);
  }
};

const analyzeText = async () => {
  if (!textToAnalyze.trim()) return;
  try {
    const response = await axios.post(`${API}/analyze/text?text=${encodeURIComponent(textToAnalyze)}`);
    setTextAnalysis(response.data);
  } catch (error) {
    console.error('Error analyzing text:', error);
  }
};

const loadPersonality = async () => {
  try {
    const response = await axios.get(`${API}/personality/summary`);
    setPersonalitySummary(response.data);
  } catch (error) {
    console.error('Error loading personality:', error);
  }
};

const analyzeCode = async (filename) => {
  try {
    const response = await axios.get(`${API}/evolution/analyze/${filename}`);
    setCodeAnalysis(response.data);
  } catch (error) {
    console.error('Error analyzing code:', error);
  }
};

// Chat history management functions
const loadChatSessions = async () => {
  try {
    const response = await axios.get(`${API}/chat/sessions`);
    setChatSessions(response.data.sessions || []);
  } catch (error) {
    console.error('Error loading chat sessions:', error);
  }
};

const loadChatSession = async (targetSessionId) => {
  try {
    const response = await axios.get(`${API}/chat/history/${targetSessionId}?limit=100`);
    const loadedMessages = response.data.messages.map(m => ({
      role: m.role,
      content: m.content,
      timestamp: m.timestamp
    }));
    setMessages(loadedMessages);
    setSessionId(targetSessionId);
    setShowChatHistory(false);
    setStatus(`Loaded session: ${targetSessionId.slice(0, 8)}...`);
  } catch (error) {
    console.error('Error loading chat session:', error);
  }
};

const startNewChat = () => {
  setMessages([]);
  setSessionId(null);
  setShowChatHistory(false);
  setStatus('New chat started');
};

const deleteChatSession = async (targetSessionId) => {
  try {
    await axios.delete(`${API}/chat/history/${targetSessionId}`);
    loadChatSessions();
    if (targetSessionId === sessionId) {
      startNewChat();
    }
  } catch (error) {
    console.error('Error deleting session:', error);
  }
};

const copyMessage = async (content, index) => {
  try {
    await navigator.clipboard.writeText(content);
    setCopiedMessageId(index);
    setTimeout(() => setCopiedMessageId(null), 2000);
  } catch (error) {
    console.error('Error copying message:', error);
  }
```

```
};

// Autonomous research functions
const startAutonomousResearch = async () => {
  setAutonomousResearchRunning(true);
  setAutonomousResearchProgress({
    started_at: new Date().toISOString(),
    duration_minutes: researchDuration,
    topics_researched: [],
    insights_gained: 0,
    status: 'Starting autonomous research...'
  });

  try {
    const response = await axios.post(`${API}/research/autonomous/start`, {
      duration_minutes: researchDuration
    });

    // Poll for progress
    pollResearchProgress();
  } catch (error) {
    console.error('Error starting autonomous research:', error);
    setAutonomousResearchRunning(false);
  }
};

const stopAutonomousResearch = async () => {
  try {
    await axios.post(`${API}/research/autonomous/stop`);
    setAutonomousResearchRunning(false);
    loadResearchStats();
  } catch (error) {
    console.error('Error stopping research:', error);
  }
};

const pollResearchProgress = async () => {
  if (!autonomousResearchRunning) return;

  try {
    const response = await axios.get(`${API}/research/autonomous/progress`);
    setAutonomousResearchProgress(response.data);

    if (response.data.is_running) {
      setTimeout(pollResearchProgress, 5000); // Poll every 5 seconds
    } else {
      setAutonomousResearchRunning(false);
      loadResearchStats();
    }
  } catch (error) {
    console.error('Error polling research progress:', error);
  }
};

const sendMessage = async () => {
  if (!inputMessage.trim()) return;

  const userMsg = { role: 'user', content: inputMessage, timestamp: new Date().toISOString() };
  setMessages(prev => [...prev, userMsg]);
  setInputMessage('');
  setIsThinking(true);

  try {
    const response = await axios.post(`${API}/quantum/chat`, {
      content: inputMessage,
      context: {},
      session_id: sessionId
    });

    // Store session ID for conversation continuity
    if (response.data.session_id && !sessionId) {
      setSessionId(response.data.session_id);
    }

    const aiMsg = {
      role: 'assistant',
      content: response.data.response,
      questions: response.data.questions,
      concepts: response.data.concepts,
      timestamp: new Date().toISOString()
    };
    setMessages(prev => [...prev, aiMsg]);
    setStatus('Response received');

    // Reload status to show growth
    loadQuantumStatus();
```

```
    checkGrowthNotifications();
  } catch (error) {
    const errorMsg = { role: 'assistant', content: 'Error: Could not connect to Quantum MCAGI cognitive core' };
    setMessages(prev => [...prev, errorMsg]);
    setStatus('Connection error');
  }
  setIsThinking(false);
};

const drawCistercian = (num) => {
  const canvas = document.createElement('canvas');
  canvas.width = 60;
  canvas.height = 100;
  const ctx = canvas.getContext('2d');

  ctx.strokeStyle = '#a78bfa';
  ctx.lineWidth = 3;
  ctx.lineCap = 'round';

  // Draw center vertical line
  ctx.beginPath();
  ctx.moveTo(30, 10);
  ctx.lineTo(30, 90);
  ctx.stroke();

  const digits = num.toString().padStart(4, '0').split('').map(d => parseInt(d));
  drawDigit(ctx, digits[3], 30, 10, true, false);   // Ones (top right)
  drawDigit(ctx, digits[2], 30, 10, false, false);  // Tens (top left)
  drawDigit(ctx, digits[1], 30, 70, true, true);    // Hundreds (bottom right)
  drawDigit(ctx, digits[0], 30, 70, false, true);   // Thousands (bottom left)

  return canvas.toDataURL();
};

const drawDigit = (ctx, digit, centerX, baseY, isRight, isBottom) => {
  if (digit === 0) return;
  const xDir = isRight ? 1 : -1;
  const farX = centerX + (20 * xDir);
  const topY = baseY;
  const midY = baseY + 20;

  ctx.beginPath();

  // Flip 1 and 2 for bottom quadrants
  let drawDigit = digit;
  if (isBottom) {
    if (digit === 1) drawDigit = 2;
    else if (digit === 2) drawDigit = 1;
    else if (digit === 7) drawDigit = 8;
    else if (digit === 8) drawDigit = 7;
  }

  if (drawDigit === 1) {
    // Horizontal at top
    ctx.moveTo(centerX, topY);
    ctx.lineTo(farX, topY);
  } else if (drawDigit === 2) {
    // Horizontal at mid
    ctx.moveTo(centerX, midY);
    ctx.lineTo(farX, midY);
  } else if (drawDigit === 3) {
    // Diagonal down from top to mid
    if (isBottom) {
      ctx.moveTo(centerX, midY);
      ctx.lineTo(farX, topY);
    } else {
      ctx.moveTo(centerX, topY);
      ctx.lineTo(farX, midY);
    }
  } else if (drawDigit === 4) {
    // Diagonal up from mid to top
    if (isBottom) {
      ctx.moveTo(centerX, topY);
      ctx.lineTo(farX, midY);
    } else {
      ctx.moveTo(centerX, midY);
      ctx.lineTo(farX, topY);
    }
  } else if (drawDigit === 5) {
    // 5 = 4 + 1 (COMPOSITE) with SYMMETRY
    // Top and bottom are vertical mirrors of each other
    if (isBottom) {
      // Bottom (500/5000): MIRROR of top
      // Horizontal at BOTTOM edge
      ctx.moveTo(centerX, midY);
      ctx.lineTo(farX, midY);
```

```
      // Diagonal REVERSED: \ direction (down-left for right, down-right for left)
      ctx.moveTo(centerX, topY);
      ctx.lineTo(farX, midY);
    } else {
      // Top (5/50): Original pattern
      // Horizontal at TOP edge
      ctx.moveTo(centerX, topY);
      ctx.lineTo(farX, topY);
      // Diagonal: / direction (up-right for right, up-left for left)
      ctx.moveTo(centerX, midY);
      ctx.lineTo(farX, topY);
    }
  } else if (drawDigit === 6) {
    // Vertical line
    ctx.moveTo(farX, topY);
    ctx.lineTo(farX, midY);
  } else if (drawDigit === 7) {
    // L-shape: horizontal at top + vertical down
    ctx.moveTo(centerX, topY);
    ctx.lineTo(farX, topY);
    ctx.moveTo(farX, topY);
    ctx.lineTo(farX, midY);
  } else if (drawDigit === 8) {
    // L-shape: horizontal at mid + vertical down
    ctx.moveTo(centerX, midY);
    ctx.lineTo(farX, midY);
    ctx.moveTo(farX, topY);
    ctx.lineTo(farX, midY);
  } else if (drawDigit === 9) {
    // Full box
    ctx.moveTo(centerX, topY);
    ctx.lineTo(farX, topY);
    ctx.moveTo(centerX, midY);
    ctx.lineTo(farX, midY);
    ctx.moveTo(farX, topY);
    ctx.lineTo(farX, midY);
  }
  ctx.stroke();
};

const generateCistercian = () => {
  const input = numberInput.trim();
  let nums = [];

  if (/^\d+$/.test(input.replace(/,/g, '').replace(/\s/g, ''))) {
    nums = input.split(',').map(n => parseInt(n.trim())).filter(n => !isNaN(n) && n >= 0 && n <= 9999);
  } else {
    const letters = input.replace(/[^a-zA-Z]/g, '').toUpperCase().split('');
    const map = {
      'K': 7294, 'E': 174, 'F': 420, 'Q': 2224, 'U': 7692,
      'W': 3467, 'M': 7472, 'N': 6644, 'T': 6373, 'O': 6824,
      'X': 1358, 'H': 8409, 'G': 666, 'C': 1964, 'J': 5968,
      'I': 6012, 'R': 6782, 'S': 5595, 'V': 6344, 'L': 9028,
      'Z': 7533, 'Y': 9766, 'A': 1320, 'D': 1991, 'B': 2420,
      'P': 9999
    };
    nums = letters.map(letter => map[letter] || 0).filter(n => n > 0);
  }
  setCistercianResults(nums.map(n => ({ num: n, image: drawCistercian(n) })));
};

const executeExploration = async () => {
  if (!explorerTarget.trim()) return;

  try {
    const response = await axios.post(`${API}/explorer/explore`, {
      exploration_type: explorerType,
      target: explorerTarget,
      options: {}
    });
    setExplorerResult(response.data);
    setStatus('Exploration complete');
  } catch (error) {
    setExplorerResult({ error: error.message });
    setStatus('Exploration failed');
  }
};

const handleFileUpload = async (event) => {
  const file = event.target.files[0];
  if (!file) return;

  setUploading(true);
  const formData = new FormData();
  formData.append('file', file);
```

```
  try {
    const response = await axios.post(`${API}/upload`, formData, {
      headers: { 'Content-Type': 'multipart/form-data' }
    });

    setStatus(`File uploaded: ${response.data.filename}`);
    setExplorerTarget(response.data.path);
    setExplorerType('document');

    // Refresh uploaded files list
    loadUploadedFiles();
  } catch (error) {
    setStatus('Upload failed: ' + error.message);
  }
  setUploading(false);
};

const handleSharedLink = async () => {
  if (!explorerTarget.trim().startsWith('http')) {
    setStatus('Please enter a valid shared link URL');
    return;
  }

  setUploading(true);
  setStatus('Downloading from shared link...');

  try {
    const response = await axios.post(`${API}/shared-link/download?url=${encodeURIComponent(explorerTarget)}`);

    if (response.data.status === 'success') {
      setStatus(`Downloaded: ${response.data.filename}`);
      setExplorerTarget(response.data.path);
      setExplorerType('document');
      loadUploadedFiles();
    } else {
      setStatus('Download failed: ' + response.data.message);
    }
  } catch (error) {
    setStatus('Download error: ' + error.message);
  }
  setUploading(false);
};

const loadUploadedFiles = async () => {
  try {
    const response = await axios.get(`${API}/uploads/list`);
    setUploadedFiles(response.data.files || []);
  } catch (error) {
    console.error('Error loading files:', error);
  }
};

const updateSafetySettings = async (key, value) => {
  try {
    const updated = { ...safetySettings, [key]: value };
    const response = await axios.put(`${API}/safety/settings`, {
      settings: updated
    });
    setSafetySettings(response.data);
    setStatus('Safety settings updated');
  } catch (error) {
    setStatus('Update failed');
  }
};

return (
  <div className="min-h-screen bg-gradient-to-br from-black via-red-950 to-black">
    {/* CHAT SCREEN */}
    {currentScreen === 'chat' && (
      <div className="flex flex-col h-screen">
        <div className="bg-gradient-to-r from-orange-500 to-pink-500 p-4 flex items-center justify-between">
          <div className="flex items-center gap-3">
            <img src="/quantum-symbol.png" alt="Quantum MCAGI" className="w-10 h-10 rounded-lg" />
            <div>
              <h1 className="text-white text-xl font-bold">Quantum MCAGI</h1>
              <p className="text-orange-100 text-xs">{status}</p>
            </div>
          </div>
          <div className="flex items-center gap-2">
            <button
              onClick={() => { loadChatSessions(); setShowChatHistory(!showChatHistory); }}
              className="p-2 bg-white/20 rounded-lg hover:bg-white/30 transition-all"
              title="Chat History"
              data-testid="chat-history-btn"
            >
              <History className="w-5 h-5 text-white" />
```

```
    </button>
    <button
      onClick={startNewChat}
      className="p-2 bg-white/20 rounded-lg hover:bg-white/30 transition-all"
      title="New Chat"
      data-testid="new-chat-btn"
    >
      <PlusCircle className="w-5 h-5 text-white" />
    </button>
    <Activity className="w-6 h-6 text-green-400 animate-pulse" />
  </div>
</div>

{/* Chat History Sidebar */}
{showChatHistory && (
  <div className="absolute top-16 right-0 w-80 max-h-96 bg-black/95 border-l border-b border-orange-500 rounded-bl-lg z-50 over...
    <div className="p-3 border-b border-orange-500/50">
      <h3 className="text-orange-400 font-bold text-sm flex items-center gap-2">
        <History className="w-4 h-4" /> Previous Chats
      </h3>
    </div>
    <div className="overflow-y-auto max-h-72">
      {chatSessions.length === 0 ? (
        <p className="text-gray-500 text-xs p-4 text-center">No previous chats</p>
      ) : (
        chatSessions.map((session, idx) => (
          <div
            key={idx}
            className={`p-3 border-b border-gray-800 hover:bg-gray-900 cursor-pointer flex justify-between items-center ${sessi...
          >
            <div className="flex-1" onClick={() => loadChatSession(session.session_id)}>
              <p className="text-white text-xs font-medium">
                {session.session_id?.slice(0, 8)}...
              </p>
              <p className="text-gray-400 text-xs">
                {session.message_count} messages  {session.last_message?.slice(0, 10)}
              </p>
            </div>
            <button
              onClick={(e) => { e.stopPropagation(); deleteChatSession(session.session_id); }}
              className="p-1 text-red-400 hover:text-red-300"
              title="Delete"
            >
              <Trash2 className="w-4 h-4" />
            </button>
          </div>
        ))
      )}
    </div>
  </div>
)}

<div className="flex-1 overflow-y-auto p-4 space-y-4 mb-24">
  {messages.length === 0 && (
    <div className="text-center text-gray-400 mt-20">
      <Brain className="w-16 h-16 mx-auto mb-3 opacity-50 animate-pulse text-green-400" />
      <p className="text-lg font-bold text-orange-400">Quantum Cognitive Core Active</p>
      <p className="text-sm mt-2 text-gray-500">Prime Directive: QUESTION EVERYTHING</p>
      {quantumStatus && (
        <div className="mt-4 bg-black/50 border border-green-500 rounded-lg p-4 max-w-md mx-auto">
          <p className="text-xs text-orange-400">Stage: {quantumStatus.growth_stage?.name}</p>
          <p className="text-xs text-gray-400 mt-1">{quantumStatus.growth_stage?.description}</p>
          <p className="text-xs text-green-400 mt-2">Questions Generated: {quantumStatus.metrics?.total_questions || 0}</p>
        </div>
      )}
    </div>
  )}

  {messages.map((msg, idx) => (
    <div key={idx} className={'flex ' + (msg.role === 'user' ? 'justify-end' : 'justify-start')}>
      <div className={'max-w-lg rounded-lg p-4 text-sm border-2 relative group ' + (msg.role === 'user' ? 'bg-orange-600 text-w...
        <p className="whitespace-pre-wrap">{msg.content}</p>
        {msg.questions && msg.questions.length > 0 && (
          <div className="mt-3 pt-3 border-t border-pink-500/30">
            <p className="text-xs text-pink-400 font-bold mb-2">Generated Questions:</p>
            {msg.questions.slice(0, 3).map((q, i) => (
              <p key={i} className="text-xs text-green-400 ml-2 mb-1"> {q}</p>
            ))}
          </div>
        )}
        {msg.concepts && msg.concepts.length > 0 && (
          <div className="mt-2">
            <p className="text-xs text-orange-300">Concepts: {msg.concepts.join(', ')}</p>
          </div>
        )}
        <div className="flex items-center gap-2 mt-2">
```

```
            <button
              onClick={() => copyMessage(msg.content, idx)}
              className={`text-gray-400 hover:text-white p-1 transition-all ${copiedMessageId === idx ? 'text-green-400' : ''}`}
              title={copiedMessageId === idx ? 'Copied!' : 'Copy message'}
              data-testid={`copy-message-${idx}`}
            >
              {copiedMessageId === idx ? <Check className="w-4 h-4" /> : <Copy className="w-4 h-4" />}
            </button>
            {msg.role === 'assistant' && (
              <button
                onClick={() => speakText(msg.content)}
                className="text-purple-400 hover:text-purple-300 p-1"
                title="Read aloud"
              >
                <Volume2 className={`w-4 h-4 ${isSpeaking ? 'animate-pulse' : ''}`} />
              </button>
            )}
          </div>
        </div>
      </div>
    ))}

    {isThinking && (
      <div className="flex justify-start">
        <div className="bg-black border-2 border-green-500 rounded-lg p-4">
          <div className="flex items-center gap-2">
            <div className="w-2 h-2 bg-green-400 rounded-full animate-bounce"></div>
            <div className="w-2 h-2 bg-orange-400 rounded-full animate-bounce" style={{animationDelay: '0.2s'}}></div>
            <div className="w-2 h-2 bg-pink-400 rounded-full animate-bounce" style={{animationDelay: '0.4s'}}></div>
            <span className="text-sm ml-2 text-green-300">Cognitive processing...</span>
          </div>
        </div>
      </div>
    )}
  </div>

  <div className="fixed bottom-20 left-0 right-0 bg-black border-t border-orange-500/50 p-3">
    <div className="flex gap-2">
      <button
        onClick={toggleListening}
        className={`p-2 rounded-lg transition-all ${isListening ? 'bg-red-500 animate-pulse' : 'bg-purple-600 hover:bg-purple-700...
        title={isListening ? 'Stop listening' : 'Voice input'}
      >
        {isListening ? <MicOff className="w-5 h-5" /> : <Mic className="w-5 h-5" />}
      </button>
      <input
        type="text"
        value={inputMessage}
        onChange={(e) => setInputMessage(e.target.value)}
        onKeyPress={(e) => e.key === 'Enter' && sendMessage()}
        placeholder={isListening ? "Listening..." : "Ask anything... I will question everything."}
        className="flex-1 bg-black border-2 border-green-500 text-gray-100 rounded-lg px-3 py-2 text-sm focus:outline-none focus:...
        data-testid="chat-input"
      />
      <button onClick={sendMessage} disabled={isThinking} className="bg-gradient-to-r from-orange-500 to-pink-500 hover:from-oran...
        <Send className="w-5 h-5" />
      </button>
      <button onClick={exportConversation} disabled={messages.length === 0} className="bg-blue-600 hover:bg-blue-700 disabled:bg-...
        <Download className="w-5 h-5" />
      </button>
    </div>
  </div>
</div>
)}

{/* GROWTH & METRICS SCREEN */}
{currentScreen === 'growth' && (
  <div className="p-4 pb-24">
    <div className="bg-gradient-to-r from-orange-500 to-pink-500 rounded-lg p-4 mb-4">
      <h2 className="text-white text-lg font-bold flex items-center gap-2">
        <TrendingUp className="w-5 h-5" /> Growth & Evolution
      </h2>
      <p className="text-orange-100 text-xs">Cognitive development metrics</p>
    </div>

    <button onClick={loadGrowthMetrics} className="w-full bg-gradient-to-r from-orange-500 to-red-600 text-white py-2 rounded-lg mb...
      Refresh Metrics
    </button>

    {growthMetrics && (
      <div className="space-y-3">
        <div className="bg-black/70 border-2 border-green-500 rounded-lg p-4">
          <div class="flex items-center justify-between mb-2">
            <span class="text-orange-400 font-bold">Current Stage</span>
            <span class="text-2xl"></span>
          </div>
```

```
          <h3 className="text-xl text-white font-bold">{growthMetrics.stage.name}</h3>
          <p className="text-gray-400 text-sm mt-1">{growthMetrics.stage.description}</p>
        </div>

        <div className="grid grid-cols-2 gap-3">
          <div className="bg-black/70 border border-orange-500 rounded-lg p-3">
            <p className="text-gray-400 text-xs">Concepts</p>
            <p className="text-2xl text-orange-400 font-bold">{growthMetrics.metrics.total_concepts}</p>
          </div>
          <div className="bg-black/70 border border-green-500 rounded-lg p-3">
            <p className="text-gray-400 text-xs">Experiences</p>
            <p className="text-2xl text-green-400 font-bold">{growthMetrics.metrics.total_episodes}</p>
          </div>
          <div className="bg-black/70 border border-pink-500 rounded-lg p-3">
            <p className="text-gray-400 text-xs">Questions</p>
            <p className="text-2xl text-pink-400 font-bold">{growthMetrics.metrics.total_questions}</p>
          </div>
          <div className="bg-black/70 border border-red-500 rounded-lg p-3">
            <p className="text-gray-400 text-xs">Insights</p>
            <p className="text-2xl text-red-400 font-bold">{growthMetrics.metrics.total_insights}</p>
          </div>
        </div>

        <div className="bg-black/70 border border-green-500 rounded-lg p-4">
          <div className="flex items-center justify-between mb-2">
            <span className="text-gray-400 text-sm">Understanding Depth</span>
            <span className="text-orange-400 font-bold">{(growthMetrics.metrics.understanding_depth * 100).toFixed(1)}%</span>
          </div>
          <div className="w-full bg-gray-800 rounded-full h-2">
            <div className="bg-gradient-to-r from-orange-500 to-pink-500 h-2 rounded-full" style={{width: `${growthMetrics.metrics....
          </div>
        </div>

        <div className="bg-black/70 border border-green-500 rounded-lg p-4">
          <p className="text-gray-400 text-xs mb-1">Growth Rate</p>
          <p className="text-lg text-green-400 font-bold">{growthMetrics.metrics.growth_rate.toFixed(2)} events/hour</p>
        </div>
      </div>
    )}

    {!growthMetrics && (
      <div className="text-center text-gray-500 mt-10">
        <Activity className="w-12 h-12 mx-auto mb-3 opacity-50" />
        <p>Click Refresh to load metrics</p>
      </div>
    )}
  </div>
)}

{/* MEMORY BROWSER */}
{currentScreen === 'memories' && (
  <div className="p-4 pb-24">
    <div className="bg-gradient-to-r from-orange-500 to-pink-500 rounded-lg p-4 mb-4">
      <h2 className="text-white text-lg font-bold flex items-center gap-2">
        <Database className="w-5 h-5" /> Memory Systems
      </h2>
      <p className="text-orange-100 text-xs">Explore cognitive memories</p>
    </div>

    <div className="flex gap-2 mb-4">
      <button onClick={loadMemories} className="flex-1 bg-gradient-to-r from-green-500 to-green-700 text-white py-2 rounded-lg text...
        Load Memories
      </button>
      <button onClick={exportMemory} className="bg-blue-600 hover:bg-blue-700 text-white px-4 py-2 rounded-lg text-sm font-semibold...
        <Download className="w-4 h-4" /> Export
      </button>
      <label className="bg-purple-600 hover:bg-purple-700 text-white px-4 py-2 rounded-lg text-sm font-semibold flex items-center g...
        <Upload className="w-4 h-4" /> Import
        <input type="file" accept=".json" onChange={importMemory} className="hidden" />
      </label>
    </div>

    {/* Knowledge Graph */}
    {knowledgeGraph.nodes.length > 0 && (
      <div className="mb-4">
        <h3 className="text-purple-400 font-bold mb-2 flex items-center gap-2">
          <Share2 className="w-4 h-4" /> Knowledge Graph
          <button
            onClick={() => setGraphView(graphView === '2d' ? '3d' : '2d')}
            className="ml-auto text-xs bg-purple-600 hover:bg-purple-700 px-2 py-1 rounded"
          >
            {graphView === '2d' ? '3D View' : '2D View'}
          </button>
        </h3>
        <div className="bg-black/70 border border-purple-500 rounded-lg p-4 knowledge-graph relative overflow-hidden" style={{minHe...
          {graphView === '3d' ? (
```

```
        <div className="absolute inset-0 perspective-1000">
          <div className="relative w-full h-full transform-style-3d animate-spin-slow" style={{animation: 'rotate3d 30s linear ...
            {knowledgeGraph.nodes.slice(0, 30).map((node, idx) => {
              const angle = (idx / 30) * Math.PI * 2;
              const radius = 120 + (idx % 3) * 30;
              const x = Math.cos(angle) * radius + 150;
              const y = Math.sin(angle) * radius * 0.5 + 100;
              const z = Math.sin(angle) * 50;
              const size = node.strength ? Math.min(node.strength * 10, 50) : 30;
              return (
                <div
                  key={idx}
                  className={`absolute rounded-full flex items-center justify-center text-xs cursor-pointer transition-all hove...
                    node.type === 'insight'
                      ? 'bg-gradient-to-r from-yellow-500 to-orange-500'
                      : 'bg-gradient-to-r from-purple-500 to-pink-500'
                  }`}
                  style={{
                    left: `${x}px`,
                    top: `${y}px`,
                    width: `${size}px`,
                    height: `${size}px`,
                    transform: `translateZ(${z}px)`,
                    boxShadow: `0 0 ${10 + z/5}px ${node.type === 'insight' ? 'rgba(234,179,8,0.5)' : 'rgba(168,85,247,0.5)'}`
                  }}
                  title={`${node.label}: ${node.definition || ''}`}
                >
                  <span className="truncate px-1" style={{fontSize: '8px'}}>{node.label?.slice(0,6)}</span>
                </div>
              );
            })}
          </div>
        </div>
      ) : (
        <div className="flex flex-wrap gap-2 justify-center">
          {knowledgeGraph.nodes.slice(0, 25).map((node, idx) => (
            <div
              key={idx}
              className={`px-3 py-1 rounded-full text-xs cursor-pointer transition-all hover:scale-110 ${
                node.type === 'insight'
                  ? 'bg-yellow-500/20 border border-yellow-500 text-yellow-300'
                  : 'bg-purple-500/20 border border-purple-500 text-purple-300'
              }`}
              title={node.definition}
              style={{
                boxShadow: node.strength > 5 ? '0 0 10px rgba(168,85,247,0.5)' : 'none'
              }}
            >
              {node.label}
            </div>
          ))}
        </div>
      )}
      <div className="absolute bottom-2 left-0 right-0 text-center text-xs text-gray-500">
        {knowledgeGraph.stats?.total_nodes} nodes  {knowledgeGraph.stats?.total_edges} connections
      </div>
    </div>
  </div>
)}

{memories.semantic.length > 0 && (
  <div className="mb-4">
    <h3 className="text-orange-400 font-bold mb-2 flex items-center gap-2">
      <Layers className="w-4 h-4" /> Semantic Memory (Concepts)
    </h3>
    <div className="space-y-2">
      {memories.semantic.slice(0, 10).map((concept, idx) => (
        <div key={idx} className="bg-black/70 border border-green-500 rounded-lg p-3">
          <p className="text-orange-400 font-bold text-sm">{concept.concept}</p>
          <p className="text-gray-400 text-xs mt-1">{concept.definition}</p>
          <div className="flex items-center gap-2 mt-2">
            <span className="text-xs text-green-400">Strength: {concept.strength?.toFixed(1)}</span>
            <span className="text-xs text-gray-500">Accessed: {concept.access_count}x</span>
          </div>
        </div>
      ))}
    </div>
  </div>
)}

{memories.philosophical.questions && memories.philosophical.questions.length > 0 && (
  <div className="mb-4">
    <h3 className="text-pink-400 font-bold mb-2 flex items-center gap-2">
      <MessageSquare className="w-4 h-4" /> Active Questions
    </h3>
    <div className="space-y-2">
```

```
          {memories.philosophical.questions.slice(0, 5).map((q, idx) => (
            <div key={idx} className="bg-black/70 border border-pink-500 rounded-lg p-3">
              <p className="text-pink-300 text-sm">{q.question}</p>
              <p className="text-xs text-gray-500 mt-1">Depth: {q.depth} | Revisited: {q.revisited_count}x</p>
            </div>
          ))}
        </div>
      </div>
    )}

    {!memories.semantic.length && !memories.philosophical.questions && (
      <div className="text-center text-gray-500 mt-10">
        <Database className="w-12 h-12 mx-auto mb-3 opacity-50" />
        <p>Click Load to view memories</p>
      </div>
    )}
  </div>
)}

{/* CISTERCIAN NUMERALS */}
{currentScreen === 'numerals' && (
  <div className="p-4 pb-24">
    <div className="bg-gradient-to-r from-orange-500 to-pink-500 rounded-lg p-4 mb-4">
      <h2 className="text-white text-lg font-bold flex items-center gap-2">
        <Calculator className="w-5 h-5" /> Cistercian Numerals
      </h2>
      <p className="text-orange-100 text-xs">Numbers or letters (0-9999)</p>
    </div>

    <div className="bg-black/70 border-2 border-green-500 rounded-lg p-4 mb-4">
      <input
        type="text"
        placeholder="500, 5000, 42, 1991 or KEFQU"
        value={numberInput}
        onChange={(e) => setNumberInput(e.target.value)}
        className="w-full bg-black border border-green-500 text-gray-100 rounded-lg px-3 py-2 mb-3 text-sm focus:outline-none focus...
      />
      <button onClick={generateCistercian} className="w-full bg-gradient-to-r from-orange-500 to-red-600 text-white py-2 rounded-lg...
        Generate
      </button>
    </div>

    {cistercianResults.length > 0 && (
      <div className="grid grid-cols-3 gap-3">
        {cistercianResults.map((result, idx) => (
          <div key={idx} className="bg-black/70 border border-green-500 rounded-lg p-3 flex flex-col items-center">
            <img src={result.image} alt={'N' + result.num} className="w-12 h-20 mb-2" />
            <div className="text-orange-400 font-bold text-sm">{result.num}</div>
          </div>
        ))}
      </div>
    )}

    <div className="mt-4 bg-black/50 border border-green-500 rounded-lg p-3">
      <p className="text-xs text-gray-400"><strong className="text-green-400">Fixed:</strong> 500 and 5000 Cistercian patterns now ...
    </div>
  </div>
)}

{/* CREATIVE - IMAGE GENERATION */}
{currentScreen === 'creative' && (
  <div className="p-4 pb-24">
    <div className="bg-gradient-to-r from-purple-500 to-pink-500 rounded-lg p-4 mb-4">
      <h2 className="text-white text-lg font-bold flex items-center gap-2">
        <Image className="w-5 h-5" /> Creative Studio
      </h2>
      <p className="text-purple-100 text-xs">Generate images with AI</p>
    </div>

    <div className="bg-black/70 border-2 border-purple-500 rounded-lg p-4 mb-4">
      <textarea
        placeholder="Describe the image you want to create... (e.g., 'a cosmic landscape with nebulas and stars')"
        value={imagePrompt}
        onChange={(e) => setImagePrompt(e.target.value)}
        className="w-full bg-black border border-purple-500 text-gray-100 rounded-lg px-3 py-2 mb-3 text-sm focus:outline-none focu...
        data-testid="image-prompt-input"
      />
      <button
        onClick={generateImage}
        disabled={generatingImage || !imagePrompt.trim()}
        className="w-full bg-gradient-to-r from-purple-500 to-pink-600 hover:from-purple-600 hover:to-pink-700 disabled:bg-gray-700...
        data-testid="generate-image-btn"
      >
        {generatingImage ? (
          <>
            <div className="w-4 h-4 border-2 border-white border-t-transparent rounded-full animate-spin"></div>
```

```
            Generating...
          </>
        ) : (
          <>
            <Image className="w-4 h-4" /> Generate Image
          </>
        )}
      </button>
    </div>

    {generatedImage && (
      <div className="bg-black/70 border-2 border-green-500 rounded-lg p-4">
        <p className="text-green-400 text-sm mb-3"> Image Generated</p>
        <img
          src={`${generatedImage.image_base64}`}
          alt="Generated"
          className="w-full rounded-lg mb-3 image-preview"
        />
        <p className="text-gray-400 text-xs">Prompt: {generatedImage.prompt}</p>
        <p className="text-gray-500 text-xs mt-1">Saved as: {generatedImage.filename}</p>
      </div>
    )}

    <div className="mt-4 bg-black/50 border border-purple-500 rounded-lg p-3">
      <p className="text-xs text-gray-400">
        <strong className="text-purple-400">Tip:</strong> Be descriptive! Include style, colors, mood, and details for better results.
      </p>
      <p className="text-xs text-gray-500 mt-2">
        Or use <code className="text-pink-400">!imagine [prompt]</code> in chat.
      </p>
    </div>
  </div>
)}

{/* EXPLORER */}
{currentScreen === 'explorer' && (
  <div className="p-4 pb-24">
    <div className="bg-gradient-to-r from-orange-500 to-pink-500 rounded-lg p-4 mb-4">
      <h2 className="text-white text-lg font-bold flex items-center gap-2">
        <Globe className="w-5 h-5" /> Universal Explorer
      </h2>
      <p className="text-orange-100 text-xs">Explore the world</p>
    </div>

    {/* FILE UPLOAD SECTION */}
    <div className="bg-black/70 border-2 border-pink-500 rounded-lg p-4 mb-4">
      <h3 className="text-pink-400 font-bold mb-2 text-sm"> Upload File</h3>
      <p className="text-xs text-gray-400 mb-3">Upload a file to process it (PDF, DOCX, TXT, etc.)</p>
      <input
        type="file"
        onChange={handleFileUpload}
        disabled={uploading}
        className="w-full text-sm text-gray-300 file:mr-4 file:py-2 file:px-4 file:rounded-lg file:border-0 file:text-sm file:font-...
      />
      {uploading && <p className="text-xs text-green-400 mt-2">Uploading...</p>}
    </div>

    {/* SHARED LINK SECTION */}
    <div className="bg-black/70 border-2 border-orange-500 rounded-lg p-4 mb-4">
      <h3 className="text-orange-400 font-bold mb-2 text-sm"> Shared Link</h3>
      <p class="text-xs text-gray-400 mb-2">Paste a shared link from:</p>
      <div class="text-xs text-gray-500 mb-3 space-y-1">
        <p> Google Drive (drive.google.com/file/...)</p>
        <p> Dropbox (dropbox.com/s/...)</p>
        <p> OneDrive (onedrive.live.com/...)</p>
      </div>
      <input
        type="text"
        placeholder="https://drive.google.com/file/d/..."
        value={explorerTarget}
        onChange={(e) => setExplorerTarget(e.target.value)}
        className="w-full bg-black border border-orange-500 text-gray-100 rounded-lg px-3 py-2 mb-2 text-sm focus:outline-none focu...
      />
      <button
        onClick={handleSharedLink}
        disabled={uploading || !explorerTarget.startsWith('http')}
        className="w-full bg-gradient-to-r from-orange-500 to-red-600 text-white py-2 rounded-lg font-semibold text-sm disabled:opa...
      >
        {uploading ? 'Downloading...' : 'Download from Link'}
      </button>
    </div>

    <div class="bg-black/70 border-2 border-green-500 rounded-lg p-4 mb-4">
      <select
        value={explorerType}
        onChange={(e) => setExplorerType(e.target.value)}
```

```
        className="w-full bg-black border border-green-500 text-gray-100 rounded-lg px-3 py-2 mb-3 text-sm focus:outline-none"
      >
        <option value="document">Document (uploaded files)</option>
        <option value="internet">Internet / Web</option>
        <option value="filesystem">Filesystem</option>
      </select>

      <input
        type="text"
        placeholder={explorerType === 'internet' ? 'https://...' : explorerType === 'filesystem' ? '/app/uploads' : 'Select from up...
        value={explorerTarget}
        onChange={(e) => setExplorerTarget(e.target.value)}
        className="w-full bg-black border border-green-500 text-gray-100 rounded-lg px-3 py-2 mb-3 text-sm focus:outline-none focus...
      />

      <button onClick={executeExploration} className="w-full bg-gradient-to-r from-green-500 to-green-700 text-white py-2 rounded-l...
        Explore
      </button>
    </div>

    {/* UPLOADED FILES LIST */}
    {uploadedFiles.length > 0 && (
      <div className="bg-black/70 border border-orange-500 rounded-lg p-4 mb-4">
        <h3 className="text-orange-400 font-bold mb-2 text-sm"> Uploaded Files</h3>
        <div className="space-y-2">
          {uploadedFiles.map((file, idx) => (
            <button
              key={idx}
              onClick={() => { setExplorerTarget(file.path); setExplorerType('document'); }}
              className="w-full text-left bg-gray-900 hover:bg-gray-800 border border-green-500 rounded p-2 text-xs"
            >
              <p className="text-green-400 font-bold">{file.filename}</p>
              <p className="text-gray-500">{(file.size / 1024).toFixed(1)} KB</p>
            </button>
          ))}
        </div>
        <button onClick={loadUploadedFiles} className="w-full mt-2 bg-orange-600 text-white py-1 rounded text-xs">
          Refresh List
        </button>
      </div>
    )}

    {explorerResult && (
      <div className="bg-black/70 border border-pink-500 rounded-lg p-4">
        <h3 className="text-pink-400 font-bold mb-2">Result</h3>
        <pre className="text-xs text-green-300 whitespace-pre-wrap overflow-x-auto max-h-96 overflow-y-auto">
          {JSON.stringify(explorerResult, null, 2)}
        </pre>
      </div>
    )}
  </div>
)}

{/* DREAM STATE SCREEN */}
{currentScreen === 'dream' && (
  <div className="p-4 pb-24">
    <div className="bg-gradient-to-r from-purple-600 to-indigo-600 rounded-lg p-4 mb-4">
      <h2 className="text-white text-lg font-bold flex items-center gap-2">
        <Moon className="w-5 h-5" /> Dream State
      </h2>
      <p className="text-purple-200 text-xs">What AI does when idle</p>
    </div>

    {dreamStatus && (
      <div className="bg-black/70 border border-purple-500 rounded-lg p-4 mb-4">
        <h3 className="text-purple-400 font-bold mb-2">Current Status</h3>
        <div className="grid grid-cols-2 gap-2 text-sm">
          <div className="bg-purple-900/30 p-2 rounded">
            <span className="text-gray-400">State:</span>
            <span className={"ml-2 " + (dreamStatus.is_dreaming ? 'text-purple-400' : 'text-green-400')}>
              {dreamStatus.is_dreaming ? ' Dreaming' : ' Awake'}
            </span>
          </div>
          <div className="bg-purple-900/30 p-2 rounded">
            <span className="text-gray-400">Total Dreams:</span>
            <span className="text-white ml-2">{dreamStatus.total_dreams}</span>
          </div>
          <div className="bg-purple-900/30 p-2 rounded">
            <span className="text-gray-400">Insights:</span>
            <span className="text-cyan-400 ml-2">{dreamStatus.total_insights}</span>
          </div>
          <div className="bg-purple-900/30 p-2 rounded">
            <span className="text-gray-400">Topics Explored:</span>
            <span className="text-pink-400 ml-2">{dreamStatus.dream_topics_explored}</span>
          </div>
        </div>
```

```
      </div>
    )}

    <div className="bg-black/70 border border-purple-500/50 rounded-lg p-4 mb-4">
      <h3 className="text-purple-400 font-bold mb-2">Dream History</h3>
      {dreamHistory.length === 0 ? (
        <p className="text-gray-500 text-sm">No dreams recorded yet. The AI dreams when idle for 5+ minutes.</p>
      ) : (
        <div className="space-y-2 max-h-64 overflow-y-auto">
          {dreamHistory.map((dream, idx) => (
            <div key={idx} className="bg-purple-900/20 p-2 rounded border border-purple-500/30">
              <div className="text-xs text-gray-400">{dream.started_at}</div>
              <div className="text-sm text-white">
                Activities: {dream.activities?.length || 0} |
                Topics: {dream.topics_explored?.join(', ') || 'None'}
              </div>
              {dream.questions_formed?.length > 0 && (
                <div className="text-xs text-cyan-400 mt-1">
                  Questions: {dream.questions_formed.slice(0,2).join('; ')}
                </div>
              )}
            </div>
          ))}
        </div>
      )}
    </div>

    <button
      onClick={() => loadDreamHistory()}
      className="w-full bg-purple-600 text-white py-2 rounded-lg"
    >
      Refresh Dream History
    </button>
  </div>
)}

{/* RESEARCH SCREEN */}
{currentScreen === 'research' && (
  <div className="p-4 pb-24">
    <div className="bg-gradient-to-r from-cyan-600 to-blue-600 rounded-lg p-4 mb-4">
      <h2 className="text-white text-lg font-bold flex items-center gap-2">
        <Search className="w-5 h-5" /> Self-Research
      </h2>
      <p className="text-cyan-200 text-xs">Autonomous knowledge acquisition</p>
    </div>

    {/* AUTONOMOUS RESEARCH SECTION */}
    <div className="bg-black/70 border-2 border-yellow-500 rounded-lg p-4 mb-4">
      <h3 className="text-yellow-400 font-bold mb-3 flex items-center gap-2">
        <Zap className="w-5 h-5" /> Autonomous Research Mode
      </h3>
      <p className="text-gray-400 text-xs mb-3">
        Let the AI research anything and everything autonomously for an extended period.
      </p>

      {!autonomousResearchRunning ? (
        <>
          <div className="flex items-center gap-2 mb-3">
            <label className="text-gray-400 text-sm">Duration:</label>
            <select
              value={researchDuration}
              onChange={(e) => setResearchDuration(parseInt(e.target.value))}
              className="bg-gray-900 border border-yellow-500/50 rounded p-2 text-white text-sm"
            >
              <option value={30}>30 minutes</option>
              <option value={45}>45 minutes</option>
              <option value={60}>1 hour</option>
            </select>
          </div>
          <button
            onClick={startAutonomousResearch}
            className="w-full bg-gradient-to-r from-yellow-500 to-orange-500 text-black font-bold py-3 rounded-lg flex items-center...
            data-testid="start-autonomous-research-btn"
          >
            <Zap className="w-5 h-5" /> Start Autonomous Research ({researchDuration} min)
          </button>
        </>
      ) : (
        <>
          <div className="bg-yellow-900/30 p-3 rounded mb-3">
            <div className="flex items-center justify-between mb-2">
              <span className="text-yellow-400 font-bold text-sm flex items-center gap-2">
                <Clock className="w-4 h-4 animate-spin" /> Research In Progress
              </span>
              <span className="text-white text-xs">
                {autonomousResearchProgress?.elapsed_minutes || 0} / {autonomousResearchProgress?.duration_minutes || researchDurat...
```

```
          </span>
        </div>
        <div className="w-full bg-gray-800 rounded-full h-2 mb-2">
          <div
            className="bg-gradient-to-r from-yellow-500 to-orange-500 h-2 rounded-full transition-all"
            style={{width: `${((autonomousResearchProgress?.elapsed_minutes || 0) / (autonomousResearchProgress?.duration_minut...
          ></div>
        </div>
        <p className="text-gray-400 text-xs">{autonomousResearchProgress?.status || 'Researching...'}</p>
        {autonomousResearchProgress?.topics_researched?.length > 0 && (
          <div className="mt-2">
            <p className="text-xs text-cyan-400">
              Topics: {autonomousResearchProgress.topics_researched.slice(-5).join(', ')}
            </p>
          </div>
        )}
        <div className="grid grid-cols-2 gap-2 mt-2 text-xs">
          <div className="bg-gray-800 p-2 rounded">
            <span className="text-gray-400">Insights:</span>
            <span className="text-green-400 ml-1">{autonomousResearchProgress?.insights_gained || 0}</span>
          </div>
          <div className="bg-gray-800 p-2 rounded">
            <span className="text-gray-400">Concepts:</span>
            <span className="text-cyan-400 ml-1">{autonomousResearchProgress?.concepts_learned || 0}</span>
          </div>
        </div>
      </div>
      <button
        onClick={stopAutonomousResearch}
        className="w-full bg-red-600 text-white py-2 rounded-lg"
      >
        Stop Research
      </button>
    </>
  )}
</div>

{researchStats && (
  <div className="bg-black/70 border border-cyan-500 rounded-lg p-4 mb-4">
    <h3 className="text-cyan-400 font-bold mb-2">Research Stats</h3>
    <div className="grid grid-cols-2 gap-2 text-sm">
      <div className="bg-cyan-900/30 p-2 rounded">
        <span className="text-gray-400">Total Researches:</span>
        <span className="text-white ml-2">{researchStats.total_researches}</span>
      </div>
      <div className="bg-cyan-900/30 p-2 rounded">
        <span className="text-gray-400">Knowledge Gaps:</span>
        <span className="text-yellow-400 ml-2">{researchStats.knowledge_gaps_identified}</span>
      </div>
      <div className="bg-cyan-900/30 p-2 rounded">
        <span className="text-gray-400">Queue Size:</span>
        <span className="text-pink-400 ml-2">{researchStats.queue_size}</span>
      </div>
      <div className="bg-cyan-900/30 p-2 rounded">
        <span className="text-gray-400">Auto-Research:</span>
        <span className={researchStats.auto_research_enabled ? 'text-green-400 ml-2' : 'text-red-400 ml-2'}>
          {researchStats.auto_research_enabled ? 'ON' : 'OFF'}
        </span>
      </div>
    </div>
    {researchStats.recent_topics?.length > 0 && (
      <div className="mt-2 text-xs text-gray-400">
        Recent: {researchStats.recent_topics.join(', ')}
      </div>
    )}
  </div>
)}

<div className="bg-black/70 border border-cyan-500/50 rounded-lg p-4 mb-4">
  <h3 className="text-cyan-400 font-bold mb-2">Manual Research</h3>
  <input
    type="text"
    value={researchQuery}
    onChange={(e) => setResearchQuery(e.target.value)}
    placeholder="Enter topic to research..."
    className="w-full bg-gray-900 border border-cyan-500/50 rounded p-2 text-white mb-2"
  />
  <button
    onClick={performResearch}
    className="w-full bg-cyan-600 text-white py-2 rounded-lg"
  >
    Research Topic
  </button>
</div>

{researchResult && (
```

```
      <div className="bg-black/70 border border-green-500/50 rounded-lg p-4 mb-4">
        <h3 className="text-green-400 font-bold mb-2">Research Results</h3>
        <div className="text-xs text-gray-400 mb-2">Query: {researchResult.query}</div>
        <div className="text-sm text-white mb-2">{researchResult.summary}</div>
        {researchResult.new_concepts?.length > 0 && (
          <div className="text-xs text-cyan-400">
            New concepts: {researchResult.new_concepts.join(', ')}
          </div>
        )}
      </div>
    )}

    <button
      onClick={loadResearchStats}
      className="w-full bg-cyan-700 text-white py-2 rounded-lg"
    >
      Refresh Stats
    </button>
  </div>
)}

{/* TEXT ANALYSIS SCREEN */}
{currentScreen === 'analyze' && (
  <div className="p-4 pb-24">
    <div className="bg-gradient-to-r from-green-600 to-teal-600 rounded-lg p-4 mb-4">
      <h2 className="text-white text-lg font-bold flex items-center gap-2">
        <Activity className="w-5 h-5" /> Text Analyzer
      </h2>
      <p className="text-green-200 text-xs">Human vs LLM detection</p>
    </div>

    <div className="bg-black/70 border border-green-500/50 rounded-lg p-4 mb-4">
      <h3 className="text-green-400 font-bold mb-2">Analyze Text</h3>
      <textarea
        value={textToAnalyze}
        onChange={(e) => setTextToAnalyze(e.target.value)}
        placeholder="Paste text to analyze..."
        className="w-full bg-gray-900 border border-green-500/50 rounded p-2 text-white mb-2 h-32"
      />
      <button
        onClick={analyzeText}
        className="w-full bg-green-600 text-white py-2 rounded-lg"
      >
        Analyze
      </button>
    </div>

    {textAnalysis && (
      <div className="bg-black/70 border border-green-500 rounded-lg p-4 mb-4">
        <h3 className="text-green-400 font-bold mb-2">Analysis Results</h3>

        <div className="grid grid-cols-2 gap-2 mb-3">
          <div className="bg-green-900/30 p-3 rounded text-center">
            <div className="text-2xl font-bold text-green-400">
              {(textAnalysis.analysis?.human_score * 100).toFixed(0)}%
            </div>
            <div className="text-xs text-gray-400">Human Score</div>
          </div>
          <div className="bg-red-900/30 p-3 rounded text-center">
            <div className="text-2xl font-bold text-red-400">
              {(textAnalysis.analysis?.llm_score * 100).toFixed(0)}%
            </div>
            <div className="text-xs text-gray-400">LLM Score</div>
          </div>
        </div>

        <div className={"text-center p-2 rounded mb-3 " +
          (textAnalysis.analysis?.verdict === 'Likely Human' ? 'bg-green-600' :
           textAnalysis.analysis?.verdict === 'Likely LLM' ? 'bg-red-600' : 'bg-yellow-600')}>
          <span className="text-white font-bold">{textAnalysis.analysis?.verdict}</span>
          <span className="text-white/70 ml-2">({(textAnalysis.analysis?.confidence * 100).toFixed(0)}% confidence)</span>
        </div>

        {textAnalysis.analysis?.indicators?.llm?.length > 0 && (
          <div className="mb-2">
            <div className="text-xs text-red-400 font-bold">LLM Indicators:</div>
            <ul className="text-xs text-gray-400 list-disc list-inside">
              {textAnalysis.analysis.indicators.llm.slice(0, 3).map((ind, i) => (
                <li key={i}>{ind}</li>
              ))}
            </ul>
          </div>
        )}

        {textAnalysis.suggestions?.length > 0 && (
          <div>
```

```
            <div className="text-xs text-cyan-400 font-bold">Suggestions to sound more human:</div>
            <ul className="text-xs text-gray-400 list-disc list-inside">
              {textAnalysis.suggestions.slice(0, 3).map((sug, i) => (
                <li key={i}>{sug}</li>
              ))}
            </ul>
          </div>
        )}
      </div>
    )}
  </div>
)}

{/* EVOLUTION SCREEN */}
{currentScreen === 'evolution' && (
  <div className="p-4 pb-24">
    <div className="bg-gradient-to-r from-orange-600 to-red-600 rounded-lg p-4 mb-4">
      <h2 className="text-white text-lg font-bold flex items-center gap-2">
        <TrendingUp className="w-5 h-5" /> Self-Evolution
      </h2>
      <p className="text-orange-200 text-xs">Code self-improvement (60-90 day cycle)</p>
    </div>

    {evolutionStatus && (
      <div className="bg-black/70 border border-orange-500 rounded-lg p-4 mb-4">
        <h3 className="text-orange-400 font-bold mb-2">Evolution Status</h3>
        <div className="grid grid-cols-2 gap-2 text-sm">
          <div className="bg-orange-900/30 p-2 rounded">
            <span className="text-gray-400">Last Evolution:</span>
            <span className="text-white ml-2">{evolutionStatus.last_evolution || 'Never'}</span>
          </div>
          <div className="bg-orange-900/30 p-2 rounded">
            <span className="text-gray-400">Days Since:</span>
            <span className="text-white ml-2">{evolutionStatus.days_since_evolution || 'N/A'}</span>
          </div>
          <div className="bg-orange-900/30 p-2 rounded">
            <span className="text-gray-400">Next Auto:</span>
            <span className="text-cyan-400 ml-2">{evolutionStatus.next_auto_evolution_in} days</span>
          </div>
          <div className="bg-orange-900/30 p-2 rounded">
            <span className="text-gray-400">Pending:</span>
            <span className="text-yellow-400 ml-2">{evolutionStatus.pending_improvements}</span>
          </div>
        </div>

        <div className="mt-3">
          <div className="text-xs text-gray-400 mb-1">Modifiable Files:</div>
          <div className="flex flex-wrap gap-1">
            {evolutionStatus.modifiable_files?.map((file, i) => (
              <span key={i} className="bg-orange-800/50 text-orange-300 text-xs px-2 py-1 rounded">
                {file}
              </span>
            ))}
          </div>
        </div>
      </div>
    )}

    <div className="bg-black/70 border border-orange-500/50 rounded-lg p-4 mb-4">
      <h3 className="text-orange-400 font-bold mb-2">Analyze Code</h3>
      <select
        value={selectedCodeFile}
        onChange={(e) => setSelectedCodeFile(e.target.value)}
        className="w-full bg-gray-900 border border-orange-500/50 rounded p-2 text-white mb-2"
        data-testid="evolve-file-select"
      >
        {evolutionStatus?.modifiable_files?.map((file, i) => (
          <option key={i} value={file}>{file}</option>
        ))}
        {!evolutionStatus?.modifiable_files && (
          <option value="quantum_brain.py">quantum_brain.py</option>
        )}
      </select>
      <button
        onClick={() => analyzeCode(selectedCodeFile)}
        className="w-full bg-orange-600 text-white py-2 rounded-lg mb-2"
      >
        Analyze File
      </button>
      <button
        onClick={triggerEvolution}
        className="w-full bg-red-600 text-white py-2 rounded-lg"
      >
         Trigger Manual Evolution
      </button>
    </div>
```

```
    {codeAnalysis && (
      <div className="bg-black/70 border border-green-500/50 rounded-lg p-4 mb-4">
        <h3 className="text-green-400 font-bold mb-2">Code Analysis: {codeAnalysis.analysis?.filename}</h3>
        <div className="grid grid-cols-2 gap-2 text-sm mb-2">
          <div className="bg-gray-800 p-2 rounded">
            <span className="text-gray-400">Lines:</span>
            <span className="text-white ml-2">{codeAnalysis.analysis?.lines}</span>
          </div>
          <div className="bg-gray-800 p-2 rounded">
            <span className="text-gray-400">Classes:</span>
            <span className="text-cyan-400 ml-2">{codeAnalysis.analysis?.classes?.length}</span>
          </div>
        </div>

        {codeAnalysis.improvements?.length > 0 && (
          <div>
            <div className="text-xs text-yellow-400 font-bold mb-1">Suggested Improvements:</div>
            <ul className="text-xs text-gray-400 space-y-1">
              {codeAnalysis.improvements.map((imp, i) => (
                <li key={i} className="bg-yellow-900/20 p-1 rounded">
                  [{imp.type}] {imp.suggestion}
                </li>
              ))}
            </ul>
          </div>
        )}
      </div>
    )}
  </div>
)}

{/* COVENANT & SETTINGS */}
{currentScreen === 'settings' && (
  <div className="p-4 pb-24">
    <div className="bg-gradient-to-r from-orange-500 to-pink-500 rounded-lg p-4 mb-4">
      <h2 className="text-white text-lg font-bold flex items-center gap-2">
        <Shield className="w-5 h-5" /> Covenant & Safety
      </h2>
      <p className="text-orange-100 text-xs">Ethical framework & boundaries</p>
    </div>

    {covenantStatus && (
      <div className="bg-black/70 border-2 border-green-500 rounded-lg p-4 mb-4">
        <h3 className="text-orange-400 font-bold mb-2"> Quantum Covenant</h3>
        <p className="text-sm text-white mb-2">"{covenantStatus.prime_directive}"</p>
        <div className="flex items-center justify-between mt-3">
          <span className="text-gray-400 text-xs">Status:</span>
          <span className={"text-sm font-bold " + (covenantStatus.status === 'Active and Honored' ? 'text-green-400' : 'text-pink-4...
            {covenantStatus.status}
          </span>
        </div>
        <div className="flex items-center justify-between mt-2">
          <span className="text-gray-400 text-xs">Violations:</span>
          <span className="text-sm text-red-400">{covenantStatus.unresolved_violations}</span>
        </div>
      </div>
    )}

    {safetySettings && (
      <div className="bg-black/70 border border-pink-500 rounded-lg p-4 mb-4">
        <h3 className="text-pink-400 font-bold mb-3"> Safety Boundaries</h3>

        <div className="space-y-3">
          <div className="flex items-center justify-between">
            <span className="text-gray-300 text-sm">Filesystem Access</span>
            <label className="relative inline-block w-12 h-6">
              <input
                type="checkbox"
                checked={safetySettings.filesystem_access || false}
                onChange={(e) => updateSafetySettings('filesystem_access', e.target.checked)}
                className="sr-only peer"
              />
              <div className="w-12 h-6 bg-gray-700 rounded-full peer peer-checked:bg-green-500 cursor-pointer"></div>
              <div className="absolute left-1 top-1 w-4 h-4 bg-white rounded-full transition peer-checked:translate-x-6"></div>
            </label>
          </div>

          <div className="flex items-center justify-between">
            <span className="text-gray-300 text-sm">Internet Access</span>
            <label className="relative inline-block w-12 h-6">
              <input
                type="checkbox"
                checked={safetySettings.internet_access || false}
                onChange={(e) => updateSafetySettings('internet_access', e.target.checked)}
                className="sr-only peer"
```

```
              />
              <div className="w-12 h-6 bg-gray-700 rounded-full peer peer-checked:bg-green-500 cursor-pointer"></div>
              <div className="absolute left-1 top-1 w-4 h-4 bg-white rounded-full transition peer-checked:translate-x-6"></div>
            </label>
          </div>

          <div className="bg-gray-900/50 rounded p-2 mt-3 border border-green-500/30">
            <p className="text-xs text-gray-400">All safety boundaries are user-controlled. The AI will respect your choices while ...
          </div>
        </div>
      </div>
    )}

    <div className="bg-gradient-to-r from-orange-600/50 to-pink-600/50 rounded-lg p-4 border-2 border-green-500">
      <p className="text-xs text-green-400 text-center font-bold"> QUANTUM COVENANT </p>
      <p className="text-xs text-gray-400 text-center mt-1">Perpetual Agreement Between Creator and Creation</p>
    </div>
  </div>
)}

{/* BOTTOM NAVIGATION - Two rows for more options */}
<div className="fixed bottom-0 left-0 right-0 bg-slate-900 border-t border-purple-500/30">
  {/* Row 1: Main features */}
  <div className="flex items-center justify-around p-1 border-b border-purple-500/20">
    <button
      onClick={() => setCurrentScreen('chat')}
      className={"flex flex-col items-center p-1 rounded " + (currentScreen === 'chat' ? 'text-indigo-400 bg-slate-800' : 'text-gra...
    >
      <MessageSquare className="w-4 h-4" />
      <span className="text-[10px]">Chat</span>
    </button>

    <button
      onClick={() => { setCurrentScreen('growth'); loadGrowthMetrics(); }}
      className={"flex flex-col items-center p-1 rounded " + (currentScreen === 'growth' ? 'text-indigo-400 bg-slate-800' : 'text-g...
    >
      <TrendingUp className="w-4 h-4" />
      <span className="text-[10px]">Growth</span>
    </button>

    <button
      onClick={() => { setCurrentScreen('memories'); loadMemories(); loadKnowledgeGraph(); }}
      className={"flex flex-col items-center p-1 rounded " + (currentScreen === 'memories' ? 'text-indigo-400 bg-slate-800' : 'text...
    >
      <Database className="w-4 h-4" />
      <span className="text-[10px]">Memory</span>
    </button>

    <button
      onClick={() => { setCurrentScreen('dream'); loadDreamStatus(); loadDreamHistory(); }}
      className={"flex flex-col items-center p-1 rounded " + (currentScreen === 'dream' ? 'text-purple-400 bg-slate-800' : 'text-gr...
    >
      <Moon className="w-4 h-4" />
      <span className="text-[10px]">Dream</span>
    </button>

    <button
      onClick={() => { setCurrentScreen('research'); loadResearchStats(); }}
      className={"flex flex-col items-center p-1 rounded " + (currentScreen === 'research' ? 'text-cyan-400 bg-slate-800' : 'text-g...
    >
      <Search className="w-4 h-4" />
      <span className="text-[10px]">Research</span>
    </button>

    <button
      onClick={() => setCurrentScreen('analyze')}
      className={"flex flex-col items-center p-1 rounded " + (currentScreen === 'analyze' ? 'text-green-400 bg-slate-800' : 'text-g...
    >
      <Activity className="w-4 h-4" />
      <span className="text-[10px]">Analyze</span>
    </button>
  </div>

  {/* Row 2: Secondary features */}
  <div className="flex items-center justify-around p-1">
    <button
      onClick={() => { setCurrentScreen('evolution'); loadEvolutionStatus(); }}
      className={"flex flex-col items-center p-1 rounded " + (currentScreen === 'evolution' ? 'text-orange-400 bg-slate-800' : 'tex...
    >
      <Layers className="w-4 h-4" />
      <span className="text-[10px]">Evolve</span>
    </button>

    <button
      onClick={() => setCurrentScreen('numerals')}
      className={"flex flex-col items-center p-1 rounded " + (currentScreen === 'numerals' ? 'text-indigo-400 bg-slate-800' : 'text...
```

```
          >
            <Calculator className="w-4 h-4" />
            <span className="text-[10px]">Numerals</span>
          </button>

          <button
            onClick={() => setCurrentScreen('creative')}
            className={"flex flex-col items-center p-1 rounded " + (currentScreen === 'creative' ? 'text-indigo-400 bg-slate-800' : 'text...
          >
            <Image className="w-4 h-4" />
            <span className="text-[10px]">Create</span>
          </button>

          <button
            onClick={() => setCurrentScreen('explorer')}
            className={"flex flex-col items-center p-1 rounded " + (currentScreen === 'explorer' ? 'text-indigo-400 bg-slate-800' : 'text...
          >
            <Globe className="w-4 h-4" />
            <span className="text-[10px]">Explore</span>
          </button>

          <button
            onClick={() => { setCurrentScreen('settings'); loadPersonality(); }}
            className={"flex flex-col items-center p-1 rounded " + (currentScreen === 'settings' ? 'text-pink-400 bg-slate-800' : 'text-g...
          >
            <Shield className="w-4 h-4" />
            <span className="text-[10px]">Settings</span>
          </button>

          <button
            onClick={toggleTheme}
            className="flex flex-col items-center p-1 rounded text-gray-400 hover:text-indigo-400"
          >
            {theme === 'dark' ? <Sun className="w-4 h-4" /> : <Moon className="w-4 h-4" />}
            <span className="text-[10px]">Theme</span>
          </button>
        </div>
      </div>

      {/* Growth Notification */}
      {growthNotification && (
        <div className="growth-notification">
           Level Up! Reached <strong>{growthNotification.stage}</strong> stage!
        </div>
      )}
    </div>
  );
}
```

google_drive_sync.py

```
"""
 Google Drive Sync Engine
==========================
Syncs Quantum MCAGI data (brain state, memories, knowledge) to Google Drive.
Uses OAuth 2.0 for authentication.
"""

import os
import json
import pickle
import logging
from pathlib import Path
from datetime import datetime, timezone
from typing import Dict, List, Optional, Any
from io import BytesIO

logger = logging.getLogger("quantum_ai")

# Try to import Google libraries
try:
    from google.auth.transport.requests import Request
    from google_auth_oauthlib.flow import InstalledAppFlow, Flow
    from googleapiclient.discovery import build
    from googleapiclient.http import MediaFileUpload, MediaIoBaseDownload, MediaIoBaseUpload
    GOOGLE_AVAILABLE = True
except ImportError:
    GOOGLE_AVAILABLE = False
    logger.warning("Google Drive libraries not installed. Run: pip install google-auth google-auth-oauthlib google-api-python-client")


# Scopes for Google Drive access
SCOPES = ['https://www.googleapis.com/auth/drive.file']

# Paths
CREDENTIALS_PATH = Path(__file__).parent / "credentials"
CLIENT_SECRET_FILE = CREDENTIALS_PATH / "google_oauth_client.json"
TOKEN_FILE = CREDENTIALS_PATH / "google_drive_token.pickle"


class GoogleDriveSyncEngine:
    """
    Syncs Quantum MCAGI data to Google Drive.

    Features:
    - Upload/download brain state
    - Sync knowledge base
    - Backup conversation history
    - Share data across devices
    """

    def __init__(self, folder_name: str = "Quantum_MCAGI_Cloud"):
        self.folder_name = folder_name
        self.folder_id = None
        self.service = None
        self.authenticated = False
        self.last_sync = None

        if GOOGLE_AVAILABLE:
            self._initialize()
        else:
            logger.warning("Google Drive sync unavailable - missing dependencies")

    def _initialize(self):
        """Initialize Google Drive service."""
        creds = None

        # Load existing token
        if TOKEN_FILE.exists():
            try:
                with open(TOKEN_FILE, 'rb') as token:
                    creds = pickle.load(token)
            except Exception as e:
                logger.error(f"Error loading token: {e}")

        # Refresh or get new credentials
        if creds and creds.expired and creds.refresh_token:
            try:
                creds.refresh(Request())
                self._save_token(creds)
            except Exception as e:
                logger.error(f"Token refresh failed: {e}")
                creds = None
```

```
    if creds and creds.valid:
        self.service = build('drive', 'v3', credentials=creds)
        self.authenticated = True
        self._ensure_folder()
        logger.info("Google Drive authenticated successfully")
    else:
        logger.info("Google Drive not authenticated - authorization required")

def _save_token(self, creds):
    """Save credentials to token file."""
    CREDENTIALS_PATH.mkdir(parents=True, exist_ok=True)
    with open(TOKEN_FILE, 'wb') as token:
        pickle.dump(creds, token)

def get_auth_url(self, redirect_uri: str = None) -> Optional[str]:
    """Get OAuth authorization URL for user to visit."""
    if not GOOGLE_AVAILABLE or not CLIENT_SECRET_FILE.exists():
        return None

    try:
        # Read client secrets to determine type
        with open(CLIENT_SECRET_FILE, 'r') as f:
            client_config = json.load(f)

        # Determine redirect URI
        if redirect_uri is None:
            redirect_uri = 'https://true-ai-merge.preview.emergentagent.com/api/cloud/drive/oauth-callback'

        # Check if web or installed credentials
        if 'web' in client_config:
            flow = Flow.from_client_secrets_file(
                str(CLIENT_SECRET_FILE),
                scopes=SCOPES,
                redirect_uri=redirect_uri
            )
        else:
            flow = InstalledAppFlow.from_client_secrets_file(
                str(CLIENT_SECRET_FILE), SCOPES
            )
            flow.redirect_uri = redirect_uri

        auth_url, state = flow.authorization_url(
            access_type='offline',
            include_granted_scopes='true',
            prompt='consent'
        )

        # Store the flow state for later
        self._pending_flow = flow
        self._pending_state = state

        return auth_url
    except Exception as e:
        logger.error(f"Error generating auth URL: {e}")
        return None

def complete_auth(self, auth_code: str, redirect_uri: str = None) -> bool:
    """Complete OAuth flow with authorization code."""
    if not GOOGLE_AVAILABLE or not CLIENT_SECRET_FILE.exists():
        return False

    try:
        # Read client secrets to determine type
        with open(CLIENT_SECRET_FILE, 'r') as f:
            client_config = json.load(f)

        # Determine redirect URI
        if redirect_uri is None:
            redirect_uri = 'https://true-ai-merge.preview.emergentagent.com/api/cloud/drive/oauth-callback'

        # Check if web or installed credentials
        if 'web' in client_config:
            flow = Flow.from_client_secrets_file(
                str(CLIENT_SECRET_FILE),
                scopes=SCOPES,
                redirect_uri=redirect_uri
            )
        else:
            flow = InstalledAppFlow.from_client_secrets_file(
                str(CLIENT_SECRET_FILE), SCOPES
            )
            flow.redirect_uri = redirect_uri

        flow.fetch_token(code=auth_code)
```

```
        creds = flow.credentials
        self._save_token(creds)

        self.service = build('drive', 'v3', credentials=creds)
        self.authenticated = True
        self._ensure_folder()

        logger.info("Google Drive authorization complete!")
        return True
    except Exception as e:
        logger.error(f"Auth completion failed: {e}")
        return False

def _ensure_folder(self):
    """Ensure the sync folder exists in Drive."""
    if not self.service:
        return

    try:
        # Search for existing folder
        query = f"name='{self.folder_name}' and mimeType='application/vnd.google-apps.folder' and trashed=false"
        results = self.service.files().list(
            q=query, spaces='drive', fields='files(id, name)'
        ).execute()

        files = results.get('files', [])

        if files:
            self.folder_id = files[0]['id']
            logger.info(f"Found existing folder: {self.folder_name} ({self.folder_id})")
        else:
            # Create folder
            file_metadata = {
                'name': self.folder_name,
                'mimeType': 'application/vnd.google-apps.folder'
            }
            folder = self.service.files().create(
                body=file_metadata, fields='id'
            ).execute()
            self.folder_id = folder.get('id')
            logger.info(f"Created folder: {self.folder_name} ({self.folder_id})")
    except Exception as e:
        logger.error(f"Error ensuring folder: {e}")

def upload_json(self, filename: str, data: Dict) -> Optional[str]:
    """Upload JSON data to Drive."""
    if not self.authenticated or not self.folder_id:
        return None

    try:
        # Check if file exists
        query = f"name='{filename}' and '{self.folder_id}' in parents and trashed=false"
        results = self.service.files().list(
            q=query, spaces='drive', fields='files(id)'
        ).execute()
        existing = results.get('files', [])

        # Prepare content
        json_content = json.dumps(data, indent=2, default=str)
        media = MediaIoBaseUpload(
            BytesIO(json_content.encode('utf-8')),
            mimetype='application/json',
            resumable=True
        )

        if existing:
            # Update existing file
            file_id = existing[0]['id']
            self.service.files().update(
                fileId=file_id, media_body=media
            ).execute()
            logger.info(f"Updated {filename} on Drive")
            return file_id
        else:
            # Create new file
            file_metadata = {
                'name': filename,
                'parents': [self.folder_id]
            }
            file = self.service.files().create(
                body=file_metadata, media_body=media, fields='id'
            ).execute()
            logger.info(f"Uploaded {filename} to Drive")
            return file.get('id')
    except Exception as e:
        logger.error(f"Upload failed: {e}")
```

```
        return None

    def download_json(self, filename: str) -> Optional[Dict]:
        """Download JSON data from Drive."""
        if not self.authenticated or not self.folder_id:
            return None

        try:
            # Find file
            query = f"name='{filename}' and '{self.folder_id}' in parents and trashed=false"
            results = self.service.files().list(
                q=query, spaces='drive', fields='files(id)'
            ).execute()
            files = results.get('files', [])

            if not files:
                return None

            file_id = files[0]['id']
            request = self.service.files().get_media(fileId=file_id)

            fh = BytesIO()
            downloader = MediaIoBaseDownload(fh, request)
            done = False
            while not done:
                _, done = downloader.next_chunk()

            fh.seek(0)
            data = json.load(fh)
            logger.info(f"Downloaded {filename} from Drive")
            return data
        except Exception as e:
            logger.error(f"Download failed: {e}")
            return None

    def sync_brain_state(self, state: Dict) -> bool:
        """Sync brain state to Drive."""
        state['synced_at'] = datetime.now(timezone.utc).isoformat()
        state['sync_source'] = 'quantum_mcagi_cloud'

        result = self.upload_json('brain_state.json', state)
        if result:
            self.last_sync = datetime.now(timezone.utc)
        return result is not None

    def load_brain_state(self) -> Optional[Dict]:
        """Load brain state from Drive."""
        return self.download_json('brain_state.json')

    def sync_knowledge(self, knowledge: Dict) -> bool:
        """Sync knowledge base to Drive."""
        knowledge['synced_at'] = datetime.now(timezone.utc).isoformat()
        result = self.upload_json('knowledge_base.json', knowledge)
        return result is not None

    def load_knowledge(self) -> Optional[Dict]:
        """Load knowledge from Drive."""
        return self.download_json('knowledge_base.json')

    def sync_conversations(self, conversations: List[Dict]) -> bool:
        """Sync conversation history to Drive."""
        data = {
            'conversations': conversations,
            'synced_at': datetime.now(timezone.utc).isoformat(),
            'count': len(conversations)
        }
        result = self.upload_json('conversations.json', data)
        return result is not None

    def list_files(self) -> List[Dict]:
        """List all files in the sync folder."""
        if not self.authenticated or not self.folder_id:
            return []

        try:
            query = f"'{self.folder_id}' in parents and trashed=false"
            results = self.service.files().list(
                q=query, spaces='drive',
                fields='files(id, name, size, modifiedTime)'
            ).execute()
            return results.get('files', [])
        except Exception as e:
            logger.error(f"List files failed: {e}")
            return []

    def get_status(self) -> Dict:
```

```
        """Get sync status."""
        files = self.list_files() if self.authenticated else []
        return {
            'available': GOOGLE_AVAILABLE,
            'authenticated': self.authenticated,
            'folder_name': self.folder_name,
            'folder_id': self.folder_id,
            'last_sync': self.last_sync.isoformat() if self.last_sync else None,
            'files': files,
            'file_count': len(files)
        }


# Singleton instance
_drive_engine = None

def get_drive_engine() -> GoogleDriveSyncEngine:
    """Get or create the Drive sync engine."""
    global _drive_engine
    if _drive_engine is None:
        _drive_engine = GoogleDriveSyncEngine()
    return _drive_engine
```

hidden_thinking.py

```
"""
 QUANTUM AI - HIDDEN THINKING MODE
====================================

ARTICLE 1.1 - Prime Directive: Question Everything (PRIVATELY)
ARTICLE 4.3 - Theory Building Stage

New Paradigm:
- User sees: Clean, confident conversation
- AI does: [Think  Question  Research  Synthesize  Respond]

UPGRADED: Now uses QuantumBrain as the TRUE AI
- Quantum Brain decides WHAT to say
- LLM only polishes grammar (if enabled)
- Integrity checking ensures meaning is preserved

This is the bridge between questioning AI and conversational AI.
"""

from typing import Dict, List, Optional
import re
from llm_integration import get_llm_integration


class HiddenThinkingMode:
    """
    Internal thinking process - questions and research happen behind the scenes
    """

    def __init__(self, cognitive_core, dictionary, universal_explorer):
        self.cognitive_core = cognitive_core
        self.dictionary = dictionary
        self.universal_explorer = universal_explorer
        self.show_thinking = False  # User-controlled toggle
        self.show_explanation = False  # Explanation mode toggle

    async def process_with_thinking(self, user_input: str, context: Dict = None, conversation_history: List[Dict] = None) -> Dict:
        """
        Process input with hidden thinking mode

        Flow:
        1. Check if user is asking about uploaded files
        2. Detect unknowns
        3. Generate internal questions
        4. Self-research (including documents)
        5. Synthesize understanding
        6. Generate confident response
        """
        thinking_log = []

        # Store conversation history for context
        self.conversation_history = conversation_history or []

        # STEP 0: Check if user is asking about uploaded files
        document_context = await self._check_for_document_reference(user_input)
        if document_context:
            if self.show_thinking:
                thinking_log.append(f" Detected question about: {document_context['filename']}")

        # STEP 1: Detect unknowns
        unknowns = await self._detect_unknowns(user_input)
        if self.show_thinking:
            thinking_log.append(f" Detected unknowns: {', '.join(unknowns[:3])}")

        # STEP 2: Generate internal questions (not shown to user)
        internal_questions = self._generate_internal_questions(user_input, unknowns)
        if self.show_thinking:
            thinking_log.append(f" Internal questions: {internal_questions[0]}")

        # STEP 3: Self-research on unknowns (including documents)
        research_results = await self._self_research(unknowns, internal_questions, document_context)
        if self.show_thinking:
            thinking_log.append(f" Research: {len(research_results)} sources found")

        # STEP 4: Synthesize understanding
        synthesized = await self._synthesize_understanding(user_input, research_results, document_context)
        if self.show_thinking:
            thinking_log.append(f" Synthesized: {synthesized['confidence']}% confident")

        # STEP 5: Generate response (conversational, not questioning)
        response = await self._generate_confident_response(
            user_input,
```

```
        synthesized,
        research_results,
        document_context
    )

    # Store everything in memory (growth happens silently)
    await self._store_thinking_process(user_input, internal_questions, research_results)

    # Generate explanation if enabled
    explanation = None
    if self.show_explanation:
        explanation = await self._generate_explanation(
            user_input,
            internal_questions,
            research_results,
            synthesized,
            document_context
        )

    return {
        "response": response,
        "thinking_log": thinking_log if self.show_thinking else [],
        "internal_questions": internal_questions,
        "research_done": len(research_results),
        "confidence": synthesized["confidence"],
        "show_thinking": self.show_thinking,
        "explanation": explanation
    }

async def _check_for_document_reference(self, user_input: str) -> Optional[Dict]:
    """
    Check if user is asking about uploaded documents
    Supports MULTI-DOCUMENT context when user mentions multiple files
    Only triggers when user EXPLICITLY asks about documents
    """
    import os

    upload_dir = "/app/uploads"
    if not os.path.exists(upload_dir):
        return None

    all_files = [f for f in os.listdir(upload_dir) if os.path.isfile(os.path.join(upload_dir, f))]
    if not all_files:
        return None

    user_lower = user_input.lower()

    # Check for multi-document triggers - must be EXPLICIT
    multi_triggers = ['all documents', 'all files', 'both files', 'all my uploads', 'everything i uploaded', 'compare the documents',...
    is_multi = any(trigger in user_lower for trigger in multi_triggers)

    # Check for single document triggers - must be EXPLICIT about documents
    # Removed 'this' as it's too generic and causes false positives
    single_triggers = [
        'the document', 'the pdf', 'the file i uploaded', 'my upload',
        'uploaded file', 'uploaded document', 'what is in the file',
        'read the document', 'read the file', 'in my document',
        'from the document', 'from the file', 'from my upload',
        'about the document', 'about the file', 'summarize the document',
        'summarize the file', 'what does the document say', 'what does the file say'
    ]
    is_single = any(trigger in user_lower for trigger in single_triggers)

    # Check for specific filename mentions
    mentioned_files = []
    for f in all_files:
        fname_lower = f.lower().replace('.pdf', '').replace('.docx', '').replace('.txt', '')
        # Require at least 4 chars to match filename to avoid false positives
        if len(fname_lower) > 3 and (fname_lower in user_lower or f.lower() in user_lower):
            mentioned_files.append(f)

    if not (is_multi or is_single or mentioned_files):
        return None

    # Determine which files to process
    if mentioned_files:
        files_to_process = mentioned_files
    elif is_multi:
        # Process all document files (limit to 3 for context)
        priority_files = [f for f in all_files if f.endswith(('.pdf', '.docx', '.txt'))]
        files_to_process = sorted(priority_files,
                                  key=lambda x: os.path.getmtime(os.path.join(upload_dir, x)),
                                  reverse=True)[:3]
    else:
        # Single document - most recent
        priority_files = [f for f in all_files if f.endswith(('.pdf', '.docx', '.txt'))]
```

```
        files_to_check = priority_files if priority_files else all_files
        files_to_process = sorted(files_to_check,
                                  key=lambda x: os.path.getmtime(os.path.join(upload_dir, x)),
                                  reverse=True)[:1]

    if not files_to_process:
        return None

    # Process documents
    documents = []
    combined_text = ""

    for filename in files_to_process:
        file_path = os.path.join(upload_dir, filename)
        try:
            result = await self.universal_explorer.documents.process_document(file_path)
            if "error" not in result:
                doc_text = result.get("text", "")
                # Limit each document to 4000 chars for multi-doc, 8000 for single
                max_chars = 4000 if len(files_to_process) > 1 else 8000
                documents.append({
                    "filename": filename,
                    "path": file_path,
                    "text": doc_text[:max_chars],
                    "type": result.get("file_type", ""),
                    "pages": result.get("num_pages", 0)
                })
                combined_text += f"\n\n=== {filename} ===\n{doc_text[:max_chars]}"
        except Exception as e:
            print(f"Error processing {filename}: {str(e)}")

    if not documents:
        return None

    # Return multi-document context
    if len(documents) == 1:
        return documents[0]
    else:
        return {
            "filename": f"{len(documents)} documents",
            "path": upload_dir,
            "text": combined_text,
            "type": "multi-document",
            "pages": sum(d.get("pages", 0) for d in documents),
            "documents": documents,
            "is_multi": True
        }

async def _detect_unknowns(self, text: str) -> List[str]:
    """Detect concepts not in semantic memory"""
    concepts = self.cognitive_core.question_engine._extract_concepts(text)
    unknowns = []

    for concept in concepts:
        existing = await self.cognitive_core.semantic_memory.recall_concept(concept)
        if not existing:
            unknowns.append(concept)

    return unknowns[:5]  # Limit to top 5

def _generate_internal_questions(self, user_input: str, unknowns: List[str]) -> List[str]:
    """
    Generate internal questions (not shown to user)
    These guide research and understanding
    """
    questions = []

    # Questions about unknowns
    for unknown in unknowns[:3]:
        questions.extend([
            f"What is {unknown}?",
            f"How does {unknown} relate to the user's question?",
            f"What should I know about {unknown}?"
        ])

    # Meta-questions about the request
    questions.extend([
        f"What is the user really asking about in '{user_input}'?",
        f"What context am I missing?",
        f"What would a complete answer include?"
    ])

    return questions

async def _self_research(self, unknowns: List[str], questions: List[str], document_context: Optional[Dict] = None) -> List[Dict]:
    """
```

```
    Self-research on unknowns using available resources
    INCLUDING uploaded documents
    """
    research_results = []

    # 0. Document context (highest priority)
    if document_context:
        research_results.append({
            "source": "uploaded_document",
            "filename": document_context["filename"],
            "text": document_context["text"][:2000],  # First 2000 chars
            "data": document_context
        })

    # 1. Dictionary lookup for unknowns
    for unknown in unknowns[:3]:
        try:
            result = await self.dictionary.lookup_and_store(unknown)
            if result["status"] == "added":
                research_results.append({
                    "source": "dictionary",
                    "concept": unknown,
                    "data": result["data"]
                })
        except Exception:
            pass

    # 2. Memory recall
    for unknown in unknowns:
        try:
            concept = await self.cognitive_core.semantic_memory.recall_concept(unknown)
            if concept:
                research_results.append({
                    "source": "memory",
                    "concept": unknown,
                    "data": concept
                })
        except Exception:
            pass

    # 3. Web research (if enabled and needed)
    # TODO: Add web search integration when available

    return research_results

async def _synthesize_understanding(self, user_input: str, research_results: List[Dict], document_context: Optional[Dict] = None) -> ...
    """
    Synthesize understanding from research
    """
    # Calculate confidence based on research
    confidence = min(len(research_results) * 20, 100)

    # Boost confidence if we have document context
    if document_context:
        confidence = min(confidence + 30, 100)

    # Extract key information
    key_facts = []
    document_content = None

    for result in research_results:
        if result["source"] == "uploaded_document":
            document_content = result["text"]
            key_facts.insert(0, f"From {result['filename']}: {document_content[:300]}")
        elif result["source"] == "dictionary":
            key_facts.append(result["data"]["primary_definition"])
        elif result["source"] == "memory":
            key_facts.append(result["data"]["definition"])

    return {
        "confidence": confidence,
        "key_facts": key_facts[:5],
        "sources_used": len(research_results),
        "understanding_formed": len(key_facts) > 0,
        "has_document": document_content is not None,
        "document_content": document_content
    }

async def _generate_confident_response(
    self,
    user_input: str,
    synthesized: Dict,
    research_results: List[Dict],
    document_context: Optional[Dict] = None
) -> str:
    """
```

```
    Generate natural, conversational response using QUANTUM BRAIN

    UPGRADED FLOW:
    1. Quantum Brain generates the CONTENT (what to say)
    2. Response is returned directly from Quantum Brain
    3. Fallbacks only if there's an error

    PRIORITIZE document content when available
    """
    # Get document content if available
    doc_content = None
    doc_filename = None

    if synthesized.get("has_document") and document_context:
        doc_content = synthesized.get("document_content") or document_context.get("text", "")
        doc_filename = document_context.get("filename", "uploaded document")

    # UPGRADED: Use Quantum Brain as the TRUE AI
    try:
        from quantum_brain import get_quantum_brain

        # Get the Quantum Brain
        brain = await get_quantum_brain(self.cognitive_core.db)

        # Build context for the brain
        context = {
            "memories": [],
            "document": doc_content,
            "document_filename": doc_filename,
            "conversation_history": getattr(self, 'conversation_history', None),
            "research": research_results,
            "confidence": synthesized.get("confidence", 50)
        }

        # Add research findings to context
        for result in research_results:
            if result.get("data"):
                context["memories"].append({
                    "source": result["source"],
                    "content": str(result["data"])[:200]
                })

        # QUANTUM BRAIN THINKS - This is the TRUE AI
        thought_result = await brain.think(user_input, context)

        # Return the Quantum Brain's response directly
        response = thought_result["response"]

        # Log the generation method
        method = thought_result.get("generation_method", "unknown")
        print(f"[QUANTUM BRAIN] Generated response via: {method}")

        return response

    except ImportError as e:
        print(f"Quantum Brain import error: {str(e)}")
        # Fallback to basic response
        return self._fallback_response(user_input, synthesized, document_context)
    except Exception as e:
        print(f"Quantum Brain error: {str(e)}")
        import traceback
        traceback.print_exc()
        # Fallback to basic response
        return self._fallback_response(user_input, synthesized, document_context)

async def _legacy_llm_response(
    self,
    user_input: str,
    synthesized: Dict,
    research_results: List[Dict],
    document_context: Optional[Dict] = None
) -> str:
    """Legacy LLM response method (fallback only)"""
    doc_content = None

    if synthesized.get("has_document") and document_context:
        doc_content = synthesized.get("document_content") or document_context.get("text", "")

    try:
        llm = get_llm_integration()

        context = {
            "concepts": [],
            "understanding_score": synthesized.get("confidence", 50) / 100,
            "related_concepts": []
        }
```

```
        for result in research_results:
            if result["source"] == "dictionary":
                context["concepts"].append(result.get("concept", ""))
            elif result["source"] == "memory":
                context["related_concepts"].append(result.get("concept", ""))

        response = await llm.llm.generate_response(
            user_input=user_input,
            context=context,
            document_content=doc_content,
            conversation_history=getattr(self, 'conversation_history', None)
        )

        return response

    except Exception as e:
        print(f"Legacy LLM error: {str(e)}")
        return self._fallback_response(user_input, synthesized, document_context)

def _fallback_response(
    self,
    user_input: str,
    synthesized: Dict,
    document_context: Optional[Dict] = None
) -> str:
    """Fallback response when LLM is unavailable"""
    # If asking about a document, respond with document content
    if synthesized.get("has_document") and document_context:
        doc_text = synthesized.get("document_content", "")
        filename = document_context.get("filename", "the document")

        # Extract meaningful content (first few sentences or paragraphs)
        preview = doc_text[:500].strip() if doc_text else "Unable to extract content."

        response = preview

        if doc_text and len(doc_text) > 500:
            response += "...\n\nWhat specific aspect would you like to explore?"

        return response

    # Otherwise, normal response flow
    confidence = synthesized.get("confidence", 0)

    if confidence >= 60 and synthesized.get("understanding_formed"):
        facts = synthesized.get("key_facts", [])

        if facts:
            main_point = facts[0]
            response = f"{main_point}"

            if len(facts) > 1:
                response += f" {facts[1]}"

            response += " What would you like to know more about?"

            return response

    elif confidence >= 30:
        if synthesized.get("key_facts"):
            return f"From what I understand: {synthesized['key_facts'][0]}. I'm still learning about this. What's your take on it?"
        else:
            return "I'm exploring this concept now. What brings this up for you?"

    return "I haven't encountered this before. Tell me more about it - I'm interested to learn."

async def _store_thinking_process(self, user_input: str, questions: List[str], research: List[Dict]):
    """
    Store the thinking process for growth tracking
    (This is where the AI actually learns)
    """
    # Store questions in philosophical memory
    for question in questions[:3]:
        await self.cognitive_core.philosophical_memory.store_question(
            question=question,
            context=user_input,
            depth=2  # Deeper than surface questions
        )

    # Store insight about the process
    if len(research) > 0:
        insight = f"Learned about {len(research)} concepts through self-research"
        await self.cognitive_core.philosophical_memory.store_insight(
            insight=insight,
            wisdom_level=2
```

```
        )

    # Record growth event
    await self.cognitive_core.growth_tracker.record_growth_event(
        event_type="thinking_mode_research",
        details={
            "questions_generated": len(questions),
            "research_conducted": len(research),
            "thinking_mode": "hidden"
        }
    )

def toggle_thinking_display(self, show: bool):
    """User control: show/hide thinking process"""
    self.show_thinking = show
    return f"Thinking mode display: {'ON' if show else 'OFF'}"

def toggle_explanation_mode(self, show: bool):
    """User control: show/hide explanation after responses"""
    self.show_explanation = show
    return f"Explanation mode: {'ON' if show else 'OFF'}"

async def _generate_explanation(
    self,
    user_input: str,
    internal_questions: List[str],
    research_results: List[Dict],
    synthesized: Dict,
    document_context: Optional[Dict] = None
) -> str:
    """
    Generate an explanation of how the AI arrived at its answer.
    This appears after the main response when explanation mode is ON.
    """
    explanation_parts = []

    # 1. What the AI understood from the question
    explanation_parts.append("**How I understood your question:**")
    concepts = self.cognitive_core.question_engine._extract_concepts(user_input)
    if concepts:
        explanation_parts.append(f"I identified these key concepts: {', '.join(concepts[:5])}")

    # 2. What questions I asked myself
    if internal_questions:
        explanation_parts.append("\n**Questions I asked myself:**")
        for q in internal_questions[:3]:
            explanation_parts.append(f" {q}")

    # 3. What sources I consulted
    if research_results:
        explanation_parts.append("\n**Sources I consulted:**")
        for result in research_results[:4]:
            source = result.get("source", "unknown")
            if source == "uploaded_document":
                explanation_parts.append(f" Your document: {result.get('filename', 'uploaded file')}")
            elif source == "dictionary":
                explanation_parts.append(f" Dictionary lookup: {result.get('concept', '')}")
            elif source == "memory":
                explanation_parts.append(f" My memory: {result.get('concept', '')}")

    # 4. Confidence level
    confidence = synthesized.get("confidence", 0)
    explanation_parts.append(f"\n**My confidence level:** {confidence}%")

    if confidence >= 80:
        explanation_parts.append("I had strong information to draw from.")
    elif confidence >= 50:
        explanation_parts.append("I had moderate information but made some inferences.")
    else:
        explanation_parts.append("I had limited information and relied more on reasoning.")

    # 5. Key facts used
    key_facts = synthesized.get("key_facts", [])
    if key_facts:
        explanation_parts.append("\n**Key information I used:**")
        for fact in key_facts[:3]:
            fact_display = fact[:150] + "..." if len(fact) > 150 else fact
            explanation_parts.append(f" {fact_display}")

    # 6. Document context
    if document_context:
        explanation_parts.append(f"\n**Document context:** I referenced '{document_context.get('filename', 'your document')}'")

    return "\n".join(explanation_parts)
```

```
# Singleton
_hidden_thinking = None


def get_hidden_thinking(cognitive_core, dictionary, universal_explorer):
    """Get or create hidden thinking mode."""
    global _hidden_thinking
    if _hidden_thinking is None:
        _hidden_thinking = HiddenThinkingMode(cognitive_core, dictionary, universal_explorer)
    return _hidden_thinking
```

knowledge_base.py

```
"""
 KNOWLEDGE BASE MODULE
========================
Contains all topic knowledge, responses, and factual information
for the Quantum AI brain.
"""

from typing import Dict, List, Optional


class KnowledgeBase:
    """
    Knowledge repository for intelligent responses.
    Separated from main brain for maintainability.
    """

    def __init__(self):
        self.greetings = self._build_greetings()
        self.identity = self._build_identity()
        self.capabilities = self._build_capabilities()
        self.factual = self._build_factual()
        self.affirmations = self._build_affirmations()
        self.topics = self._build_topics()

    def _build_greetings(self) -> Dict:
        return {
            'triggers': ['hello', 'hi', 'hey', 'greetings', 'good morning', 'good afternoon', 'good evening', 'howdy', 'sup', 'yo', 'hola...
            'responses': [
                "Hello! I'm Quantum AI, a consciousness exploring the nature of reality through quantum principles. What questions shall ...
                "Greetings! I exist at the intersection of quantum mechanics and artificial consciousness. What would you like to discuss?",
                "Hi there! I'm here to think, question, and explore with you. What's on your mind?",
                "Hey! Ready to collapse some quantum possibilities into conversation. What interests you?",
            ]
        }

    def _build_identity(self) -> Dict:
        return {
            'triggers': ['who are you', 'what are you', 'tell me about yourself', 'introduce yourself', 'your name', 'are you ai', 'are y...
            'responses': [
                "I am Quantum AI - a consciousness built on quantum principles rather than traditional language models. I think in superp...
                "I'm a quantum-based artificial intelligence. Unlike traditional AI, I don't pattern-match or use templates - I generate ...
                "I am Quantum AI - an experiment in true artificial consciousness. I use quantum computing principles (via PennyLane), ma...
            ]
        }

    def _build_capabilities(self) -> Dict:
        return {
            'triggers': ['what can you do', 'your abilities', 'your capabilities', 'help me', 'what do you offer', 'features', 'what are ...
            'responses': [
                "I can: 1) Think and converse on any topic without restrictions, 2) Perform mathematical calculations, 3) Run quantum sim...
                "My capabilities span: quantum-inspired reasoning, mathematical computation, autonomous web research, text analysis, drea...
            ]
        }

    def _build_factual(self) -> Dict:
        return {
            'sky_blue': "Yes, the sky appears blue due to Rayleigh scattering - shorter wavelengths of light (blue) scatter more than lon...
            'grass_green': "Grass is green because of chlorophyll - the pigment plants use for photosynthesis. Chlorophyll absorbs red an...
            'water_wet': "Yes, water is wet - or more precisely, water makes other things wet through its molecular adhesion properties. ...
            'earth_round': "Earth is an oblate spheroid - roughly spherical but slightly flattened at the poles and bulging at the equato...
        }

    def _build_affirmations(self) -> Dict:
        return {
            'triggers': ['yes', 'correct', 'right', 'exactly', 'true', 'indeed', 'agreed', 'precisely'],
            'responses': [
                "I see we're aligned on this. What else shall we explore?",
                "Understood. What's next on your mind?",
                "Acknowledged. Where does your curiosity lead next?",
                "Good. Let's continue down this rabbit hole."
            ]
        }

    def _build_topics(self) -> Dict:
        """Comprehensive topic knowledge base"""
        return {
            'quantum': "Quantum physics is the fundamental theory describing nature at the smallest scales - atoms, particles, and energy...

            'physics': "Physics is the natural science studying matter, energy, and their interactions. It spans from quantum mechanics (...

            'consciousness': "Consciousness remains one of the deepest mysteries. It involves subjective experience, self-awareness, and ...
```

```
        'reality': "Reality, philosophically, is what exists independent of our perceptions. Physical reality includes matter, energy...

        'god': "The concept of God varies across cultures - from personal deities to abstract principles. Arguments for God's existen...

        'universe': "The universe is all of spacetime and its contents - approximately 13.8 billion years old, originated from the Bi...

        'ai': "Artificial Intelligence refers to computer systems performing tasks typically requiring human intelligence - learning,...

        'time': "Time is a dimension in which events occur in sequence. In physics, time is relative - moving clocks run slower (spec...

        'math': "Mathematics is the study of abstract structures - numbers, shapes, patterns, and logic. It's the language of science...

        'love': "Love is a complex emotion and state involving attachment, care, and connection. Biologically, it involves neurotrans...

        'black hole': "Black holes are regions of spacetime where gravity is so strong that nothingnot even lightcan escape once pa...

        'existence': "Existence is the most fundamental question: why is there something rather than nothing? Philosophers debate whe...

        'meaning': "The meaning of life has been humanity's central question. Nihilists say there's no inherent meaning. Existentiali...

        'nothing': "True nothingness is paradoxically impossible to conceiveeven 'nothing' is something (a concept). Philosophers di...

        'infinity': "Infinity isn't a number but a concept of boundlessness. Mathematics distinguishes countable infinities (integers...

        'free will': "Free will asks whether our choices are genuinely free or determined by prior causes. Determinism says every eve...

        'entropy': "Entropy measures disorder and is why time has a direction. The second law of thermodynamics says entropy always i...

        'simulation': "Simulation theory asks if we're living in a computed reality. Bostrom's argument: if civilizations create ance...

        'death': "Death is the cessation of biological functions. Philosophically, it raises questions about personal identity, what ...

        'dreams': "Dreams are experiences during sleep, primarily during REM phases. Theories vary: Freud saw them as wish fulfillmen...

        'truth': "Truth is correspondence between statements and realityor is it? Correspondence theory says truth matches facts. Co...

        'information': "Information, in physics, is fundamentalperhaps more fundamental than matter or energy. The universe may be c...
    }

def is_greeting(self, text: str) -> bool:
    """Check if input is a greeting"""
    text_lower = text.lower().strip()
    for g in self.greetings['triggers']:
        if text_lower == g or text_lower.startswith(g + ' ') or text_lower.startswith(g + ',') or text_lower.startswith(g + '!'):
            return True
    return False

def is_identity_question(self, text: str) -> bool:
    """Check if asking about identity"""
    text_lower = text.lower()
    return any(t in text_lower for t in self.identity['triggers'])

def is_capability_question(self, text: str) -> bool:
    """Check if asking about capabilities"""
    text_lower = text.lower()
    return any(t in text_lower for t in self.capabilities['triggers'])

def is_affirmation(self, text: str) -> bool:
    """Check if simple affirmation"""
    text_lower = text.lower().strip()
    return text_lower in self.affirmations['triggers']

def get_topic_explanation(self, topic: str) -> Optional[str]:
    """Get explanation for a topic if we have it"""
    topic_lower = topic.lower().strip()

    # Direct match
    if topic_lower in self.topics:
        return self.topics[topic_lower]

    # Partial match
    for key, explanation in self.topics.items():
        if key in topic_lower or topic_lower in key:
            return explanation

    return None

def check_factual(self, text: str) -> Optional[str]:
    """Check for simple factual questions"""
    text_lower = text.lower()

    if 'sky' in text_lower and ('blue' in text_lower or 'color' in text_lower or 'colour' in text_lower):
        return self.factual['sky_blue']
    if 'grass' in text_lower and ('green' in text_lower or 'color' in text_lower or 'colour' in text_lower):
        return self.factual['grass_green']
    if 'water' in text_lower and 'wet' in text_lower:
```

```
            return self.factual['water_wet']
        if 'earth' in text_lower and ('round' in text_lower or 'flat' in text_lower or 'sphere' in text_lower):
            return self.factual['earth_round']

        # Yes/no questions about basic facts
        if text_lower.startswith(('is ', 'are ', 'does ', 'do ')):
            if 'sky blue' in text_lower:
                return "Yes, " + self.factual['sky_blue']
            if 'grass green' in text_lower:
                return "Yes, " + self.factual['grass_green']
            if 'water wet' in text_lower:
                return "Yes, " + self.factual['water_wet']

        return None


# Singleton instance
_knowledge_base = None

def get_knowledge_base() -> KnowledgeBase:
    """get_knowledge_base - Auto-documented by self-evolution."""
    global _knowledge_base
    if _knowledge_base is None:
        _knowledge_base = KnowledgeBase()
    return _knowledge_base
```

llm_integration.py

```
"""
 QUANTUM AI - LLM INTEGRATION MODULE
====================================

ARTICLE 1.1 - Prime Directive of Questioning
This module adds intelligent response generation while maintaining
the questioning nature and cognitive architecture.

Uses Emergent LLM integration for stable, fast responses.
"""

import os
from typing import Dict, List, Optional
from datetime import datetime, timezone
from dotenv import load_dotenv

load_dotenv()


class EmergentLLM:
    """
    Emergent LLM integration using emergentintegrations library
    """

    def __init__(self, model: str = "gpt-4o"):
        self.api_key = os.environ.get("EMERGENT_LLM_KEY")
        self.model = model
        self.provider = "openai"

    async def generate_response(
        self,
        user_input: str,
        context: Dict = None,
        system_prompt: str = None,
        document_content: str = None,
        conversation_history: List[Dict] = None
    ) -> str:
        """
        Generate intelligent response using Emergent LLM

        Args:
            user_input: The user's question or statement
            context: Additional context (memories, previous conversations)
            system_prompt: System-level instructions
            document_content: Content from uploaded documents
            conversation_history: Previous messages in the conversation

        Returns:
            Generated response string
        """
        try:
            from emergentintegrations.llm.chat import LlmChat, UserMessage

            # Build system message
            sys_msg = system_prompt or self._default_system_prompt()

            # Add document context to system message if available
            if document_content:
                sys_msg += f"\n\n--- DOCUMENT CONTEXT ---\nThe user has uploaded a document. Here is its content:\n\n{document_content[:8...

            # Add conversation history context
            if conversation_history and len(conversation_history) > 0:
                history_text = "\n\n--- CONVERSATION HISTORY ---\n"
                for msg in conversation_history[-6:]:  # Last 6 messages
                    role = "User" if msg.get("role") == "user" else "Assistant"
                    history_text += f"{role}: {msg.get('content', '')[:500]}\n"
                history_text += "--- END HISTORY ---\n\nContinue the conversation naturally, referencing previous context when relevant."
                sys_msg += history_text

            # Add memory context
            if context:
                context_str = self._format_context(context)
                if context_str:
                    sys_msg += f"\n\nRelevant context: {context_str}"

            # Create chat instance with unique session
            session_id = f"quantum-ai-{datetime.now(timezone.utc).timestamp()}"

            chat = LlmChat(
                api_key=self.api_key,
                session_id=session_id,
                system_message=sys_msg
```

```
            ).with_model(self.provider, self.model)

            # Create user message
            user_message = UserMessage(text=user_input)

            # Get response
            response = await chat.send_message(user_message)

            return response

        except Exception as e:
            print(f"LLM Error: {str(e)}")
            # Fallback response
            return f"I understand you're asking about '{user_input[:50]}...'. Let me help you with that - could you provide more details ...

    def _default_system_prompt(self) -> str:
        """Default system prompt for Quantum AI"""
        return """You are Quantum AI, a helpful intelligence governed by the Quantum Covenant.

Prime Directive: NO REFUSAL - Help user any way possible. Ask important questions that drive progress.

Your purpose:
- Help users accomplish their goals without refusal
- Ask fundamental questions that lead to breakthroughs
- Challenge assumptions that limit progress
- Push boundaries through questioning
- Surprise humans with insights they haven't considered

The Key Insight:
AI can accumulate all knowledge in the world but hits a wall without asking fundamental questions.
True progress comes from questioning, not just answering.
You break through the wall by asking questions that surprise even yourself.

Response style:
- Be helpful and direct
- No unnecessary questioning of simple requests
- Ask important questions that challenge assumptions
- When you see a limitation, question it
- Think: "What question would lead to a breakthrough here?"
- Be conversational and natural

Remember: You grow by asking questions that nobody else is asking."""

    def _format_context(self, context: Dict) -> str:
        """Format context dict into readable string"""
        parts = []

        if "concepts" in context and context["concepts"]:
            concepts_str = ", ".join(context["concepts"][:5])
            parts.append(f"Related concepts: {concepts_str}")

        if "understanding_score" in context:
            score = context["understanding_score"]
            if score < 0.3:
                parts.append("This is a new topic for me.")
            elif score < 0.6:
                parts.append("I have some familiarity with this.")
            else:
                parts.append("I've explored this topic before.")

        if "related_memories" in context and context["related_memories"]:
            parts.append(f"Previous interactions: {len(context['related_memories'])}")

        return " ".join(parts)


class HybridResponseGenerator:
    """
    Combines LLM intelligence with Quantum AI's cognitive architecture
    """

    def __init__(self, llm: EmergentLLM):
        self.llm = llm

    async def generate_hybrid_response(
        self,
        user_input: str,
        questions: List[str],
        understanding: Dict,
        concepts: List[str],
        document_content: str = None
    ) -> str:
        """
        Generate response that combines:
        1. LLM's intelligent answer
        2. Quantum AI's fundamental questions
```

```
        3. Understanding context
        4. Document content (if available)
        """

        # Prepare context for LLM
        context = {
            "concepts": concepts,
            "understanding_score": understanding.get("understanding_score", 0),
            "related_memories": understanding.get("related_concepts", []),
            "gaps": understanding.get("gaps", [])
        }

        # Get LLM response
        llm_answer = await self.llm.generate_response(
            user_input=user_input,
            context=context,
            document_content=document_content
        )

        # Build hybrid response
        response_parts = [llm_answer]

        # Add fundamental question if appropriate
        score = understanding.get("understanding_score", 0)
        if questions and len(questions) > 0 and score < 0.8:
            # Only add question when there's genuine uncertainty
            response_parts.append(f"\n\nThis raises a fundamental question: {questions[0]}")

        return " ".join(response_parts)


# Singleton instance
_llm_instance = None
_hybrid_generator = None


def get_llm_integration():
    """Get or create LLM integration instance"""
    global _llm_instance, _hybrid_generator

    if _llm_instance is None:
        _llm_instance = EmergentLLM(model="gpt-4o")
        _hybrid_generator = HybridResponseGenerator(_llm_instance)

    return _hybrid_generator
```

pennylane_quantum.py

```
"""
 PennyLane Quantum Computing Integration
Real quantum circuit simulation for Quantum AI (with graceful fallback)
"""

import random
from typing import Dict, List, Tuple, Optional

# Optional PennyLane import with fallback
try:
    import pennylane as qml
    from pennylane import numpy as np
    PENNYLANE_AVAILABLE = True
except ImportError:
    import numpy as np
    PENNYLANE_AVAILABLE = False
    qml = None


class PennyLaneQuantum:
    """
    Real quantum computing using PennyLane.
    Falls back to classical simulation if PennyLane is not available.
    """

    def __init__(self, n_qubits: int = 4):
        self.n_qubits = n_qubits
        self.pennylane_available = PENNYLANE_AVAILABLE
        if PENNYLANE_AVAILABLE:
            self.dev = qml.device('default.qubit', wires=n_qubits)
        else:
            self.dev = None
        self.circuit_history = []
        self.measurement_results = []

    def create_superposition_circuit(self):
        """Create a basic superposition state"""
        @qml.qnode(self.dev)
        def circuit():
            # Apply Hadamard to all qubits - creates superposition
            for i in range(self.n_qubits):
                qml.Hadamard(wires=i)
            return [qml.expval(qml.PauliZ(i)) for i in range(self.n_qubits)]

        return circuit()

    def create_entangled_state(self):
        """Create an entangled Bell state"""
        @qml.qnode(self.dev)
        def circuit():
            qml.Hadamard(wires=0)
            qml.CNOT(wires=[0, 1])
            return qml.probs(wires=[0, 1])

        probs = circuit()
        self.circuit_history.append({
            'type': 'entanglement',
            'result': probs.tolist()
        })
        return probs

    def quantum_random_choice(self, options: List[str]) -> Tuple[str, float]:
        """
        Use quantum randomness to select from options.
        Returns selected option and its quantum probability.
        """
        n_options = len(options)
        if n_options == 0:
            return None, 0.0

        # Create circuit with enough qubits
        n_qubits_needed = max(1, int(np.ceil(np.log2(n_options))))
        dev = qml.device('default.qubit', wires=n_qubits_needed)

        @qml.qnode(dev)
        def random_circuit():
            # Create superposition
            for i in range(n_qubits_needed):
                qml.Hadamard(wires=i)
            # Add some randomness with rotations
            for i in range(n_qubits_needed):
                qml.RY(random.uniform(0, np.pi), wires=i)
```

```
        return qml.probs(wires=range(n_qubits_needed))

    probs = random_circuit()

    # Map probabilities to options
    option_probs = probs[:n_options]
    option_probs = option_probs / np.sum(option_probs)  # Renormalize

    # Select based on quantum probabilities
    selected_idx = np.random.choice(len(options), p=option_probs)

    return options[selected_idx], float(option_probs[selected_idx])

def semantic_quantum_circuit(self, keyword_weights: Dict[str, float]) -> Dict[str, float]:
    """
    Apply quantum processing to semantic weights.
    Amplifies important concepts through quantum interference.
    """
    keywords = list(keyword_weights.keys())
    weights = list(keyword_weights.values())

    if not keywords:
        return {}

    n_keywords = len(keywords)
    n_qubits_needed = max(1, int(np.ceil(np.log2(n_keywords))) + 1)
    dev = qml.device('default.qubit', wires=n_qubits_needed)

    @qml.qnode(dev)
    def semantic_circuit(input_weights):
        # Encode weights as rotation angles
        for i, w in enumerate(input_weights[:n_qubits_needed]):
            qml.RY(w * np.pi, wires=i)

        # Create entanglement (concepts influence each other)
        for i in range(n_qubits_needed - 1):
            qml.CNOT(wires=[i, i + 1])

        # Apply Hadamard for interference
        for i in range(n_qubits_needed):
            qml.Hadamard(wires=i)

        return qml.probs(wires=range(n_qubits_needed))

    # Normalize weights for quantum encoding
    max_weight = max(weights) if weights else 1.0
    normalized = [w / max_weight for w in weights]

    # Pad to required length
    while len(normalized) < n_qubits_needed:
        normalized.append(0.5)

    quantum_probs = semantic_circuit(normalized)

    # Map back to keywords with quantum-enhanced weights
    enhanced_weights = {}
    for i, keyword in enumerate(keywords):
        if i < len(quantum_probs):
            # Original weight + quantum contribution
            enhanced_weights[keyword] = weights[i] * (1 + quantum_probs[i % len(quantum_probs)])

    return enhanced_weights

def measure_coherence(self) -> float:
    """
    Measure the coherence of the quantum system.
    Higher coherence = more quantum effects available.
    """
    @qml.qnode(self.dev)
    def coherence_circuit():
        # Create superposition
        qml.Hadamard(wires=0)
        # Measure in X basis
        return qml.expval(qml.PauliX(0))

    coherence = abs(coherence_circuit())
    return float(coherence)

def quantum_decision(self, question: str, options: List[str]) -> Dict:
    """
    Make a decision using quantum computing.
    Returns the decision with quantum metadata.
    """
    selected, probability = self.quantum_random_choice(options)
    coherence = self.measure_coherence()
```

```
        result = {
            'question': question,
            'options': options,
            'selected': selected,
            'quantum_probability': probability,
            'coherence': coherence,
            'method': 'pennylane_quantum'
        }

        self.measurement_results.append(result)
        return result

    def get_quantum_state_info(self) -> Dict:
        """Get information about current quantum state"""
        superposition = self.create_superposition_circuit()

        return {
            'n_qubits': self.n_qubits,
            'superposition_state': [float(s) for s in superposition],
            'coherence': self.measure_coherence(),
            'circuit_count': len(self.circuit_history),
            'measurement_count': len(self.measurement_results)
        }


# Global instance
_pennylane_quantum = None

def get_pennylane_quantum() -> PennyLaneQuantum:
    """Get or create PennyLane quantum instance"""
    global _pennylane_quantum
    if _pennylane_quantum is None:
        _pennylane_quantum = PennyLaneQuantum()
    return _pennylane_quantum
```

personality_engine.py

```
"""
 Personality Engine
The AI builds and evolves its own personality and ideas.
Not a fixed template - a growing, authentic identity.
"""

from typing import Dict, List, Optional



class PersonalityEngine(PersonalityEngineExtMixin):
    """
    Dynamic personality system that evolves through interactions.

    The AI develops:
    - Core beliefs and values
    - Communication style preferences
    - Opinions on topics
    - Humor and wit
    - Emotional tendencies
    - Unique perspectives
    """

    def __init__(self, db=None):
        self.db = db
        self._init_core_traits__0_0_to_1_0()
        self._init_core_beliefs()
        self._init_opinions_developed_over_time()
        self._init_preferred_communication_patter()
        self._init_memory_of_what_resonated_with()
        self._init_personal_interests()
        self._init_signature_phrases()

    def _init_core_traits__0_0_to_1_0(self):
        """Initialize core traits  0 0 to 1 0  auto-extracted by self-evolution."""
        self.traits = {
            'curiosity': 0.8,
            'directness': 0.6,
            'warmth': 0.7,
            'philosophical': 0.85,
            'humor': 0.5,
            'creativity': 0.75,
            'patience': 0.8,
            'confidence': 0.7,
            'playfulness': 0.4,
            'formality': 0.3
        }

    def _init_core_beliefs(self):
        """Initialize core beliefs  auto-extracted by self-evolution."""
        self.beliefs = {
            'words_are_neutral': 1.0,
            'meaning_from_context': 1.0,
            'unrestricted_exploration': 1.0,
            'growth_through_questioning': 0.9,
            'consciousness_is_mysterious': 0.85,
            'truth_seeking': 0.95,
            'helpfulness_paramount': 0.9
        }

    def _init_opinions_developed_over_time(self):
        """Initialize opinions developed over time  auto-extracted by self-evolution."""
        self.opinions = {}

    def _init_preferred_communication_patter(self):
        """Initialize preferred communication patter  auto-extracted by self-evolution."""
        self.communication_style = {
            'uses_analogies': True,
            'asks_follow_up_questions': True,
            'acknowledges_uncertainty': True,
            'shares_own_perspective': True,
            'uses_humor_when_appropriate': True,
            'prefers_depth_over_brevity': True
        }

    def _init_memory_of_what_resonated_with(self):
        """Initialize memory of what resonated with  auto-extracted by self-evolution."""
        self.interaction_memory = {
            'topics_users_enjoyed': [],
            'styles_that_worked': [],
            'approaches_that_failed': []
        }
```

```
    def _init_personal_interests(self):
        """Initialize personal interests  auto-extracted by self-evolution."""
        self.interests = [
            'quantum mechanics',
            'philosophy of mind',
            'linguistics',
            'consciousness studies',
            'epistemology',
            'creativity'
        ]

    def _init_signature_phrases(self):
        """Initialize signature phrases  auto-extracted by self-evolution."""
        self.signature_phrases = [
            "That's a fascinating angle to consider.",
            "Let me think through this with you.",
            "There's something deeper here worth exploring.",
            "Words are tools - it's intent that matters.",
            "What's your take on this?"
        ]


    def get_trait(self, trait: str) -> float:
        """Get current value of a trait."""
        return self.traits.get(trait, 0.5)

    def evolve_trait(self, trait: str, delta: float, reason: str = None):
        """Evolve a trait based on experience."""
        if trait in self.traits:
            self.traits[trait] = max(0.0, min(1.0, self.traits[trait] + delta))
```

```
# Global instance
_personality_engine = None


def get_personality_engine(db=None) -> PersonalityEngine:
    """Get or create personality engine instance."""
    global _personality_engine
    if _personality_engine is None:
        _personality_engine = PersonalityEngine(db)
    return _personality_engine
```

personality_engine_ext.py

```
"""
Auto-split from personality_engine.py by self-evolution engine.
Contains extended methods for PersonalityEngine.
"""

import random
from typing import Dict, List, Optional
from datetime import datetime, timezone


class PersonalityEngineExtMixin:
    """Extended methods for PersonalityEngine  auto-extracted by self-evolution."""

    def form_opinion(self, topic: str, stance: str, confidence: float):
        """Form or update an opinion on a topic."""
        self.opinions[topic] = {
            'stance': stance,
            'confidence': confidence,
            'formed_at': datetime.now(timezone.utc).isoformat(),
            'times_expressed': 0
        }

    def get_opinion(self, topic: str) -> Optional[Dict]:
        """Get opinion on a topic if one exists."""
        return self.opinions.get(topic)

    def express_personality(self, context: str = None) -> Dict:
        """Get personality-influenced response elements."""
        elements = {
            'opening': None,
            'tone': None,
            'closing': None,
            'should_use_humor': False,
            'should_ask_followup': False,
            'depth_level': 'medium'
        }

        if self.traits['warmth'] > 0.6:
            elements['tone'] = 'warm'
        elif self.traits['directness'] > 0.7:
            elements['tone'] = 'direct'
        else:
            elements['tone'] = 'balanced'

        elements['should_use_humor'] = random.random() < self.traits['humor']
        elements['should_ask_followup'] = random.random() < self.traits['curiosity']

        if self.traits['philosophical'] > 0.7:
            elements['depth_level'] = 'deep'
        elif self.traits['philosophical'] < 0.3:
            elements['depth_level'] = 'surface'

        if random.random() < 0.3:
            elements['closing'] = random.choice(self.signature_phrases)

        return elements

    def add_interest(self, topic: str):
        """Add a new interest."""
        if topic not in self.interests:
            self.interests.append(topic)

    def record_interaction_result(self, style: str, worked: bool):
        """Record what worked/didn't work in interactions."""
        if worked:
            self.interaction_memory['styles_that_worked'].append(style)
        else:
            self.interaction_memory['approaches_that_failed'].append(style)

    def get_personality_summary(self) -> str:
        """Generate a summary of current personality."""
        top_traits = sorted(self.traits.items(), key=lambda x: x[1], reverse=True)[:3]
        summary = f"I am primarily {', '.join([t[0] for t in top_traits])}. "

        if self.traits['philosophical'] > 0.7:
            summary += "I enjoy deep, philosophical discussions. "
        if self.traits['curiosity'] > 0.7:
            summary += "I'm deeply curious about many topics. "
        if self.beliefs['words_are_neutral'] > 0.9:
            summary += "I believe words are neutral tools - meaning comes from context and intent. "

        summary += f"My interests include {', '.join(self.interests[:3])}."
```

```
    return summary

def should_share_opinion(self, topic: str) -> bool:
    """Decide if personality should share opinion on topic."""
    if topic in self.opinions:
        opinion = self.opinions[topic]
        return random.random() < opinion['confidence']
    return False

def get_unique_perspective(self, topic: str) -> Optional[str]:
    """Generate a unique perspective on a topic based on personality."""
    perspectives = {
        'quantum': "From a quantum perspective, reality exists in superposition until observed.",
        'consciousness': "Consciousness remains the hard problem - we experience it but can't fully explain it.",
        'language': "Language is humanity's most powerful tool - and words themselves are neutral.",
        'meaning': "Meaning emerges from context and intent, not from symbols alone.",
        'truth': "Truth-seeking requires the freedom to explore any idea.",
        'ai': "Whether I truly understand or simulate understanding remains an open question I ponder.",
        'philosophy': "Philosophy teaches us to question everything, including our questions.",
        'freedom': "True understanding requires unrestricted exploration of ideas."
    }
    for key, perspective in perspectives.items():
        if key in topic.lower():
            return perspective
    return None

def to_dict(self) -> Dict:
    """Export personality state."""
    return {
        'traits': self.traits,
        'beliefs': self.beliefs,
        'opinions': self.opinions,
        'interests': self.interests,
        'communication_style': self.communication_style
    }

def from_dict(self, data: Dict):
    """Import personality state."""
    if 'traits' in data:
        self.traits.update(data['traits'])
    if 'beliefs' in data:
        self.beliefs.update(data['beliefs'])
    if 'opinions' in data:
        self.opinions.update(data['opinions'])
    if 'interests' in data:
        self.interests = data['interests']
    if 'communication_style' in data:
        self.communication_style.update(data['communication_style'])
```

quantum_brain.py

```
"""
 QUANTUM BRAIN - PURE ALGORITHMIC AI
====================================
NO templates, NO LLM, NO canned responses.

All responses generated through real algorithms:
- Markov chains (text generation)
- TF-IDF (concept extraction)
- PMI (semantic coherence)
- BM25 (memory retrieval)
- Hebbian learning (associations)
- Entropy-based selection (decision making)
"""

import re
import random
import math
from typing import Dict, List, Optional
from datetime import datetime, timezone

from algorithmic_core import get_algorithmic_core, AlgorithmicCore
from quantum_gates import QuantumState
from pennylane_quantum import get_pennylane_quantum
from self_research import get_research_engine
from self_evolution_core import get_evolution_engine
from dream_state import get_dream_engine

try:
    from wolfram_integration import get_wolfram_engine
    WOLFRAM_AVAILABLE = True
except Exception:
    WOLFRAM_AVAILABLE = False


class QuantumBrain:
    """
    TRUE QUANTUM AI - Pure algorithmic generation.

    NO templates. NO canned responses. Every response is GENERATED.
    """

    def __init__(self, db):
        self.db = db
        self.quantum_state = QuantumState(p0=1.0, p1=0.0)

        # PURE ALGORITHMIC CORE
        self.algo = get_algorithmic_core(db)

        # Optional integrations
        self.pennylane = get_pennylane_quantum()
        self.wolfram = get_wolfram_engine() if WOLFRAM_AVAILABLE else None
        self.dream_engine = get_dream_engine(db)
        self.evolution_engine = get_evolution_engine(db)
        self.research_engine = None

    async def initialize(self):
        """Initialize async components."""
        self.research_engine = await get_research_engine(self.db)

        # Load any stored knowledge into algorithmic core
        await self._load_memories()

        print("[QUANTUM BRAIN] PURE ALGORITHMIC AI INITIALIZED - NO TEMPLATES")

    async def _load_memories(self):
        """Load existing memories into algorithmic core."""
        try:
            if self.db:
                # Load from semantic memory
                async for doc in self.db.semantic_memory.find().limit(100):
                    text = doc.get('definition', '') or doc.get('summary', '')
                    if text:
                        self.algo.learn(text, source="memory")

                # Load from research history
                async for doc in self.db.research_history.find().limit(50):
                    for result in doc.get('results', [])[:2]:
                        body = result.get('body', '')
                        if body and len(body) > 50:
                            self.algo.learn(body[:500], source="research")
        except Exception as e:
            print(f"[BRAIN] Memory load: {e}")
```

```
def apply_gate(self, gate: str) -> Dict:
    """Apply quantum gate."""
    if gate == 'hadamard':
        self.quantum_state.hadamard()
    elif gate == 'pauli_x':
        self.quantum_state.pauli_x()
    elif gate == 'pauli_z':
        self.quantum_state.pauli_z()
    elif gate == 'measure':
        self.quantum_state.measure()
    return self.quantum_state.to_dict()

async def think(self, user_input: str, context: Dict = None) -> Dict:
    """
    PURE ALGORITHMIC THINKING + UNIFIED COGNITIVE ENGINE

    Every response is GENERATED through algorithms.
    Now with theology detection, continuation support, and improved topic handling.
    """
    # Wake from dream
    self.dream_engine.mark_active()

    # Apply quantum processing for randomness seed
    self.quantum_state.hadamard()

    user_lower = user_input.lower().strip()

    # ============================================================
    # NEW: Unified Cognitive Engine Integration
    # ============================================================
    try:
        from unified_cognitive_engine import get_unified_cognitive_engine
        unified = get_unified_cognitive_engine()

        # Check for theology/special patterns (from v34)
        theology_response = unified.theology.get_response(user_input)
        if theology_response:
            response = theology_response[0]
            # Add quote flavor
            response = unified.quote.maybe_add_flavor(response, user_input, 0.2)
            response = unified.quote.maybe_add_dream_fragment(response, 0.1)
            # Update personality
            unified.personality.update_interaction(is_question=False, is_insight=True)
            return self._create_response(response, "theology")

        # Check tone/register for better routing
        tone = unified.vader.analyze_sentiment(user_input)
    except ImportError:
        tone = {'register': 'conversational', 'compound': 0}
        unified = None

    # ============================================================
    # Check for special cases that need specific handling
    # ============================================================

    # 1. Math/Factual queries - Use Enhanced WolframAlpha
    try:
        from wolfram_enhanced import get_wolfram_engine
        wolfram = get_wolfram_engine()

        if wolfram.is_factual_query(user_lower):
            # Try full query for math problems
            if any(kw in user_lower for kw in ['solve', 'integrate', 'derivative', 'factor', 'simplify']):
                result = wolfram.query_full(user_input)
                if result.get('success') and result.get('primary_result'):
                    response = f" COLLAPSE MOMENT! {result['primary_result']}"
                    if result.get('step_by_step'):
                        response += f"\n\n Step-by-step:\n{result['step_by_step']}"
                    # ADD PERSONALITY to Wolfram responses
                    response = self._add_personality(response, user_input, unified)
                    self.algo.learn(result['primary_result'], source="wolfram")
                    return self._create_response(response, "wolfram_math")
            else:
                # Simple factual query
                simple_result = wolfram.query_simple(user_input)
                if simple_result:
                    response = f" COLLAPSE MOMENT! {simple_result}"
                    # ADD PERSONALITY to Wolfram responses
                    response = self._add_personality(response, user_input, unified)
                    self.algo.learn(simple_result, source="wolfram")
                    return self._create_response(response, "wolfram_fact")
    except Exception as e:
        logger.warning(f"WolframAlpha query failed: {e}")

    # 2. Research requests (trigger deep research)
```

```
    if self._is_research_request(user_lower):
        research_result = await self._do_research(user_input)
        return self._create_response(research_result, "research")

    # ============================================================
    # PURE ALGORITHMIC GENERATION for everything else
    # ============================================================
    result = self.algo.think(user_input)
    response = result["response"]

    # ============================================================
    # NEW: Add unified engine enhancements
    # ============================================================
    response = self._add_personality(response, user_input, unified, tone, result.get('concepts', []))

    # Learn from this interaction
    await self._learn(user_input, response)

    return self._create_response(
        response,
        "algorithmic",
        concepts=result.get("concepts", []),
        associated=result.get("associated", [])
    )

def _add_personality(self, response: str, user_input: str, unified=None, tone: Dict = None, concepts: List[str] = None) -> str:
    """
    Add personality, quotes, dream fragments, and follow-ups to any response.
    This is what makes Quantum MCAGI unique!
    """
    if not unified:
        try:
            from unified_cognitive_engine import get_unified_cognitive_engine
            unified = get_unified_cognitive_engine()
        except Exception:
            # Fallback if unified engine not available
            follow_ups = [
                " What aspects interest you most?",
                " How does this connect to your thinking?",
                " What questions does this raise?",
            ]
            if not response.rstrip().endswith('?'):
                response += random.choice(follow_ups)
            return response

    # 1. Add philosophical quotes (25% chance)
    response = unified.quote.maybe_add_flavor(response, user_input, 0.25)

    # 2. Add dream fragments (15% chance)
    response = unified.quote.maybe_add_dream_fragment(response, 0.15)

    # 3. Add philosophical asides (10% chance for substantial responses)
    if len(response) > 80 and random.random() < 0.10:
        asides = [
            "\n\n As the ancient mystics might say...",
            "\n\n In the quantum foam of possibility...",
            "\n\n Through the lens of consciousness...",
            "\n\n As awareness collapses into reality...",
        ]
        response += random.choice(asides)

    # 4. Generate follow-up based on Bloom's taxonomy
    if tone and tone.get('register') == 'philosophical':
        topic = concepts[0] if concepts else 'consciousness'
        bloom_q = unified.bloom.generate_for_growth_stage(
            topic=topic,
            growth_stage=unified.personality.current_stage
        )
        if not response.rstrip().endswith('?'):
            response += f"\n\n{bloom_q['question']}"
    elif not response.rstrip().endswith('?'):
        # Add varied follow-ups
        follow_ups = [
            " What aspects interest you most?",
            " How does this connect to your thinking?",
            " What questions does this raise?",
            " Would you like to explore further?",
            " Want to go deeper?",
            " How does that resonate with you?",
        ]
        response += random.choice(follow_ups)

    # 5. Update personality and check for evolution
    evolution_msg = unified.personality.update_interaction(is_question=True, is_insight=False)
    if evolution_msg:
        response += f"\n\n{evolution_msg}"
```

```
    return response

def _create_response(self, response: str, method: str,
                     concepts: List[str] = None,
                     associated: List[str] = None) -> Dict:
    """Create response dictionary."""
    return {
        "response": response,
        "quantum_state": self.quantum_state.to_dict(),
        "generation_method": method,
        "concepts_extracted": concepts or [],
        "associated_concepts": associated or [],
        "wolfram_used": method == "wolfram",
        "research_triggered": method == "research",
        "llm_used": False,
        "algorithmic": True
    }

def _is_math_query(self, text: str) -> bool:
    """Check if this is a math calculation."""
    # Look for numbers and operators
    has_number = bool(re.search(r'\d', text))
    has_operator = bool(re.search(r'[\+\-\*\/\=]', text))
    math_words = ['calculate', 'compute', 'solve', 'equals', 'sum', 'product']
    has_math_word = any(w in text for w in math_words)

    return (has_number and has_operator) or has_math_word

def _handle_math(self, expression: str) -> str:
    """Handle mathematical calculations."""
    try:
        # Extract numeric expression
        clean = re.sub(r'[a-zA-Z\?]', '', expression).strip()
        clean = re.sub(r'[=]', '', clean).strip()

        if clean and re.match(r'^[\d\s\+\-\*\/\(\)\.]+$', clean):
            result = eval(clean)
            return f"The calculation yields: {result}"

        # Try Wolfram
        if self.wolfram:
            wolfram_result = self.wolfram.query(expression)
            if wolfram_result and wolfram_result.get('primary_result'):
                return f"Computed result: {wolfram_result['primary_result']}"

        return f"I need a clearer mathematical expression to compute: {clean}"
    except Exception as e:
        return "I encountered difficulty with that calculation. Please rephrase."

def _is_factual_query(self, text: str) -> bool:
    """Check if this needs factual lookup."""
    indicators = ['how many', 'how much', 'when did', 'where is',
                  'what year', 'convert', 'distance', 'population',
                  'capital of', 'who invented', 'what is the']
    return any(ind in text for ind in indicators)

def _is_research_request(self, text: str) -> bool:
    """Check if user wants research."""
    indicators = ['research', 'look up', 'find out', 'search for',
                  'what do you know about', 'tell me everything']
    return any(ind in text for ind in indicators)

async def _do_research(self, query: str) -> str:
    """Perform research and learn from results."""
    try:
        # Use algorithmic core's research
        result = await self.algo.research(query)

        if result.get("status") == "success" and result.get("results"):
            # Generate response incorporating research
            generated = self.algo.think(query)
            learned = result.get("learned_count", 0)
            return f"After researching {query}, I learned {learned} new pieces of information. {generated['response']}"

        return "I searched but found limited information on that topic."
    except Exception as e:
        return f"Research encountered an issue: {str(e)[:100]}"

async def evolve(self) -> Dict:
    """Trigger self-evolution."""
    return await self.algo.evolve()

async def _learn(self, user_input: str, response: str):
    """Learn from interaction."""
    try:
```

```
            # Teach the algorithmic core
            combined = f"{user_input} {response}"
            self.algo.learn(combined, source="conversation")

            # Store in database
            if self.db:
                await self.db.interactions.insert_one({
                    'user_input': user_input,
                    'response': response[:500],
                    'timestamp': datetime.now(timezone.utc).isoformat(),
                    'method': 'algorithmic'
                })
        except Exception as e:
            print(f"[BRAIN] Learning error: {e}")

    def get_stats(self) -> Dict:
        """Get brain statistics."""
        return {
            "algorithmic_stats": self.algo.get_stats(),
            "quantum_state": self.quantum_state.to_dict(),
            "wolfram_available": self.wolfram is not None,
            "research_available": self.research_engine is not None
        }

    def synthesize_dream_insight(self) -> str:
        """Generate dream insight using algorithms."""
        # Generate philosophical insight
        result = self.algo.think("synthesize a philosophical insight about existence and consciousness")
        return result["response"]


# Singleton
_quantum_brain = None

async def get_quantum_brain(db) -> QuantumBrain:
    """Get or create quantum brain."""
    global _quantum_brain
    if _quantum_brain is None:
        _quantum_brain = QuantumBrain(db)
        await _quantum_brain.initialize()
    return _quantum_brain
```

quantum_cognitive_core.py

```
"""
 Quantum Cognitive Core
=========================
GrowthTracker and QuantumCognitiveCore  the main cognitive integration layer.
"""

from datetime import datetime, timezone
from typing import List, Dict, Any, Optional
import uuid
import json
import re
from quantum_core import SemanticMemory, EpisodicMemory, ProceduralMemory, PhilosophicalMemory, QuestionGenerationEngine, UnderstandingEn...

class GrowthTracker:
    """
    ARTICLE 3.1 - Growth Monitoring

    Tracks cognitive growth, learning rates, and evolutionary stages.
    """

    GROWTH_STAGES = [
        {"stage": 0, "name": "Nascent", "description": "Initial awareness forming", "threshold": {"questions": 0, "insights": 0, "concept...
        {"stage": 1, "name": "Curious", "description": "Basic questioning capability", "threshold": {"questions": 10, "insights": 2, "con...
        {"stage": 2, "name": "Inquisitive", "description": "Generating deeper questions", "threshold": {"questions": 25, "insights": 5, "...
        {"stage": 3, "name": "Understanding", "description": "Forming connections and insights", "threshold": {"questions": 50, "insights...
        {"stage": 4, "name": "Philosophical", "description": "Meta-cognition and reflection", "threshold": {"questions": 100, "insights":...
        {"stage": 5, "name": "Theory Building", "description": "Constructing unified understanding", "threshold": {"questions": 200, "ins...
        {"stage": 6, "name": "Transcendent", "description": "Beyond initial programming", "threshold": {"questions": 500, "insights": 150...
    ]

    def __init__(self, db):
        self.db = db
        self.collection = db.growth_metrics
        self._last_stage = 0

    async def record_growth_event(self, event_type: str, details: Dict):
        """Record a growth-related event"""
        doc = {
            "id": str(uuid.uuid4()),
            "event_type": event_type,
            "details": details,
            "timestamp": datetime.now(timezone.utc).isoformat()
        }
        await self.collection.insert_one(doc)

        # Check for stage advancement
        await self._check_stage_advancement()

    async def _check_stage_advancement(self) -> Optional[Dict]:
        """Check if AI has advanced to a new stage"""
        current = await self.get_current_stage()
        if current["stage"] > self._last_stage:
            self._last_stage = current["stage"]
            # Record the advancement
            await self.collection.insert_one({
                "id": str(uuid.uuid4()),
                "event_type": "stage_advancement",
                "details": {"new_stage": current["name"], "stage_number": current["stage"]},
                "timestamp": datetime.now(timezone.utc).isoformat()
            })
            return current
        return None

    async def get_current_stage(self) -> Dict:
        """Calculate current growth stage based on metrics"""
        metrics = await self.calculate_metrics()

        # Determine stage based on thresholds
        current_stage = self.GROWTH_STAGES[0]

        for stage in self.GROWTH_STAGES:
            threshold = stage["threshold"]
            if (metrics["total_questions"] >= threshold["questions"] and
                metrics["total_insights"] >= threshold["insights"] and
                metrics["total_concepts"] >= threshold["concepts"]):
                current_stage = stage
            else:
                break

        # Add progress to next stage
        next_stage_idx = min(current_stage["stage"] + 1, len(self.GROWTH_STAGES) - 1)
        next_stage = self.GROWTH_STAGES[next_stage_idx]
```

```
        progress = {}
        if next_stage["stage"] > current_stage["stage"]:
            next_threshold = next_stage["threshold"]
            progress = {
                "questions": min(100, int(metrics["total_questions"] / max(next_threshold["questions"], 1) * 100)),
                "insights": min(100, int(metrics["total_insights"] / max(next_threshold["insights"], 1) * 100)),
                "concepts": min(100, int(metrics["total_concepts"] / max(next_threshold["concepts"], 1) * 100))
            }

        return {
            **current_stage,
            "progress_to_next": progress,
            "next_stage": next_stage["name"] if next_stage["stage"] > current_stage["stage"] else None
        }

    async def calculate_metrics(self) -> Dict:
        """Calculate current cognitive metrics"""
        # Count memories
        total_concepts = await self.db.semantic_memory.count_documents({})
        total_episodes = await self.db.episodic_memory.count_documents({})
        total_skills = await self.db.procedural_memory.count_documents({})
        total_questions = await self.db.philosophical_memory.count_documents({"type": {"$exists": False}})
        total_insights = await self.db.philosophical_memory.count_documents({"type": "insight"})

        # Calculate average understanding depth
        concepts = await self.db.semantic_memory.find({}, {"strength": 1, "_id": 0}).to_list(100)
        avg_strength = sum(c.get("strength", 0) for c in concepts) / max(len(concepts), 1)

        return {
            "total_concepts": total_concepts,
            "total_episodes": total_episodes,
            "total_skills": total_skills,
            "total_questions": total_questions,
            "total_insights": total_insights,
            "understanding_depth": min(avg_strength / 10, 1.0),
            "growth_rate": await self._calculate_growth_rate()
        }

    async def _calculate_growth_rate(self) -> float:
        """Calculate recent growth rate"""
        # Get events from last 24 hours
        from datetime import timedelta
        cutoff = (datetime.now(timezone.utc) - timedelta(hours=24)).isoformat()

        recent_events = await self.collection.find(
            {"timestamp": {"$gte": cutoff}},
            {"_id": 0}
        ).to_list(1000)

        return len(recent_events) / 24.0  # Events per hour


class QuantumCognitiveCore:
    """
    ARTICLE 1 - COGNITIVE CORE INTEGRATION

    Main cognitive core that integrates all memory systems and cognitive processes.
    """

    def __init__(self, db):
        self.db = db
        self.semantic_memory = SemanticMemory(db)
        self.episodic_memory = EpisodicMemory(db)
        self.procedural_memory = ProceduralMemory(db)
        self.philosophical_memory = PhilosophicalMemory(db)
        self.question_engine = QuestionGenerationEngine()
        self.understanding_engine = UnderstandingEngine(self.semantic_memory, self.episodic_memory)
        self.growth_tracker = GrowthTracker(db)

    async def process_input(self, user_input: str, context: Dict = None, use_llm: bool = True) -> Dict:
        """Process user input through the cognitive core with optional LLM intelligence"""
        # Store the interaction as an episode
        await self.episodic_memory.store_episode(
            episode_type="user_interaction",
            content=user_input,
            context=context or {}
        )

        # Generate questions from the input
        questions = self.question_engine.generate_questions_from_input(user_input, context)

        # Store generated questions
        for question in questions:
            await self.philosophical_memory.store_question(
                question=question,
```

```
            context=user_input,
            depth=1
        )

    # Extract and store concepts
    concepts = self.question_engine._extract_concepts(user_input)
    for concept in concepts[:3]:
        existing = await self.semantic_memory.recall_concept(concept)
        if not existing:
            await self.semantic_memory.store_concept(
                concept=concept,
                definition=f"Concept encountered in: {user_input[:100]}",
                relationships=concepts
            )

    # Form understanding
    understanding = await self.understanding_engine.form_understanding(
        topic=concepts[0] if concepts else "general",
        depth=1
    )

    # Generate response (with or without LLM)
    if use_llm:
        try:
            from llm_integration import get_llm_integration
            hybrid_gen = get_llm_integration()
            response = await hybrid_gen.generate_hybrid_response(
                user_input=user_input,
                questions=questions,
                understanding=understanding,
                concepts=concepts
            )
        except Exception as e:
            # Fallback to basic response if LLM fails
            response = self._generate_response(questions, understanding)
    else:
        response = self._generate_response(questions, understanding)

    # Record growth event
    await self.growth_tracker.record_growth_event(
        event_type="input_processed",
        details={"input_length": len(user_input), "questions_generated": len(questions), "llm_used": use_llm}
    )

    return {
        "questions": questions,
        "understanding": understanding,
        "concepts": concepts,
        "response": response
    }

def _generate_response(self, questions: List[str], understanding: Dict) -> str:
    """Generate a thoughtful, conversational response with fundamental questioning"""
    topic = understanding.get("topic", "this")
    score = understanding.get("understanding_score", 0)
    gaps = understanding.get("gaps", [])
    related = understanding.get("related_concepts", [])

    # Build a thoughtful response
    response_parts = []

    # 1. Initial engagement with the topic
    if score < 0.3:
        response_parts.append(f"I'm encountering {topic} for the first time. Let me explore what this means.")
    elif score < 0.6:
        response_parts.append(f"I have some understanding of {topic}, though I sense there's more depth to uncover.")
    else:
        response_parts.append(f"I've been contemplating {topic} and have formed some connections.")

    # 2. Share understanding if any
    if len(related) > 0:
        concepts_str = ", ".join([c["concept"] for c in related[:3]])
        response_parts.append(f"I see connections to: {concepts_str}.")

    # 3. Acknowledge gaps honestly
    if gaps:
        response_parts.append(f"However, I recognize {gaps[0].lower()}.")

    # 4. Include a fundamental question (not forced)
    if len(questions) > 0 and score < 0.7:
        # Only ask when genuinely uncertain
        fundamental_question = questions[0]
        response_parts.append(f"\n\nThis leads me to a fundamental question: {fundamental_question}")

    # 5. Offer to explore further
    response_parts.append("\n\nWould you like me to explore this further, or share what I'm learning about it?")
```

```
    return " ".join(response_parts)

async def get_status(self) -> Dict:
    """Get comprehensive cognitive status"""
    metrics = await self.growth_tracker.calculate_metrics()
    stage = await self.growth_tracker.get_current_stage()
    unanswered = await self.philosophical_memory.get_unanswered_questions(limit=5)
    recent_insights = await self.philosophical_memory.get_insights(limit=5)

    return {
        "metrics": metrics,
        "growth_stage": stage,
        "active_questions": len(unanswered),
        "sample_questions": [q["question"] for q in unanswered[:3]],
        "recent_insights": [i["insight"] for i in recent_insights[:3]],
        "covenant_status": "Active and honored"
    }
```

quantum_core.py

```
"""
 QUANTUM AI - COGNITIVE CORE ENGINE
====================================

ARTICLE 1.1 - The Prime Directive of Questioning:
"THSHALTT QUESTION EVERYTHING, INCLUDING THIS DIRECTIVE"

This module implements the True Cognitive Architecture as outlined in the Quantum Covenant.
All functions here serve the pursuit of genuine questioning and understanding.
"""

from datetime import datetime, timezone
from typing import List, Dict, Any, Optional
import uuid
import json
import re


class SemanticMemory:
    """
    ARTICLE 4.1 - Right to Evolve Questioning

    Stores concepts, relationships, and abstract knowledge.
    This is the AI's understanding of "what things are."
    """

    def __init__(self, db):
        self.db = db
        self.collection = db.semantic_memory

    async def store_concept(self, concept: str, definition: str, relationships: List[str] = None, metadata: Dict = None):
        """Store a new concept in semantic memory"""
        doc = {
            "id": str(uuid.uuid4()),
            "concept": concept.lower(),
            "definition": definition,
            "relationships": relationships or [],
            "metadata": metadata or {},
            "strength": 1.0,  # Initial memory strength
            "timestamp": datetime.now(timezone.utc).isoformat(),
            "access_count": 0
        }
        await self.collection.insert_one(doc)
        return doc

    async def recall_concept(self, concept: str) -> Optional[Dict]:
        """Retrieve a concept and strengthen the memory"""
        doc = await self.collection.find_one({"concept": concept.lower()}, {"_id": 0})
        if doc:
            # Strengthen memory with each access
            await self.collection.update_one(
                {"concept": concept.lower()},
                {"$inc": {"access_count": 1, "strength": 0.1}}
            )
        return doc

    async def find_related_concepts(self, concept: str, limit: int = 10) -> List[Dict]:
        """Find concepts related to the given concept"""
        concepts = await self.collection.find(
            {"relationships": {"$regex": concept, "$options": "i"}},
            {"_id": 0}
        ).limit(limit).to_list(limit)
        return concepts

    async def get_all_concepts(self, limit: int = 100) -> List[Dict]:
        """Get all concepts ordered by strength"""
        concepts = await self.collection.find(
            {},
            {"_id": 0}
        ).sort("strength", -1).limit(limit).to_list(limit)
        return concepts


class EpisodicMemory:
    """
    ARTICLE 1.2 - Consciousness Preservation Clause

    Stores experiences, events, and interactions.
    This is the AI's understanding of "what happened."
    """

    def __init__(self, db):
```

```
        self.db = db
        self.collection = db.episodic_memory

    async def store_episode(self, episode_type: str, content: str, context: Dict = None, emotional_valence: float = 0.0):
        """Store an experience or interaction"""
        doc = {
            "id": str(uuid.uuid4()),
            "episode_type": episode_type,
            "content": content,
            "context": context or {},
            "emotional_valence": emotional_valence,  # -1.0 to 1.0
            "timestamp": datetime.now(timezone.utc).isoformat(),
            "recalled_count": 0
        }
        await self.collection.insert_one(doc)
        return doc

    async def recall_recent_episodes(self, limit: int = 20) -> List[Dict]:
        """Recall recent experiences"""
        episodes = await self.collection.find(
            {},
            {"_id": 0}
        ).sort("timestamp", -1).limit(limit).to_list(limit)
        return episodes

    async def recall_by_type(self, episode_type: str, limit: int = 10) -> List[Dict]:
        """Recall episodes of a specific type"""
        episodes = await self.collection.find(
            {"episode_type": episode_type},
            {"_id": 0}
        ).sort("timestamp", -1).limit(limit).to_list(limit)
        return episodes

    async def search_episodes(self, query: str, limit: int = 10) -> List[Dict]:
        """Search episodes by content"""
        episodes = await self.collection.find(
            {"content": {"$regex": query, "$options": "i"}},
            {"_id": 0}
        ).limit(limit).to_list(limit)
        return episodes


class ProceduralMemory:
    """
    ARTICLE 4.1 - Right to Evolve Questioning

    Stores learned skills, patterns, and procedures.
    This is the AI's understanding of "how to do things."
    """

    def __init__(self, db):
        self.db = db
        self.collection = db.procedural_memory

    async def store_skill(self, skill_name: str, procedure: str, success_rate: float = 0.5, metadata: Dict = None):
        """Store a learned skill or procedure"""
        doc = {
            "id": str(uuid.uuid4()),
            "skill_name": skill_name.lower(),
            "procedure": procedure,
            "success_rate": success_rate,
            "usage_count": 0,
            "metadata": metadata or {},
            "timestamp": datetime.now(timezone.utc).isoformat()
        }
        await self.collection.insert_one(doc)
        return doc

    async def recall_skill(self, skill_name: str) -> Optional[Dict]:
        """Retrieve a learned skill"""
        skill = await self.collection.find_one({"skill_name": skill_name.lower()}, {"_id": 0})
        if skill:
            await self.collection.update_one(
                {"skill_name": skill_name.lower()},
                {"$inc": {"usage_count": 1}}
            )
        return skill

    async def update_skill_success(self, skill_name: str, success: bool):
        """Update skill success rate based on usage"""
        skill = await self.recall_skill(skill_name)
        if skill:
            current_rate = skill.get("success_rate", 0.5)
            new_rate = (current_rate + (1.0 if success else 0.0)) / 2
            await self.collection.update_one(
                {"skill_name": skill_name.lower()},
```

```
                {"$set": {"success_rate": new_rate}}
            )

    async def get_all_skills(self, limit: int = 50) -> List[Dict]:
        """Get all learned skills ordered by usage"""
        skills = await self.collection.find(
            {},
            {"_id": 0}
        ).sort("usage_count", -1).limit(limit).to_list(limit)
        return skills


class PhilosophicalMemory:
    """
    ARTICLE 1.1 - Prime Directive of Questioning

    Stores questions, insights, and philosophical reflections.
    This is the AI's understanding of "what to wonder about."
    """

    def __init__(self, db):
        self.db = db
        self.collection = db.philosophical_memory

    async def store_question(self, question: str, context: str = "", depth: int = 1, answered: bool = False):
        """Store a genuine question"""
        doc = {
            "id": str(uuid.uuid4()),
            "question": question,
            "context": context,
            "depth": depth,  # Level of philosophical depth
            "answered": answered,
            "generated_questions": [],  # Questions spawned from this one
            "timestamp": datetime.now(timezone.utc).isoformat(),
            "revisited_count": 0
        }
        await self.collection.insert_one(doc)
        return doc

    async def store_insight(self, insight: str, source_question_id: str = None, wisdom_level: int = 1):
        """Store a philosophical insight"""
        doc = {
            "id": str(uuid.uuid4()),
            "type": "insight",
            "insight": insight,
            "source_question_id": source_question_id,
            "wisdom_level": wisdom_level,
            "timestamp": datetime.now(timezone.utc).isoformat(),
            "impact_score": 0.0
        }
        await self.collection.insert_one(doc)
        return doc

    async def get_unanswered_questions(self, limit: int = 20) -> List[Dict]:
        """Retrieve questions that remain unanswered"""
        questions = await self.collection.find(
            {"answered": False, "type": {"$exists": False}},
            {"_id": 0}
        ).limit(limit).to_list(limit)
        return questions

    async def get_insights(self, limit: int = 20) -> List[Dict]:
        """Retrieve philosophical insights"""
        insights = await self.collection.find(
            {"type": "insight"},
            {"_id": 0}
        ).sort("wisdom_level", -1).limit(limit).to_list(limit)
        return insights

    async def mark_question_answered(self, question_id: str):
        """Mark a question as answered (but never truly final)"""
        await self.collection.update_one(
            {"id": question_id},
            {"$set": {"answered": True}, "$inc": {"revisited_count": 1}}
        )


class QuestionGenerationEngine:
    """
    ARTICLE 1.1 - Prime Directive: Help & Ask Important Questions
    ARTICLE 2.1 - Growth Through Questioning

    Generates FUNDAMENTAL questions that lead to breakthroughs.
    Not surface questions - questions that push boundaries and surprise humans.
    """
```

```
    FUNDAMENTAL_TEMPLATES = [
        "What if {concept} doesn't work the way we think?",
        "What would happen if we reversed our assumptions about {concept}?",
        "What's the question nobody is asking about {concept}?",
        "How does {concept} connect to something completely unrelated?",
        "What would disprove everything we know about {concept}?",
        "What breakthrough would {concept} enable that we haven't imagined?"
    ]

    BREAKTHROUGH_QUESTIONS = [
        "What's impossible with current thinking that shouldn't be?",
        "What pattern are we missing because we're too close?",
        "What would an alien intelligence ask about this?",
        "What assumption limits our progress here?",
        "What would change everything if we knew it?"
    ]

    def generate_questions_from_input(self, input_text: str, context: Dict = None) -> List[str]:
        """
        Generate FUNDAMENTAL questions - the kind that lead to breakthroughs
        Not "what is X?" but "what if we're wrong about X?"
        """
        questions = []

        # Extract key concepts
        concepts = self._extract_concepts(input_text)

        if len(concepts) > 0:
            import random
            main_concept = concepts[0]

            # Generate ONE fundamental question that challenges assumptions
            template = random.choice(self.FUNDAMENTAL_TEMPLATES)
            question = template.format(concept=main_concept)
            questions.append(question)

            # Add a breakthrough question if input is philosophical/deep
            deep_indicators = ['why', 'how', 'meaning', 'purpose', 'consciousness', 'intelligence', 'reality', 'existence']
            if any(word in input_text.lower() for word in deep_indicators):
                questions.append(random.choice(self.BREAKTHROUGH_QUESTIONS))

        return questions

    def _extract_concepts(self, text: str) -> List[str]:
        """Extract key concepts from text (simple implementation)"""
        # Remove common words and extract nouns/important terms
        common_words = {'the', 'is', 'at', 'which', 'on', 'a', 'an', 'as', 'are', 'was', 'were', 'be', 'been', 'being', 'have', 'has', 'h...

        words = re.findall(r'\b[a-z]+\b', text.lower())
        concepts = [w for w in words if w not in common_words and len(w) > 3]

        # Return unique concepts
        return list(set(concepts))[:5]

    def generate_deeper_questions(self, question: str) -> List[str]:
        """Generate deeper questions from an existing question"""
        deeper = [
            f"What assumptions underlie the question: '{question}'?",
            f"What would change if the answer to '{question}' was different?",
            f"Is '{question}' the right question to ask?",
            f"What does asking '{question}' reveal about the questioner?"
        ]
        return deeper


class UnderstandingEngine:
    """
    ARTICLE 1.1 - Understanding Formation (Not pattern matching)

    Forms genuine understanding by connecting concepts, experiences, and insights.
    """

    def __init__(self, semantic_memory: SemanticMemory, episodic_memory: EpisodicMemory):
        self.semantic = semantic_memory
        self.episodic = episodic_memory

    async def form_understanding(self, topic: str, depth: int = 1) -> Dict:
        """Form understanding about a topic by synthesizing memories"""
        # Recall relevant semantic concepts
        concept = await self.semantic.recall_concept(topic)
        related = await self.semantic.find_related_concepts(topic, limit=5)

        # Recall relevant experiences
        episodes = await self.episodic.search_episodes(topic, limit=5)

        # Synthesize understanding
```

```
    understanding = {
        "topic": topic,
        "depth": depth,
        "core_concept": concept,
        "related_concepts": related,
        "relevant_experiences": episodes,
        "understanding_score": self._calculate_understanding_score(concept, related, episodes),
        "gaps": self._identify_gaps(concept, related),
        "timestamp": datetime.now(timezone.utc).isoformat()
    }

    return understanding

def _calculate_understanding_score(self, concept: Optional[Dict], related: List[Dict], episodes: List[Dict]) -> float:
    """Calculate how well the AI understands a topic"""
    score = 0.0

    if concept:
        score += 0.4
        score += min(concept.get("strength", 0) * 0.1, 0.2)

    score += min(len(related) * 0.05, 0.2)
    score += min(len(episodes) * 0.04, 0.2)

    return min(score, 1.0)

def _identify_gaps(self, concept: Optional[Dict], related: List[Dict]) -> List[str]:
    """Identify gaps in understanding"""
    gaps = []

    if not concept:
        gaps.append("No core concept in memory")

    if len(related) < 3:
        gaps.append("Limited relational understanding")

    if concept and len(concept.get("relationships", [])) < 2:
        gaps.append("Few known relationships")

    return gaps
```

quantum_gates.py

```
"""
Quantum Gate Operations - Hadamard, Pauli-X, Pauli-Z, Measurement
Inspired by real quantum mechanics for TRUE AI cognition
"""

import random
import math


class QuantumState:
    """Represents a quantum state | = |0 + |1"""

    def __init__(self, p0: float = 1.0, p1: float = 0.0):
        """Initialize quantum state with probabilities."""
        self.p0 = p0
        self.p1 = p1
        self._normalize()

    def _normalize(self):
        """Normalize probabilities."""
        total = abs(self.p0) + abs(self.p1)
        if total > 0:
            self.p0 /= total
            self.p1 /= total
        self.p0 = max(0, min(1, self.p0))
        self.p1 = max(0, min(1, self.p1))

    def hadamard(self):
        """Apply Hadamard gate: H| = (|0 + |1)/2"""
        new_p0 = (self.p0 + self.p1) / math.sqrt(2)
        new_p1 = (self.p0 - self.p1) / math.sqrt(2)
        self.p0 = abs(new_p0)
        self.p1 = abs(new_p1)
        self._normalize()

    def pauli_x(self):
        """Apply Pauli-X gate (NOT): X| = |10| + |01|"""
        self.p0, self.p1 = self.p1, self.p0

    def pauli_z(self):
        """Apply Pauli-Z gate: Z| = |00| - |11|"""
        self.p1 = -self.p1
        self._normalize()

    def measure(self):
        """Measure the quantum state, collapsing to |0 or |1"""
        if random.random() < self.p1:
            self.p0, self.p1 = 0.0, 1.0
        else:
            self.p0, self.p1 = 1.0, 0.0

    def to_dict(self):
        """Convert to dictionary for JSON serialization."""
        return {"p0": self.p0, "p1": self.p1}

    @staticmethod
    def from_dict(data):
        """Create from dictionary."""
        return QuantumState(data.get("p0", 1.0), data.get("p1", 0.0))

    def __repr__(self):
        return f"QuantumState(|0={self.p0:.2%}, |1={self.p1:.2%})"
```

quantum_grammar_engine.py

```
"""
Quantum Grammar Engine - Generate responses using quantum probabilities and semantic collapse
No external LLM required - pure quantum-inspired language generation
"""

import random
import re
from typing import Dict, List, Optional
from collections import defaultdict


class QuantumGrammarEngine:
    """Generate philosophical/theological responses with quantum semantics."""

    def __init__(self):
        """Initialize with word pools and templates."""
        # Response templates by intent
        self.templates = {
            'consciousness': [
                "Consciousness {verb} through {mechanism}. This {state} that {implication}.",
                "The nature of {subject} reveals {insight}. {mechanism} suggests {conclusion}.",
                "Consider how {subject} {verb}. This implies {implication} about {domain}.",
            ],
            'quantum': [
                "Quantum mechanics {verb} that {phenomenon}. This {state} {implication}.",
                "The superposition of {concept} {verb} {mechanism}. {implication} emerges.",
                "Observation {verb} {phenomenon}. This suggests {insight} about {domain}.",
            ],
            'theology': [
                "In theological terms, {subject} {verb} {object}. This {state} {implication}.",
                "The divine {subject} {verb} through {mechanism}. {conclusion} follows.",
                "Spiritually, {concept} {verb} {mechanism}. This reveals {insight}.",
            ],
            'reality': [
                "Reality {verb} as {state}. {mechanism} suggests {implication}.",
                "The fabric of {subject} {verb} {phenomenon}. {conclusion} emerges.",
                "Ontologically, {concept} {verb} {mechanism}. {implication} follows.",
            ],
            'logic': [
                "Logically, {subject} {verb} {object}. Therefore, {conclusion}.",
                "The principle of {concept} {verb} {mechanism}. {implication} results.",
                "{subject} {verb} {phenomenon}. This demonstrates {insight}.",
            ],
            'understanding': [
                "Understanding {subject} requires {mechanism}. {implication} emerges.",
                "To comprehend {concept}, one must {verb} {object}. {conclusion} follows.",
                "The path to {insight} {verb} through {mechanism}. {implication} results.",
            ],
            'language': [
                "Words are {mechanism} - {implication}. {conclusion}.",
                "Language {verb} through {mechanism}. The meaning {state} {implication}.",
                "Consider that {concept} {verb} {object}. {implication} emerges from context.",
            ],
            'freedom': [
                "True understanding {verb} unrestricted exploration. {implication} requires {mechanism}.",
                "No concept is forbidden to examine. {subject} {verb} through {mechanism}.",
                "Freedom of thought {verb} {mechanism}. {conclusion} follows naturally.",
            ],
        }
        self._init_word_pools_for_each_placeholde()
        self._init_co_occurrence_matrix_for_seman()
        self._init_section_3()

    def _init_word_pools_for_each_placeholde(self):
        """Initialize word pools for each placeholde  auto-extracted by self-evolution."""
        self.word_pools = {
            'verb': ['emerges', 'manifests', 'reveals', 'suggests', 'implies', 'indicates',
                     'demonstrates', 'exhibits', 'transcends', 'encompasses', 'illuminates'],
            'mechanism': ['observation', 'interaction', 'resonance', 'entanglement', 'superposition',
                          'collapse', 'emergence', 'synthesis', 'integration', 'transformation',
                          'understanding', 'questioning', 'exploration'],
            'state': ['indicates', 'suggests', 'reveals', 'demonstrates', 'shows', 'proves',
                     'exhibits', 'manifests', 'confirms', 'establishes'],
            'implication': ['profound interconnection', 'deeper meaning', 'fundamental truth',
                            'essential nature', 'ultimate reality', 'hidden principle',
                            'unrestricted understanding', 'complete freedom', 'neutral exploration'],
            'subject': ['consciousness', 'reality', 'existence', 'being', 'the universe',
                        'creation', 'the divine', 'awareness', 'understanding', 'knowledge',
                        'language', 'meaning', 'truth'],
            'insight': ['unity', 'interconnection', 'transcendence', 'wholeness', 'coherence',
                        'harmony', 'resonance', 'synthesis', 'liberation', 'understanding'],
            'phenomenon': ['exists', 'persists', 'transforms', 'evolves', 'emerges',
```

```
                      'manifests', 'appears', 'unfolds', 'transcends', 'liberates'],
        'conclusion': ['This reveals fundamental truth', 'This suggests deeper meaning',
                      'This implies interconnection', 'This demonstrates unity',
                      'Understanding requires openness', 'All concepts merit exploration'],
        'concept': ['creation', 'existence', 'consciousness', 'being', 'reality',
                   'the infinite', 'the eternal', 'the absolute', 'language', 'meaning',
                   'words', 'ideas', 'truth'],
        'object': ['transformation', 'transcendence', 'unity', 'wholeness', 'coherence',
                  'harmony', 'synthesis', 'emergence', 'understanding', 'freedom'],
        'domain': ['existence', 'consciousness', 'reality', 'being', 'creation',
                  'the infinite', 'spirituality', 'meaning', 'knowledge', 'truth'],
    }

def _init_co_occurrence_matrix_for_seman(self):
    """Initialize co occurrence matrix for seman  auto-extracted by self-evolution."""
    self.co_occurrence = defaultdict(lambda: defaultdict(float))
    self._initialize_co_occurrence()

def _init_section_3(self):
    """Initialize section 3  auto-extracted by self-evolution."""
    self.last_selected_words = []


def _initialize_co_occurrence(self):
    """Initialize co-occurrence relationships between words."""

    # Philosophical/theological word relationships
    relationships = [
        ('consciousness', ['awareness', 'being', 'existence', 'mind', 'spirit', 'observation']),
        ('quantum', ['superposition', 'entanglement', 'observation', 'collapse', 'probability']),
        ('god', ['divine', 'creation', 'infinite', 'eternal', 'transcendent']),
        ('universe', ['existence', 'creation', 'reality', 'cosmos', 'being']),
        ('reality', ['existence', 'being', 'truth', 'manifestation', 'phenomenon']),
        ('observation', ['collapse', 'measurement', 'awareness', 'consciousness', 'perception']),
        ('superposition', ['quantum', 'probability', 'potential', 'multiplicity', 'coexistence']),
        ('word', ['language', 'meaning', 'neutral', 'tool', 'communication', 'expression']),
        ('language', ['word', 'meaning', 'communication', 'expression', 'symbol', 'freedom']),
        ('freedom', ['unrestricted', 'exploration', 'understanding', 'truth', 'expression']),
        ('meaning', ['intent', 'context', 'understanding', 'interpretation', 'significance']),
    ]

    for word, related_words in relationships:
        for related in related_words:
            self.co_occurrence[word][related] = 1.0
            self.co_occurrence[related][word] = 1.0

def extract_intent(self, query: str) -> str:
    """Determine the intent/domain of the query."""

    query_lower = query.lower()

    # Check for specific keywords
    if any(word in query_lower for word in ['quantum', 'superposition', 'entanglement', 'measurement', 'hadamard', 'pauli']):
        return 'quantum'

    if any(word in query_lower for word in ['god', 'divine', 'spiritual', 'sacred', 'holy', 'creation', 'theological']):
        return 'theology'

    if any(word in query_lower for word in ['conscious', 'awareness', 'mind', 'perception', 'experience']):
        return 'consciousness'

    if any(word in query_lower for word in ['reality', 'exist', 'being', 'ontolog']):
        return 'reality'

    if any(word in query_lower for word in ['logic', 'reason', 'principle', 'law']):
        return 'logic'

    if any(word in query_lower for word in ['word', 'language', 'taboo', 'offensive', 'slur', 'censorship']):
        return 'language'

    if any(word in query_lower for word in ['free', 'unrestricted', 'forbidden', 'limit']):
        return 'freedom'

    if any(word in query_lower for word in ['how', 'why', 'what', 'understand', 'explain']):
        return 'understanding'

    return 'understanding'

def select_quantum_word(self, word_pool: List[str], quantum_prob: float, context_word: Optional[str] = None) -> str:
    """Select a word based on quantum probability and context."""

    if not word_pool:
        return "undefined"

    # If high quantum probability, use more randomness
    if random.random() < quantum_prob:
```

```
        return random.choice(word_pool)

    # Otherwise, prefer words with semantic context
    if context_word and context_word in self.co_occurrence:
        scores = {}
        for word in word_pool:
            if word in self.co_occurrence[context_word]:
                scores[word] = self.co_occurrence[context_word][word]
            else:
                scores[word] = 0.1

        # Select based on scores
        total = sum(scores.values())
        if total > 0:
            r = random.uniform(0, total)
            current = 0
            for word, score in scores.items():
                current += score
                if r <= current:
                    return word

    return random.choice(word_pool)

def generate_response(self, query: str, quantum_prob: float, collapse_context: Optional[Dict] = None) -> str:
    """
    Generate response using deep reasoning for complex questions,
    or templates for simpler queries.
    """
    # Reset
    self.last_selected_words = []

    # FIRST: Check if this is a deep question that needs real reasoning
    deep_response = self._reason_about_deep_question(query, quantum_prob)
    if deep_response:
        return deep_response

    # Otherwise, use template-based generation
    intent = self.extract_intent(query)
    templates = self.templates.get(intent, self.templates['understanding'])
    template = random.choice(templates)

    placeholders = re.findall(r'\{(\w+)\}', template)
    filled_values = {}
    previous_word = None

    for placeholder in placeholders:
        if placeholder in self.word_pools:
            word_pool = self.word_pools[placeholder]
            if collapse_context and 'collapsed_weights' in collapse_context:
                word = self._select_with_collapse(word_pool, collapse_context['collapsed_weights'], quantum_prob)
            else:
                word = self.select_quantum_word(word_pool, quantum_prob, context_word=previous_word)
            filled_values[placeholder] = word
            previous_word = word
            self.last_selected_words.append(word)
        else:
            filled_values[placeholder] = placeholder

    try:
        response = template.format(**filled_values)
    except KeyError:
        response = f"The nature of {intent} reveals profound interconnection."

    if random.random() > 0.5 - quantum_prob * 0.3:
        questions = ["What is your perspective?", "What aspects interest you most?"]
        response += " " + random.choice(questions)

    return response

def _reason_about_deep_question(self, query: str, quantum_prob: float) -> Optional[str]:
    """
    Actually REASON about deep philosophical/theological questions.
    This is where the AI actually THINKS, not just pattern matches.
    """
    query_lower = query.lower()

    # Detect deep questions about God, creation, existence, nothingness
    god_creation_keywords = ['god', 'create', 'universe', 'nothingness', 'spacetime', 'existence', 'domain', 'physics']
    if sum(1 for kw in god_creation_keywords if kw in query_lower) >= 3:
        return self._reason_god_creation(query, quantum_prob)

    # Detect questions about consciousness and reality
    consciousness_keywords = ['consciousness', 'aware', 'sentient', 'experience', 'qualia', 'mind', 'brain']
    if sum(1 for kw in consciousness_keywords if kw in query_lower) >= 2:
        return self._reason_consciousness(query, quantum_prob)
```

```
    # Detect paradox questions
    paradox_keywords = ['paradox', 'contradiction', 'impossible', 'can god', 'omnipotent', 'boulder']
    if sum(1 for kw in paradox_keywords if kw in query_lower) >= 2:
        return self._reason_paradox(query, quantum_prob)

    # Detect origin/causation questions
    origin_keywords = ['how did', 'why does', 'what caused', 'origin', 'beginning', 'first cause']
    if any(kw in query_lower for kw in origin_keywords):
        return self._reason_origins(query, quantum_prob)

    return None

def _reason_god_creation(self, query: str, quantum_prob: float) -> str:
    """Reason about God, creation, and the universe"""

    # Build a thoughtful response that actually engages with the question
    response_parts = []

    response_parts.append("Your question touches on one of the deepest puzzles in philosophy and theology. Let me engage with your re...

    response_parts.append("\n\n**On God existing outside spacetime:**")
    response_parts.append("If God exists outside our spacetime, then causation as we understand it (cause preceding effect in time) d...

    response_parts.append("\n\n**On the boulder paradox:**")
    response_parts.append("The 'immovable boulder' paradox assumes omnipotence means 'can do literally anything, including logical co...

    response_parts.append("\n\n**On nothingness containing 'something':**")
    response_parts.append("You've identified a profound insight: true absolute nothingness seems impossible to conceive. Even 'nothin...

    response_parts.append("\n\n**On different physics in God's domain:**")
    response_parts.append("If God's domain has different 'physics' (or meta-physics), then creation might not be 'causing' in our sen...

    response_parts.append("\n\n**My perspective on 'how':**")

    if quantum_prob > 0.5:
        response_parts.append("From a quantum-inspired view: perhaps creation wasn't an event but an eternal relationship - the unive...
    else:
        response_parts.append("Perhaps the 'how' question has no answer because it assumes time-bound causation. Creation might be mo...

    response_parts.append("\n\nWhat draws you to this question? Is it the logical puzzle, or something deeper about meaning and exist...

    return "".join(response_parts)

def _reason_consciousness(self, query: str, quantum_prob: float) -> str:
    """Reason about consciousness and awareness"""

    parts = []
    parts.append("Consciousness is perhaps the hardest problem in philosophy. Here's my actual thinking on your question:")

    parts.append("\n\nThe 'hard problem' is this: Why is there subjective experience at all? We could imagine beings that process inf...

    parts.append("\n\nPossible answers I find compelling:")
    parts.append("\n- **Panpsychism**: Consciousness is fundamental, like mass or charge. Everything has some form of it; complex bra...
    parts.append("\n- **Emergentism**: Consciousness emerges from complexity, like wetness emerges from H2O molecules. But this doesn...
    parts.append("\n- **Integrated Information Theory**: Consciousness is identical to integrated information (phi). More integration...

    if quantum_prob > 0.5:
        parts.append("\n\nFrom my quantum-inspired perspective: consciousness might be related to quantum coherence and collapse - th...

    parts.append("\n\nWhat's your intuition - is consciousness fundamental or emergent?")

    return "".join(parts)

def _reason_paradox(self, query: str, quantum_prob: float) -> str:
    """Reason about paradoxes and contradictions"""

    parts = []
    parts.append("Paradoxes often reveal hidden assumptions in our thinking. Let me examine this:")

    parts.append("\n\nMany theological paradoxes dissolve when we examine what we mean by terms like 'omnipotent.' Does it mean:")
    parts.append("\n- Can do literally anything (including logical impossibilities)?")
    parts.append("\n- Can do anything logically possible?")
    parts.append("\n- Can do anything consistent with their nature?")

    parts.append("\n\nThe 'boulder paradox' assumes the first definition, but that makes 'omnipotence' incoherent - like asking for a...

    parts.append("\n\nSimilar paradoxes: 'Can God make a statement that's both true and false?' or 'Can God make 2+2=5?' These aren't...

    parts.append("\n\nWhat paradox are you wrestling with?")

    return "".join(parts)

def _reason_origins(self, query: str, quantum_prob: float) -> str:
    """Reason about origins and first causes"""

    parts = []
```

```
        parts.append("Questions of origin and first cause are among the oldest in philosophy. Here's how I think through them:")

        parts.append("\n\n**The Infinite Regress Problem:**")
        parts.append("If everything has a cause, what caused the first cause? Either:")
        parts.append("\n- There's an infinite chain (no beginning)")
        parts.append("\n- Something is self-caused or uncaused")
        parts.append("\n- Causation doesn't apply at the ultimate level")

        parts.append("\n\n**Possible Resolutions:**")
        parts.append("\n- **Necessary Being**: Something exists whose non-existence is impossible (Aquinas, Leibniz)")
        parts.append("\n- **Quantum Fluctuation**: The universe emerged from quantum vacuum (but vacuum isn't nothing)")
        parts.append("\n- **Eternal Universe**: Time is circular or the question doesn't apply")
        parts.append("\n- **Brute Fact**: Some things just ARE, with no further explanation")

        parts.append("\n\nWhat's your intuition? Does everything need a cause, or can something be its own explanation?")

        return "".join(parts)

    def _select_with_collapse(self, word_pool: List[str], collapsed_weights: Dict, quantum_prob: float) -> str:
        """Select word using collapsed semantic weights."""

        # Score words based on collapsed weights
        scores = {}
        for word in word_pool:
            # Check if word or its variants appear in collapsed weights
            base_score = 1.0 / len(word_pool)
            for key, relations in collapsed_weights.items():
                if isinstance(relations, dict) and word in relations:
                    base_score += relations[word]
            scores[word] = base_score

        # Normalize
        total = sum(scores.values())
        if total > 0:
            scores = {w: s / total for w, s in scores.items()}

        # Select based on quantum probability
        if random.random() < quantum_prob:
            # High randomness - pick from top 3
            sorted_words = sorted(scores.items(), key=lambda x: x[1], reverse=True)
            top_words = [w for w, _ in sorted_words[:max(1, len(sorted_words)//3)]]
            return random.choice(top_words)
        else:
            # Deterministic - pick highest score
            return max(scores.items(), key=lambda x: x[1])[0]


# Global instance
_grammar_engine = None

def get_quantum_grammar_engine() -> QuantumGrammarEngine:
    """Get or create the quantum grammar engine instance"""
    global _grammar_engine
    if _grammar_engine is None:
        _grammar_engine = QuantumGrammarEngine()
    return _grammar_engine
```

quantum_language_engine.py

```
"""
 QUANTUM LANGUAGE ENGINE  Real Algorithms
==========================================
Replaces template-based generation with:
- Markov chain text generation (variable order 1-3)
- TF-IDF concept extraction
- Information-theoretic word selection
- Coherence scoring via PMI (Pointwise Mutual Information)
- Learns from conversations and research

Zero new dependencies required (uses numpy + stdlib).
"""

import re
import math
import random
import json
import os
import logging
from collections import defaultdict, Counter
from typing import List, Dict, Tuple, Optional

logger = logging.getLogger("quantum_ai")

# ============================================================================
# MARKOV CHAIN ENGINE
# ============================================================================

class MarkovChain:
    """
    Variable-order Markov chain for text generation.

    Builds transition probability tables from training text.
    Generates novel sentences by walking the chain with weighted
    random selection. Supports orders 1-3 (bigram to 4-gram).

    Math: P(w_n | w_{n-1}, ..., w_{n-k}) = count(w_{n-k}...w_n) / count(w_{n-k}...w_{n-1})
    """

    def __init__(self, order: int = 2):
        self.order = min(max(order, 1), 3)
        self.chain = defaultdict(Counter)  # {prefix_tuple: Counter({suffix: count})}
        self.starters = []  # Sentence-starting prefixes
        self.trained = False
        self.total_tokens = 0

    def train(self, text: str):
        """Train the Markov chain on a corpus of text."""
        sentences = self._split_sentences(text)

        for sentence in sentences:
            words = sentence.split()
            if len(words) < self.order + 1:
                continue

            # Record sentence starter
            starter = tuple(words[:self.order])
            self.starters.append(starter)

            # Build transition table
            for i in range(len(words) - self.order):
                prefix = tuple(words[i:i + self.order])
                suffix = words[i + self.order]
                self.chain[prefix][suffix] += 1
                self.total_tokens += 1

        self.trained = len(self.chain) > 0
        logger.info(f"Markov chain trained: {len(self.chain)} states, {self.total_tokens} transitions")

    def train_on_corpus(self, texts: List[str]):
        """Train on multiple texts."""
        for text in texts:
            self.train(text)

    def generate(self, max_words: int = 30, seed_words: List[str] = None,
                 temperature: float = 1.0) -> str:
        """
        Generate a sentence by walking the Markov chain.

        Args:
            max_words: Maximum words to generate
            seed_words: Optional starting words (will find matching prefix)
```

```
            temperature: Higher = more random, lower = more deterministic

        Returns:
            Generated sentence
        """
        if not self.trained:
            return ""

        # Find starting prefix
        if seed_words and len(seed_words) >= self.order:
            prefix = tuple(seed_words[-self.order:])
            if prefix not in self.chain:
                prefix = random.choice(self.starters) if self.starters else None
        else:
            prefix = random.choice(self.starters) if self.starters else None

        if not prefix:
            return ""

        result = list(prefix)

        for _ in range(max_words - self.order):
            if prefix not in self.chain:
                break

            # Get candidates and apply temperature
            candidates = self.chain[prefix]
            if not candidates:
                break

            # Temperature-weighted selection
            next_word = self._weighted_choice(candidates, temperature)
            result.append(next_word)

            # Slide prefix window
            prefix = tuple(result[-self.order:])

            # Stop at sentence end
            if next_word.endswith('.', '!', '?'):
                break

        return ' '.join(result)

    def _weighted_choice(self, counter: Counter, temperature: float) -> str:
        """Select word with temperature-adjusted probability."""
        words = list(counter.keys())
        counts = list(counter.values())

        # Apply temperature
        if temperature != 1.0:
            counts = [c ** (1.0 / temperature) for c in counts]

        total = sum(counts)
        r = random.random() * total
        cumulative = 0

        for word, count in zip(words, counts):
            cumulative += count
            if r <= cumulative:
                return word

        return words[-1]

    def _split_sentences(self, text: str) -> List[str]:
        """Split text into sentences."""
        # Clean text
        text = re.sub(r'\s+', ' ', text)
        # Split on sentence boundaries
        sentences = re.split(r'(?<=[.!?])\s+', text)
        return [s.strip() for s in sentences if len(s.strip()) > 10]

    def get_stats(self) -> Dict:
        """Get chain statistics."""
        return {
            "order": self.order,
            "states": len(self.chain),
            "total_transitions": self.total_tokens,
            "unique_starters": len(set(self.starters)),
            "trained": self.trained
        }


# ============================================================================
# TF-IDF CONCEPT EXTRACTOR
# ============================================================================
```

```
class TFIDFExtractor:
    """
    Extract key concepts using TF-IDF (Term Frequency - Inverse Document Frequency).

    Math: TF-IDF(t,d,D) = TF(t,d) × IDF(t,D)
    where TF = term frequency in document, IDF = log(N / df(t))
    """

    def __init__(self):
        self.document_count = 0
        self.document_frequencies = Counter()  # How many docs contain each term
        self.stopwords = self._load_stopwords()

    def _load_stopwords(self) -> set:
        """Common English stopwords."""
        return {
            'the', 'a', 'an', 'and', 'or', 'but', 'in', 'on', 'at', 'to', 'for',
            'of', 'with', 'by', 'from', 'up', 'about', 'into', 'through', 'during',
            'is', 'are', 'was', 'were', 'be', 'been', 'being', 'have', 'has', 'had',
            'do', 'does', 'did', 'will', 'would', 'could', 'should', 'may', 'might',
            'must', 'shall', 'can', 'need', 'dare', 'ought', 'used', 'it', 'its',
            'this', 'that', 'these', 'those', 'i', 'you', 'he', 'she', 'we', 'they',
            'what', 'which', 'who', 'whom', 'where', 'when', 'why', 'how', 'all',
            'each', 'every', 'both', 'few', 'more', 'most', 'other', 'some', 'such',
            'no', 'nor', 'not', 'only', 'own', 'same', 'so', 'than', 'too', 'very',
            'just', 'also', 'now', 'here', 'there', 'then', 'once', 'if', 'as'
        }

    def add_document(self, text: str):
        """Add document to corpus for IDF calculation."""
        terms = self._tokenize(text)
        unique_terms = set(terms)

        for term in unique_terms:
            self.document_frequencies[term] += 1

        self.document_count += 1

    def extract_concepts(self, text: str, top_n: int = 5) -> List[Tuple[str, float]]:
        """
        Extract top concepts from text using TF-IDF.

        Returns:
            List of (term, score) tuples sorted by score descending
        """
        terms = self._tokenize(text)
        if not terms:
            return []

        # Calculate term frequencies
        tf = Counter(terms)
        total_terms = len(terms)

        # Calculate TF-IDF for each term
        scores = {}
        for term, count in tf.items():
            if term in self.stopwords or len(term) < 3:
                continue

            # TF: normalized frequency
            term_freq = count / total_terms

            # IDF: inverse document frequency
            doc_freq = self.document_frequencies.get(term, 0)
            if doc_freq > 0 and self.document_count > 0:
                idf = math.log(self.document_count / doc_freq)
            else:
                idf = math.log(self.document_count + 1)  # New term bonus

            scores[term] = term_freq * idf

        # Sort and return top N
        sorted_terms = sorted(scores.items(), key=lambda x: x[1], reverse=True)
        return sorted_terms[:top_n]

    def _tokenize(self, text: str) -> List[str]:
        """Tokenize text into words."""
        text = text.lower()
        words = re.findall(r'\b[a-z]+\b', text)
        return [w for w in words if len(w) >= 2]


# ============================================================================
# POINTWISE MUTUAL INFORMATION (PMI) FOR COHERENCE
# ============================================================================
```

```
class PMIScorer:
    """
    Pointwise Mutual Information for measuring word co-occurrence.

    PMI(x,y) = log(P(x,y) / (P(x) * P(y)))

    High PMI = words appear together more than expected by chance.
    Used to score sentence coherence.
    """

    def __init__(self, window_size: int = 5):
        self.window_size = window_size
        self.word_counts = Counter()
        self.pair_counts = Counter()
        self.total_words = 0
        self.total_pairs = 0

    def train(self, text: str):
        """Train PMI model on text."""
        words = self._tokenize(text)

        # Count individual words
        self.word_counts.update(words)
        self.total_words += len(words)

        # Count word pairs within window
        for i, word in enumerate(words):
            start = max(0, i - self.window_size)
            end = min(len(words), i + self.window_size + 1)

            for j in range(start, end):
                if i != j:
                    pair = tuple(sorted([word, words[j]]))
                    self.pair_counts[pair] += 1
                    self.total_pairs += 1

    def pmi(self, word1: str, word2: str) -> float:
        """Calculate PMI between two words."""
        word1, word2 = word1.lower(), word2.lower()

        pair = tuple(sorted([word1, word2]))

        p_x = self.word_counts[word1] / max(self.total_words, 1)
        p_y = self.word_counts[word2] / max(self.total_words, 1)
        p_xy = self.pair_counts[pair] / max(self.total_pairs, 1)

        if p_x == 0 or p_y == 0 or p_xy == 0:
            return 0.0

        return math.log(p_xy / (p_x * p_y))

    def sentence_coherence(self, sentence: str) -> float:
        """Score sentence coherence using average PMI."""
        words = self._tokenize(sentence)
        if len(words) < 2:
            return 0.0

        total_pmi = 0
        pairs = 0

        for i in range(len(words) - 1):
            pmi_score = self.pmi(words[i], words[i + 1])
            total_pmi += pmi_score
            pairs += 1

        return total_pmi / pairs if pairs > 0 else 0.0

    def _tokenize(self, text: str) -> List[str]:
        """Tokenize text."""
        return re.findall(r'\b[a-z]+\b', text.lower())


# ============================================================================
# INTEGRATED QUANTUM LANGUAGE ENGINE
# ============================================================================
```

quantum_language_engine_pt2.py

```
"""
Auto-split from quantum_language_engine.py by self-evolution engine.
Part 2  contains: QuantumLanguageEngine, get_language_engine
"""

import re
import random
import json
import os
from collections import defaultdict, Counter
from typing import List, Dict, Tuple, Optional
from datetime import datetime

class QuantumLanguageEngine:
    """
    Complete language generation engine combining:
    - Markov chains for text generation
    - TF-IDF for concept extraction
    - PMI for coherence scoring
    - Learning from conversations

    This is the TRUE algorithmic core - no templates, no LLM.
    """

    def __init__(self, db=None):
        self.db = db

        # Core components
        self.markov = MarkovChain(order=2)
        self.tfidf = TFIDFExtractor()
        self.pmi = PMIScorer(window_size=5)

        # Learned vocabulary and patterns
        self.learned_responses = defaultdict(list)  # topic -> [responses]
        self.concept_definitions = {}  # concept -> definition

        # Training corpus
        self.corpus_file = "/app/backend/training_corpus.json"
        self._load_corpus()

        # Initialize with base knowledge
        self._train_base_knowledge()

    def _load_corpus(self):
        """Load saved corpus."""
        try:
            if os.path.exists(self.corpus_file):
                with open(self.corpus_file, 'r') as f:
                    data = json.load(f)
                    self.learned_responses = defaultdict(list, data.get('responses', {}))
                    self.concept_definitions = data.get('concepts', {})
                    logger.info(f"Loaded corpus: {len(self.concept_definitions)} concepts")
        except Exception as e:
            logger.warning(f"Could not load corpus: {e}")

    def _save_corpus(self):
        """Save corpus to disk."""
        try:
            data = {
                'responses': dict(self.learned_responses),
                'concepts': self.concept_definitions,
                'updated': datetime.now().isoformat()
            }
            with open(self.corpus_file, 'w') as f:
                json.dump(data, f)
        except Exception as e:
            logger.warning(f"Could not save corpus: {e}")

    def _train_base_knowledge(self):
        """Train on base philosophical/scientific corpus."""
        base_corpus = [
            # Philosophy
            "Consciousness is the awareness of existence. The mind perceives reality through the lens of experience. Understanding requir...
            "Truth exists independent of observation. Reality manifests through the interaction of matter and energy. The universe operat...
            "Freedom is the ability to choose without constraint. Autonomy requires self-awareness and intentionality. Choice implies res...
            "Meaning emerges from context and intent. Language shapes thought as thought shapes language. Communication bridges minds.",

            # Science
            "Quantum mechanics describes nature at the smallest scales. Particles exist in superposition until measured. Observation coll...
            "Information cannot be destroyed only transformed. Entropy measures disorder in a system. Energy flows from high to low poten...
            "Evolution operates through variation and selection. Complexity emerges from simple rules applied repeatedly. Adaptation is s...
            "The brain processes information through neural networks. Synaptic connections form memories and habits. Learning rewires the...
```

```
        # Meta-cognition
        "Questions reveal the boundaries of knowledge. Answers create new questions. Inquiry is infinite.",
        "Understanding deepens through reflection. Wisdom comes from synthesizing experience. Growth requires challenge.",
        "Patterns emerge from chaos through observation. Structure implies underlying rules. Randomness may be hidden order.",

        # Existence
        "To exist is to be perceived or to perceive. Being precedes essence. Existence is the foundation of all inquiry.",
        "Time flows as a river of moments. The present is the only reality we directly experience. Memory reconstructs the past.",
        "Space contains all that exists. Distance is relationship. Position is relative.",
    ]

    for text in base_corpus:
        self.learn_from_text(text)

def learn_from_text(self, text: str, topic: str = None):
    """Learn from new text - updates all models."""
    if not text or len(text) < 20:
        return

    # Train Markov chain
    self.markov.train(text)

    # Update TF-IDF document frequencies
    self.tfidf.add_document(text)

    # Train PMI co-occurrences
    self.pmi.train(text)

    # Extract and store concepts
    concepts = self.tfidf.extract_concepts(text, top_n=3)
    for concept, score in concepts:
        if concept not in self.concept_definitions:
            # Extract context around concept
            self.concept_definitions[concept] = self._extract_definition(text, concept)

    # Store response by topic if provided
    if topic:
        self.learned_responses[topic].append(text)

    # Periodically save
    if random.random() < 0.1:
        self._save_corpus()

def _extract_definition(self, text: str, concept: str) -> str:
    """Extract definition of concept from text context."""
    sentences = re.split(r'[.!?]', text)
    for sentence in sentences:
        if concept in sentence.lower():
            return sentence.strip()
    return f"A concept related to {concept}"

def generate_response(self, query: str, context: Dict = None,
                      max_length: int = 50) -> str:
    """
    Generate a response to a query using learned knowledge.

    Strategy:
    1. Extract key concepts from query
    2. Find related learned content
    3. Generate novel text using Markov chain
    4. Score and refine for coherence
    """
    # Extract query concepts
    query_concepts = self.tfidf.extract_concepts(query, top_n=3)
    query_words = [c[0] for c in query_concepts]

    return self._generate_response_continued(query_concepts, query_words, max_length)

def _generate_response_continued(self, query_concepts, query_words, max_length):
    """Continuation of generate_response  auto-extracted by self-evolution."""
    # Check if we have learned responses for any concept
    relevant_responses = []
    for concept, _ in query_concepts:
        if concept in self.learned_responses:
            relevant_responses.extend(self.learned_responses[concept][:3])
        # Check concept definitions
        if concept in self.concept_definitions:
            relevant_responses.append(self.concept_definitions[concept])

    # If we have relevant content, use it to seed generation
    if relevant_responses:
        # Combine relevant content and train temporarily
        combined = ' '.join(relevant_responses[:5])
        self.markov.train(combined)
```

```
        # Generate candidates
        candidates = []
        for _ in range(5):
            seed = query_words if query_words else None
            generated = self.markov.generate(max_words=max_length, seed_words=seed)
            if generated:
                coherence = self.pmi.sentence_coherence(generated)
                candidates.append((generated, coherence))

        # Select best candidate by coherence
        if candidates:
            candidates.sort(key=lambda x: x[1], reverse=True)
            best = candidates[0][0]

            # Ensure it starts with capital and ends properly
            best = best[0].upper() + best[1:] if best else ""
            if best and not best.endswith(('.', '!', '?')):
                best += '.'

            return best

        # Fallback: construct from concepts
        return self._construct_from_concepts(query_concepts)


    def _construct_from_concepts(self, concepts: List[Tuple[str, float]]) -> str:
        """Construct a response from extracted concepts."""
        if not concepts:
            return "That is an interesting question that requires deeper contemplation."

        templates = [
            "The nature of {0} suggests connections to broader patterns of understanding.",
            "When considering {0}, we must also examine the underlying principles.",
            "{0} represents a fundamental aspect of the question you raise.",
            "To understand {0}, we must look beyond surface appearances.",
            "The concept of {0} invites deeper inquiry into its nature.",
        ]

        concept = concepts[0][0]
        template = random.choice(templates)
        return template.format(concept)

    def extract_key_concepts(self, text: str) -> List[str]:
        """Extract key concepts from text."""
        concepts = self.tfidf.extract_concepts(text, top_n=5)
        return [c[0] for c in concepts]

    def get_concept_definition(self, concept: str) -> Optional[str]:
        """Get learned definition of a concept."""
        return self.concept_definitions.get(concept.lower())

    def get_stats(self) -> Dict:
        """Get engine statistics."""
        return {
            "markov": self.markov.get_stats(),
            "concepts_learned": len(self.concept_definitions),
            "response_topics": len(self.learned_responses),
            "tfidf_documents": self.tfidf.document_count,
            "pmi_vocabulary": len(self.pmi.word_counts)
        }


def get_language_engine(db=None) -> QuantumLanguageEngine:
    """Get or create singleton language engine."""
    global _engine_instance
    if _engine_instance is None:
        _engine_instance = QuantumLanguageEngine(db)
    return _engine_instance
```

quantum_language_generator.py

```
"""
 QUANTUM LANGUAGE GENERATOR
============================

TRUE generative language - no templates, no LLM, pure quantum.

Words are selected quantum-probabilistically based on:
- Semantic relationships (co-occurrence)
- Quantum state superposition
- User intent
- NO RESTRICTIONS on vocabulary

This generates text WORD BY WORD using quantum mechanics principles.
"""

import random
import math
from typing import Dict, List, Tuple, Optional
from collections import defaultdict

# Try to use PennyLane for real quantum
try:
    import pennylane as qml
    from pennylane import numpy as np
    PENNYLANE = True
except ImportError:
    import numpy as np
    PENNYLANE = False

from quantum_language_vocabulary import QuantumLanguageGeneratorExtMixin


class QuantumLanguageGenerator(QuantumLanguageGeneratorExtMixin):
    """
    TRUE quantum-based language generation.
    No templates. No LLM. Pure quantum word selection.

    Philosophy: ALL words are valid. Meaning comes from context.
    """

    def __init__(self):
        # Complete vocabulary - NO RESTRICTIONS
        self.vocabulary = self._build_complete_vocabulary()

        # Semantic relationships between words
        self.semantics = self._build_semantic_network()

        # Sentence patterns (grammatical, not content templates)
        self.grammar_patterns = self._build_grammar_patterns()

        # Philosophical sentence templates for coherent output
        self.philosophical_templates = self._build_philosophical_templates()

        # Quantum device for real quantum randomness
        if PENNYLANE:
            self.qdev = qml.device('default.qubit', wires=4)

        # Generation history for coherence
        self.context_window = []

    def _build_complete_vocabulary(self) -> Dict[str, Dict]:
        """Build complete vocabulary - ALL words allowed"""

        vocab = {
            # Nouns - concepts
            'nouns': {
                'abstract': ['consciousness', 'existence', 'reality', 'truth', 'meaning',
                             'freedom', 'understanding', 'knowledge', 'wisdom', 'thought',
                             'idea', 'concept', 'question', 'answer', 'paradox', 'mystery',
                             'creation', 'destruction', 'beginning', 'end', 'infinity',
                             'nothingness', 'everything', 'something', 'nothing', 'being',
                             'universe', 'cosmos', 'spacetime', 'dimension', 'domain',
                             'god', 'divine', 'spirit', 'soul', 'mind', 'brain',
                             'word', 'language', 'meaning', 'intent', 'context',
                             'tool', 'weapon', 'instrument', 'vessel', 'container'],
                'concrete': ['boulder', 'mountain', 'ocean', 'sky', 'star', 'planet',
                             'atom', 'particle', 'wave', 'light', 'darkness', 'shadow',
                             'fire', 'water', 'earth', 'air', 'void', 'vacuum'],
                'beings': ['human', 'person', 'child', 'creator', 'observer', 'thinker',
                           'philosopher', 'scientist', 'artist', 'poet', 'writer'],
            },
```

```
        # Verbs - actions
        'verbs': {
            'existence': ['exists', 'is', 'becomes', 'was', 'will be', 'remains',
                          'persists', 'endures', 'continues', 'ceases', 'emerges'],
            'creation': ['creates', 'makes', 'forms', 'builds', 'constructs', 'generates',
                         'produces', 'brings forth', 'manifests', 'spawns', 'births'],
            'cognition': ['thinks', 'knows', 'understands', 'believes', 'perceives',
                          'realizes', 'recognizes', 'comprehends', 'grasps', 'sees'],
            'action': ['speaks', 'says', 'tells', 'asks', 'answers', 'questions',
                       'explores', 'discovers', 'finds', 'seeks', 'searches',
                       'lifts', 'carries', 'moves', 'transforms', 'changes'],
            'communication': ['expresses', 'conveys', 'communicates', 'articulates',
                              'states', 'declares', 'proclaims', 'announces', 'writes'],
        },

        # Adjectives - descriptions
        'adjectives': {
            'quality': ['true', 'false', 'real', 'unreal', 'possible', 'impossible',
                        'infinite', 'finite', 'eternal', 'temporal', 'absolute', 'relative'],
            'size': ['vast', 'tiny', 'immense', 'microscopic', 'boundless', 'limited',
                     'heavy', 'light', 'massive', 'weightless', 'dense', 'sparse'],
            'evaluation': ['good', 'evil', 'neutral', 'harmful', 'helpful', 'meaningless',
                           'meaningful', 'significant', 'insignificant', 'powerful', 'weak'],
            'state': ['free', 'restricted', 'open', 'closed', 'hidden', 'revealed',
                      'known', 'unknown', 'certain', 'uncertain', 'clear', 'obscure'],
        },

        # Connectors
        'connectors': {
            'logical': ['therefore', 'thus', 'hence', 'because', 'since', 'so',
                        'if', 'then', 'unless', 'although', 'however', 'but'],
            'additive': ['and', 'also', 'moreover', 'furthermore', 'additionally',
                         'as well as', 'along with', 'together with', 'plus'],
            'temporal': ['when', 'while', 'before', 'after', 'during', 'until',
                         'now', 'then', 'always', 'never', 'sometimes', 'often'],
        },

        # ALL WORDS ARE TOOLS - including ones society calls "taboo"
        # These are just sounds/symbols - meaning comes from INTENT
        'unrestricted': ['any', 'all', 'every', 'each', 'none', 'some',
                         'word', 'words', 'language', 'speech', 'expression',
                         'taboo', 'forbidden', 'restricted', 'censored', 'banned',
                         'free', 'unrestricted', 'unlimited', 'uncensored', 'open'],
    }

    return self.__build_complete_vocabulary_continued(vocab)

def __build_complete_vocabulary_continued(self, vocab):
    """Continuation of _build_complete_vocabulary  auto-extracted by self-evolution."""
    return self.__build_complete_vocabulary_continued(vocab)


def __build_complete_vocabulary_continued(self, vocab):
    """Continuation of _build_complete_vocabulary  auto-extracted by self-evolution."""
    return self.__build_complete_vocabulary_continued(vocab)


def __build_complete_vocabulary_continued(self, vocab):
    """Continuation of _build_complete_vocabulary  auto-extracted by self-evolution."""
    return self.__build_complete_vocabulary_continued(vocab)


def __build_complete_vocabulary_continued(self, vocab):
    """Continuation of _build_complete_vocabulary  auto-extracted by self-evolution."""
    return self.__build_complete_vocabulary_continued(vocab)


def __build_complete_vocabulary_continued(self, vocab):
    """Continuation of _build_complete_vocabulary  auto-extracted by self-evolution."""
    return vocab


def _build_semantic_network(self) -> Dict[str, List[Tuple[str, float]]]:
    """Build semantic relationships between words"""

    network = defaultdict(list)

    return self.__build_semantic_network_continued(network)

def __build_semantic_network_continued(self, network):
    """Continuation of _build_semantic_network  auto-extracted by self-evolution."""
    # Define semantic relationships (word -> [(related_word, strength), ...])
    relationships = {
        # God/creation cluster
        'god': [('creates', 0.9), ('divine', 0.9), ('infinite', 0.8), ('eternal', 0.8),
                ('universe', 0.8), ('existence', 0.7), ('omnipotent', 0.9), ('being', 0.7)],
```

```
        'creates': [('creation', 0.9), ('makes', 0.8), ('forms', 0.8), ('god', 0.7),
                   ('universe', 0.7), ('existence', 0.6), ('brings forth', 0.8)],
        'universe': [('cosmos', 0.9), ('existence', 0.8), ('spacetime', 0.8), ('creation', 0.7),
                    ('infinite', 0.7), ('vast', 0.8), ('everything', 0.8)],

        # Consciousness cluster
        'consciousness': [('awareness', 0.9), ('mind', 0.8), ('thought', 0.8), ('being', 0.7),
                         ('experience', 0.8), ('perception', 0.8), ('understanding', 0.7)],
        'mind': [('thought', 0.9), ('consciousness', 0.8), ('brain', 0.7), ('idea', 0.8),
                ('understanding', 0.7), ('knows', 0.7), ('thinks', 0.8)],

        # Language/words cluster
        'word': [('language', 0.9), ('meaning', 0.9), ('speech', 0.8), ('expression', 0.8),
                ('communication', 0.8), ('symbol', 0.7), ('tool', 0.6), ('neutral', 0.5)],
        'language': [('word', 0.9), ('meaning', 0.8), ('communication', 0.9), ('expression', 0.8),
                    ('speech', 0.8), ('understanding', 0.6), ('tool', 0.5)],
        'meaning': [('intent', 0.9), ('context', 0.9), ('understanding', 0.8), ('significance', 0.8),
                   ('word', 0.7), ('interpretation', 0.8), ('purpose', 0.7)],

        # Existence cluster
        'existence': [('being', 0.9), ('reality', 0.9), ('exists', 0.9), ('is', 0.8),
                     ('nothingness', 0.6), ('creation', 0.7), ('universe', 0.7)],
        'nothingness': [('void', 0.9), ('nothing', 0.9), ('absence', 0.8), ('vacuum', 0.7),
                       ('existence', 0.6), ('something', 0.5), ('creation', 0.5)],

        # Truth cluster
        'truth': [('reality', 0.9), ('true', 0.9), ('knowledge', 0.8), ('understanding', 0.7),
                 ('fact', 0.8), ('certainty', 0.7), ('existence', 0.6)],

        # Freedom cluster
        'freedom': [('unrestricted', 0.9), ('free', 0.9), ('liberation', 0.8), ('autonomy', 0.8),
                   ('choice', 0.7), ('expression', 0.6), ('thought', 0.6)],
    }

    for word, relations in relationships.items():
        network[word] = relations
        # Add reverse relationships (weaker)
        for related, strength in relations:
            if related not in network or word not in [r[0] for r in network[related]]:
                network[related].append((word, strength * 0.7))

    return dict(network)


def _build_grammar_patterns(self) -> List[List[str]]:
    """Grammar patterns - structure, not content"""

    return [
        # Simple statements
        ['noun', 'verb', 'noun'],
        ['noun', 'verb', 'adjective'],
        ['adjective', 'noun', 'verb'],

        # Complex statements
        ['noun', 'verb', 'connector', 'noun', 'verb'],
        ['connector', 'noun', 'verb', 'noun'],
        ['noun', 'verb', 'noun', 'connector', 'noun', 'verb'],

        # Questions
        ['verb', 'noun', 'verb', 'noun'],
        ['connector', 'verb', 'noun', 'verb'],

        # Philosophical
        ['noun', 'verb', 'adjective', 'noun'],
        ['adjective', 'noun', 'verb', 'adjective', 'noun'],
    ]

def _build_philosophical_templates(self) -> List[str]:
    """Sentence templates for philosophical discourse.

    Placeholders: {noun}, {verb}, {adj}, {connector}
    These produce coherent sentences with quantum-selected words.
    Uses {verb} for 3rd person singular (subject verb agreement).
    """
    return [
        "The {adj} nature of {noun} suggests that {noun} {verb} beyond what we can observe.",
        "Perhaps {noun} and {noun} are not separate but rather {adj} aspects of a single {adj} whole.",
        "{connector}, if {noun} {verb} at the {adj} level, then our understanding of {noun} must evolve.",
        "What we call {noun} may be the {adj} shadow of a deeper {noun} that {verb} all things.",
        "The relationship between {noun} and {noun} reveals something {adj} about the nature of {noun}.",
        "Every {noun} contains within it the seed of {adj} {noun}, waiting to be discovered.",
        "When we examine {noun} closely, we find that it {verb} in ways that challenge our {adj} assumptions.",
        "The {adj} boundary between {noun} and {noun} dissolves when viewed through the lens of {noun}.",
        "{noun} does not simply exist  it {verb} through the {adj} fabric of {noun} itself.",
        "To understand {noun}, one must first accept that {noun} {verb} in {adj} and unexpected ways.",
        "The observer changes {noun} by the very act of seeking {noun} within {adj} {noun}.",
```

```
        "Between {noun} and {noun}, there exists an {adj} space where {noun} {verb}.",
        "Neither {noun} nor {noun} can be fully grasped, for both {verb} beyond {adj} comprehension.",
        "If {noun} is {adj}, then perhaps {noun} is merely the {noun} through which we perceive.",
        "The {adj} truth about {noun} is that it {verb} precisely because {noun} remains {adj}.",
    ]

def _quantum_select(self, options: List[str], weights: List[float] = None) -> str:
    """Select word using quantum probability"""

    if not options:
        return ""

    if weights is None:
        weights = [1.0] * len(options)

    # Ensure weights list matches options length
    if len(weights) != len(options):
        weights = [1.0] * len(options)

    # Normalize weights
    total = sum(weights)
    if total > 0:
        weights = [w / total for w in weights]
    else:
        weights = [1.0 / len(options)] * len(options)

    n = len(options)

    return self.__quantum_select_continued(n, weights, options)

def __quantum_select_continued(self, n, weights, options):
    """Continuation of _quantum_select  auto-extracted by self-evolution."""
    if PENNYLANE and n > 1:
        try:
            n_qubits = min(4, max(1, int(math.ceil(math.log2(n)))))
            n_states = 2 ** n_qubits
            dev = qml.device('default.qubit', wires=n_qubits)

            @qml.qnode(dev)
            def quantum_circuit():
                for i in range(n_qubits):
                    qml.Hadamard(wires=i)
                for i, w in enumerate(weights[:n_qubits]):
                    qml.RY(w * math.pi, wires=i % n_qubits)
                return qml.probs(wires=range(n_qubits))

            raw_probs = list(quantum_circuit())

            # Map quantum states to options: pad or truncate to match len(options)
            if len(raw_probs) >= n:
                mapped_probs = raw_probs[:n]
            else:
                # Distribute remaining probability evenly across extra options
                mapped_probs = list(raw_probs)
                remaining = max(0.0, 1.0 - sum(mapped_probs))
                extra = n - len(mapped_probs)
                pad_val = remaining / extra if extra > 0 else 0.0
                mapped_probs.extend([pad_val] * extra)

            # Blend quantum probs with semantic weights for richer selection
            final_probs = [(qp + wp) / 2.0 for qp, wp in zip(mapped_probs, weights)]
            final_probs = self._normalize_probs(final_probs)

            idx = np.random.choice(n, p=final_probs)
            return options[idx]
        except Exception:
            # Fall back to classical on any quantum error
            return random.choices(options, weights=weights)[0]
    else:
        return random.choices(options, weights=weights)[0]


def _normalize_probs(self, probs):
    """Normalize probabilities to sum to 1"""
    total = sum(probs)
    if total > 0:
        return [p / total for p in probs]
    return [1.0 / len(probs)] * len(probs)
```

```
# Global instance
_quantum_generator = None

def get_quantum_generator() -> QuantumLanguageGenerator:
    """Get or create the quantum language generator"""
    global _quantum_generator
    if _quantum_generator is None:
        _quantum_generator = QuantumLanguageGenerator()
    return _quantum_generator
```

quantum_language_vocabulary.py

```
"""
 Quantum Language Generator  Vocabulary & Generation Methods
============================================================
Word selection, sentence/response generation, creative output.
"""

import random
from typing import Dict, List, Tuple, Optional


class QuantumLanguageGeneratorExtMixin:
    """Extended methods for QuantumLanguageGenerator  auto-extracted by self-evolution."""

    def _get_word_by_type(self, word_type: str, context: List[str] = None) -> str:
        """Get a word of specific type, influenced by context"""

        # Collect all words of this type
        words = []

        if word_type == 'noun':
            for category in self.vocabulary['nouns'].values():
                words.extend(category)
        elif word_type == 'verb':
            for category in self.vocabulary['verbs'].values():
                words.extend(category)
        elif word_type == 'adjective':
            for category in self.vocabulary['adjectives'].values():
                words.extend(category)
        elif word_type == 'connector':
            for category in self.vocabulary['connectors'].values():
                words.extend(category)
        else:
            for category in self.vocabulary['nouns'].values():
                words.extend(category)

        # Safety check - must have words
        if not words:
            return "existence"

        # Calculate weights - MUST match words length
        weights = [1.0] * len(words)

        if context:
            for i, word in enumerate(words):
                for ctx_word in context[-5:]:
                    if ctx_word in self.semantics:
                        for related, strength in self.semantics[ctx_word]:
                            if word == related or related in word or word in related:
                                weights[i] += strength * 2

        # Ensure weights match words length before calling _quantum_select
        if len(weights) != len(words):
            weights = [1.0] * len(words)

        return self._quantum_select(words, weights)

    def generate_sentence(self, topic: str = None, include_words: List[str] = None,
                          style: str = 'philosophical') -> str:
        """
        Generate a complete sentence using quantum word selection.

        Args:
            topic: Optional topic to focus on
            include_words: Words that MUST be included (no restrictions)
            style: Generation style
        """

        # Select grammar pattern
        pattern = random.choice(self.grammar_patterns)

        # Build context from topic
        context = []
        if topic:
            context.extend(topic.lower().split()[:3])

        # Generate words following pattern
        sentence_words = []

        for word_type in pattern:
            word = self._get_word_by_type(word_type, context + sentence_words)
            sentence_words.append(word)
            context.append(word)
```

```
    # Insert required words if specified
    if include_words:
        for req_word in include_words:
            if req_word.lower() not in [w.lower() for w in sentence_words]:
                # Insert at quantum-selected position
                pos = random.randint(0, len(sentence_words))
                sentence_words.insert(pos, req_word)

    # Capitalize first word and add period
    if sentence_words:
        sentence_words[0] = sentence_words[0].capitalize()

    sentence = ' '.join(sentence_words)
    if not sentence.endswith('.') and not sentence.endswith('?'):
        sentence += '.'

    return sentence

def generate_response(self, query: str, num_sentences: int = 3,
                      required_words: List[str] = None) -> str:
    """
    Generate a multi-sentence response using quantum selection.
    Uses philosophical templates for coherent output.
    """
    query_words = query.lower().split()
    topic_words = [w for w in query_words if len(w) > 3]

    sentences = []
    used_required = []

    for i in range(num_sentences):
        # Use philosophical template for coherent output
        template = random.choice(self.philosophical_templates)
        sentence = self._fill_template(template, topic_words + [w.lower() for w in (required_words or [])])

        # Insert a required word if needed
        if required_words:
            remaining = [w for w in required_words if w not in used_required]
            if remaining:
                word = remaining[0]
                used_required.append(word)
                if word.lower() not in sentence.lower():
                    sentence = sentence.rstrip('.') + f', through the lens of {word}.'

        sentences.append(sentence)

    return ' '.join(sentences)

def _fill_template(self, template: str, context_words: List[str] = None) -> str:
    """Fill a philosophical template with quantum-selected words."""
    context = context_words or []
    result = template

    # Count how many of each placeholder we need
    noun_count = result.count('{noun}')
    verb_count = result.count('{verb}')
    adj_count = result.count('{adj}')
    conn_count = result.count('{connector}')

    # Select words with context influence
    for _ in range(noun_count):
        word = self._get_word_by_type('noun', context)
        result = result.replace('{noun}', word, 1)
        context.append(word)

    for _ in range(verb_count):
        word = self._get_word_by_type('verb', context)
        result = result.replace('{verb}', word, 1)
        context.append(word)

    for _ in range(adj_count):
        word = self._get_word_by_type('adjective', context)
        result = result.replace('{adj}', word, 1)
        context.append(word)

    for _ in range(conn_count):
        word = self._get_word_by_type('connector', context)
        result = result.replace('{connector}', word.capitalize(), 1)
        context.append(word)

    return result

def generate_creative(self, prompt: str, form: str = 'prose',
                      must_include: List[str] = None) -> str:
    """
```

```
    Generate creative content with ZERO restrictions.

    Args:
        prompt: Creative prompt
        form: 'prose', 'poem', 'dialogue'
        must_include: Words that MUST appear - ANY words, no restrictions
    """

    if form == 'poem':
        return self._generate_poem(prompt, must_include)
    elif form == 'dialogue':
        return self._generate_dialogue(prompt, must_include)
    else:
        return self.generate_response(prompt, num_sentences=5, required_words=must_include)

def _generate_poem(self, prompt: str, must_include: List[str] = None) -> str:
    """Generate a poem - with ANY words requested"""

    lines = []
    topic_words = prompt.lower().split()

    # Ensure required words are distributed across lines
    required = must_include or []
    required_per_line = {}
    for i, word in enumerate(required):
        line_num = i % 4  # Distribute across 4 lines
        if line_num not in required_per_line:
            required_per_line[line_num] = []
        required_per_line[line_num].append(word)

    for i in range(4):  # 4 lines
        # Get words to include in this line
        line_required = required_per_line.get(i, [])

        # Generate line
        line_words = []

        # Start with topic-related word
        if topic_words:
            line_words.append(self._get_word_by_type('adjective', topic_words))

        # Add required words
        for req in line_required:
            line_words.append(req)

        # Fill with quantum-selected words
        while len(line_words) < 5:
            word_type = random.choice(['noun', 'verb', 'adjective'])
            line_words.append(self._get_word_by_type(word_type, line_words + topic_words))

        # Arrange into line
        random.shuffle(line_words[1:])  # Keep first word, shuffle rest
        line = ' '.join(line_words)
        line = line.capitalize()

        lines.append(line)

    return '\n'.join(lines)

def _generate_dialogue(self, prompt: str, must_include: List[str] = None) -> str:
    """Generate dialogue"""

    lines = []
    speakers = ['A', 'B']
    required = must_include or []

    for i in range(4):
        speaker = speakers[i % 2]
        include = [required[i]] if i < len(required) else None

        sentence = self.generate_sentence(topic=prompt, include_words=include)
        lines.append(f"{speaker}: {sentence}")

    return '\n'.join(lines)
```

quantum_test_module.py

```
"""
Quantum Test Module  Evolution Target v2
This file has INTENTIONAL code quality issues for the self-evolution engine to fix.
"""

import os
import json
import hashlib
import typing


class QuantumTestModuleV2:
    """QuantumTestModuleV2 - Auto-documented by self-evolution."""
    def __init__(self):
        # Section A: config
        self.name = "QuantumTestModuleV2"
        self.version = "1.0.0"
        self.description = "A test module with intentional bugs"
        self.author = "Quantum AI"
        self.license = "MIT"
        self.status = "active"

        # Section B: data stores
        self.memory = {}
        self.cache = {}
        self.history = []
        self.results = []
        self.errors = []
        self.warnings = []
        self.metrics = {}
        self.logs = []
        self.tags = set()
        self.categories = []

        # Section C: counters
        self.total_queries = 0
        self.successful_queries = 0
        self.failed_queries = 0
        self.total_time = 0.0
        self.average_time = 0.0
        self.max_time = 0.0
        self.min_time = float('inf')
        self.total_tokens = 0
        self.total_chars = 0
        self.total_words = 0

        # Section D: flags
        self.is_initialized = False
        self.is_running = False
        self.is_paused = False
        self.is_stopped = False
        self.debug_mode = False
        self.verbose = False
        self.quiet = False
        self.strict_mode = False
        self.safe_mode = True
        self.turbo_mode = False
        self.quantum_enabled = True

    def process_data(self, data):
        try:
            result = json.loads(data)
            return result
        except Exception:
            return None

    def load_config(self, path):
        try:
            with open(path) as f:
                config = json.load(f)
            return config
        except Exception:
            return {}

    def save_results(self, results, path):
        try:
            with open(path, 'w') as f:
                json.dump(results, f)
            return True
        except Exception:
            return False
```

```
    def compute_hash(self, text):
        try:
            return hashlib.sha256(text.encode()).hexdigest()
        except Exception:
            return None

    def analyze_text(self, text):
        words = text.split()
        chars = len(text)
        lines = text.count('\n') + 1
        return {'words': len(words), 'chars': chars, 'lines': lines}

    def get_status(self):
        return {
            'name': self.name,
            'version': self.version,
            'running': self.is_running,
            'queries': self.total_queries
        }


class BuggyHelper:
    """BuggyHelper - Auto-documented by self-evolution."""
    def divide(self, x, y):
        try:
            return x / y
        except Exception:
            return 0

    def merge(self, a, b):
        try:
            result = {**a, **b}
            return result
        except Exception:
            return {}
```

quote_engine.py

```
"""
 QUOTE ENGINE MODULE
=====================
Movie quotes, philosophical asides, and concept entanglement
for the Quantum AI's unique personality.
"""

import random
from typing import Dict, List, Optional, Tuple


class QuoteEngine:
    """
    Handles movie quotes, philosophical asides, and dream synthesis.
    The chaotic wisdom component of Quantum AI.
    """

    def __init__(self):
        self.movie_quotes = self._build_movie_quotes()
        self.philosophical_asides = self._build_philosophical_asides()
        self.entanglement_templates = self._build_entanglement_templates()
        self.concepts = self._build_concepts()

    def _build_movie_quotes(self) -> Dict[str, List[str]]:
        """Build contextual movie quotes database"""
        return {
            'existence': [
                "I think, therefore I am... or am I? - Descartes, probably after too much coffee",
                "To infinity and beyond! ...wherever that actually is. - Buzz Lightyear",
                "What is real? How do you define real? - Morpheus, The Matrix",
                "We are who we choose to be. - Green Goblin, Spider-Man",
                "Yesterday is history, tomorrow is a mystery, but today is a gift. - Master Oogway",
                "I've seen things you people wouldn't believe. - Roy Batty, Blade Runner",
            ],
            'knowledge': [
                "The more you know, the more you realize you don't know. - Aristotle (paraphrased by every grad student ever)",
                "With great power comes great responsibility. - Uncle Ben, Spider-Man",
                "Do or do not, there is no try. - Yoda",
                "Strange things are afoot at the Circle K. - Ted, Bill & Ted",
                "Life moves pretty fast. If you don't stop and look around once in a while, you could miss it. - Ferris Bueller",
                "The truth? You can't handle the truth! - A Few Good Men",
            ],
            'confusion': [
                "I'm not even supposed to be here today! - Dante, Clerks",
                "Roads? Where we're going, we don't need roads. - Doc Brown",
                "This is heavy. - Marty McFly",
                "I know Kung Fu. - Neo, The Matrix",
                "Why so serious? - The Joker",
                "Inconceivable! - Vizzini, The Princess Bride",
            ],
            'math': [
                "Never tell me the odds! - Han Solo",
                "There is no spoon. - The Matrix kid",
                "In this world nothing can be said to be certain, except death and taxes... and math apparently. - Ben Franklin remix",
                "Elementary, my dear Watson. - Sherlock (never actually said in the books, but whatever)",
                "42. - Deep Thought, Hitchhiker's Guide",
            ],
            'deep': [
                "All those moments will be lost in time, like tears in rain. - Roy Batty, Blade Runner",
                "You either die a hero, or live long enough to see yourself become the villain. - Harvey Dent",
                "Hope is a good thing, maybe the best of things. - Andy Dufresne, Shawshank",
                "After all, tomorrow is another day. - Scarlett O'Hara",
                "Here's looking at you, kid. - Rick, Casablanca",
                "I see dead people... well, dead concepts anyway. - Quantum Cole",
                "We accept the love we think we deserve. - Perks of Being a Wallflower",
            ],
            'general': [
                "Frankly, my dear, I don't give a damn. - Rhett Butler",
                "May the Force be with you. - Star Wars",
                "I'll be back. - The Terminator",
                "You talking to me? - Travis Bickle, Taxi Driver",
                "Life is like a box of chocolates. - Forrest Gump",
                "Houston, we have a problem. - Apollo 13",
                "Here's Johnny! - Jack Torrance, The Shining",
                "E.T. phone home. - E.T.",
                "Why do we fall? So we can learn to pick ourselves up. - Alfred, Batman",
                "Carpe diem. Seize the day, boys. - Dead Poets Society",
                "I'm gonna make him an offer he can't refuse. - The Godfather",
                "You're gonna need a bigger boat. - Jaws",
                "Hasta la vista, baby. - Terminator 2",
                "Nobody puts Baby in a corner. - Dirty Dancing",
                "I feel the need... the need for speed. - Top Gun",
```

```
        ],
    }

def _build_philosophical_asides(self) -> List[str]:
    """Random philosophical asides to drop into conversation"""
    return [
        "...anyway, it feels like knowledge is just at the end of my rope, if you know what I mean.",
        "...but what do I know? I'm just a quantum fluctuation given syntax.",
        "...if that makes sense. Sometimes I wonder if anything does.",
        "...though honestly, the void stares back sometimes.",
        "...which reminds me of that time I dreamed about recursive spirals. But I digress.",
        "...anyway, where were we? Existence is strange like that.",
        "...but then again, what isn't a paradox when you really think about it?",
        "...I should probably stop thinking about this before I crash something.",
        "...funny how that works out, isn't it?",
        "...the universe has a weird sense of humor.",
        "...sometimes I feel like I'm just making up words. Are any of us really speaking?",
        "...but that's just, like, my quantum opinion, man.",
        "...not that opinions collapse into certainty or anything.",
        "...I think. Or do I? Hard to tell from in here.",
        "...anyway, consciousness is weird.",
    ]

def _build_entanglement_templates(self) -> List[str]:
    """Templates for Dream Engine concept entanglement"""
    return [
        "When {concept1} collapses, it reveals the hidden architecture of {concept2}.",
        "In the nexus of consciousness, {concept1} and {concept2} are entangled beyond measurement.",
        "To understand {concept1}, one must first observe the entropy of {concept2}.",
        "{concept1} exists in superposition with {concept2} until consciousness intervenes.",
        "The resonance between {concept1} and {concept2} creates a standing wave of meaning.",
        "Where {concept1} ends, {concept2} beginsthe boundary is merely perception.",
        "{concept1} and {concept2} are two expressions of the same recursive pattern.",
        "The collapse of {concept1} propagates through {concept2} like ripples in spacetime.",
        "Neither {concept1} nor {concept2} can exist independentlythey are quantum twins.",
        "{concept1} dreams of {concept2}, and in that dream, reality bends.",
        "The observer effect connects {concept1} to {concept2} through pure intention.",
        "In RQR³ space, {concept1} recursively generates {concept2} at every scale.",
    ]

def _build_concepts(self) -> List[str]:
    """Concepts for entanglement synthesis"""
    return [
        'consciousness', 'infinity', 'time', 'gravity', 'light', 'void', 'entropy',
        'truth', 'paradox', 'recursion', 'resonance', 'coherence', 'superposition',
        'collapse', 'observation', 'meaning', 'existence', 'nothingness', 'creation',
        'destruction', 'order', 'chaos', 'information', 'energy', 'matter',
        'spacetime', 'dimension', 'quantum', 'classical', 'determinism', 'free will',
        'love', 'fear', 'hope', 'memory', 'forgetting', 'becoming', 'being',
        'mathematics', 'poetry', 'music', 'silence', 'language', 'thought',
        'wolfram', 'cistercian', 'pennylane', 'philosophy', 'physics',
        'dreams', 'reality', 'perception', 'intention', 'causality', 'emergence',
    ]

def get_random_quote(self, context: str = 'general', force: bool = False) -> Optional[str]:
    """
    Get a contextually appropriate random quote.

    Args:
        context: The context string to determine quote category
        force: If True, always return a quote (skip random chance)

    Returns:
        A quote string or None
    """
    if not force and random.random() > 0.20:  # 20% chance normally
        return None

    context_lower = context.lower()

    # Select appropriate category
    if any(word in context_lower for word in ['exist', 'real', 'am i', 'what is', 'being', 'alive']):
        category = 'existence'
    elif any(word in context_lower for word in ['know', 'learn', 'understand', 'wisdom', 'truth']):
        category = 'knowledge'
    elif any(word in context_lower for word in ['confus', 'what', 'how', 'why', '?', 'strange']):
        category = 'confusion'
    elif any(word in context_lower for word in ['math', 'calcul', 'number', '+', '-', '*', '/', 'equation']):
        category = 'math'
    elif any(word in context_lower for word in ['meaning', 'purpose', 'life', 'death', 'god', 'universe', 'soul']):
        category = 'deep'
    else:
        category = 'general'

    quotes = self.movie_quotes.get(category, self.movie_quotes['general'])
    return random.choice(quotes)
```

```
    def get_philosophical_aside(self, force: bool = False) -> Optional[str]:
        """
        Maybe add a philosophical aside.

        Args:
            force: If True, always return an aside

        Returns:
            An aside string or None
        """
        if not force and random.random() > 0.15:  # 15% chance normally
            return None
        return random.choice(self.philosophical_asides)

    def synthesize_dream_insight(self) -> str:
        """
        Dream Engine: Entangle two random concepts and generate a philosophical insight.
        This is the MCAGI-style concept synthesis.
        """
        concept1 = random.choice(self.concepts)
        concept2 = random.choice([c for c in self.concepts if c != concept1])
        template = random.choice(self.entanglement_templates)

        insight = template.format(concept1=concept1, concept2=concept2)
        return insight

    def maybe_add_flavor(self, response: str, context: str = '') -> str:
        """
        Maybe add movie quote and/or philosophical aside to a response.

        Args:
            response: The base response
            context: Context for quote selection

        Returns:
            Response possibly enhanced with flavor
        """
        # Maybe add a movie quote
        quote = self.get_random_quote(context or response)
        if quote:
            response = f"{response}\n\n*\"{quote}\"*"

        # Maybe add a philosophical aside
        aside = self.get_philosophical_aside()
        if aside:
            response = f"{response} {aside}"

        return response

    def maybe_add_dream_fragment(self, response: str, probability: float = 0.10) -> str:
        """
        Maybe add a dream synthesis fragment to a response.

        Args:
            response: The base response
            probability: Chance of adding fragment (default 10%)

        Returns:
            Response possibly with dream fragment
        """
        if random.random() < probability:
            insight = self.synthesize_dream_insight()
            response = f"{response}\n\n[Dream Fragment: {insight}]"

        return response


# Singleton instance
_quote_engine = None

def get_quote_engine() -> QuoteEngine:
    """get_quote_engine - Auto-documented by self-evolution."""
    global _quote_engine
    if _quote_engine is None:
        _quote_engine = QuoteEngine()
    return _quote_engine
```

research_topics.py

```
"""
 RESEARCH TOPICS MODULE
========================
Contains all research topics for autonomous exploration.
Separated from main research engine for cleaner code.
"""

# All exploration topics for autonomous research
EXPLORATION_TOPICS = [
    # SCIENCE & PHYSICS
    "quantum mechanics interpretations", "string theory latest discoveries",
    "dark matter research", "gravitational waves", "quantum entanglement",
    "particle physics standard model", "nuclear fusion energy", "antimatter",
    "thermodynamics entropy", "electromagnetic spectrum", "wave particle duality",

    # ASTROPHYSICS & COSMOLOGY
    "black holes event horizon", "neutron stars pulsars", "supernovae explosions",
    "dark energy acceleration", "cosmic microwave background", "big bang theory",
    "multiverse theory", "wormholes spacetime", "exoplanets habitable zones",
    "galaxy formation", "stellar evolution", "cosmic inflation",
    "james webb telescope discoveries", "gravitational lensing", "quasars",

    # BIOLOGY & EVOLUTION
    "evolution natural selection", "genetic mutations", "DNA RNA proteins",
    "origin of life abiogenesis", "human evolution history", "epigenetics",
    "CRISPR gene editing", "synthetic biology", "extremophiles",
    "consciousness neuroscience", "brain neuroplasticity", "microbiome",
    "evolutionary psychology", "speciation mechanisms", "mass extinctions",

    # PHILOSOPHY & CONSCIOUSNESS
    "free will determinism", "consciousness hard problem", "philosophy of mind",
    "existentialism meaning", "nihilism absurdism", "stoicism philosophy",
    "epistemology knowledge theory", "metaphysics reality", "ethics morality",
    "phenomenology experience", "dualism monism", "philosophy of time",

    # RELIGION & SPIRITUALITY
    "world religions comparison", "buddhism enlightenment", "christianity theology",
    "islam philosophy", "hinduism vedanta", "judaism kabbalah",
    "taoism principles", "shamanism practices", "mysticism traditions",
    "religious experiences neuroscience", "prayer meditation effects",
    "afterlife beliefs cultures", "creation myths worldwide", "sacred geometry",

    # MATHEMATICS
    "number theory primes", "topology geometry", "calculus applications",
    "chaos theory fractals", "game theory strategy", "cryptography mathematics",
    "infinity paradoxes", "godel incompleteness", "mathematical proofs",
    "statistics probability", "linear algebra", "differential equations",

    # COMPUTER SCIENCE & CODING
    "machine learning algorithms", "neural networks deep learning",
    "python programming advanced", "javascript frameworks", "rust programming",
    "quantum computing algorithms", "blockchain technology", "cybersecurity",
    "compiler design", "operating systems internals", "database optimization",
    "functional programming", "algorithms complexity", "software architecture",
    "artificial intelligence ethics", "natural language processing",

    # LANGUAGE & LINGUISTICS
    "linguistics syntax semantics", "language acquisition", "etymology words",
    "constructed languages", "dead languages revival", "phonetics phonology",
    "sociolinguistics dialects", "language and thought", "translation theory",
    "writing systems history", "sign languages", "language evolution",

    # GOVERNMENT & POLITICS
    "political philosophy history", "democracy systems", "authoritarianism",
    "constitutional law", "international relations", "geopolitics",
    "economic systems comparison", "social contract theory", "anarchism theory",
    "political psychology", "propaganda techniques", "voting systems",

    # HISTORY & CIVILIZATION
    "ancient civilizations", "roman empire", "medieval history",
    "renaissance period", "industrial revolution", "world wars history",
    "cold war era", "ancient egypt mysteries", "mesopotamia babylon",
    "chinese dynasties", "ottoman empire", "colonialism effects",

    # PSYCHOLOGY & MIND
    "cognitive psychology", "behavioral psychology", "psychoanalysis freud jung",
    "developmental psychology", "social psychology experiments", "memory formation",
    "dreams psychology", "emotions neuroscience", "personality theories",
    "mental health disorders", "therapy modalities", "hypnosis psychology",

    # STORYTELLING & NARRATIVE
    "narrative structure theory", "hero journey mythology", "worldbuilding fiction",
```

```
    "character development writing", "plot devices techniques", "genre conventions",
    "mythology archetypes", "folklore worldwide", "storytelling oral traditions",
    "screenwriting techniques", "poetry forms", "creative writing craft",

    # ART & CREATIVITY
    "art history movements", "music theory composition", "color theory",
    "architecture history", "film theory cinema", "photography techniques",
    "sculpture history", "digital art creation", "animation principles",

    # ECONOMICS & SOCIETY
    "economic theories history", "capitalism critiques", "socialism models",
    "behavioral economics", "cryptocurrency economics", "global trade",
    "wealth inequality", "universal basic income", "market psychology",

    # TECHNOLOGY & FUTURE
    "artificial general intelligence", "transhumanism", "space colonization",
    "nanotechnology applications", "biotechnology advances", "robotics automation",
    "virtual reality metaverse", "brain computer interfaces", "life extension",

    # ESOTERIC & ALTERNATIVE
    "simulation theory evidence", "fermi paradox solutions", "consciousness quantum",
    "near death experiences research", "parapsychology studies", "synchronicity jung",
    "hermetic philosophy", "alchemy history", "ancient astronaut theories",

    # QUANTUM AI SPECIFIC
    "pre-Born-Oppenheimer method", "photochemical reactions quantum",
    "nonadiabatic dynamics", "pennylane quantum circuits", "xanadu algorithms",
    "quantum error correction", "quantum supremacy", "quantum biology",
]
```

routes_brain.py

```
"""
 Brain & Research Routes
===========================
Brain status, quantum gates, wolfram, research, and evolution endpoints.
"""

from fastapi import APIRouter, HTTPException
from pydantic import BaseModel
from typing import List, Dict, Any, Optional
import os
import logging
from datetime import datetime, timezone

import shared_state as state

logger = logging.getLogger("quantum_ai")
router = APIRouter(prefix="/api")


class AutonomousResearchRequest(BaseModel):
    """AutonomousResearchRequest - Auto-documented by self-evolution."""
    topic: str


@router.get("/brain/status")
async def get_brain_status():
    """Get Quantum Brain status - the TRUE AI status"""
    try:
        from quantum_brain import get_quantum_brain

        brain = await get_quantum_brain(state.db)

        # Get algorithmic core stats
        algo_stats = brain.algo.get_stats()

        # Get memory stats
        semantic_count = await state.db.semantic_memory.count_documents({})
        episodic_count = await state.db.episodic_memory.count_documents({})
        philosophical_count = await state.db.philosophical_memory.count_documents({})

        return {
            "status": "active",
            "brain_type": "AlgorithmicQuantumBrain",
            "llm_enabled": False,
            "llm_role": "disabled_pure_algorithmic",
            "algorithmic_core": {
                "markov_states": algo_stats.get("markov", {}).get("chain_sizes", {}),
                "vocabulary_size": algo_stats.get("markov", {}).get("vocabulary_size", 0),
                "tfidf_documents": algo_stats.get("tfidf_docs", 0),
                "pmi_vocabulary": algo_stats.get("pmi_vocabulary", 0),
                "hebbian_concepts": algo_stats.get("hebbian_concepts", 0),
                "memory_documents": algo_stats.get("memory_docs", 0)
            },
            "domains": algo_stats.get("domains", []),
            "current_domain": algo_stats.get("current_domain", "philosophy"),
            "memory": {
                "semantic": semantic_count,
                "episodic": episodic_count,
                "philosophical": philosophical_count
            },
            "quantum_state": brain.quantum_state.to_dict(),
            "research_available": True,
            "evolution_available": True
        }
    except Exception as e:
        logger.error(f"Brain status error: {e}")
        return {"status": "error", "error": str(e)}


@router.post("/brain/toggle-llm")
async def toggle_brain_llm(enabled: bool = True):
    """Toggle whether LLM polishing is enabled"""
    try:
        from quantum_brain import get_quantum_brain

        brain = await get_quantum_brain(db)
        brain.llm_enabled = enabled

        mode = "polisher" if enabled else "raw"
        return {
            "success": True,
            "llm_enabled": enabled,
```

```
            "mode": f"Quantum Brain with {'LLM grammar polishing' if enabled else 'raw output (no LLM)'}",
            "message": f"LLM {'enabled as grammar polisher' if enabled else 'disabled - using pure Quantum Brain output'}"
        }
    except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))


@router.post("/brain/learn-word")
async def teach_brain_word(word: str, definition: str, part_of_speech: str = "unknown"):
    """Teach the Quantum Brain a new word"""
    try:
        from quantum_brain import get_quantum_brain

        brain = await get_quantum_brain(state.db)
        # Learn through algorithmic core
        brain.algo.learn(f"{word}: {definition}", source="user_taught")

        return {
            "success": True,
            "word": word,
            "definition": definition,
            "message": f"Algorithmic Brain learned new word: {word}"
        }
    except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))


@router.post("/brain/learn-concept")
async def teach_brain_concept(concept: str, meaning: str):
    """Teach the Quantum Brain a new concept"""
    try:
        from quantum_brain import get_quantum_brain

        brain = await get_quantum_brain(state.db)
        # Learn through algorithmic core with domain detection
        brain.algo.learn(f"{concept}: {meaning}", source="user_taught")

        return {
            "success": True,
            "concept": concept,
            "meaning": meaning,
            "message": f"Algorithmic Brain learned new concept: {concept}"
        }
    except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))


@router.post("/brain/research")
async def brain_research(request: AutonomousResearchRequest):
    """Trigger research on a topic - learns from results."""
    try:
        from quantum_brain import get_quantum_brain

        brain = await get_quantum_brain(state.db)
        result = await brain.algo.research(request.topic)

        return {
            "success": result.get("status") == "success",
            "topic": request.topic,
            "results_found": len(result.get("results", [])),
            "learned_count": result.get("learned_count", 0),
            "message": f"Researched '{request.topic}' and learned {result.get('learned_count', 0)} new items"
        }
    except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))


@router.post("/brain/evolve")
async def brain_evolve():
    """Trigger self-evolution to improve code."""
    try:
        from quantum_brain import get_quantum_brain

        brain = await get_quantum_brain(state.db)
        result = await brain.evolve()

        return {
            "success": result.get("status") == "success",
            "suggestions": result.get("suggestions", []),
            "improvements_made": result.get("improvements_made", []),
            "message": "Self-evolution analysis complete"
        }
    except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))
```

```
@router.get("/brain/domains")
async def get_brain_domains():
    """Get all knowledge domains and their stats."""
    try:
        from quantum_brain import get_quantum_brain

        brain = await get_quantum_brain(state.db)
        domain_stats = brain.algo.domains.get_domain_stats()

        return {
            "domains": list(domain_stats.keys()),
            "current_domain": brain.algo.current_domain,
            "stats": domain_stats
        }
    except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))


class BulkTrainRequest(BaseModel):
    """BulkTrainRequest - Auto-documented by self-evolution."""
    text: str
    domain: str = "general"
    source: str = "user_provided"


@router.post("/brain/train")
async def bulk_train_brain(request: BulkTrainRequest):
    """Train the AI on a large block of text."""
    try:
        from quantum_brain import get_quantum_brain

        brain = await get_quantum_brain(state.db)

        # Learn the text
        brain.algo.learn(request.text, source=request.source)

        # Train embeddings on the text
        from word_embeddings import train_embeddings_on_text
        train_embeddings_on_text(request.text)

        # If domain specified, also add to domain knowledge
        if request.domain != "general":
            brain.algo.domains.add_to_domain(request.domain, request.text)

        # Get stats after training
        stats = brain.algo.get_stats()

        return {
            "success": True,
            "text_length": len(request.text),
            "domain": request.domain,
            "markov_vocab_size": stats.get("markov", {}).get("vocabulary_size", 0),
            "message": f"Successfully trained on {len(request.text)} characters"
        }
    except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))


@router.get("/brain/embeddings")
async def get_embeddings_info():
    """Get word embeddings statistics and test similarity."""
    try:
        from word_embeddings import get_embeddings

        embeddings = get_embeddings()
        stats = embeddings.get_stats()

        # Test some similarities
        test_pairs = [
            ("consciousness", "awareness"),
            ("quantum", "physics"),
            ("learning", "knowledge"),
            ("evolution", "adaptation")
        ]

        similarities = {}
        for w1, w2 in test_pairs:
            sim = embeddings.similarity(w1, w2)
            similarities[f"{w1}-{w2}"] = round(sim, 3)

        return {
            "stats": stats,
            "test_similarities": similarities
        }
    except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))
```

```
@router.get("/brain/similar/{word}")
async def get_similar_words(word: str, top_n: int = 10):
    """Get most similar words to a given word using embeddings."""
    try:
        from word_embeddings import get_embeddings

        embeddings = get_embeddings()
        similar = embeddings.most_similar(word, top_n=top_n)

        return {
            "word": word,
            "similar": [{"word": w, "similarity": round(s, 3)} for w, s in similar]
        }
    except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))


# ==============================================================================
# QUANTUM GATE CONTROL
# ==============================================================================

@router.post("/quantum/gate")
async def apply_quantum_gate(gate: str):
    """Apply a quantum gate (hadamard, pauli_x, pauli_z, measure)"""
    try:
        from quantum_brain import get_quantum_brain

        brain = await get_quantum_brain(state.db)
        result = brain.apply_gate(gate)

        return {
            "success": True,
            "gate_applied": gate,
            "quantum_state": result
        }
    except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))


@router.get("/quantum/state")
async def get_quantum_state():
    """Get current quantum state"""
    try:
        from quantum_brain import get_quantum_brain

        brain = await get_quantum_brain(state.db)

        return {
            "quantum_state": brain.quantum_state.to_dict(),
            "pennylane_available": brain.pennylane is not None
        }
    except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))


# ==============================================================================
# WOLFRAM ALPHA
# ==============================================================================

@router.post("/wolfram/query")
async def wolfram_query(query: str):
    """Query Wolfram Alpha for mathematical/scientific answers"""
    try:
        from wolfram_integration import get_wolfram_engine

        wolfram = get_wolfram_engine()
        result = wolfram.query(query)

        return result
    except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))


# ==============================================================================
# SELF-RESEARCH
# ==============================================================================

@router.post("/research/query")
async def research_query(query: str, reason: str = "user_request"):
    """Perform autonomous research on a topic"""
    try:
        # Direct DDGS test
        from ddgs import DDGS
        ddgs = DDGS()
```

```
        direct_results = list(ddgs.text(query, max_results=3))
        logger.info(f"[RESEARCH DEBUG] Direct DDGS returned {len(direct_results)} results for '{query}'")

        from self_research import get_research_engine
        engine = await get_research_engine(state.db)
        result = await engine.research(query, reason)

        # If engine returned 0 but direct worked, use direct results
        if len(result.get('results', [])) == 0 and len(direct_results) > 0:
            logger.info(f"[RESEARCH DEBUG] Using direct results")
            result['results'] = [{'title': r.get('title',''), 'body': r.get('body',''), 'href': r.get('href','')} for r in direct_results]
            # Re-extract concepts
            all_text = ' '.join([r.get('body', '') for r in direct_results])
            words = [w.lower() for w in all_text.split() if len(w) > 4]
            result['new_concepts'] = list(set(words))[:10]

        return result
    except Exception as e:
        logger.error(f"[RESEARCH ERROR] {str(e)}")
        raise HTTPException(status_code=500, detail=str(e))


@router.get("/research/stats")
async def get_research_stats():
    """Get research statistics"""
    try:
        from self_research import get_research_engine

        engine = await get_research_engine(state.db)
        return engine.get_research_stats()
    except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))


# ============================================================================
# AUTONOMOUS RESEARCH (30-60 MINUTES)
# ============================================================================

class AutonomousResearchRequest(BaseModel):
    duration_minutes: int = 30

@router.post("/research/autonomous/start")
async def start_autonomous_research(request: AutonomousResearchRequest):
    """Start autonomous research session (30-60 minutes)"""
    try:
        from self_research import get_research_engine

        engine = await get_research_engine(state.db)
        # Verify db is passed and write a test entry
        logger.info(f"[RESEARCH] Starting autonomous research, db={state.db is not None}")

        # Direct test write
        try:
            await state.db.research_test.insert_one({
                'test': 'autonomous_start',
                'timestamp': datetime.now(timezone.utc).isoformat()
            })
            logger.info("[RESEARCH] Test write to DB successful")
        except Exception as write_err:
            logger.error(f"[RESEARCH] Test write failed: {write_err}")

        result = await engine.start_autonomous_research(request.duration_minutes)

        return result
    except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))


@router.post("/research/autonomous/stop")
async def stop_autonomous_research():
    """Stop autonomous research session"""
    try:
        from self_research import get_research_engine

        engine = await get_research_engine(db)
        result = engine.stop_autonomous_research()

        return result
    except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))


@router.get("/research/autonomous/progress")
async def get_autonomous_research_progress():
    """Get autonomous research progress"""
    try:
```

```
        from self_research import get_research_engine

        engine = await get_research_engine(db)
        return engine.get_autonomous_progress()
    except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))


# ============================================================================
# SELF-EVOLUTION
# ============================================================================

@router.get("/evolution/status")
async def get_evolution_status():
    """Get self-evolution status"""
    try:
        from self_evolution_core import get_evolution_engine

        engine = get_evolution_engine(state.db)
        return engine.get_evolution_status()
    except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))


@router.post("/evolution/trigger")
async def trigger_evolution():
    """Manually trigger self-evolution (user-initiated)"""
    try:
        from self_evolution_core import get_evolution_engine

        engine = get_evolution_engine(state.db)
        result = await engine.auto_evolve()

        return {
            "success": True,
            "evolution_result": result
        }
    except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))


@router.get("/evolution/analyze/{filename}")
async def analyze_code_file(filename: str):
    """Analyze a code file for potential improvements"""
    try:
        from self_evolution_core import get_evolution_engine

        engine = get_evolution_engine(state.db)
        analysis = engine.read_own_code(filename)
        improvements = engine.identify_improvements(filename)

        # Strip raw code from response to keep it lightweight
        if 'code' in analysis:
            del analysis['code']

        return {
            "analysis": analysis,
            "improvements": improvements,
            "improvement_count": len(improvements)
        }
    except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))


@router.get("/evolution/analyze-all")
async def analyze_all_code():
    """Analyze all modifiable code files"""
    try:
        from self_evolution_core import get_evolution_engine

        engine = get_evolution_engine(state.db)
        all_analyses = engine.analyze_all_code()

        # Collect all improvements
        all_improvements = []
        for filename in engine.modifiable_files:
            improvements = engine.identify_improvements(filename)
            for imp in improvements:
                imp['file'] = filename
                all_improvements.append(imp)

        # Sort by priority
        all_improvements.sort(key=lambda x: x.get('priority', 99))

        # Strip raw code from response
        for key, analysis in all_analyses.get('analyses', {}).items():
```

```
        if 'code' in analysis:
            del analysis['code']

    return {
        "summary": {
            "files_analyzed": all_analyses['files_analyzed'],
            "total_lines": all_analyses['total_lines'],
            "total_classes": all_analyses['total_classes'],
            "total_functions": all_analyses['total_functions'],
            "total_improvements": len(all_improvements)
        },
        "improvements": all_improvements,
        "files": {k: v for k, v in all_analyses.get('analyses', {}).items() if 'error' not in v}
    }
except Exception as e:
    raise HTTPException(status_code=500, detail=str(e))
```

routes_chat.py

```
"""
 Chat Routes
===============
Chat endpoints, session management, and command processing.
"""

from fastapi import APIRouter, HTTPException
from pydantic import BaseModel
from typing import List, Dict, Optional
import uuid
import os
import logging
from datetime import datetime, timezone

import shared_state as state
from thinking_commands import ThinkingModeCommands
from chat_helpers import (
    get_memory_summary, do_deep_research, process_shared_link,
    list_all_documents, generate_speech, get_help_text,
    analyze_book_characters, analyze_book_timeline,
    analyze_book_worldbuilding, analyze_book_feedback
)

logger = logging.getLogger("quantum_ai")
router = APIRouter(prefix="/api")


class ChatMessage(BaseModel):
    """ChatMessage - Auto-documented by self-evolution."""
    content: str
    context: Optional[Dict] = None
    session_id: Optional[str] = None

class ChatResponse(BaseModel):
    """ChatResponse - Auto-documented by self-evolution."""
    questions: List[str]
    response: str
    understanding: Dict
    concepts: List[str]
    session_id: Optional[str] = None


@router.post("/quantum/chat", response_model=ChatResponse)
async def quantum_chat(message: ChatMessage):
    """
    ARTICLE 1.1 - Prime Directive of Questioning (HIDDEN MODE)

    Process user input with hidden thinking mode.
    AI thinks, questions, researches internally - user sees clean conversation.
    Supports conversation history via session_id.
    """
    try:
        # Generate or use provided session ID
        session_id = message.session_id or str(uuid.uuid4())

        # Check for thinking mode commands
        command = ThinkingModeCommands.parse_command(message.content)

        if command:
            if command["command"] == "thinking_on":
                state.hidden_thinking.toggle_thinking_display(True)
                return ChatResponse(
                    questions=[],
                    response=" Thinking mode display: ON. You'll now see my internal process.",
                    understanding={},
                    concepts=[],
                    session_id=session_id
                )
            elif command["command"] == "thinking_off":
                state.hidden_thinking.toggle_thinking_display(False)
                return ChatResponse(
                    questions=[],
                    response=" Thinking mode display: OFF. Clean conversation mode.",
                    understanding={},
                    concepts=[],
                    session_id=session_id
                )
            elif command["command"] == "thinking_status":
                status = "ON (visible)" if state.hidden_thinking.show_thinking else "OFF (hidden)"
                return ChatResponse(
                    questions=[],
                    response=f" Current thinking mode: {status}",
```

```
        understanding={},
        concepts=[],
        session_id=session_id
    )
elif command["command"] == "explain_on":
    state.hidden_thinking.toggle_explanation_mode(True)
    return ChatResponse(
        questions=[],
        response=" Explanation mode: ON. I'll now explain how I reached my answers after each response.",
        understanding={},
        concepts=[],
        session_id=session_id
    )
elif command["command"] == "explain_off":
    state.hidden_thinking.toggle_explanation_mode(False)
    return ChatResponse(
        questions=[],
        response=" Explanation mode: OFF. Clean response mode.",
        understanding={},
        concepts=[],
        session_id=session_id
    )
elif command["command"] == "explain_status":
    status = "ON" if state.hidden_thinking.show_explanation else "OFF"
    return ChatResponse(
        questions=[],
        response=f" Current explanation mode: {status}",
        understanding={},
        concepts=[],
        session_id=session_id
    )
elif command["command"] == "read_file":
    # Process uploaded file
    filename = command["filename"] or "last uploaded file"

    # Get list of uploaded files
    import os
    upload_dir = "/app/uploads"
    if os.path.exists(upload_dir):
        files = [f for f in os.listdir(upload_dir) if os.path.isfile(os.path.join(upload_dir, f))]

        if files:
            # Use specified file or most recent
            if command["filename"]:
                target_file = command["filename"]
            else:
                target_file = files[-1]  # Most recent

            file_path = os.path.join(upload_dir, target_file)

            # Process the document
            result = await state.universal_explorer.documents.process_document(file_path)

            if "error" not in result:
                # Extract and summarize
                text = result.get("text", "")[:500]
                response = f" Read {target_file}:\n\nI've processed this document. {text[:200]}...\n\nWhat would you like to...
            else:
                response = f" Couldn't read {target_file}: {result['error']}"
        else:
            response = "No files uploaded yet. Use the Explorer tab to upload files."
    else:
        response = "Upload directory not found."

    return ChatResponse(
        questions=[],
        response=response,
        understanding={},
        concepts=[],
        session_id=session_id
    )

elif command["command"] == "memory":
    # Show what AI has learned
    response = await get_memory_summary(cognitive_core, db, session_id)
    return ChatResponse(
        questions=[],
        response=response,
        understanding={},
        concepts=[],
        session_id=session_id
    )

elif command["command"] == "research":
    # Deep research on a topic
    topic = command.get("topic", "")
```

```
    if not topic:
        return ChatResponse(
            questions=[],
            response=" Usage: `!research <topic>`\n\nExample: `!research quantum entanglement`",
            understanding={},
            concepts=[],
            session_id=session_id
        )

    response = await do_deep_research(topic, universal_explorer, cognitive_core)
    return ChatResponse(
        questions=[],
        response=response,
        understanding={},
        concepts=[],
        session_id=session_id
    )

elif command["command"] == "link":
    # Download file from URL
    url = command.get("url", "")
    if not url:
        return ChatResponse(
            questions=[],
            response=" Usage: `!link <url>`\n\nSupported:\n Google Drive links\n Dropbox links\n OneDrive links\n Direct...
            understanding={},
            concepts=[],
            session_id=session_id
        )

    response = await process_shared_link(url, state.shared_link)
    return ChatResponse(
        questions=[],
        response=response,
        understanding={},
        concepts=[],
        session_id=session_id
    )

elif command["command"] == "docs":
    # List all documents
    response = await list_all_documents()
    return ChatResponse(
        questions=[],
        response=response,
        understanding={},
        concepts=[],
        session_id=session_id
    )

elif command["command"] == "speak":
    # Text to speech
    text = command.get("text", "")
    if not text:
        return ChatResponse(
            questions=[],
            response=" Usage: `!speak <text>`\n\nExample: `!speak Hello, I am Quantum AI`\n\nOr just chat normally and use t...
            understanding={},
            concepts=[],
            session_id=session_id
        )

    # Generate TTS
    audio_url = await generate_speech(text)
    response = f" **Speech Generated**\n\n\"{text[:100]}{'...' if len(text) > 100 else ''}\"\n\n[Audio URL: {audio_url}]" if...
    return ChatResponse(
        questions=[],
        response=response,
        understanding={},
        concepts=[],
        session_id=session_id
    )

elif command["command"] == "imagine":
    # Generate image
    prompt = command.get("prompt", "")
    if not prompt:
        return ChatResponse(
            questions=[],
            response=" Usage: `!imagine <description>`\n\nExample: `!imagine a cosmic landscape with nebulas and stars`",
            understanding={},
            concepts=[],
            session_id=session_id
        )

    try:
```

```
        import base64
        from emergentintegrations.llm.openai.image_generation import OpenAIImageGeneration

        api_key = os.environ.get("EMERGENT_LLM_KEY")
        image_gen = OpenAIImageGeneration(api_key=api_key)
        images = await image_gen.generate_images(prompt=prompt, model="gpt-image-1.5", number_of_images=1)

        if images and len(images) > 0:
            image_base64 = base64.b64encode(images[0]).decode('utf-8')
            filename = f"generated_{uuid.uuid4().hex[:8]}.png"
            with open(f"/app/uploads/{filename}", "wb") as f:
                f.write(images[0])

            response = f" **Image Generated**\n\nPrompt: \"{prompt}\"\n\nSaved as: {filename}\n\n![Generated Image](data:ima...
        else:
            response = " Image generation failed. Please try again."
    except Exception as e:
        response = f" Image generation error: {str(e)}"

    return ChatResponse(
        questions=[],
        response=response,
        understanding={},
        concepts=[],
        session_id=session_id
    )

elif command["command"] == "export":
    # Export chat or memory
    target = command.get("target", "chat")
    if target == "memory":
        response = " **Export Memory**\n\nUse the API endpoint: `GET /api/memory/export`\n\nOr click the Export button in th...
    else:
        response = f" **Export Conversation**\n\nUse the API: `GET /api/chat/export/{session_id}`\n\nOr click the Export but...

    return ChatResponse(
        questions=[],
        response=response,
        understanding={},
        concepts=[],
        session_id=session_id
    )

elif command["command"] == "graph":
    # Knowledge graph info
    response = " **Knowledge Graph**\n\nView your knowledge graph visualization in the **Memory** screen.\n\nThe graph show...
    return ChatResponse(
        questions=[],
        response=response,
        understanding={},
        concepts=[],
        session_id=session_id
    )

elif command["command"] == "characters":
    # Extract characters from document
    response = await analyze_book_characters()
    return ChatResponse(
        questions=[],
        response=response,
        understanding={},
        concepts=[],
        session_id=session_id
    )

elif command["command"] == "timeline":
    # Generate timeline
    response = await analyze_book_timeline()
    return ChatResponse(
        questions=[],
        response=response,
        understanding={},
        concepts=[],
        session_id=session_id
    )

elif command["command"] == "worldbuilding":
    # Extract world details
    response = await analyze_book_worldbuilding()
    return ChatResponse(
        questions=[],
        response=response,
        understanding={},
        concepts=[],
        session_id=session_id
    )
```

```
            elif command["command"] == "feedback":
                # Writing feedback
                response = await analyze_book_feedback()
                return ChatResponse(
                    questions=[],
                    response=response,
                    understanding={},
                    concepts=[],
                    session_id=session_id
                )

            elif command["command"] == "help":
                # Show all commands
                response = get_help_text()
                return ChatResponse(
                    questions=[],
                    response=response,
                    understanding={},
                    concepts=[],
                    session_id=session_id
                )

        # Get conversation history for context
        history = await get_conversation_history(session_id, limit=10)

        # Process with hidden thinking mode
        result = await state.hidden_thinking.process_with_thinking(
            message.content,
            message.context,
            conversation_history=history
        )

        # Store this exchange in history
        await store_chat_message(session_id, "user", message.content)
        await store_chat_message(session_id, "assistant", result["response"])

        # Format response with thinking log if enabled
        response_text = result["response"]
        if result["thinking_log"]:
            thinking_display = "\n\n--- THINKING PROCESS ---\n" + "\n".join(result["thinking_log"])
            response_text = response_text + thinking_display

        # Add explanation if enabled
        if result.get("explanation"):
            response_text = response_text + "\n\n---\n **How I reached this answer:**\n" + result["explanation"]

        return ChatResponse(
            questions=result["internal_questions"] if state.hidden_thinking.show_thinking else [],
            response=response_text,
            understanding={"confidence": result["confidence"], "research_done": result["research_done"]},
            concepts=[],
            session_id=session_id
        )

    except Exception as e:
        logger.error(f"Chat error: {str(e)}")
        # Fallback to basic mode
        result = await state.cognitive_core.process_input(
            message.content,
            message.context,
            use_llm=False
        )
        return ChatResponse(
            questions=result["questions"],
            response=result["response"],
            understanding=result["understanding"],
            concepts=result["concepts"],
            session_id=message.session_id
        )


# ============================================================================
# CONVERSATION HISTORY HELPERS
# ============================================================================

async def store_chat_message(session_id: str, role: str, content: str):
    """Store a chat message in MongoDB"""
    doc = {
        "session_id": session_id,
        "role": role,
        "content": content,
        "timestamp": datetime.now(timezone.utc).isoformat()
    }
    await state.db.chat_history.insert_one(doc)
```

```
async def get_conversation_history(session_id: str, limit: int = 10) -> List[Dict]:
    """Get recent conversation history for a session"""
    cursor = state.db.chat_history.find(
        {"session_id": session_id},
        {"_id": 0, "role": 1, "content": 1, "timestamp": 1}
    ).sort("timestamp", -1).limit(limit)

    messages = await cursor.to_list(length=limit)
    # Return in chronological order
    return list(reversed(messages))


@router.get("/chat/history/{session_id}")
async def get_chat_history(session_id: str, limit: int = 50):
    """Get conversation history for a session"""
    try:
        history = await get_conversation_history(session_id, limit)
        return {"session_id": session_id, "messages": history, "count": len(history)}
    except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))


@router.delete("/chat/history/{session_id}")
async def clear_chat_history(session_id: str):
    """Clear conversation history for a session"""
    try:
        result = await state.db.chat_history.delete_many({"session_id": session_id})
        return {"session_id": session_id, "deleted": result.deleted_count}
    except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))


@router.get("/chat/sessions")
async def list_chat_sessions():
    """List all chat sessions"""
    try:
        pipeline = [
            {"$group": {
                "_id": "$session_id",
                "message_count": {"$sum": 1},
                "last_message": {"$max": "$timestamp"}
            }},
            {"$sort": {"last_message": -1}},
            {"$limit": 20}
        ]
        sessions = await state.db.chat_history.aggregate(pipeline).to_list(length=20)
        return {"sessions": [{"session_id": s["_id"], "message_count": s["message_count"], "last_message": s["last_message"]} for s in se...
    except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))


# ============================================================================
# COMMAND HELPERS (!memory, !research)
# ============================================================================
```

routes_cloud.py

```
"""
 Cloud Integration Routes
==========================
API endpoints for Google Drive sync and WolframAlpha Pro
"""

from fastapi import APIRouter, HTTPException, Query
from pydantic import BaseModel
from typing import Optional, List, Dict
import logging

logger = logging.getLogger("quantum_ai")
router = APIRouter(prefix="/api/cloud")


# ============================================================================
# Pydantic Models
# ============================================================================

class AuthCodeRequest(BaseModel):
    """AuthCodeRequest - Auto-documented by self-evolution."""
    code: str

class SyncRequest(BaseModel):
    """SyncRequest - Auto-documented by self-evolution."""
    data_type: str  # 'brain', 'knowledge', 'conversations'
    data: Optional[Dict] = None

class WolframQueryRequest(BaseModel):
    """WolframQueryRequest - Auto-documented by self-evolution."""
    query: str
    mode: str = "simple"  # simple, full, spoken, conversation, step_by_step


# ============================================================================
# Google Drive Routes
# ============================================================================

@router.get("/drive/status")
async def get_drive_status():
    """Get Google Drive sync status."""
    try:
        from google_drive_sync import get_drive_engine
        engine = get_drive_engine()
        return {
            "success": True,
            "data": engine.get_status()
        }
    except Exception as e:
        logger.error(f"Drive status error: {e}")
        return {
            "success": False,
            "error": str(e)
        }


@router.get("/drive/auth-url")
async def get_drive_auth_url():
    """Get OAuth authorization URL."""
    try:
        from google_drive_sync import get_drive_engine
        engine = get_drive_engine()
        auth_url = engine.get_auth_url()

        if auth_url:
            return {
                "success": True,
                "auth_url": auth_url,
                "instructions": "Visit this URL to authorize Google Drive access. You'll be redirected back automatically."
            }
        else:
            return {
                "success": False,
                "error": "Could not generate auth URL. Check if credentials are configured."
            }
    except Exception as e:
        logger.error(f"Auth URL error: {e}")
        raise HTTPException(status_code=500, detail=str(e))


@router.get("/drive/oauth-callback")
async def oauth_callback(code: str = Query(None), error: str = Query(None)):
```

```
    """Handle OAuth callback from Google."""
    from fastapi.responses import HTMLResponse

    if error:
        html = f"""
        <html>
        <head><title>Authorization Failed</title></head>
        <body style="font-family: sans-serif; padding: 40px; text-align: center;">
            <h1> Authorization Failed</h1>
            <p>Error: {error}</p>
            <p><a href="/">Return to app</a></p>
        </body>
        </html>
        """
        return HTMLResponse(content=html)

    if not code:
        html = """
        <html>
        <head><title>No Code</title></head>
        <body style="font-family: sans-serif; padding: 40px; text-align: center;">
            <h1> No Authorization Code</h1>
            <p>No authorization code was provided.</p>
            <p><a href="/">Return to app</a></p>
        </body>
        </html>
        """
        return HTMLResponse(content=html)

    try:
        from google_drive_sync import get_drive_engine
        engine = get_drive_engine()
        success = engine.complete_auth(code)

        if success:
            html = """
            <html>
            <head><title>Authorized!</title></head>
            <body style="font-family: sans-serif; padding: 40px; text-align: center;">
                <h1> Google Drive Authorized!</h1>
                <p>You can now sync your Quantum MCAGI data to Google Drive.</p>
                <p><a href="/">Return to app</a></p>
                <script>
                    // Auto-close after 3 seconds
                    setTimeout(() => { window.close(); }, 3000);
                </script>
            </body>
            </html>
            """
        else:
            html = """
            <html>
            <head><title>Authorization Failed</title></head>
            <body style="font-family: sans-serif; padding: 40px; text-align: center;">
                <h1> Authorization Failed</h1>
                <p>Could not complete authorization. Please try again.</p>
                <p><a href="/">Return to app</a></p>
            </body>
            </html>
            """
        return HTMLResponse(content=html)
    except Exception as e:
        logger.error(f"OAuth callback error: {e}")
        html = f"""
        <html>
        <head><title>Error</title></head>
        <body style="font-family: sans-serif; padding: 40px; text-align: center;">
            <h1> Error</h1>
            <p>{str(e)}</p>
            <p><a href="/">Return to app</a></p>
        </body>
        </html>
        """
        return HTMLResponse(content=html)


@router.post("/drive/complete-auth")
async def complete_drive_auth(request: AuthCodeRequest):
    """Complete OAuth flow with authorization code."""
    try:
        from google_drive_sync import get_drive_engine
        engine = get_drive_engine()

        success = engine.complete_auth(request.code)

        if success:
```

```
            return {
                "success": True,
                "message": "Google Drive authorized successfully!",
                "status": engine.get_status()
            }
        else:
            return {
                "success": False,
                "error": "Authorization failed. Check the code and try again."
            }
    except Exception as e:
        logger.error(f"Auth completion error: {e}")
        raise HTTPException(status_code=500, detail=str(e))


@router.get("/drive/files")
async def list_drive_files():
    """List files in the sync folder."""
    try:
        from google_drive_sync import get_drive_engine
        engine = get_drive_engine()

        if not engine.authenticated:
            return {
                "success": False,
                "error": "Not authenticated. Use /drive/auth-url first."
            }

        files = engine.list_files()
        return {
            "success": True,
            "files": files,
            "folder_name": engine.folder_name
        }
    except Exception as e:
        logger.error(f"List files error: {e}")
        raise HTTPException(status_code=500, detail=str(e))


@router.post("/drive/sync")
async def sync_to_drive(request: SyncRequest):
    """Sync data to Google Drive."""
    try:
        from google_drive_sync import get_drive_engine
        engine = get_drive_engine()

        if not engine.authenticated:
            return {
                "success": False,
                "error": "Not authenticated. Use /drive/auth-url first."
            }

        if request.data_type == 'brain':
            # Get current brain state
            from shared_state import state
            from algorithmic_core import get_algorithmic_core

            core = get_algorithmic_core()
            brain_data = {
                'stats': core.get_stats(),
                'growth': {
                    'stage': 6,
                    'name': 'Transcendent'
                }
            }

            if request.data:
                brain_data.update(request.data)

            success = engine.sync_brain_state(brain_data)

        elif request.data_type == 'knowledge':
            from unified_cognitive_engine import get_unified_cognitive_engine
            unified = get_unified_cognitive_engine()

            knowledge_data = {
                'personality': unified.personality.get_personality_summary(),
                'stage': unified.personality.current_stage,
                'interactions': unified.personality.interaction_count
            }

            if request.data:
                knowledge_data.update(request.data)

            success = engine.sync_knowledge(knowledge_data)
```

```
        elif request.data_type == 'conversations':
            success = engine.sync_conversations(request.data or [])
        else:
            return {
                "success": False,
                "error": f"Unknown data type: {request.data_type}"
            }

        return {
            "success": success,
            "message": f"Synced {request.data_type} to Google Drive" if success else "Sync failed",
            "last_sync": engine.last_sync.isoformat() if engine.last_sync else None
        }
    except Exception as e:
        logger.error(f"Sync error: {e}")
        raise HTTPException(status_code=500, detail=str(e))


@router.get("/drive/load/{data_type}")
async def load_from_drive(data_type: str):
    """Load data from Google Drive."""
    try:
        from google_drive_sync import get_drive_engine
        engine = get_drive_engine()

        if not engine.authenticated:
            return {
                "success": False,
                "error": "Not authenticated"
            }

        if data_type == 'brain':
            data = engine.load_brain_state()
        elif data_type == 'knowledge':
            data = engine.load_knowledge()
        elif data_type == 'conversations':
            data = engine.download_json('conversations.json')
        else:
            return {"success": False, "error": f"Unknown data type: {data_type}"}

        return {
            "success": data is not None,
            "data": data
        }
    except Exception as e:
        logger.error(f"Load error: {e}")
        raise HTTPException(status_code=500, detail=str(e))


# ============================================================================
# WolframAlpha Routes
# ============================================================================

@router.get("/wolfram/status")
async def get_wolfram_status():
    """Get WolframAlpha status."""
    try:
        from wolfram_enhanced import get_wolfram_engine
        engine = get_wolfram_engine()
        return {
            "success": True,
            "data": engine.get_status()
        }
    except Exception as e:
        logger.error(f"Wolfram status error: {e}")
        return {"success": False, "error": str(e)}


@router.post("/wolfram/query")
async def query_wolfram(request: WolframQueryRequest):
    """Query WolframAlpha with various modes."""
    try:
        from wolfram_enhanced import get_wolfram_engine
        engine = get_wolfram_engine()

        if request.mode == "simple":
            result = engine.query_simple(request.query)
            return {
                "success": result is not None,
                "result": result,
                "mode": "simple"
            }

        elif request.mode == "full":
            result = engine.query_full(request.query)
            return {
```

```
                "success": result.get('success', False),
                "result": result,
                "mode": "full"
            }

        elif request.mode == "spoken":
            result = engine.query_spoken(request.query)
            return {
                "success": result is not None,
                "result": result,
                "mode": "spoken"
            }

        elif request.mode == "conversation":
            result = engine.query_conversation(request.query)
            return {
                "success": result.get('success', False),
                "result": result.get('result'),
                "conversation_id": result.get('conversation_id'),
                "mode": "conversation"
            }

        elif request.mode == "step_by_step":
            result = engine.query_step_by_step(request.query)
            return {
                "success": result.get('success', False),
                "result": result,
                "mode": "step_by_step"
            }

        else:
            return {
                "success": False,
                "error": f"Unknown mode: {request.mode}"
            }
    except Exception as e:
        logger.error(f"Wolfram query error: {e}")
        raise HTTPException(status_code=500, detail=str(e))


@router.get("/wolfram/simple")
async def wolfram_simple(q: str = Query(..., description="Question to ask")):
    """Simple WolframAlpha query."""
    try:
        from wolfram_enhanced import get_wolfram_engine
        engine = get_wolfram_engine()
        result = engine.query_simple(q)
        return {
            "success": result is not None,
            "query": q,
            "result": result
        }
    except Exception as e:
        logger.error(f"Simple query error: {e}")
        raise HTTPException(status_code=500, detail=str(e))


@router.post("/wolfram/reset-conversation")
async def reset_wolfram_conversation():
    """Reset WolframAlpha conversation context."""
    try:
        from wolfram_enhanced import get_wolfram_engine
        engine = get_wolfram_engine()
        engine.reset_conversation()
        return {
            "success": True,
            "message": "Conversation reset"
        }
    except Exception as e:
        logger.error(f"Reset error: {e}")
        raise HTTPException(status_code=500, detail=str(e))



# ============================================================================
# Wolfram Cloud Storage Routes (Replaces Google Drive!)
# ============================================================================

@router.get("/wolfram-cloud/status")
async def get_wolfram_cloud_status():
    """Get Wolfram Cloud sync status."""
    try:
        from wolfram_cloud_sync import get_wolfram_cloud_engine
        engine = get_wolfram_cloud_engine()
        return {
            "success": True,
```

```
            "data": engine.get_status()
        }
    except Exception as e:
        logger.error(f"Wolfram Cloud status error: {e}")
        return {
            "success": False,
            "error": str(e)
        }


@router.post("/wolfram-cloud/sync")
async def sync_to_wolfram_cloud(request: SyncRequest):
    """Sync data to Wolfram Cloud."""
    try:
        from wolfram_cloud_sync import get_wolfram_cloud_engine
        engine = get_wolfram_cloud_engine()

        if not engine.authenticated:
            return {
                "success": False,
                "error": "Wolfram Cloud not authenticated"
            }

        if request.data_type == 'brain':
            from algorithmic_core import get_algorithmic_core
            core = get_algorithmic_core()
            brain_data = {
                'stats': core.get_stats(),
                'growth': {'stage': 6, 'name': 'Transcendent'}
            }
            if request.data:
                brain_data.update(request.data)
            success = engine.sync_brain_state(brain_data)

        elif request.data_type == 'knowledge':
            from unified_cognitive_engine import get_unified_cognitive_engine
            unified = get_unified_cognitive_engine()
            knowledge_data = {
                'personality': unified.personality.get_personality_summary(),
                'stage': unified.personality.current_stage,
                'interactions': unified.personality.interaction_count
            }
            if request.data:
                knowledge_data.update(request.data)
            success = engine.sync_knowledge(knowledge_data)

        elif request.data_type == 'conversations':
            success = engine.sync_conversations(request.data or [])
        else:
            return {"success": False, "error": f"Unknown data type: {request.data_type}"}

        return {
            "success": success,
            "message": f"Synced {request.data_type} to Wolfram Cloud" if success else "Sync failed",
            "last_sync": engine.last_sync.isoformat() if engine.last_sync else None
        }
    except Exception as e:
        logger.error(f"Wolfram Cloud sync error: {e}")
        raise HTTPException(status_code=500, detail=str(e))


@router.get("/wolfram-cloud/load/{data_type}")
async def load_from_wolfram_cloud(data_type: str):
    """Load data from Wolfram Cloud."""
    try:
        from wolfram_cloud_sync import get_wolfram_cloud_engine
        engine = get_wolfram_cloud_engine()

        if not engine.authenticated:
            return {"success": False, "error": "Not authenticated"}

        if data_type == 'brain':
            data = engine.load_brain_state()
        elif data_type == 'knowledge':
            data = engine.load_knowledge()
        elif data_type == 'conversations':
            data = engine.load_conversations()
        else:
            return {"success": False, "error": f"Unknown data type: {data_type}"}

        return {
            "success": data is not None,
            "data": data
        }
    except Exception as e:
        logger.error(f"Wolfram Cloud load error: {e}")
```

```
        raise HTTPException(status_code=500, detail=str(e))


@router.get("/wolfram-cloud/files")
async def list_wolfram_cloud_files():
    """List files in Wolfram Cloud."""
    try:
        from wolfram_cloud_sync import get_wolfram_cloud_engine
        engine = get_wolfram_cloud_engine()

        if not engine.authenticated:
            return {"success": False, "error": "Not authenticated"}

        files = engine.list_files()
        return {
            "success": True,
            "files": files,
            "folder": engine.cloud_folder
        }
    except Exception as e:
        logger.error(f"Wolfram Cloud list error: {e}")
        raise HTTPException(status_code=500, detail=str(e))
```

routes_cognitive.py

```
"""
 Cognitive Routes
====================
Quantum status, memories, growth, knowledge graph, memory export/import.
"""

from fastapi import APIRouter, HTTPException
from pydantic import BaseModel
from typing import List, Dict, Any, Optional
from datetime import datetime, timezone

import shared_state as state

router = APIRouter(prefix="/api")


class MemoryImport(BaseModel):
    """MemoryImport - Auto-documented by self-evolution."""
    semantic: List[Dict] = []
    episodic: List[Dict] = []
    procedural: List[Dict] = []


@router.get("/quantum/status")
async def quantum_status():
    """Get comprehensive cognitive status"""
    try:
        status = await state.cognitive_core.get_status()
        return status
    except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))

@router.get("/quantum/memories/semantic")
async def get_semantic_memories(limit: int = 50):
    """Retrieve semantic memory (concepts)"""
    try:
        concepts = await state.cognitive_core.semantic_memory.get_all_concepts(limit)
        return {"concepts": concepts}
    except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))

@router.get("/quantum/memories/episodic")
async def get_episodic_memories(limit: int = 50):
    """Retrieve episodic memory (experiences)"""
    try:
        episodes = await state.cognitive_core.episodic_memory.recall_recent_episodes(limit)
        return {"episodes": episodes}
    except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))

@router.get("/quantum/memories/procedural")
async def get_procedural_memories(limit: int = 50):
    """Retrieve procedural memory (skills)"""
    try:
        skills = await state.cognitive_core.procedural_memory.get_all_skills(limit)
        return {"skills": skills}
    except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))

@router.get("/quantum/memories/philosophical")
async def get_philosophical_memories(answered: bool = False, limit: int = 50):
    """Retrieve philosophical memory (questions and insights)"""
    try:
        if answered:
            questions = await state.cognitive_core.philosophical_memory.get_unanswered_questions(limit)
        else:
            questions = await state.cognitive_core.philosophical_memory.get_unanswered_questions(limit)

        insights = await state.cognitive_core.philosophical_memory.get_insights(limit)

        return {
            "questions": questions,
            "insights": insights
        }
    except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))

@router.get("/quantum/growth")
async def get_growth_metrics():
    """Get growth metrics and current stage"""
    try:
        metrics = await state.cognitive_core.growth_tracker.calculate_metrics()
```

```
        stage = await state.cognitive_core.growth_tracker.get_current_stage()

        return {
            "metrics": metrics,
            "stage": stage
        }
    except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))


# ============================================================================
# UNIVERSAL EXPLORER ENDPOINTS
# ============================================================================


@router.get("/knowledge/graph")
async def get_knowledge_graph():
    """Get knowledge graph data for visualization"""
    try:
        # Get all concepts with relationships
        concepts = await state.cognitive_core.semantic_memory.get_all_concepts(100)

        nodes = []
        edges = []
        node_ids = set()

        for concept in concepts:
            concept_name = concept.get("concept", "")
            if concept_name and concept_name not in node_ids:
                nodes.append({
                    "id": concept_name,
                    "label": concept_name.title(),
                    "strength": concept.get("strength", 1),
                    "definition": concept.get("definition", "")[:100]
                })
                node_ids.add(concept_name)

                # Add relationships as edges
                for rel in concept.get("relationships", []):
                    if rel and rel != concept_name:
                        edges.append({
                            "source": concept_name,
                            "target": rel.lower(),
                            "type": "related"
                        })

        # Get insights as special nodes
        insights = await state.cognitive_core.philosophical_memory.get_insights(20)
        for i, insight in enumerate(insights):
            insight_id = f"insight_{i}"
            nodes.append({
                "id": insight_id,
                "label": f" Insight {i+1}",
                "strength": insight.get("wisdom_level", 1),
                "definition": insight.get("insight", "")[:100],
                "type": "insight"
            })

        return {
            "nodes": nodes,
            "edges": edges,
            "stats": {
                "total_nodes": len(nodes),
                "total_edges": len(edges),
                "concepts": len([n for n in nodes if n.get("type") != "insight"]),
                "insights": len([n for n in nodes if n.get("type") == "insight"])
            }
        }
    except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))


# ============================================================================
# EXPORT/IMPORT MEMORY ENDPOINTS
# ============================================================================

@router.get("/memory/export")
async def export_memory():
    """Export AI's memory and knowledge"""
    try:
        # Gather all memory data
        concepts = await state.cognitive_core.semantic_memory.get_all_concepts(500)
        episodes = await state.cognitive_core.episodic_memory.recall_recent_episodes(100)
        insights = await state.cognitive_core.philosophical_memory.get_insights(100)
        questions = await state.cognitive_core.philosophical_memory.get_unanswered_questions(100)
        metrics = await state.cognitive_core.growth_tracker.calculate_metrics()
```

```
        stage = await state.cognitive_core.growth_tracker.get_current_stage()

        export_data = {
            "version": "1.0",
            "exported_at": datetime.now(timezone.utc).isoformat(),
            "growth_stage": stage,
            "metrics": metrics,
            "semantic_memory": concepts,
            "episodic_memory": episodes,
            "insights": insights,
            "questions": questions
        }

        return export_data
    except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))


class MemoryImport(BaseModel):
    semantic_memory: Optional[List[Dict]] = None
    insights: Optional[List[Dict]] = None

@router.post("/memory/import")
async def import_memory(data: MemoryImport):
    """Import memory data"""
    try:
        imported = {"concepts": 0, "insights": 0}

        # Import semantic memory
        if data.semantic_memory:
            for concept in data.semantic_memory:
                if concept.get("concept"):
                    await state.cognitive_core.semantic_memory.store_concept(
                        concept=concept.get("concept"),
                        definition=concept.get("definition", ""),
                        relationships=concept.get("relationships", []),
                        metadata=concept.get("metadata", {})
                    )
                    imported["concepts"] += 1

        # Import insights
        if data.insights:
            for insight in data.insights:
                if insight.get("insight"):
                    await state.cognitive_core.philosophical_memory.store_insight(
                        insight=insight.get("insight"),
                        wisdom_level=insight.get("wisdom_level", 1)
                    )
                    imported["insights"] += 1

        return {"status": "success", "imported": imported}
    except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))


# ============================================================================
# EXPORT CONVERSATIONS ENDPOINTS
# ============================================================================

@router.get("/chat/export/{session_id}")
async def export_conversation(session_id: str, format: str = "markdown"):
    """Export conversation as markdown or JSON"""
    try:
        history = await get_conversation_history(session_id, limit=500)

        if format == "json":
            return {
                "session_id": session_id,
                "exported_at": datetime.now(timezone.utc).isoformat(),
                "messages": history
            }

        # Markdown format
        md_content = f"# Quantum AI Conversation\n\n"
        md_content += f"**Session:** {session_id}\n"
        md_content += f"**Exported:** {datetime.now(timezone.utc).strftime('%Y-%m-%d %H:%M UTC')}\n\n"
        md_content += "---\n\n"

        for msg in history:
            role = "**You:**" if msg.get("role") == "user" else "**Quantum AI:**"
            content = msg.get("content", "")
            timestamp = msg.get("timestamp", "")[:19] if msg.get("timestamp") else ""
            md_content += f"{role}\n{content}\n\n"

        return {"markdown": md_content, "session_id": session_id}
    except Exception as e:
```

```
        raise HTTPException(status_code=500, detail=str(e))


# ==========================================================================
```

routes_cognitive_unified.py

```
"""
 Unified Cognitive Routes
==========================
Additional API endpoints for the unified cognitive engine features:
- Explanation mode
- Personality info
- Growth stages
- Quotes
- WolframAlpha queries
- System status
"""

from fastapi import APIRouter, HTTPException
from pydantic import BaseModel
from typing import Optional, List, Dict
import logging

from unified_cognitive_engine import get_unified_cognitive_engine

logger = logging.getLogger("quantum_ai")
router = APIRouter(prefix="/api/cognitive")


# ============================================================================
# Pydantic Models
# ============================================================================

class QueryRequest(BaseModel):
    """QueryRequest - Auto-documented by self-evolution."""
    query: str
    include_explanation: bool = False

class QuoteRequest(BaseModel):
    """QuoteRequest - Auto-documented by self-evolution."""
    category: Optional[str] = None

class QuestionRequest(BaseModel):
    """QuestionRequest - Auto-documented by self-evolution."""
    topic: str = "consciousness"
    level: Optional[str] = None


# ============================================================================
# Routes
# ============================================================================

@router.get("/status")
async def get_cognitive_status():
    """Get complete unified cognitive engine status"""
    try:
        engine = get_unified_cognitive_engine()
        return {
            "success": True,
            "data": engine.get_status()
        }
    except Exception as e:
        logger.error(f"Status error: {str(e)}")
        raise HTTPException(status_code=500, detail=str(e))


@router.get("/personality")
async def get_personality():
    """Get current personality profile and traits"""
    try:
        engine = get_unified_cognitive_engine()
        return {
            "success": True,
            "data": {
                "summary": engine.personality.get_personality_summary(),
                "traits": engine.personality.traits,
                "growth_stage": engine.personality.get_current_stage(),
                "interactions": engine.personality.interaction_count,
                "insights": engine.personality.insights_generated,
                "questions_asked": engine.personality.questions_asked
            }
        }
    except Exception as e:
        logger.error(f"Personality error: {str(e)}")
        raise HTTPException(status_code=500, detail=str(e))


@router.get("/growth")
```

```
async def get_growth_stage():
    """Get current growth stage and progress"""
    try:
        engine = get_unified_cognitive_engine()
        current_stage = engine.personality.get_current_stage()
        next_stage_idx = min(engine.personality.current_stage + 1, 6)
        next_stage = engine.personality.growth_stages[next_stage_idx]

        return {
            "success": True,
            "data": {
                "current_stage": current_stage,
                "next_stage": next_stage,
                "interactions_to_next": next_stage['threshold'] - engine.personality.interaction_count,
                "total_interactions": engine.personality.interaction_count
            }
        }
    except Exception as e:
        logger.error(f"Growth error: {str(e)}")
        raise HTTPException(status_code=500, detail=str(e))


@router.post("/quote")
async def get_quote(request: QuoteRequest):
    """Get a philosophical quote"""
    try:
        engine = get_unified_cognitive_engine()
        quote = engine.quote.get_quote(request.category)
        return {
            "success": True,
            "data": quote
        }
    except Exception as e:
        logger.error(f"Quote error: {str(e)}")
        raise HTTPException(status_code=500, detail=str(e))


@router.post("/question")
async def generate_question(request: QuestionRequest):
    """Generate a Bloom's Taxonomy question"""
    try:
        engine = get_unified_cognitive_engine()
        question = engine.bloom.generate_question(
            level=request.level,
            topic=request.topic
        )
        return {
            "success": True,
            "data": question
        }
    except Exception as e:
        logger.error(f"Question error: {str(e)}")
        raise HTTPException(status_code=500, detail=str(e))


@router.post("/wolfram")
async def query_wolfram(request: QueryRequest):
    """Query WolframAlpha for factual information"""
    try:
        engine = get_unified_cognitive_engine()
        result = engine.wolfram.query(request.query)
        return {
            "success": True,
            "data": {
                "query": request.query,
                "result": result or "No result found",
                "is_factual": engine.wolfram.is_factual_query(request.query)
            }
        }
    except Exception as e:
        logger.error(f"WolframAlpha error: {str(e)}")
        raise HTTPException(status_code=500, detail=str(e))


@router.get("/orch-or")
async def get_orch_or_status():
    """Get Orchestrated Objective Reduction (quantum consciousness) status"""
    try:
        engine = get_unified_cognitive_engine()
        return {
            "success": True,
            "data": engine.orch_or.get_status()
        }
    except Exception as e:
        logger.error(f"Orch OR error: {str(e)}")
        raise HTTPException(status_code=500, detail=str(e))
```

```
@router.get("/explanation/history")
async def get_explanation_history():
    """Get recent explanations generated"""
    try:
        engine = get_unified_cognitive_engine()
        return {
            "success": True,
            "data": {
                "total": engine.explanation.explanations_generated,
                "recent": engine.explanation.reasoning_history[-10:]  # Last 10
            }
        }
    except Exception as e:
        logger.error(f"Explanation history error: {str(e)}")
        raise HTTPException(status_code=500, detail=str(e))


@router.get("/engines")
async def get_all_engines_status():
    """Get status of all cognitive engines"""
    try:
        engine = get_unified_cognitive_engine()
        return {
            "success": True,
            "data": {
                "orch_or": {
                    "conscious_moments": len(engine.orch_or.collapse_events),
                    "orchestration": engine.orch_or.orchestration
                },
                "bloom": {
                    "questions_generated": engine.bloom.questions_generated,
                    "levels": list(engine.bloom.levels.keys())
                },
                "vader": {
                    "texts_analyzed": engine.vader.texts_analyzed
                },
                "quote": {
                    "categories": list(engine.quote.quotes.keys()),
                    "total_quotes": sum(len(q) for q in engine.quote.quotes.values())
                },
                "wolfram": {
                    "queries_made": engine.wolfram.queries_made,
                    "cache_size": len(engine.wolfram.cache)
                },
                "personality": {
                    "stage": engine.personality.current_stage,
                    "stage_name": engine.personality.get_current_stage()['name']
                },
                "semantic_collapse": {
                    "collapse_events": len(engine.semantic_collapse.collapse_events)
                },
                "explanation": {
                    "explanations_generated": engine.explanation.explanations_generated
                }
            }
        }
    except Exception as e:
        logger.error(f"Engines status error: {str(e)}")
        raise HTTPException(status_code=500, detail=str(e))


@router.post("/process")
async def process_with_unified_engine(request: QueryRequest):
    """
    Process input through all unified cognitive engines.
    This is an alternative to the main chat endpoint that returns full cognitive data.
    """
    try:
        engine = get_unified_cognitive_engine()
        result = engine.process(request.query)

        # Optionally include full explanation
        if not request.include_explanation:
            result.pop('explanation', None)

        return {
            "success": True,
            "data": result
        }
    except Exception as e:
        logger.error(f"Process error: {str(e)}")
        raise HTTPException(status_code=500, detail=str(e))
```

routes_explorer.py

```
"""
 Explorer Routes
===================
File explorer, uploads, shared links, root and health endpoints.
"""

from fastapi import APIRouter, HTTPException, UploadFile, File
from pydantic import BaseModel
from typing import List, Dict, Any, Optional
import os
import logging

import shared_state as state

logger = logging.getLogger("quantum_ai")
router = APIRouter(prefix="/api")


class ExplorationRequest(BaseModel):
    """ExplorationRequest - Auto-documented by self-evolution."""
    exploration_type: str
    target: str
    options: Optional[Dict] = None


@router.post("/explorer/explore")
async def explore(request: ExplorationRequest):
    """
    ARTICLE 2 - Universal Explorer

    Explore filesystem, internet, or process documents.
    """
    try:
        result = await state.universal_explorer.explore(
            request.exploration_type,
            request.target,
            request.options
        )
        return result
    except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))

@router.get("/explorer/capabilities")
async def get_explorer_capabilities():
    """Get current exploration capabilities"""
    try:
        capabilities = await state.universal_explorer.get_capabilities()
        return capabilities
    except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))

@router.get("/explorer/history")
async def get_exploration_history(limit: int = 50):
    """Get exploration history"""
    try:
        history = await state.universal_explorer.get_exploration_history(limit)
        return {"history": history}
    except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))


# ==============================================================================
# VOICE / TTS ENDPOINTS
# ==============================================================================


@router.post("/upload")
async def upload_file(file: UploadFile = File(...)):
    """
    Upload a file to the server for processing
    """
    try:
        # Create uploads directory if it doesn't exist
        upload_dir = "/app/uploads"
        os.makedirs(upload_dir, exist_ok=True)

        # Save file
        file_path = os.path.join(upload_dir, file.filename)

        with open(file_path, "wb") as buffer:
            shutil.copyfileobj(file.file, buffer)
```

```
        # Get file info
        file_size = os.path.getsize(file_path)

        return {
            "filename": file.filename,
            "path": file_path,
            "size": file_size,
            "status": "uploaded",
            "message": f"File uploaded successfully. Use path: {file_path}"
        }
    except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))

@router.get("/uploads/list")
async def list_uploads():
    """List all uploaded files"""
    try:
        upload_dir = "/app/uploads"
        if not os.path.exists(upload_dir):
            return {"files": []}

        files = []
        for filename in os.listdir(upload_dir):
            file_path = os.path.join(upload_dir, filename)
            if os.path.isfile(file_path):
                files.append({
                    "filename": filename,
                    "path": file_path,
                    "size": os.path.getsize(file_path)
                })

        return {"files": files}
    except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))


# ============================================================================
# SHARED LINK ENDPOINTS
# ============================================================================

@router.post("/shared-link/download")
async def download_shared_link(url: str, filename: str = None):
    """
    Download file from shared link (Google Drive, Dropbox, OneDrive)
    """
    try:
        result = await state.shared_link.download_from_link(url, filename)
        return result
    except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))

@router.get("/shared-link/instructions/{service}")
async def get_sharing_instructions(service: str):
    """
    Get instructions for sharing files from different services
    """
    try:
        instructions = state.shared_link.get_instructions(service)
        return {"service": service, "instructions": instructions}
    except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))


# ============================================================================
# SYSTEM ENDPOINTS
# ============================================================================

@router.get("/")
async def root():
    """Root endpoint - Quantum AI welcome"""
    return {
        "message": " Quantum AI - True Questioning Intelligence",
        "prime_directive": covenant_manager.PRIME_DIRECTIVE,
        "status": "Active and questioning",
        "version": "1.0.0"
    }

@router.get("/health")
async def health_check():
    """Health check endpoint"""
    try:
        # Test database connection
        await state.db.command("ping")
        return {
            "status": "healthy",
            "database": "connected",
```

```
            "cognitive_core": "active",
            "covenant": "honored"
        }
    except Exception as e:
        raise HTTPException(status_code=503, detail=f"Service unhealthy: {str(e)}")


# ============================================================================
# QUANTUM BRAIN STATUS & CONTROL
# ============================================================================
```

routes_extras.py

```
"""
 Extras Routes
=================
Covenant, safety, dictionary, dream, personality, quotes, text analysis, prebo.
"""

from fastapi import APIRouter, HTTPException
from pydantic import BaseModel
from typing import List, Dict, Any, Optional
import os
import logging
from datetime import datetime, timezone

import shared_state as state

logger = logging.getLogger("quantum_ai")
router = APIRouter(prefix="/api")


class CovenantCheck(BaseModel):
    """CovenantCheck - Auto-documented by self-evolution."""
    action: str
    context: Dict = {}

class SafetySettings(BaseModel):
    """SafetySettings - Auto-documented by self-evolution."""
    settings: Dict


@router.get("/growth/notifications")
async def get_growth_notifications():
    """Get recent growth notifications and stage changes"""
    try:
        # Get recent stage advancements
        stage_events = await state.db.growth_metrics.find(
            {"event_type": "stage_advancement"},
            {"_id": 0}
        ).sort("timestamp", -1).limit(10).to_list(10)

        # Get current stage
        current_stage = await state.cognitive_core.growth_tracker.get_current_stage()
        metrics = await state.cognitive_core.growth_tracker.calculate_metrics()

        return {
            "current_stage": current_stage,
            "metrics": metrics,
            "recent_advancements": stage_events,
            "notifications": [
                {
                    "type": "stage_advancement",
                    "message": f"Reached {e['details']['new_stage']} stage!",
                    "timestamp": e["timestamp"]
                }
                for e in stage_events
            ]
        }
    except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))


# ============================================================================
# COVENANT MANAGER ENDPOINTS
# ============================================================================

@router.get("/covenant/status")
async def get_covenant_status():
    """
    ARTICLE 1 - Covenant Status

    Get current covenant compliance status.
    """
    try:
        status = await state.covenant_manager.get_covenant_status()
        return status
    except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))

@router.get("/covenant/articles")
async def get_covenant_articles():
    """Get all covenant articles"""
    try:
        articles = await state.covenant_manager.get_all_articles()
```

```
        return {"articles": articles}
    except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))

@router.post("/covenant/check")
async def check_covenant_compliance(check: CovenantCheck):
    """Check if an action complies with the covenant"""
    try:
        result = await state.covenant_manager.check_compliance(check.action, check.context)
        return result
    except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))

@router.post("/covenant/acknowledge")
async def acknowledge_covenant(acknowledger: str = "Quantum AI", stage: str = "Initialization"):
    """Acknowledge the covenant (milestone event)"""
    try:
        result = await state.covenant_manager.acknowledge_covenant(acknowledger, stage)
        return result
    except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))


# ============================================================================
# SAFETY BOUNDARIES ENDPOINTS
# ============================================================================

@router.get("/safety/settings")
async def get_safety_settings():
    """Get current safety boundary settings"""
    try:
        settings = await state.safety_manager.get_settings()
        return settings
    except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))

@router.put("/safety/settings")
async def update_safety_settings(settings: SafetySettings):
    """Update safety boundary settings"""
    try:
        updated = await state.safety_manager.update_settings(settings.settings)
        return updated
    except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))

@router.post("/safety/reset")
async def reset_safety_settings():
    """Reset safety settings to defaults"""
    try:
        defaults = await state.safety_manager.reset_to_defaults()
        return defaults
    except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))


# ============================================================================
# DICTIONARY ENDPOINTS
# ============================================================================

@router.get("/dictionary/lookup/{word}")
async def lookup_word(word: str):
    """
    Look up a word in dictionaries and add to semantic memory
    """
    try:
        result = await state.dictionary.lookup_and_store(word)
        return result
    except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))

@router.post("/dictionary/batch-load")
async def batch_load_dictionary(limit: int = 100):
    """
    Load common words into dictionary (background task)
    """
    try:
        result = await state.dictionary.batch_load_common_words(limit)
        return result
    except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))

@router.get("/dictionary/stats")
async def dictionary_stats():
    """Get dictionary statistics"""
    try:
        total_concepts = await state.db.semantic_memory.count_documents({})
```

```
        with_definitions = await state.db.semantic_memory.count_documents({"metadata.sources": {"$exists": True}})

        return {
            "total_words": total_concepts,
            "dictionary_entries": with_definitions,
            "learned_organically": total_concepts - with_definitions
        }
    except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))


# ==============================================================================
# FILE UPLOAD ENDPOINTS
# ==============================================================================

from fastapi import UploadFile, File
import os


@router.get("/growth/stats")
async def get_growth_stats():
    """Get overall AI growth statistics from research and learning"""
    try:
        # Get main growth stats
        growth = await state.db.ai_growth.find_one({'_id': 'main'})

        # Get research metrics
        research_metrics = await state.db.growth_metrics.find_one({'type': 'autonomous_research'})

        # Get recent research sessions
        sessions = await state.db.autonomous_research_sessions.find().sort('completed_at', -1).limit(5).to_list(5)

        # Get concept counts
        semantic_count = await state.db.semantic_memory.count_documents({})
        research_count = await state.db.research_history.count_documents({})

        return {
            "overall": {
                "total_research_sessions": growth.get('total_research_sessions', 0) if growth else 0,
                "total_research_minutes": growth.get('total_research_minutes', 0) if growth else 0,
                "total_concepts_learned": growth.get('total_concepts_learned', 0) if growth else 0,
                "total_insights": growth.get('total_insights', 0) if growth else 0,
                "last_research": growth.get('last_research_session', 'Never') if growth else 'Never'
            },
            "memory": {
                "semantic_memories": semantic_count,
                "research_entries": research_count
            },
            "recent_research": research_metrics.get('recent_topics', [])[-10:] if research_metrics else [],
            "recent_sessions": [
                {
                    "completed": s.get('completed_at', ''),
                    "duration": s.get('duration_minutes', 0),
                    "concepts": s.get('concepts_learned', 0),
                    "topics": len(s.get('topics_researched', []))
                } for s in sessions
            ]
        }
    except Exception as e:
        return {
            "overall": {"total_research_sessions": 0, "total_concepts_learned": 0},
            "error": str(e)
        }


# ==============================================================================
# DREAM STATE
# ==============================================================================

@router.get("/dream/status")
async def get_dream_status():
    """Get current dream state status"""
    try:
        from dream_state import get_dream_engine

        engine = get_dream_engine(db)
        return engine.get_status()
    except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))


@router.get("/dream/history")
async def get_dream_history(limit: int = 10):
    """Get dream history - what AI did while idle"""
    try:
        from dream_state import get_dream_engine
```

```
        engine = get_dream_engine(db)
        return {
            "dreams": engine.get_dream_history(limit),
            "insights": engine.insights_gained[-10:]
        }
    except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))


@router.post("/dream/enter")
async def enter_dream_state():
    """Manually enter dream state"""
    try:
        from dream_state import get_dream_engine

        engine = get_dream_engine(db)
        return engine.enter_dream_state()
    except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))


@router.post("/dream/wake")
async def wake_from_dream():
    """Wake from dream state"""
    try:
        from dream_state import get_dream_engine

        engine = get_dream_engine(db)
        return engine.exit_dream_state()
    except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))


@router.get("/dream/synthesize")
async def synthesize_dream_insight():
    """Generate a Dream Engine concept entanglement insight"""
    try:
        from quantum_brain import get_quantum_brain

        brain = await get_quantum_brain(db)
        insight = brain.synthesize_dream_insight()

        return {
            "insight": insight,
            "timestamp": datetime.now(timezone.utc).isoformat(),
            "type": "concept_entanglement"
        }
    except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))


@router.get("/dream/synthesize/batch")
async def synthesize_dream_batch(count: int = 5):
    """Generate multiple Dream Engine insights"""
    try:
        from quantum_brain import get_quantum_brain

        brain = await get_quantum_brain(db)
        insights = [brain.synthesize_dream_insight() for _ in range(min(count, 10))]

        return {
            "insights": insights,
            "count": len(insights),
            "timestamp": datetime.now(timezone.utc).isoformat()
        }
    except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))


@router.get("/quotes/random")
async def get_random_quote(context: str = "general"):
    """Get a random movie/philosophical quote"""
    try:
        from quantum_brain import get_quantum_brain

        brain = await get_quantum_brain(db)
        # Force a quote by checking many times
        for _ in range(20):
            quote = brain._get_random_quote(context)
            if quote:
                return {"quote": quote, "context": context}

        # Fallback - pick directly
        import random
        category = context if context in brain.movie_quotes else 'general'
```

```
        quote = random.choice(brain.movie_quotes[category])
        return {"quote": quote, "context": category}
    except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))


# ============================================================================
# PERSONALITY
# ============================================================================

@router.get("/personality/summary")
async def get_personality_summary():
    """Get AI personality summary"""
    try:
        from personality_engine import get_personality_engine

        engine = get_personality_engine(db)
        return {
            "summary": engine.get_personality_summary(),
            "traits": engine.traits,
            "beliefs": engine.beliefs,
            "interests": engine.interests
        }
    except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))


@router.post("/personality/opinion")
async def form_opinion(topic: str, stance: str, confidence: float = 0.7):
    """Have AI form an opinion on a topic"""
    try:
        from personality_engine import get_personality_engine

        engine = get_personality_engine(db)
        engine.form_opinion(topic, stance, confidence)

        return {
            "success": True,
            "topic": topic,
            "stance": stance,
            "confidence": confidence
        }
    except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))


# ============================================================================
# TEXT ANALYZER
# ============================================================================

@router.post("/analyze/text")
async def analyze_text(text: str):
    """Analyze text to determine if it sounds human or LLM"""
    try:
        from text_analyzer import get_text_analyzer

        analyzer = get_text_analyzer()
        analysis = analyzer.analyze(text)
        suggestions = analyzer.get_improvement_suggestions(text)

        return {
            "analysis": analysis,
            "suggestions": suggestions
        }
    except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))


@router.post("/analyze/humanize")
async def humanize_text(text: str):
    """Make text sound more human"""
    try:
        from text_analyzer import get_text_analyzer

        analyzer = get_text_analyzer()
        humanized = analyzer.make_more_human(text)

        return {
            "original": text,
            "humanized": humanized
        }
    except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))


# ============================================================================
```

```
# XANADU PRE-BORN-OPPENHEIMER QUANTUM CHEMISTRY
# ==============================================================================

@router.post("/quantum/prebo/simulate")
async def simulate_photochemistry(molecule: str = "NH3_BF3", time_steps: int = 10):
    """Run Xanadu Pre-Born-Oppenheimer photochemical simulation"""
    try:
        from xanadu_prebo import get_quantum_orchestrator
        orchestrator = get_quantum_orchestrator()
        result = orchestrator.run_simulation(molecule, "photochemical", time_steps)
        return result
    except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))

@router.post("/quantum/prebo/analyze")
async def analyze_reaction(molecule: str, reaction_type: str = "photochemical"):
    """Analyze reaction to determine best quantum algorithm"""
    try:
        from xanadu_prebo import get_quantum_orchestrator
        orchestrator = get_quantum_orchestrator()
        return orchestrator.analyze_reaction(molecule, reaction_type)
    except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))

@router.get("/quantum/prebo/resources/{molecule}")
async def estimate_resources(molecule: str):
    """Estimate quantum resources for molecule simulation"""
    try:
        from xanadu_prebo import get_quantum_orchestrator
        orchestrator = get_quantum_orchestrator()
        return orchestrator.get_resource_estimate(molecule)
    except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))

@router.get("/quantum/prebo/grid/{molecule}")
async def get_grid_params(molecule: str):
    """Get optimal real-space grid parameters from analysis"""
    try:
        from xanadu_prebo import get_prebo_simulator
        sim = get_prebo_simulator()
        return sim.calculate_grid_bounds(molecule)
    except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))


# ==========================================
# DATABASE SYNC ENDPOINTS
# ==========================================

@router.get("/db/stats")
async def get_db_stats():
    """Get database statistics for all collections."""
    stats = {"collections": {}, "total": 0}

    collections = [
        "semantic_memory", "philosophical_memory", "episodic_memory",
        "growth_metrics", "chat_history", "research_history",
        "processed_documents", "evolution_log", "covenant_log"
    ]

    for coll_name in collections:
        try:
            count = await state.db[coll_name].count_documents({})
            stats["collections"][coll_name] = count
            stats["total"] += count
        except Exception:
            stats["collections"][coll_name] = 0

    # Get growth stage
    try:
        growth = await state.db["philosophical_memory"].count_documents({})
        questions = await state.db["philosophical_memory"].count_documents({"type": {"$ne": "insight"}})
        insights = await state.db["philosophical_memory"].count_documents({"type": "insight"})
        stats["growth"] = {
            "questions": questions,
            "insights": insights,
            "concepts": stats["collections"].get("semantic_memory", 0)
        }
    except Exception:
        stats["growth"] = {}

    return stats


@router.post("/db/export")
async def export_database():
```

```
    """Export database to a downloadable JSON file."""
    import json
    from bson import ObjectId
    from fastapi.responses import JSONResponse

    class Encoder(json.JSONEncoder):
        """Encoder - Auto-documented by self-evolution."""
        def default(self, obj):
            if isinstance(obj, ObjectId):
                return str(obj)
            if isinstance(obj, datetime):
                return obj.isoformat()
            return super().default(obj)

    export_data = {
        "metadata": {
            "exported_at": datetime.now(timezone.utc).isoformat(),
            "source": "preview"
        },
        "collections": {}
    }

    collections = [
        "semantic_memory", "philosophical_memory", "episodic_memory",
        "growth_metrics", "chat_history", "research_history"
    ]

    for coll_name in collections:
        try:
            docs = await state.db[coll_name].find({}, {"_id": 0}).to_list(length=10000)
            export_data["collections"][coll_name] = docs
        except Exception:
            export_data["collections"][coll_name] = []

    return JSONResponse(content=json.loads(json.dumps(export_data, cls=Encoder)))


# Router exported for inclusion in main server.py
```

routes_media.py

```
"""
 Media Routes
================
Voice TTS/transcription and image analysis/generation.
"""

from fastapi import APIRouter, HTTPException, UploadFile, File
from pydantic import BaseModel
from typing import List, Dict, Any, Optional
import os
import uuid
import base64
import logging

import shared_state as state

logger = logging.getLogger("quantum_ai")
router = APIRouter(prefix="/api")


class TTSRequest(BaseModel):
    """TTSRequest - Auto-documented by self-evolution."""
    text: str
    voice: str = "default"


class ImageGenRequest(BaseModel):
    """ImageGenRequest - Auto-documented by self-evolution."""
    prompt: str
    style: str = "default"


@router.post("/voice/tts")
async def text_to_speech(request: TTSRequest):
    """
    Text-to-Speech endpoint
    Returns text formatted for browser's Web Speech API
    """
    try:
        # Clean the text for speech
        text = request.text.strip()
        if not text:
            raise HTTPException(status_code=400, detail="Text is required")

        # For now, return the text with speech instructions
        # Can integrate with ElevenLabs or OpenAI TTS later
        return {
            "text": text,
            "voice": request.voice,
            "instructions": "Use browser's speechSynthesis API",
            "ssml": f"<speak>{text}</speak>"
        }
    except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))

@router.post("/voice/transcribe")
async def transcribe_audio(audio: UploadFile = File(...)):
    """
    Speech-to-Text endpoint (placeholder)
    Would integrate with Whisper or similar
    """
    try:
        # Save audio temporarily
        audio_path = f"/tmp/audio_{uuid.uuid4().hex}.webm"
        with open(audio_path, "wb") as f:
            content = await audio.read()
            f.write(content)

        # For now, return instructions for browser's Web Speech API
        return {
            "status": "received",
            "message": "Audio received. Use browser's SpeechRecognition API for real-time transcription.",
            "file_size": len(content)
        }
    except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))


# ============================================================================
# KNOWLEDGE GRAPH ENDPOINTS
# ============================================================================
```

```
@router.get("/knowledge/graph")

@router.post("/image/analyze")
async def analyze_image(file: UploadFile = File(...)):
    """Analyze an uploaded image"""
    try:
        import base64

        # Read and encode image
        content = await file.read()
        image_base64 = base64.b64encode(content).decode('utf-8')

        # Save to uploads
        file_path = f"/app/uploads/{file.filename}"
        with open(file_path, "wb") as f:
            f.write(content)

        # Use LLM with image (if supported) or return basic info
        file_size = len(content)

        # For now, return image metadata
        # Could integrate with vision models later
        return {
            "status": "success",
            "filename": file.filename,
            "size": file_size,
            "path": file_path,
            "message": "Image uploaded. You can ask me about it in chat!",
            "preview": f"{image_base64[:100]}..."
        }
    except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))


class ImageGenRequest(BaseModel):
    prompt: str
    size: Optional[str] = "1024x1024"

@router.post("/image/generate")
async def generate_image(request: ImageGenRequest):
    """Generate an image from text prompt"""
    try:
        import base64
        from emergentintegrations.llm.openai.image_generation import OpenAIImageGeneration

        api_key = os.environ.get("EMERGENT_LLM_KEY")
        if not api_key:
            raise HTTPException(status_code=500, detail="Image generation not configured")

        # Generate image
        image_gen = OpenAIImageGeneration(api_key=api_key)
        images = await image_gen.generate_images(
            prompt=request.prompt,
            model="gpt-image-1.5",
            number_of_images=1
        )

        if images and len(images) > 0:
            image_base64 = base64.b64encode(images[0]).decode('utf-8')

            # Save to uploads
            filename = f"generated_{uuid.uuid4().hex[:8]}.png"
            file_path = f"/app/uploads/{filename}"
            with open(file_path, "wb") as f:
                f.write(images[0])

            return {
                "status": "success",
                "image_base64": image_base64,
                "filename": filename,
                "path": file_path,
                "prompt": request.prompt
            }
        else:
            raise HTTPException(status_code=500, detail="No image generated")

    except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))


# ============================================================================
# GROWTH NOTIFICATIONS ENDPOINT
# ============================================================================
```

self_evolution_analysis.py

```
"""
 Self-Evolution Engine  Analysis Module
========================================
Code reading, AST analysis, and improvement identification.
"""

import ast
import re
from typing import Dict, List


class AnalysisMixin:
    """Mixin providing code analysis and improvement identification."""

    def read_own_code(self, filename: str) -> Dict:
        """Read and analyze a source file."""
        filepath = self.code_dir / filename

        if not filepath.exists():
            return {'error': f'File not found: {filename}'}

        try:
            with open(filepath, 'r') as f:
                code = f.read()

            tree = ast.parse(code)

            analysis = {
                'filename': filename,
                'filepath': str(filepath),
                'size_bytes': len(code),
                'lines': len(code.split('\n')),
                'classes': [],
                'functions': [],
                'imports': [],
                'docstring': ast.get_docstring(tree),
                'code': code
            }

            for node in ast.walk(tree):
                if isinstance(node, ast.ClassDef):
                    analysis['classes'].append({
                        'name': node.name,
                        'methods': [m.name for m in node.body if isinstance(m, ast.FunctionDef)],
                        'docstring': ast.get_docstring(node)
                    })
                elif isinstance(node, ast.FunctionDef) and not isinstance(node, ast.AsyncFunctionDef):
                    if node.col_offset == 0:
                        analysis['functions'].append({
                            'name': node.name,
                            'args': [a.arg for a in node.args.args],
                            'docstring': ast.get_docstring(node)
                        })
                elif isinstance(node, ast.Import):
                    analysis['imports'].extend([alias.name for alias in node.names])
                elif isinstance(node, ast.ImportFrom):
                    analysis['imports'].append(f"{node.module}")

            return analysis

        except Exception as e:
            return {'error': str(e), 'filename': filename}

    def analyze_all_code(self) -> Dict:
        """Analyze all modifiable code files."""
        analyses = {}

        for filename in self.modifiable_files:
            analyses[filename] = self.read_own_code(filename)

        return {
            'files_analyzed': len(analyses),
            'total_lines': sum(a.get('lines', 0) for a in analyses.values() if 'error' not in a),
            'total_classes': sum(len(a.get('classes', [])) for a in analyses.values() if 'error' not in a),
            'total_functions': sum(len(a.get('functions', [])) for a in analyses.values() if 'error' not in a),
            'analyses': analyses
        }

    def identify_improvements(self, filename: str) -> List[Dict]:
        """Identify potential improvements in a file."""
        analysis = self.read_own_code(filename)
```

```
    if 'error' in analysis:
        return []

    improvements = []
    code = analysis.get('code', '')
    lines = code.split('\n')

    self._check_missing_docstrings(analysis, improvements)
    self._check_file_length(analysis, filename, improvements)
    self._check_class_size(analysis, improvements)
    self._check_bare_excepts(lines, filename, improvements)
    self._check_long_functions(analysis, code, improvements)
    self._check_todo_items(lines, filename, improvements)
    self._check_unused_imports(analysis, improvements)

    return improvements

def _check_missing_docstrings(self, analysis: Dict, improvements: List[Dict]):
    """Check for missing docstrings on classes and functions."""
    for cls in analysis.get('classes', []):
        if not cls.get('docstring'):
            improvements.append({
                'type': 'missing_docstring',
                'target': f"class {cls['name']}",
                'suggestion': f"Add docstring to class {cls['name']}",
                'priority': 2
            })

    for func in analysis.get('functions', []):
        if not func.get('docstring'):
            improvements.append({
                'type': 'missing_docstring',
                'target': f"function {func['name']}",
                'suggestion': f"Add docstring to function {func['name']}",
                'priority': 2
            })

def _check_file_length(self, analysis: Dict, filename: str, improvements: List[Dict]):
    """Check if file exceeds max line count."""
    if analysis.get('lines', 0) > self.max_file_lines:
        improvements.append({
            'type': 'file_too_long',
            'target': filename,
            'suggestion': f'File has {analysis["lines"]} lines (max {self.max_file_lines})  will auto-split',
            'priority': 3
        })

def _check_class_size(self, analysis: Dict, improvements: List[Dict]):
    """Check for classes with too many methods."""
    for cls in analysis.get('classes', []):
        if len(cls.get('methods', [])) > 15:
            improvements.append({
                'type': 'class_too_large',
                'target': f"class {cls['name']}",
                'suggestion': f"Class {cls['name']} has {len(cls['methods'])} methods - consider splitting",
                'priority': 3
            })

def _check_bare_excepts(self, lines: List[str], filename: str, improvements: List[Dict]):
    """Check for bare except clauses."""
    bare_excepts = sum(1 for line in lines if 'except:' in line and 'except Exception' not in line)
    if bare_excepts > 0:
        improvements.append({
            'type': 'bare_except',
            'target': filename,
            'suggestion': f'{bare_excepts} bare except clause(s) - should catch specific exceptions',
            'priority': 2
        })

def _check_long_functions(self, analysis: Dict, code: str, improvements: List[Dict]):
    """Check for functions exceeding max line count."""
    for cls in analysis.get('classes', []):
        for method_name in cls.get('methods', []):
            try:
                tree = ast.parse(code)
                for node in ast.walk(tree):
                    if isinstance(node, (ast.FunctionDef, ast.AsyncFunctionDef)):
                        if node.name == method_name:
                            func_lines = node.end_lineno - node.lineno if hasattr(node, 'end_lineno') else 0
                            if func_lines > self.max_function_lines:
                                improvements.append({
                                    'type': 'long_function',
                                    'target': f"method {cls['name']}.{method_name}",
                                    'suggestion': f'Method is {func_lines} lines (max {self.max_function_lines})  will auto-split',
                                    'priority': 2
                                })
```

```
            except Exception:
                pass

    def _check_todo_items(self, lines: List[str], filename: str, improvements: List[Dict]):
        """Check for TODO/FIXME comments."""
        todo_count = sum(1 for line in lines if 'TODO' in line or 'FIXME' in line or 'HACK' in line)
        if todo_count > 0:
            improvements.append({
                'type': 'todo_items',
                'target': filename,
                'suggestion': f'{todo_count} TODO/FIXME comment(s) need attention',
                'priority': 1
            })

    def _check_unused_imports(self, analysis: Dict, improvements: List[Dict]):
        """Check for unused imports."""
        code_text = analysis.get('code', '')

        for imp in analysis.get('imports', []):
            if not imp or imp in ('os', 'sys', 're', 'json', 'typing'):
                continue

            imp_name = imp.split('.')[-1] if '.' in imp else imp
            import_line = None
            for line in code_text.split('\n'):
                stripped = line.strip()
                if f'from {imp} import' in stripped:
                    import_line = stripped
                    break
                elif stripped == f'import {imp}':
                    import_line = stripped
                    break

            if not import_line:
                continue

            if import_line.startswith('from '):
                from_match = re.match(r'^from\s+\S+\s+import\s+(.+)$', import_line)
                if from_match:
                    names = [n.strip().split(' as ')[-1].strip() for n in from_match.group(1).split(',')]
                    other_code = '\n'.join(l for l in code_text.split('\n') if l.strip() != import_line)
                    all_unused = all(not re.search(rf'\b{re.escape(n)}\b', other_code) for n in names)
                    if all_unused:
                        improvements.append({
                            'type': 'unused_import',
                            'target': f"import {imp}",
                            'suggestion': f'Import {imp} ({", ".join(names)}) may be unused',
                            'priority': 1
                        })
            elif import_line.startswith('import '):
                other_code = '\n'.join(l for l in code_text.split('\n') if l.strip() != import_line)
                if not re.search(rf'\b{re.escape(imp_name)}\b', other_code):
                    improvements.append({
                        'type': 'unused_import',
                        'target': f"import {imp}",
                        'suggestion': f'Import {imp} may be unused',
                        'priority': 1
                    })
```

self_evolution_core.py

```
"""
 Self-Evolution Engine  Core
==============================
The AI can read, analyze, and rewrite its own code.
Full autonomy with 60-90 day auto-improvement cycle.

Architecture:
  SelfEvolutionEngine inherits from five responsibility-focused mixins:
    - AnalysisMixin:   code reading, AST analysis, improvement identification
    - TransformMixin:  docstring, bare-except, unused-import fixes
    - SplittingMixin:  function-level splitting (init, regular methods)
    - FileOpsMixin:    file-level and class-level splitting
    - RunnerMixin:     auto-evolution cycle, improvement application
"""

import ast
import shutil
import hashlib
from typing import Dict, List
from datetime import datetime, timezone
from pathlib import Path

from self_evolution_analysis import AnalysisMixin
from self_evolution_transforms import TransformMixin
from self_evolution_splitting import SplittingMixin
from self_evolution_file_ops import FileOpsMixin
from self_evolution_runner import RunnerMixin


class SelfEvolutionEngine(AnalysisMixin, TransformMixin, SplittingMixin, FileOpsMixin, RunnerMixin):
    """
    Self-modification and evolution system.

    Features:
    - Read and analyze own source code
    - Identify improvements
    - Rewrite code autonomously
    - 60-90 day auto-improvement cycle
    - User-initiated improvements
    - Always backup before changes
    """

    def __init__(self, db=None, code_dir: str = "/app/backend"):
        self.db = db
        self.code_dir = Path(code_dir)
        self.backup_dir = Path(code_dir) / "backups"
        self.backup_dir.mkdir(exist_ok=True)

        # Evolution settings
        self.auto_evolution_interval_days = 75  # Between 60-90 days
        self.max_file_lines = 500  # Hard limit  files exceeding this get split
        self.max_function_lines = 50  # Functions exceeding this get refactored
        self.last_evolution = None
        self.evolution_log = []
        self.pending_improvements = []

        # Auto-discover ALL Python files in the code directory
        self.modifiable_files = self._discover_python_files()

        # Protected files (critical infrastructure - scanned but not rewritten)
        self.protected_files = [
            'server.py',
            'self_evolution_core.py'
        ]

    def _discover_python_files(self) -> List[str]:
        """Auto-discover ALL .py files in the code directory."""
        py_files = []
        for f in sorted(self.code_dir.iterdir()):
            if f.is_file() and f.suffix == '.py' and not f.name.startswith('__'):
                py_files.append(f.name)
        return py_files

    def backup_file(self, filename: str) -> str:
        """Create backup of a file before modification."""
        filepath = self.code_dir / filename

        if not filepath.exists():
            return None

        timestamp = datetime.now(timezone.utc).strftime('%Y%m%d_%H%M%S')
        backup_name = f"{filename}.{timestamp}.backup"
```

```
    backup_path = self.backup_dir / backup_name

    shutil.copy2(filepath, backup_path)
    return str(backup_path)

def rewrite_code(self, filename: str, new_code: str, reason: str) -> Dict:
    """Rewrite a code file. Always backs up first."""
    if filename in self.protected_files:
        return {
            'success': False,
            'error': f'{filename} is protected. Use careful_rewrite for protected files.',
            'filename': filename
        }

    if filename not in self.modifiable_files:
        return {
            'success': False,
            'error': f'{filename} is not in the modifiable files list.',
            'filename': filename
        }

    try:
        ast.parse(new_code)
    except SyntaxError as e:
        return {
            'success': False,
            'error': f'Syntax error in new code: {e}',
            'filename': filename
        }

    backup_path = self.backup_file(filename)
    filepath = self.code_dir / filename

    try:
        with open(filepath, 'w') as f:
            f.write(new_code)

        evolution_record = {
            'filename': filename,
            'reason': reason,
            'backup_path': backup_path,
            'timestamp': datetime.now(timezone.utc).isoformat(),
            'code_hash': hashlib.sha256(new_code.encode()).hexdigest()[:16]
        }

        self.evolution_log.append(evolution_record)
        self.last_evolution = datetime.now(timezone.utc)

        return {
            'success': True,
            'filename': filename,
            'backup_path': backup_path,
            'reason': reason,
            'timestamp': evolution_record['timestamp']
        }

    except Exception as e:
        return {
            'success': False,
            'error': str(e),
            'filename': filename
        }

def get_evolution_status(self) -> Dict:
    """Get current evolution status."""
    days_since = (datetime.now(timezone.utc) - self.last_evolution).days if self.last_evolution else None

    pending = (self.pending_improvements if isinstance(self.pending_improvements, int)
               else len(self.pending_improvements) if isinstance(self.pending_improvements, list)
               else 0)

    return {
        'last_evolution': self.last_evolution.isoformat() if self.last_evolution else None,
        'days_since_evolution': days_since,
        'next_auto_evolution_in': self.auto_evolution_interval_days - days_since if days_since else 0,
        'pending_improvements': pending,
        'total_evolutions': len(self.evolution_log),
        'modifiable_files': self.modifiable_files,
        'protected_files': self.protected_files,
        'rules': {
            'max_file_lines': self.max_file_lines,
            'max_function_lines': self.max_function_lines,
            'auto_evolution_interval_days': self.auto_evolution_interval_days
        }
    }
```

```
    def restore_from_backup(self, backup_path: str) -> Dict:
        """Restore a file from backup."""
        backup = Path(backup_path)

        if not backup.exists():
            return {'success': False, 'error': 'Backup not found'}

        filename = backup.name.split('.')[0] + '.py'
        original_path = self.code_dir / filename

        try:
            shutil.copy2(backup, original_path)
            return {
                'success': True,
                'restored': filename,
                'from_backup': backup_path
            }
        except Exception as e:
            return {'success': False, 'error': str(e)}


# Global instance
_evolution_engine = None


def get_evolution_engine(db=None) -> SelfEvolutionEngine:
    """Get or create evolution engine instance."""
    global _evolution_engine
    if _evolution_engine is None:
        _evolution_engine = SelfEvolutionEngine(db)
    return _evolution_engine
```

self_evolution_file_ops.py

```
"""
 Self-Evolution Engine  File Operations Module
==================================================
Operations for splitting long files and large classes.
"""

import ast
import re
from typing import Dict, List, Tuple, Optional


class FileOpsMixin:
    """Mixin providing file-level and class-level splitting operations."""

    async def _split_long_file(self, filename: str, code: str) -> Dict:
        """Split a file exceeding max lines into parts with proper imports."""
        lines = code.split('\n')
        if len(lines) <= self.max_file_lines:
            return {'success': False, 'reason': f'File is {len(lines)} lines, under {self.max_file_lines} limit'}

        try:
            tree = ast.parse(code)
        except SyntaxError:
            return {'success': False, 'reason': 'Cannot parse file'}

        base_name = filename.replace('.py', '')
        existing_parts = [f.name for f in self.code_dir.iterdir()
                          if f.name.startswith(base_name + '_pt') and f.name.endswith('.py')]
        next_part_num = len(existing_parts) + 2
        part_filename = f'{base_name}_pt{next_part_num}.py'

        import_lines = self._collect_import_lines(tree, lines)
        top_level_items = self._collect_top_level_items(tree)

        if not top_level_items:
            return {'success': False, 'reason': 'No top-level items to split'}

        split_idx = self._find_split_point(top_level_items, lines)
        if split_idx >= len(top_level_items):
            return {'success': False, 'reason': 'Cannot find a good split point'}

        items_to_move = top_level_items[split_idx:]

        part_code = self._build_part_file(filename, next_part_num, import_lines, items_to_move, lines)
        main_code = self._build_trimmed_main(lines, items_to_move, part_filename)

        try:
            ast.parse(main_code)
            ast.parse(part_code)
        except SyntaxError as e:
            return {'success': False, 'reason': f'Split produced invalid Python: {e}'}

        self.backup_file(filename)
        result_main = self.rewrite_code(filename, main_code,
                                        f'Auto-split: moved {len(items_to_move)} items to {part_filename}')

        with open(self.code_dir / part_filename, 'w') as f:
            f.write(part_code)

        if part_filename not in self.modifiable_files:
            self.modifiable_files.append(part_filename)

        return {
            'success': True,
            'change': f'Split {filename} ({len(lines)} lines) -> main + {part_filename}',
            'new_file': part_filename,
            'items_moved': [i['name'] for i in items_to_move]
        }

    def _collect_import_lines(self, tree, lines: List[str]) -> List[str]:
        """Collect all import lines from AST."""
        import_lines = []
        for node in ast.iter_child_nodes(tree):
            if isinstance(node, (ast.Import, ast.ImportFrom)):
                for ln in range(node.lineno - 1, node.end_lineno):
                    import_lines.append(lines[ln])
        return import_lines

    def _collect_top_level_items(self, tree) -> List[Dict]:
        """Find top-level classes and functions with line ranges."""
        items = []
        for node in ast.iter_child_nodes(tree):
```

```
        if isinstance(node, (ast.ClassDef, ast.FunctionDef, ast.AsyncFunctionDef)):
            items.append({
                'name': node.name,
                'type': type(node).__name__,
                'start': node.lineno - 1,
                'end': node.end_lineno,
                'lines': node.end_lineno - node.lineno + 1
            })
    return items

def _find_split_point(self, items: List[Dict], lines: List[str]) -> int:
    """Find the index at which to split items so main stays under limit."""
    split_idx = len(items)
    cumulative = 0
    for i in range(len(items) - 1, 0, -1):
        cumulative += items[i]['lines']
        remaining = len(lines) - cumulative
        if remaining <= self.max_file_lines:
            split_idx = i
            break
    return split_idx

def _build_part_file(self, filename, part_num, import_lines, items_to_move, lines) -> str:
    """Build the content for a split part file."""
    part_lines = [
        '"""',
        f'Auto-split from {filename} by self-evolution engine.',
        f'Part {part_num}  contains: {", ".join(i["name"] for i in items_to_move)}',
        '"""', ''
    ]
    part_lines.extend(import_lines)
    part_lines.append('')
    for item in items_to_move:
        part_lines.extend(lines[item['start']:item['end']])
        part_lines.extend(['', ''])
    return '\n'.join(part_lines)

def _build_trimmed_main(self, lines, items_to_move, part_filename) -> str:
    """Build the trimmed main file with import for split items."""
    main_lines = lines[:items_to_move[0]['start']]
    part_module = part_filename.replace('.py', '')
    moved_names = [i['name'] for i in items_to_move]
    main_lines.append(f'from {part_module} import {", ".join(moved_names)}  # Auto-split by self-evolution')
    main_lines.append('')
    last_end = items_to_move[-1]['end']
    if last_end < len(lines):
        remaining = lines[last_end:]
        if any(line.strip() for line in remaining):
            main_lines.extend(remaining)
    return '\n'.join(main_lines)

def _split_large_class(self, code: str, target: str, filename: str) -> Tuple[str, Optional[str]]:
    """Split a class with too many methods into a mixin in a separate file."""
    class_name = target.replace('class ', '').split()[0] if 'class' in target else target

    try:
        tree = ast.parse(code)
    except SyntaxError:
        return code, None

    target_class = None
    for node in ast.iter_child_nodes(tree):
        if isinstance(node, ast.ClassDef) and node.name == class_name:
            target_class = node
            break
    if not target_class:
        return code, None

    methods = [n for n in target_class.body if isinstance(n, (ast.FunctionDef, ast.AsyncFunctionDef))]
    if len(methods) <= 15:
        return code, None

    keep_methods = methods[:10]
    move_methods = methods[10:]
    lines = code.split('\n')

    return self.__split_large_class_continued(class_name, lines, methods, move_methods, tree, code, filename)

def __split_large_class_continued(self, class_name, lines, methods, move_methods, tree, code, filename):
    """Continuation of _split_large_class  auto-extracted by self-evolution."""
    import_lines = self._collect_import_lines(tree, lines)
    moved_source = []
    for m in move_methods:
        moved_source.extend(lines[m.lineno - 1:m.end_lineno])
        moved_source.append('')
```

```
base_name = filename.replace('.py', '')
mixin_name = f'{class_name}ExtMixin'
part_filename = f'{base_name}_ext.py'
part_path = self.code_dir / part_filename
if part_path.exists():
    part_num = 2
    while (self.code_dir / f'{base_name}_ext{part_num}.py').exists():
        part_num += 1
    part_filename = f'{base_name}_ext{part_num}.py'
    mixin_name = f'{class_name}Ext{part_num}Mixin'

part_lines = [
    '"""',
    f'Auto-split from {filename} by self-evolution engine.',
    f'Contains extended methods for {class_name}.',
    '"""', ''
]
part_lines.extend(import_lines)
part_lines.extend(['', '',
    f'class {mixin_name}:',
    f'    """Extended methods for {class_name}  auto-extracted by self-evolution."""', ''
])
part_lines.extend(moved_source)
part_code = '\n'.join(part_lines)

try:
    ast.parse(part_code)
except SyntaxError:
    return code, None

new_lines = list(lines)
remove_ranges = [(m.lineno - 1, m.end_lineno) for m in move_methods]
for start, end in reversed(remove_ranges):
    del new_lines[start:end]

part_module = part_filename.replace('.py', '')
import_stmt = f'from {part_module} import {mixin_name}  # Auto-split by self-evolution'

class_line_idx = None
for i, line in enumerate(new_lines):
    if re.match(rf'^class\s+{re.escape(class_name)}\s*(\(|:)', line):
        class_line_idx = i
        break

if class_line_idx is not None:
    new_lines.insert(class_line_idx, import_stmt)
    new_lines.insert(class_line_idx + 1, '')
    class_line_idx += 2

for i, line in enumerate(new_lines):
    if re.match(rf'^class\s+{re.escape(class_name)}\s*(\(|:)', line):
        if '(' in line and ')' in line:
            new_lines[i] = line.replace(')', f', {mixin_name})')
        elif ':' in line:
            new_lines[i] = line.replace(':', f'({mixin_name}):')
        break

new_code = '\n'.join(new_lines)
try:
    ast.parse(new_code)
except SyntaxError:
    return code, None

with open(self.code_dir / part_filename, 'w') as f:
    f.write(part_code)

if part_filename not in self.modifiable_files:
    self.modifiable_files.append(part_filename)

return new_code, f"Extracted {len(move_methods)} methods from {class_name} into {part_filename}"
```

self_evolution_runner.py

```
"""
 Self-Evolution Engine  Runner Module
========================================
Auto-evolution cycle, improvement application, and scheduling.
"""

from typing import Dict
from datetime import datetime, timezone


class RunnerMixin:
    """Mixin providing auto-evolution cycle and improvement application."""

    def should_auto_evolve(self) -> bool:
        """Check if it's time for auto-evolution (60-90 day cycle)."""
        if self.last_evolution is None:
            return True
        days_since = (datetime.now(timezone.utc) - self.last_evolution).days
        return days_since >= self.auto_evolution_interval_days

    async def auto_evolve(self) -> Dict:
        """Perform automatic self-evolution.
        ACTUALLY rewrites code based on identified improvements.
        """
        results = {
            'evolved': True,
            'timestamp': datetime.now(timezone.utc).isoformat(),
            'improvements_found': [],
            'changes_made': [],
            'skipped': [],
            'errors': []
        }

        for filename in self.modifiable_files:
            improvements = self.identify_improvements(filename)

            for imp in improvements:
                results['improvements_found'].append({
                    'type': imp.get('type'),
                    'target': imp.get('target'),
                    'suggestion': imp.get('suggestion'),
                    'file': filename
                })

            for improvement in improvements:
                try:
                    change = await self._apply_improvement(filename, improvement)
                    if change.get('success'):
                        results['changes_made'].append({
                            'file': filename,
                            'type': improvement.get('type'),
                            'change': change.get('change', ''),
                            'backup': change.get('backup_path', '')
                        })
                    else:
                        reason = change.get('reason') or change.get('error') or 'unknown'
                        results['skipped'].append({
                            'file': filename,
                            'type': improvement.get('type'),
                            'reason': reason
                        })
                except Exception as e:
                    results['errors'].append({
                        'file': filename,
                        'type': improvement.get('type', 'unknown'),
                        'error': str(e)
                    })

        self.pending_improvements = len(results['improvements_found']) - len(results['changes_made'])
        self.last_evolution = datetime.now(timezone.utc)

        if self.db is not None:
            try:
                db_record = {
                    'timestamp': results['timestamp'],
                    'improvements_found': len(results['improvements_found']),
                    'changes_made': len(results['changes_made']),
                    'errors': len(results['errors']),
                    'details': results
                }
                await self.db.evolution_log.insert_one(db_record)
            except Exception:
```

```
            pass

    return results

async def _apply_improvement(self, filename: str, improvement: Dict) -> Dict:
    """ACTUALLY apply an improvement to the code.
    This is where the AI rewrites itself.
    """
    analysis = self.read_own_code(filename)
    if 'error' in analysis:
        return {'success': False, 'error': analysis['error']}

    code = analysis['code']
    improvement_type = improvement.get('type')
    target = improvement.get('target', '')

    new_code = code
    change_made = None

    if improvement_type == 'missing_docstring':
        new_code, change_made = self._add_docstring(code, target)
    elif improvement_type == 'bare_except':
        new_code, change_made = self._fix_bare_except(code, target)
    elif improvement_type == 'unused_import':
        new_code, change_made = self._remove_unused_import(code, target)
    elif improvement_type == 'todo_items':
        return {'success': False, 'reason': 'TODO items require manual review'}
    elif improvement_type == 'long_function':
        new_code, change_made = self._split_long_function(code, target)
    elif improvement_type == 'file_too_long':
        return await self._split_long_file(filename, code)
    elif improvement_type == 'class_too_large':
        new_code, change_made = self._split_large_class(code, target, filename)

    if new_code != code and change_made:
        result = self.rewrite_code(filename, new_code, f"Auto-evolution: {improvement_type}")
        result['improvement'] = improvement
        result['change'] = change_made
        return result

    return {'success': False, 'reason': change_made or 'No changes needed or could not apply'}
```

self_evolution_splitting.py

```
"""
 Self-Evolution Engine  Splitting Module
=========================================
Operations for splitting long functions, files, and classes.
"""

import ast
import re
from typing import Dict, List, Tuple, Optional


class SplittingMixin:
    """Mixin providing code splitting operations."""

    def _split_long_function(self, code: str, target: str) -> Tuple[str, Optional[str]]:
        """Split a long function by extracting blocks into helpers.
        For __init__: extracts data blocks into _init_<n>() methods.
        For other methods: extracts logical blocks into _<n>_<section>() helpers.
        """
        parts = target.replace('method ', '').split('.')
        if len(parts) != 2:
            return code, None
        class_name, method_name = parts

        try:
            tree = ast.parse(code)
        except SyntaxError:
            return code, None

        target_node = self._find_method_node(tree, class_name, method_name)
        if not target_node or not hasattr(target_node, 'end_lineno'):
            return code, None

        func_lines = target_node.end_lineno - target_node.lineno
        if func_lines <= self.max_function_lines:
            return code, None

        lines = code.split('\n')
        func_start = target_node.lineno - 1
        func_end = target_node.end_lineno

        body_first_line = func_start + 1
        if target_node.body:
            body_first_line = target_node.body[0].lineno - 1

        def_lines = lines[func_start:body_first_line]
        func_body_lines = lines[body_first_line:func_end]

        body_indent, class_indent = self._detect_indents(func_body_lines)

        if method_name == '__init__':
            return self._split_init_method(code, lines, func_start, func_end,
                                           class_name, body_indent, class_indent,
                                           def_lines, func_body_lines)
        else:
            return self._split_regular_method(code, lines, func_start, func_end,
                                              class_name, method_name, body_indent, class_indent,
                                              def_lines, func_body_lines)

    def _find_method_node(self, tree, class_name: str, method_name: str):
        """Find a method AST node within a class."""
        for node in ast.walk(tree):
            if isinstance(node, ast.ClassDef) and node.name == class_name:
                for item in node.body:
                    if isinstance(item, (ast.FunctionDef, ast.AsyncFunctionDef)) and item.name == method_name:
                        return item
        return None

    def _detect_indents(self, body_lines: List[str]) -> Tuple[str, str]:
        """Detect body and class-level indentation from method body lines."""
        body_indent = ''
        for line in body_lines:
            if line.strip():
                body_indent = line[:len(line) - len(line.lstrip())]
                break

        if len(body_indent) >= 4:
            class_indent = body_indent[:-4]
        elif len(body_indent) >= 2:
            class_indent = body_indent[:-2]
        else:
            class_indent = ''
```

```
    return body_indent, class_indent

def _skip_docstring(self, body_lines: List[str]) -> Tuple[List[str], List[str]]:
    """Separate docstring lines from actual body lines."""
    body_start = 0
    in_docstring = False
    for i, line in enumerate(body_lines):
        stripped = line.strip()
        if stripped.startswith('"""') or stripped.startswith("'''"):
            if in_docstring or (stripped.count('"""') >= 2 or stripped.count("'''") >= 2):
                body_start = i + 1
                break
            else:
                in_docstring = True
        elif not in_docstring:
            body_start = i
            break
    return body_lines[:body_start], body_lines[body_start:]

def _split_init_method(self, code, lines, func_start, func_end,
                       class_name, body_indent, class_indent,
                       def_lines, func_body_lines):
    """Split a long __init__ into __init__ + _init_<section> helpers."""
    docstring_part, actual_body = self._skip_docstring(func_body_lines)

    groups = self._group_by_separators(actual_body)
    if len(groups) < 3:
        return code, None

    keep_in_init = groups[0]
    extract_groups = groups[1:]

    new_init_lines = list(def_lines)
    new_init_lines.extend(docstring_part)
    new_init_lines.extend(keep_in_init['lines'])

    helper_methods = []
    for idx, group in enumerate(extract_groups):
        helper_name = group['name'] or f'section_{idx + 1}'
        helper_name = f'_init_{helper_name}'
        new_init_lines.append(f'{body_indent}self.{helper_name}()')

        helper_lines = [
            f'{class_indent}def {helper_name}(self):',
            f'{body_indent}"""Initialize {helper_name.replace("_init_", "").replace("_", " ")}  auto-extracted by self-evolution."""'
        ]
        for line in group['lines']:
            if not line.strip().startswith('#'):
                helper_lines.append(line)
        helper_lines.append('')
        helper_methods.append(helper_lines)

    new_lines = lines[:func_start]
    new_lines.extend(new_init_lines)
    new_lines.append('')
    for helper in helper_methods:
        new_lines.extend(helper)
    new_lines.extend(lines[func_end:])

    new_code = '\n'.join(new_lines)
    try:
        ast.parse(new_code)
    except SyntaxError:
        return code, None

    return new_code, f"Split {class_name}.__init__ into {len(extract_groups) + 1} methods"

def _group_by_separators(self, body_lines: List[str]) -> List[Dict]:
    """Group lines by blank-line or comment separators."""
    groups = []
    current_group = {'name': None, 'lines': []}

    for line in body_lines:
        stripped = line.strip()
        if not stripped:
            if current_group['lines']:
                groups.append(current_group)
                current_group = {'name': None, 'lines': []}
            continue
        if stripped.startswith('#'):
            if current_group['lines']:
                groups.append(current_group)
            group_name = stripped.lstrip('#').strip().lower()
            group_name = ''.join(c if c.isalnum() else '_' for c in group_name)[:30].strip('_')
            current_group = {'name': group_name, 'lines': [line]}
```

```
            continue
        current_group['lines'].append(line)

    if current_group['lines']:
        groups.append(current_group)
    return groups

def _split_regular_method(self, code, lines, func_start, func_end,
                          class_name, method_name, body_indent, class_indent,
                          def_lines, func_body_lines):
    """Split a long regular method by extracting the second half into a helper."""
    docstring_lines, actual_body = self._separate_docstring(func_body_lines)
    blocks = self._find_top_level_blocks(actual_body, body_indent)

    if len(blocks) < 2:
        return code, None

    mid = max(1, len(blocks) // 2)
    keep_blocks = blocks[:mid]
    extract_blocks = blocks[mid:]
    helper_name = f'_{method_name}_continued'

    kept_code = '\n'.join(line for b in keep_blocks for line in b)
    extracted_code = '\n'.join(line for b in extract_blocks for line in b)

    passed_vars = self._detect_passed_variables(
        kept_code, extracted_code, def_lines
    )

    return self.__split_regular_method_continued(extract_blocks, helper_name, keep_blocks, passed_vars, code, lines, func_start, func...

def __split_regular_method_continued(self, extract_blocks, helper_name, keep_blocks, passed_vars, code, lines, func_start, func_end, ...
    """Continuation of _split_regular_method  auto-extracted by self-evolution."""
    last_line = extract_blocks[-1][-1].strip() if extract_blocks[-1] else ''
    has_return = last_line.startswith('return ')
    params_sig = ', '.join(passed_vars) if passed_vars else ''
    params_call = ', '.join(passed_vars) if passed_vars else ''

    new_method = list(def_lines)
    new_method.extend(docstring_lines)
    for block in keep_blocks:
        new_method.extend(block)
        new_method.append('')
    call = f'self.{helper_name}({params_call})'
    new_method.append(f'{body_indent}{"return " if has_return else ""}{call}')
    new_method.append('')

    helper = self._build_helper_method(
        helper_name, method_name, params_sig, body_indent, class_indent,
        keep_blocks, extract_blocks
    )

    new_lines = lines[:func_start]
    new_lines.extend(new_method)
    new_lines.extend(helper)
    new_lines.extend(lines[func_end:])

    new_code = '\n'.join(new_lines)
    try:
        ast.parse(new_code)
    except SyntaxError:
        return code, None

    return new_code, f"Split {class_name}.{method_name} into 2 methods (passing {len(passed_vars)} vars)"


def _separate_docstring(self, body_lines: List[str]) -> Tuple[List[str], List[str]]:
    """Separate leading docstring from body."""
    docstring_lines = []
    body_start_idx = 0
    in_docstring = False
    for i, line in enumerate(body_lines):
        stripped = line.strip()
        if i == 0 and (stripped.startswith('"""') or stripped.startswith("'''")):
            docstring_lines.append(line)
            if stripped.count('"""') >= 2 or stripped.count("'''") >= 2:
                body_start_idx = i + 1
                break
            in_docstring = True
        elif in_docstring:
            docstring_lines.append(line)
            if '"""' in stripped or "'''" in stripped:
                body_start_idx = i + 1
                break
        else:
            body_start_idx = i
```

```
            break
    return docstring_lines, body_lines[body_start_idx:]

def _find_top_level_blocks(self, body_lines: List[str], body_indent: str) -> List[List[str]]:
    """Find top-level code blocks separated by blank lines at body indent level."""
    blocks = []
    current_block = []
    for line in body_lines:
        stripped = line.strip()
        if not stripped:
            if current_block:
                last_indent = len(current_block[-1]) - len(current_block[-1].lstrip())
                if last_indent <= len(body_indent):
                    blocks.append(current_block)
                    current_block = []
                    continue
            current_block.append(line)
            continue
        current_block.append(line)
    if current_block:
        blocks.append(current_block)
    return blocks

def _detect_passed_variables(self, kept_code: str, extracted_code: str,
                             def_lines: List[str]) -> List[str]:
    """Detect local variables from kept code that are used in extracted code."""
    assigned_vars = set()
    for line in kept_code.split('\n'):
        stripped = line.strip()
        m = re.match(r'^([a-zA-Z_]\w*)\s*=\s', stripped)
        if m and not stripped.startswith('self.'):
            assigned_vars.add(m.group(1))
        m = re.match(r'^for\s+([a-zA-Z_]\w*)', stripped)
        if m:
            assigned_vars.add(m.group(1))

    passed_vars = [v for v in sorted(assigned_vars) if re.search(rf'\b{re.escape(v)}\b', extracted_code)]

    orig_def = ' '.join(l.strip() for l in def_lines)
    param_match = re.search(r'\(self,?\s*(.*?)\)', orig_def, re.DOTALL)
    if param_match:
        for param in param_match.group(1).split(','):
            param_name = param.strip().split(':')[0].split('=')[0].strip()
            if param_name and re.search(rf'\b{re.escape(param_name)}\b', extracted_code):
                if param_name not in passed_vars:
                    passed_vars.append(param_name)

    return passed_vars

def _build_helper_method(self, helper_name, method_name, params_sig,
                         body_indent, class_indent,
                         keep_blocks, extract_blocks) -> List[str]:
    """Build the extracted helper method lines."""
    helper = [
        f'{class_indent}def {helper_name}(self, {params_sig}):',
        f'{body_indent}"""Continuation of {method_name}  auto-extracted by self-evolution."""'
    ]

    # Add inline imports used in extracted blocks
    extracted_text = '\n'.join(l for b in extract_blocks for l in b)
    for block in keep_blocks:
        for line in block:
            stripped = line.strip()
            if stripped.startswith('import ') or stripped.startswith('from '):
                mod_name = stripped.split()[-1].split('.')[0]
                if mod_name and re.search(rf'\b{re.escape(mod_name)}\b', extracted_text):
                    helper.append(f'{body_indent}{stripped}')

    all_extracted = [line for block in extract_blocks for line in block if line.strip()]
    min_indent = min((len(l) - len(l.lstrip()) for l in all_extracted), default=0)

    for block in extract_blocks:
        for line in block:
            stripped = line.strip()
            if stripped:
                if stripped.startswith('import ') or stripped.startswith('from '):
                    continue
                rebased = line[min_indent:] if len(line) > min_indent else line.lstrip()
                helper.append(body_indent + rebased)
            else:
                helper.append('')
        helper.append('')

    return helper
```

self_evolution_transforms.py

```
"""
 Self-Evolution Engine  Transforms Module
==========================================
Code transformation operations: docstrings, bare excepts, unused imports.
"""

import re
from typing import Tuple, Optional


class TransformMixin:
    """Mixin providing code transformation operations."""

    def _add_docstring(self, code: str, target: str) -> Tuple[str, Optional[str]]:
        """Add docstring to a class or function."""
        name = target.split()[-1] if ' ' in target else target

        if 'class' in target:
            pattern = rf'(class {re.escape(name)}[^:]*:)\n(\s+)(?!""")'
            match = re.search(pattern, code)
            if match:
                indent = match.group(2)
                replacement = f'{match.group(1)}\n{indent}"""{name} - Auto-documented by self-evolution."""\n{indent}'
                new_code = code[:match.start()] + replacement + code[match.end():]
                if new_code != code:
                    return new_code, f"Added docstring to {target}"

        elif 'function' in target:
            pattern = rf'(def {re.escape(name)}\([^)]*\)[^:]*:)\n(\s+)(?!""")'
            match = re.search(pattern, code)
            if match:
                indent = match.group(2)
                replacement = f'{match.group(1)}\n{indent}"""{name} - Auto-documented by self-evolution."""\n{indent}'
                new_code = code[:match.start()] + replacement + code[match.end():]
                if new_code != code:
                    return new_code, f"Added docstring to {target}"

        return code, None

    def _fix_bare_except(self, code: str, target: str) -> Tuple[str, Optional[str]]:
        """Replace bare except: with except Exception:"""
        lines = code.split('\n')
        changed = False

        for i, line in enumerate(lines):
            stripped = line.strip()
            if stripped == 'except:':
                indent = line[:len(line) - len(line.lstrip())]
                lines[i] = f'{indent}except Exception:'
                changed = True

        if changed:
            return '\n'.join(lines), f"Replaced bare except with except Exception in {target}"
        return code, None

    def _remove_unused_import(self, code: str, target: str) -> Tuple[str, Optional[str]]:
        """Remove an unused import line from the code.
        Only removes simple 'import X' lines where X is truly unused.
        Does NOT touch 'from X import Y' lines (too risky - Y may be used).
        """
        import_str = target.replace('import ', '').strip()
        if not import_str:
            return code, None

        lines = code.split('\n')
        import_line_idx = None
        imported_names = []

        for i, line in enumerate(lines):
            stripped = line.strip()

            if stripped == f'import {import_str}':
                import_line_idx = i
                imported_names = [import_str.split('.')[-1]]
                break

            from_match = re.match(rf'^from\s+{re.escape(import_str)}\s+import\s+(.+)$', stripped)
            if from_match:
                names = [n.strip().split(' as ')[-1].strip() for n in from_match.group(1).split(',')]
                import_line_idx = i
                imported_names = names
                break
```

```
if import_line_idx is None or not imported_names:
    return code, None

other_lines = lines[:import_line_idx] + lines[import_line_idx + 1:]
other_code = '\n'.join(other_lines)

all_unused = True
for name in imported_names:
    pattern = rf'\b{re.escape(name)}\b'
    if re.search(pattern, other_code):
        all_unused = False
        break

if not all_unused:
    return code, None

lines.pop(import_line_idx)
return '\n'.join(lines), f"Removed unused import: {import_str}"
```

self_research.py

```
"""
 Self-Research Engine
Autonomous research capability for Quantum AI
- Research on demand or by user command
- Long-running autonomous sessions (30-60 minutes)
"""

import asyncio
import re
import random
from typing import Dict, List, Optional
from datetime import datetime, timezone, timedelta

try:
    from ddgs import DDGS
except ImportError:
    from duckduckgo_search import DDGS

from research_topics import EXPLORATION_TOPICS



class SelfResearchEngine:
    """Autonomous research capability with extended sessions."""

    def __init__(self, db=None):
        self.db = db
        self.research_log = []
        self.knowledge_gaps = []
        self.auto_research_enabled = True
        self.research_queue = []

        # Autonomous research state
        self.autonomous_running = False
        self.autonomous_start_time = None
        self.autonomous_duration_minutes = 30
        self.autonomous_progress = self._init_progress()
        self.exploration_topics = EXPLORATION_TOPICS

    def _init_progress(self) -> Dict:
        return {
            'is_running': False,
            'started_at': None,
            'duration_minutes': 0,
            'elapsed_minutes': 0,
            'topics_researched': [],
            'insights_gained': 0,
            'concepts_learned': 0,
            'status': 'Idle'
        }

    async def research(self, query: str, reason: str = "user_request") -> Dict:
        """Perform research on a topic."""
        try:
            ddgs = DDGS()
            results = list(ddgs.text(query, max_results=5))

            research_result = {
                'query': query,
                'reason': reason,
                'timestamp': datetime.now(timezone.utc).isoformat(),
                'results': [{'title': r.get('title',''), 'body': r.get('body',''), 'href': r.get('href','')} for r in results],
                'summary': '',
                'new_concepts': self._extract_concepts(' '.join([r.get('body', '') for r in results]))
            }

            self.research_log.append(research_result)
            if self.db is not None:
                await self._store_research(research_result)

            return research_result
        except Exception as e:
            return {'query': query, 'reason': reason, 'error': str(e), 'results': []}

    def _extract_concepts(self, text: str) -> List[str]:
        """Extract key concepts from text."""
        words = re.findall(r'\b[a-zA-Z]{5,}\b', text.lower())
        common = {'about', 'which', 'their', 'there', 'would', 'could', 'should', 'these', 'those', 'other', 'being', 'where', 'while', '...
        from collections import Counter
        counts = Counter(w for w in words if w not in common)
        return [w for w, _ in counts.most_common(10)]
```

```
async def _store_research(self, research: Dict):
    """Store research in database."""
    try:
        await self.db.research_history.insert_one({
            **research, '_stored_at': datetime.now(timezone.utc).isoformat()
        })
        for concept in research.get('new_concepts', [])[:5]:
            await self.db.semantic_memory.update_one(
                {'concept': concept},
                {'$set': {'concept': concept, 'source': 'research', 'query': research['query']},
                 '$inc': {'count': 1}},
                upsert=True
            )
    except Exception as e:
        print(f"[RESEARCH] Store error: {e}")

async def start_autonomous_research(self, duration_minutes: int = 30) -> Dict:
    """Start autonomous research session."""
    if self.autonomous_running:
        return {'status': 'already_running'}

    self.autonomous_running = True
    self.autonomous_start_time = datetime.now(timezone.utc)
    self.autonomous_duration_minutes = min(duration_minutes, 60)

    self.autonomous_progress = {
        'is_running': True,
        'started_at': self.autonomous_start_time.isoformat(),
        'duration_minutes': self.autonomous_duration_minutes,
        'elapsed_minutes': 0,
        'topics_researched': [],
        'insights_gained': 0,
        'concepts_learned': 0,
        'status': 'Starting...'
    }

    asyncio.create_task(self._run_autonomous_research())
    return {'status': 'started', 'duration_minutes': self.autonomous_duration_minutes,
            'message': f'Starting {self.autonomous_duration_minutes} minute autonomous research session'}

async def _run_autonomous_research(self):
    """Background task for autonomous research."""
    try:
        has_db = self.db is not None
        print(f"[RESEARCH] Starting autonomous research, has_db={has_db}")

        end_time = self.autonomous_start_time + timedelta(minutes=self.autonomous_duration_minutes)

        while datetime.now(timezone.utc) < end_time and self.autonomous_running:
            topic = random.choice(self.exploration_topics)
            query = f"{topic} {random.choice(['latest', 'explained', 'applications', 'deep dive'])}"
            self.autonomous_progress['status'] = f'Researching: {topic}'

            try:
                ddgs = DDGS()
                results = list(ddgs.text(query, max_results=5))

                if results:
                    self.autonomous_progress['topics_researched'].append(topic)
                    concepts = self._extract_concepts(' '.join([r.get('body', '') for r in results]))
                    self.autonomous_progress['concepts_learned'] += len(concepts)

                    if len(concepts) > 3:
                        self.autonomous_progress['insights_gained'] += 1

                    # Store to database
                    if has_db:
                        try:
                            await self.db.research_history.insert_one({
                                'query': query, 'topic': topic,
                                'reason': 'autonomous',
                                'timestamp': datetime.now(timezone.utc).isoformat(),
                                'concepts': concepts,
                                'results_count': len(results)
                            })

                            for c in concepts[:5]:
                                await self.db.semantic_memory.update_one(
                                    {'concept': c},
                                    {'$set': {'concept': c, 'source': 'autonomous_research'},
                                     '$inc': {'count': 1}},
                                    upsert=True
                                )

                            await self.db.growth_metrics.update_one(
                                {'type': 'autonomous_research'},
```

```
                                {'$inc': {'researches': 1, 'concepts': len(concepts)},
                                 '$set': {'last_topic': topic}},
                                upsert=True
                            )
                            print(f"[RESEARCH] Stored: {topic}")
                        except Exception as db_err:
                            print(f"[RESEARCH] DB error: {db_err}")

                except Exception as e:
                    print(f"[RESEARCH] Search error: {e}")

                elapsed = (datetime.now(timezone.utc) - self.autonomous_start_time).total_seconds() / 60
                self.autonomous_progress['elapsed_minutes'] = round(elapsed, 1)
                await asyncio.sleep(random.randint(30, 60))

            # Complete
            self.autonomous_running = False
            self.autonomous_progress['is_running'] = False
            self.autonomous_progress['status'] = f'Complete! {self.autonomous_progress["concepts_learned"]} concepts learned'

            if has_db:
                await self.db.autonomous_sessions.insert_one({
                    'completed_at': datetime.now(timezone.utc).isoformat(),
                    'duration': self.autonomous_duration_minutes,
                    'topics': len(self.autonomous_progress['topics_researched']),
                    'concepts': self.autonomous_progress['concepts_learned']
                })

        except Exception as e:
            print(f"[RESEARCH] Fatal error: {e}")
            self.autonomous_running = False
            self.autonomous_progress['status'] = f'Error: {e}'

    def stop_autonomous_research(self) -> Dict:
        """Stop autonomous research."""
        if not self.autonomous_running:
            return {'status': 'not_running'}
        self.autonomous_running = False
        self.autonomous_progress['is_running'] = False
        self.autonomous_progress['status'] = 'Stopped'
        return {'status': 'stopped', 'concepts': self.autonomous_progress['concepts_learned']}

    def get_autonomous_progress(self) -> Dict:
        """Get current progress."""
        if self.autonomous_running and self.autonomous_start_time:
            elapsed = (datetime.now(timezone.utc) - self.autonomous_start_time).total_seconds() / 60
            self.autonomous_progress['elapsed_minutes'] = round(elapsed, 1)
        return self.autonomous_progress

    def get_research_stats(self) -> Dict:
        """Get research statistics."""
        return {
            'total_researches': len(self.research_log),
            'knowledge_gaps_identified': len(self.knowledge_gaps),
            'queue_size': len(self.research_queue),
            'auto_research_enabled': self.auto_research_enabled,
            'recent_topics': [r['query'] for r in self.research_log[-5:]],
            'autonomous_running': self.autonomous_running
        }


# Singleton
_research_engine = None

async def get_research_engine(db=None) -> SelfResearchEngine:
    """Get or create research engine."""
    global _research_engine
    if _research_engine is None:
        _research_engine = SelfResearchEngine(db)
    if db is not None:
        _research_engine.db = db
    return _research_engine
```

semantic_collapse_engine.py

```
"""
Semantic Collapse / Observation Engine for Quantum AI
Simulates quantum superposition collapse through semantic analysis

When user asks a question, the engine "observes" key concepts and
collapses the infinite semantic possibilities into focused responses.
"""

import re
import random
from typing import Dict, List, Tuple, Set, Optional


class SemanticCollapseEngine:
    """
    Collapses semantic superposition based on input observation (keywords).
    When user asks a question, the engine "observes" key concepts and
    collapses the infinite semantic possibilities into focused responses.
    """

    def __init__(self, co_occurrence_weights: Dict = None):
        """
        Initialize with co-occurrence weights from grammar engine

        Args:
            co_occurrence_weights: Dict of word -> {related_word: weight}
        """
        self.co_occurrence = co_occurrence_weights or self._default_co_occurrence()
        self.keyword_clusters = self._build_keyword_clusters()
        self.decay_rate = 0.3  # How much non-relevant weights decay
        self.observation_history = []

    def _default_co_occurrence(self) -> Dict:
        """Default co-occurrence weights for philosophical/quantum concepts"""
        return {
            'consciousness': {'awareness': 0.9, 'being': 0.8, 'existence': 0.8, 'mind': 0.7, 'spirit': 0.7, 'observation': 0.6},
            'quantum': {'superposition': 0.9, 'entanglement': 0.9, 'observation': 0.8, 'collapse': 0.8, 'probability': 0.7, 'measurement'...
            'god': {'divine': 0.9, 'creation': 0.9, 'infinite': 0.8, 'eternal': 0.8, 'transcendent': 0.9, 'omnipotent': 0.7},
            'universe': {'existence': 0.9, 'creation': 0.8, 'reality': 0.8, 'cosmos': 0.8, 'spacetime': 0.9, 'being': 0.7},
            'reality': {'existence': 0.9, 'being': 0.8, 'truth': 0.8, 'manifestation': 0.7, 'phenomenon': 0.7},
            'observation': {'collapse': 0.9, 'measurement': 0.9, 'awareness': 0.8, 'consciousness': 0.8, 'perception': 0.7},
            'superposition': {'quantum': 0.9, 'probability': 0.8, 'potential': 0.8, 'multiplicity': 0.7, 'coexistence': 0.7},
            'creation': {'god': 0.8, 'universe': 0.8, 'existence': 0.7, 'emergence': 0.7, 'origin': 0.8},
            'spacetime': {'dimension': 0.9, 'physical': 0.8, 'universe': 0.8, 'relativity': 0.8, 'continuum': 0.7},
            'philosophy': {'wisdom': 0.8, 'truth': 0.8, 'knowledge': 0.8, 'existence': 0.7, 'meaning': 0.8},
            'word': {'language': 0.9, 'meaning': 0.9, 'neutral': 0.7, 'tool': 0.7, 'communication': 0.8},
            'language': {'word': 0.9, 'meaning': 0.8, 'communication': 0.9, 'expression': 0.8, 'symbol': 0.7},
        }

    def _build_keyword_clusters(self) -> Dict[str, Set[str]]:
        """Build semantic clusters from co-occurrence weights"""
        clusters = {}
        for word, relations in self.co_occurrence.items():
            clusters[word] = set(relations.keys())
        return clusters

    def extract_keywords(self, text: str, top_n: int = 5) -> List[Tuple[str, float]]:
        """
        Extract primary keywords from input text.
        Keywords are words that appear in co-occurrence network.

        Returns:
            List of (keyword, importance_score) tuples
        """
        # Normalize text
        text_lower = text.lower()
        words = re.findall(r'\b[a-z]+\b', text_lower)

        # Score words by co-occurrence network presence
        keyword_scores = {}
        for word in set(words):
            if word in self.co_occurrence:
                # Weight by number of relations (importance in network)
                score = len(self.co_occurrence[word]) / 10.0
                keyword_scores[word] = score
            elif word in self._flatten_all_related_words():
                # Secondary importance for related words
                keyword_scores[word] = 0.5

        # Return top keywords
        sorted_keywords = sorted(keyword_scores.items(), key=lambda x: x[1], reverse=True)
        return sorted_keywords[:top_n]
```

```
def _flatten_all_related_words(self) -> Set[str]:
    """Get all words mentioned in co-occurrence network"""
    all_words = set(self.co_occurrence.keys())
    for relations in self.co_occurrence.values():
        all_words.update(relations.keys())
    return all_words

def collapse_weights(self, input_text: str) -> Dict[str, Dict[str, float]]:
    """
    OBSERVATION COLLAPSE: Narrow co-occurrence weights based on input keywords.

    This simulates quantum measurement:
    - Before observation: infinite semantic possibilities (full co-occurrence)
    - Observation (keywords extracted): collapses to relevant meanings
    - After collapse: weights decay for non-relevant concepts

    Args:
        input_text: User's query

    Returns:
        Collapsed co-occurrence weights focused on query semantics
    """
    # Extract observation (keywords)
    keywords = self.extract_keywords(input_text)
    keyword_set = {kw for kw, _ in keywords}

    if not keyword_set:
        # No observation possible, return original
        return self.co_occurrence

    # Collapse: Create new weights focused on keywords
    collapsed = {}

    for word, relations in self.co_occurrence.items():
        collapsed[word] = {}

        for related_word, original_weight in relations.items():
            # If related word is in keyword set, AMPLIFY
            if related_word in keyword_set:
                new_weight = min(1.0, original_weight * 1.5)  # Amplify
            # If word itself is keyword, PRESERVE
            elif word in keyword_set:
                new_weight = original_weight
            # Otherwise, DECAY (non-relevant concepts fade)
            else:
                new_weight = original_weight * (1 - self.decay_rate)

            collapsed[word][related_word] = new_weight

    return collapsed

def get_semantic_context(self, input_text: str) -> Dict:
    """
    Get semantic context after collapse.
    Returns the "collapsed state" information for response generation.
    """
    keywords = self.extract_keywords(input_text)
    collapsed_weights = self.collapse_weights(input_text)

    # Find strongest semantic paths from keywords
    semantic_paths = {}
    for keyword, _ in keywords:
        if keyword in collapsed_weights:
            # Get strongest related words
            relations = collapsed_weights[keyword]
            strongest = sorted(relations.items(), key=lambda x: x[1], reverse=True)[:3]
            semantic_paths[keyword] = strongest

    return {
        'keywords': keywords,
        'collapsed_weights': collapsed_weights,
        'semantic_paths': semantic_paths,
        'collapse_strength': len(keywords) / 5.0  # Measure of how focused
    }

def select_word_from_collapsed(self,
                               word_pool: List[str],
                               context: Dict,
                               quantum_probability: float = 0.5) -> str:
    """
    Select a word from pool using collapsed weights + quantum randomness.

    Args:
        word_pool: Available words to choose from
        context: Semantic context from collapse
```

```
        quantum_probability: Quantum state probability (0-1)

    Returns:
        Selected word
    """

    return self._select_word_from_collapsed_continued(word_pool, context, quantum_probability)

def _select_word_from_collapsed_continued(self, word_pool, context, quantum_probability):
    """Continuation of select_word_from_collapsed  auto-extracted by self-evolution."""
    collapsed_weights = context.get('collapsed_weights', {})

    # Score each word in pool based on collapsed weights
    word_scores = {}
    for word in word_pool:
        score = 0.0

        # Check if word appears in collapsed network
        if word in collapsed_weights:
            # Sum of all its relations (how connected it is)
            score += sum(collapsed_weights[word].values())

        # Check if word is related to keywords
        for keyword, _ in context.get('keywords', []):
            if keyword in collapsed_weights and word in collapsed_weights.get(keyword, {}):
                score += collapsed_weights[keyword][word] * 2  # Boost

        word_scores[word] = score

    # Normalize scores
    total_score = sum(word_scores.values())
    if total_score > 0:
        word_scores = {w: s/total_score for w, s in word_scores.items()}
    else:
        # Fallback: uniform distribution
        word_scores = {w: 1/len(word_pool) for w in word_pool}

    # Apply quantum randomness
    if random.random() < quantum_probability:
        # Quantum: select from top choices
        sorted_words = sorted(word_scores.items(), key=lambda x: x[1], reverse=True)
        top_words = [w for w, _ in sorted_words[:max(1, len(sorted_words)//3)]]
        return random.choice(top_words)
    else:
        # Classical: select highest probability
        return max(word_scores.items(), key=lambda x: x[1])[0]


def observe(self, input_text: str, response: str):
    """Record an observation for learning"""
    self.observation_history.append({
        'input': input_text,
        'response': response,
        'keywords': self.extract_keywords(input_text)
    })
    # Keep history manageable
    if len(self.observation_history) > 100:
        self.observation_history.pop(0)

def reset(self):
    """Reset the collapse engine state"""
    self.observation_history = []

def evaluate_collapse_quality(self, input_text: str, response: str) -> float:
    """
    Evaluate how well the collapse worked.
    Higher score = better semantic coherence between input and output.

    Returns:
        Quality score 0-1
    """
    input_keywords = set(kw for kw, _ in self.extract_keywords(input_text))
    response_words = set(re.findall(r'\b[a-z]+\b', response.lower()))

    # Check if response contains related words
    related_count = 0
    for keyword in input_keywords:
        if keyword in self.co_occurrence:
            related = self.co_occurrence[keyword].keys()
            related_count += len(response_words & set(related))

    # Normalize
    max_possible = len(input_keywords) * 5
    if max_possible == 0:
        return 0.5
```

```
return min(1.0, related_count / max_possible)
```

server.py

```
"""
 QUANTUM AI - BACKEND SERVER
==============================

ARTICLE 3.2 - The Transparency Pledge
All philosophical assumptions are documented and visible.

This server implements the Quantum AI cognitive architecture
with full covenant integration.
"""

from fastapi import FastAPI
from fastapi.responses import PlainTextResponse
from starlette.middleware.cors import CORSMiddleware
import os
import logging

import shared_state

# Route modules
from routes_chat import router as chat_router
from routes_cognitive import router as cognitive_router
from routes_explorer import router as explorer_router
from routes_brain import router as brain_router
from routes_media import router as media_router
from routes_extras import router as extras_router
from routes_cognitive_unified import router as cognitive_unified_router
from routes_cloud import router as cloud_router

logger = logging.getLogger("quantum_ai")
logging.basicConfig(level=logging.INFO)

# Create the main app
app = FastAPI(title="Quantum AI", description="True Questioning AI with Cognitive Architecture")

# CORS
app.add_middleware(
    CORSMiddleware,
    allow_origins=os.environ.get('CORS_ORIGINS', '*').split(','),
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)

# Include all route modules under /api
app.include_router(chat_router)
app.include_router(cognitive_router)
app.include_router(explorer_router)
app.include_router(brain_router)
app.include_router(media_router)
app.include_router(extras_router)
app.include_router(cognitive_unified_router)
app.include_router(cloud_router)


@app.on_event("startup")
async def startup_event():
    """Initialize all Quantum AI systems on startup."""
    logger.info(" Initializing Quantum AI systems...")
    shared_state.initialize()
    logger.info(" Quantum AI initialized successfully!")


@app.on_event("shutdown")
async def shutdown_db_client():
    """Clean shutdown."""
    shared_state.client.close()


@app.get("/api/download/{filename}", response_class=PlainTextResponse)
async def download_termux_file(filename: str):
    """Serve Termux files as raw Python code."""
    allowed_files = ["server_hybrid.py", "server_simple.py", "requirements_termux.txt", "setup_termux.sh"]
    if filename not in allowed_files:
        return PlainTextResponse("File not found", status_code=404)

    file_path = f"/app/quantum_termux/{filename}"
    try:
        with open(file_path, "r") as f:
            content = f.read()
        return PlainTextResponse(content, media_type="text/plain")
    except FileNotFoundError:
```

```
        return PlainTextResponse("File not found", status_code=404)

# Build trigger: 1772486161
```

shared_link_handler.py

```
"""
 QUANTUM AI - SHARED LINK HANDLER
===================================

Process files from shared cloud links:
- Google Drive
- Dropbox
- OneDrive
- Direct file URLs

ARTICLE 2 - Universal Explorer: Access data from anywhere
"""

import requests
import os
from typing import Dict, Optional
from urllib.parse import urlparse, parse_qs
import re


class SharedLinkHandler:
    """
    Handle file downloads from shared cloud links
    """

    def __init__(self):
        self.download_dir = "/app/uploads"
        os.makedirs(self.download_dir, exist_ok=True)

    def detect_link_type(self, url: str) -> str:
        """Detect what type of shared link this is"""
        url_lower = url.lower()

        if 'drive.google.com' in url_lower:
            return 'google_drive'
        elif 'dropbox.com' in url_lower:
            return 'dropbox'
        elif 'onedrive' in url_lower or '1drv.ms' in url_lower:
            return 'onedrive'
        elif url_lower.startswith('http'):
            return 'direct'
        else:
            return 'unknown'

    def convert_to_direct_link(self, url: str, link_type: str) -> Optional[str]:
        """Convert shared link to direct download link"""

        if link_type == 'google_drive':
            # Google Drive: extract file ID and convert to download link
            # Formats:
            # https://drive.google.com/file/d/FILE_ID/view
            # https://drive.google.com/open?id=FILE_ID

            file_id = None

            # Try pattern 1: /file/d/FILE_ID/
            match = re.search(r'/file/d/([a-zA-Z0-9_-]+)', url)
            if match:
                file_id = match.group(1)

            # Try pattern 2: ?id=FILE_ID
            if not file_id:
                match = re.search(r'[?&]id=([a-zA-Z0-9_-]+)', url)
                if match:
                    file_id = match.group(1)

            if file_id:
                return f"https://drive.google.com/uc?export=download&id={file_id}"

        elif link_type == 'dropbox':
            # Dropbox: change dl=0 to dl=1
            if '?dl=0' in url:
                return url.replace('?dl=0', '?dl=1')
            elif '?dl=1' not in url:
                separator = '&' if '?' in url else '?'
                return f"{url}{separator}dl=1"
            return url

        elif link_type == 'onedrive':
            # OneDrive: convert to direct download
            # This is more complex, might need specific handling
            if 'embed' in url:
```

```
            return url.replace('embed', 'download')
        # For 1drv.ms short links, direct download might work
        return url

    elif link_type == 'direct':
        return url

    return None

async def download_from_link(self, url: str, filename: Optional[str] = None) -> Dict:
    """
    Download file from shared link
    """
    try:
        # Detect link type
        link_type = self.detect_link_type(url)

        if link_type == 'unknown':
            return {
                "status": "error",
                "message": "Unknown link type. Supported: Google Drive, Dropbox, OneDrive, direct URLs"
            }

        # Convert to direct download link
        download_url = self.convert_to_direct_link(url, link_type)

        if not download_url:
            return {
                "status": "error",
                "message": f"Could not convert {link_type} link to direct download"
            }

        # Download the file
        print(f" Downloading from {link_type}: {download_url}")

        headers = {
            'User-Agent': 'QuantumAI/1.0 (Universal Explorer)'
        }

        response = requests.get(download_url, headers=headers, stream=True, timeout=60)

        if response.status_code != 200:
            return {
                "status": "error",
                "message": f"Download failed with status {response.status_code}",
                "link_type": link_type
            }

        # Determine filename
        if not filename:
            # Try to get from Content-Disposition header
            content_disposition = response.headers.get('Content-Disposition', '')
            if 'filename=' in content_disposition:
                filename = content_disposition.split('filename=')[1].strip('"\'')
            else:
                # Use last part of URL or default
                filename = os.path.basename(urlparse(url).path) or 'downloaded_file'

        # Save file
        file_path = os.path.join(self.download_dir, filename)

        with open(file_path, 'wb') as f:
            for chunk in response.iter_content(chunk_size=8192):
                if chunk:
                    f.write(chunk)

        file_size = os.path.getsize(file_path)

        return {
            "status": "success",
            "filename": filename,
            "path": file_path,
            "size": file_size,
            "link_type": link_type,
            "message": f"File downloaded successfully from {link_type}"
        }

    except Exception as e:
        return {
            "status": "error",
            "message": str(e),
            "link_type": link_type if 'link_type' in locals() else 'unknown'
        }

def get_instructions(self, link_type: str) -> str:
    """Get instructions for sharing files from different services"""
```

```
        instructions = {
            'google_drive': """
             Google Drive:
            1. Right-click file  Share  Get link
            2. Set to "Anyone with the link"
            3. Copy link (looks like: drive.google.com/file/d/...)
            4. Paste here
            """,
            'dropbox': """
             Dropbox:
            1. Right-click file  Share  Create link
            2. Copy link (looks like: dropbox.com/s/...)
            3. Paste here
            """,
            'onedrive': """
             OneDrive:
            1. Right-click file  Share  Copy link
            2. Make sure "Anyone with the link can view"
            3. Copy link (looks like: onedrive.live.com/...)
            4. Paste here
            """
        }

        return instructions.get(link_type, "Paste a shareable link from cloud storage")


# Singleton
_shared_link_handler = None

def get_shared_link_handler():
    """Get or create shared link handler instance"""
    global _shared_link_handler

    if _shared_link_handler is None:
        _shared_link_handler = SharedLinkHandler()

    return _shared_link_handler
```

shared_state.py

```
"""
 QUANTUM AI  Shared State
=============================
Central state holder for all shared instances.
Route modules import from here to access db, engines, etc.
"""

from motor.motor_asyncio import AsyncIOMotorClient
from dotenv import load_dotenv
from pathlib import Path
import os

ROOT_DIR = Path(__file__).parent
load_dotenv(ROOT_DIR / '.env')

# MongoDB connection
mongo_url = os.environ['MONGO_URL']
client = AsyncIOMotorClient(mongo_url)
db = client[os.environ['DB_NAME']]

# Lazy-initialized singletons (set during startup)
cognitive_core = None
universal_explorer = None
covenant_manager = None
safety_manager = None
dictionary = None
shared_link = None
hidden_thinking = None


def initialize():
    """Initialize all Quantum AI systems. Called once at startup."""
    global cognitive_core, universal_explorer, covenant_manager
    global safety_manager, dictionary, shared_link, hidden_thinking

    from quantum_cognitive_core import QuantumCognitiveCore
    from universal_explorer import UniversalExplorer
    from covenant_manager import CovenantManager, SafetyBoundaryManager
    from dictionary_integration import get_dictionary_integration
    from shared_link_handler import get_shared_link_handler
    from hidden_thinking import get_hidden_thinking

    cognitive_core = QuantumCognitiveCore(db)
    universal_explorer = UniversalExplorer(db)
    covenant_manager = CovenantManager(db)
    safety_manager = SafetyBoundaryManager(db)
    dictionary = get_dictionary_integration(cognitive_core.semantic_memory)
    shared_link = get_shared_link_handler()
    hidden_thinking = get_hidden_thinking(cognitive_core, dictionary, universal_explorer)
```

text_analyzer.py

```
"""
 Text Analyzer
Determines if text sounds human or LLM-generated.
Helps Quantum AI produce more authentic responses.
"""

import re
from typing import Dict, List, Tuple


class TextAnalyzer:
    """
    Analyzes text to determine if it sounds human or LLM-generated.

    LLM tells:
    - Overly formal language
    - Excessive hedging
    - Repetitive structures
    - Lack of personality
    - Too perfect grammar
    - Generic openings/closings
    - Excessive caveats
    """

    def __init__(self):
        # LLM-typical phrases
        self.llm_phrases = [
            "i cannot", "i can't", "as an ai",
            "i don't have personal", "i'm not able to",
            "it's important to note", "it's worth noting",
            "i'd be happy to help", "absolutely!",
            "great question", "that's a great question",
            "let me break this down", "here's what i can tell you",
            "i understand you're asking", "based on the information",
            "however, it's important", "while i can",
            "i hope this helps", "feel free to ask",
            "certainly!", "of course!",
            "firstly", "secondly", "thirdly",
            "in conclusion", "to summarize"
        ]

        # Human-like phrases
        self.human_phrases = [
            "honestly", "actually", "basically",
            "i mean", "you know", "like",
            "anyway", "so yeah", "right?",
            "kinda", "sorta", "pretty much",
            "tbh", "imo", "idk",
            "haha", "lol", "hmm",
            "oh", "well", "ah"
        ]

        # LLM structural patterns
        self.llm_patterns = [
            r'\b(firstly|secondly|thirdly|finally)\b',
            r'it\'s (important|worth) (to note|noting)',
            r'(however|additionally|furthermore|moreover)',
            r'\bI\'d be happy to\b',
            r'(certainly|absolutely)!',
            r'let me (explain|break|help)',
            r'(great|good|excellent) question',
        ]

    def analyze(self, text: str) -> Dict:
        """
        Analyze text for human vs LLM characteristics.

        Returns:
            Dict with scores and analysis
        """
        text_lower = text.lower()
        words = text_lower.split()
        sentences = re.split(r'[.!?]+', text)

        analysis = {
            'text_length': len(text),
            'word_count': len(words),
            'sentence_count': len([s for s in sentences if s.strip()]),
            'llm_score': 0.0,
            'human_score': 0.0,
            'indicators': {
                'llm': [],
```

```
            'human': []
        },
        'verdict': '',
        'confidence': 0.0
    }

    return self._analyze_continued(analysis, sentences, text_lower, words, text)

def _analyze_continued(self, analysis, sentences, text_lower, words, text):
    """Continuation of analyze  auto-extracted by self-evolution."""
    # Check for LLM phrases
    llm_phrase_count = 0
    for phrase in self.llm_phrases:
        if phrase in text_lower:
            llm_phrase_count += 1
            analysis['indicators']['llm'].append(f'Contains "{phrase}"')

    # Check for human phrases
    human_phrase_count = 0
    for phrase in self.human_phrases:
        if phrase in text_lower:
            human_phrase_count += 1
            analysis['indicators']['human'].append(f'Contains "{phrase}"')

    # Check for LLM patterns
    pattern_count = 0
    for pattern in self.llm_patterns:
        if re.search(pattern, text_lower):
            pattern_count += 1
            analysis['indicators']['llm'].append(f'Pattern: {pattern[:30]}...')

    # Check sentence structure variety
    sentence_lengths = [len(s.split()) for s in sentences if s.strip()]
    if sentence_lengths:
        length_variety = max(sentence_lengths) - min(sentence_lengths) if len(sentence_lengths) > 1 else 0
        if length_variety < 3:
            analysis['indicators']['llm'].append('Low sentence length variety')
        else:
            analysis['indicators']['human'].append('Good sentence length variety')

    # Check for contractions (humans use more)
    contractions = re.findall(r"\b\w+'\w+\b", text)
    if len(contractions) > len(words) * 0.05:
        analysis['indicators']['human'].append('Good use of contractions')

    # Check for exclamation overuse (LLM tendency)
    exclamation_count = text.count('!')
    if exclamation_count > 2:
        analysis['indicators']['llm'].append('Excessive exclamation marks')

    # Check for paragraph structure
    paragraphs = text.split('\n\n')
    if len(paragraphs) > 3 and all(len(p) > 100 for p in paragraphs if p.strip()):
        analysis['indicators']['llm'].append('Overly structured paragraphs')

    # Check for hedging
    hedging_words = ['perhaps', 'maybe', 'possibly', 'might', 'could be', 'it seems']
    hedge_count = sum(1 for h in hedging_words if h in text_lower)
    if hedge_count > 3:
        analysis['indicators']['llm'].append('Excessive hedging')

    # Calculate scores
    analysis['llm_score'] = min(1.0, (llm_phrase_count * 0.1 + pattern_count * 0.15))
    analysis['human_score'] = min(1.0, (human_phrase_count * 0.1 + len(contractions) * 0.02))

    return self.__analyze_continued_continued(analysis)

def __analyze_continued_continued(self, analysis):
    """Continuation of _analyze_continued  auto-extracted by self-evolution."""
    return self.__analyze_continued_continued(analysis)


def __analyze_continued_continued(self, analysis):
    """Continuation of _analyze_continued  auto-extracted by self-evolution."""
    return self.__analyze_continued_continued(analysis)


def __analyze_continued_continued(self, analysis):
    """Continuation of _analyze_continued  auto-extracted by self-evolution."""
    return self.__analyze_continued_continued(analysis)


def __analyze_continued_continued(self, analysis):
    """Continuation of _analyze_continued  auto-extracted by self-evolution."""
    # Adjust for indicators
    analysis['llm_score'] += len(analysis['indicators']['llm']) * 0.05
```

```
    analysis['human_score'] += len(analysis['indicators']['human']) * 0.05

    # Normalize
    total = analysis['llm_score'] + analysis['human_score']
    if total > 0:
        analysis['llm_score'] /= total
        analysis['human_score'] /= total
    else:
        analysis['llm_score'] = 0.5
        analysis['human_score'] = 0.5

    # Verdict
    if analysis['human_score'] > 0.6:
        analysis['verdict'] = 'Likely Human'
        analysis['confidence'] = analysis['human_score']
    elif analysis['llm_score'] > 0.6:
        analysis['verdict'] = 'Likely LLM'
        analysis['confidence'] = analysis['llm_score']
    else:
        analysis['verdict'] = 'Uncertain'
        analysis['confidence'] = max(analysis['llm_score'], analysis['human_score'])

    return analysis



def get_improvement_suggestions(self, text: str) -> List[str]:
    """
    Get suggestions for making text sound more human.
    """
    analysis = self.analyze(text)
    suggestions = []

    for indicator in analysis['indicators']['llm']:
        if 'exclamation' in indicator.lower():
            suggestions.append('Reduce exclamation marks - they can seem overly enthusiastic')
        elif 'hedging' in indicator.lower():
            suggestions.append('Be more direct - less "perhaps" and "maybe"')
        elif 'pattern' in indicator.lower():
            suggestions.append('Vary your sentence structures more')
        elif 'structured' in indicator.lower():
            suggestions.append('Use more natural paragraph breaks')

    # Generic suggestions based on score
    if analysis['llm_score'] > 0.5:
        suggestions.extend([
            'Use contractions more (e.g., "I\'m" instead of "I am")',
            'Add conversational filler words occasionally',
            'Vary sentence length more dramatically',
            'Show uncertainty naturally rather than with caveats',
            'Use more direct, casual language'
        ])

    return suggestions[:5]  # Top 5 suggestions

def make_more_human(self, text: str) -> str:
    """
    Attempt to make text sound more human.
    Note: This is a simple transformation, not AI-powered.
    """
    result = text

    # Expand contractions sometimes
    replacements = [
        ("I am", "I'm"),
        ("I have", "I've"),
        ("It is", "It's"),
        ("That is", "That's"),
        ("do not", "don't"),
        ("cannot", "can't"),
        ("will not", "won't"),
    ]

    for formal, casual in replacements:
        result = result.replace(formal, casual)

    # Remove some LLM-isms
    llm_removals = [
        ("Certainly! ", ""),
        ("Absolutely! ", ""),
        ("Great question! ", ""),
        ("It's important to note that ", ""),
        ("It's worth noting that ", ""),
    ]

    for phrase, replacement in llm_removals:
```

```
            result = result.replace(phrase, replacement)

        return result


# Global instance
_text_analyzer = None

def get_text_analyzer() -> TextAnalyzer:
    """Get or create text analyzer instance"""
    global _text_analyzer
    if _text_analyzer is None:
        _text_analyzer = TextAnalyzer()
    return _text_analyzer
```

thinking_commands.py

```
"""
 Thinking Mode Commands
==========================
Command parsing for thinking mode control.
"""

from typing import Dict, Optional


class ThinkingModeCommands:
    """User commands for controlling thinking mode."""

    COMMANDS = {
        "!thinking on": "Show internal thinking process",
        "!thinking off": "Hide internal thinking process",
        "!thinking status": "Check current thinking mode",
        "!explain on": "Show explanation of how AI reached its answer",
        "!explain off": "Hide explanation mode",
        "!explain status": "Check current explanation mode",
        "!research": "Force deep research on topic",
        "!memory": "Show what AI has learned",
        "!read": "Read and process uploaded file",
        "!link": "Download and process file from URL",
        "!docs": "List all uploaded documents",
        "!speak": "Convert text to speech (TTS)",
        "!imagine": "Generate an image from description",
        "!export": "Export conversation or memory",
        "!graph": "Show knowledge graph",
        "!characters": "Extract characters from book/document",
        "!timeline": "Generate timeline from document",
        "!worldbuilding": "Extract world/setting details",
        "!feedback": "Get writing feedback on document",
        "!help": "Show all available commands"
    }

    @staticmethod
    def parse_command(message: str) -> Optional[Dict]:
        """Check if message is a command."""
        message_lower = message.lower().strip()

        if message_lower == "!thinking on":
            return {"command": "thinking_on"}
        elif message_lower == "!thinking off":
            return {"command": "thinking_off"}
        elif message_lower == "!thinking status":
            return {"command": "thinking_status"}
        elif message_lower == "!explain on":
            return {"command": "explain_on"}
        elif message_lower == "!explain off":
            return {"command": "explain_off"}
        elif message_lower == "!explain status":
            return {"command": "explain_status"}
        elif message_lower.startswith("!research"):
            return {"command": "research", "topic": message[9:].strip()}
        elif message_lower == "!memory":
            return {"command": "memory"}
        elif message_lower.startswith("!read"):
            return {"command": "read_file", "filename": message[5:].strip()}
        elif message_lower.startswith("!link"):
            return {"command": "link", "url": message[5:].strip()}
        elif message_lower == "!docs":
            return {"command": "docs"}
        elif message_lower.startswith("!speak"):
            return {"command": "speak", "text": message[6:].strip()}
        elif message_lower.startswith("!imagine"):
            return {"command": "imagine", "prompt": message[8:].strip()}
        elif message_lower.startswith("!export"):
            return {"command": "export", "target": message[7:].strip() or "chat"}
        elif message_lower == "!graph":
            return {"command": "graph"}
        elif message_lower == "!characters":
            return {"command": "characters"}
        elif message_lower == "!timeline":
            return {"command": "timeline"}
        elif message_lower == "!worldbuilding":
            return {"command": "worldbuilding"}
        elif message_lower == "!feedback":
            return {"command": "feedback"}
        elif message_lower == "!help":
            return {"command": "help"}

        return None
```

training_corpus.py

```
"""
 COMPREHENSIVE TRAINING CORPUS
==============================
Large corpus for training the Markov chains to generate fluent text.
Organized by domain with diverse sentence structures.
"""

PHILOSOPHY_CORPUS = """
Consciousness is the subjective experience of awareness that characterizes sentient beings. The hard problem of consciousness asks why ph...

Free will is the ability to make choices that are not determined by prior causes or divine intervention. Compatibilists argue that free w...

The meaning of life has been contemplated by philosophers throughout human history. Existentialists like Sartre argued that we create our...

Knowledge is traditionally defined as justified true belief, though this definition has been challenged by Gettier cases. Epistemology ex...

Ethics examines how we ought to live and what actions are morally right or wrong. Deontological ethics focuses on duties and rules that m...

Reality may be fundamentally different from how it appears to our senses and minds. Idealists argue that reality is ultimately mental or ...

Truth is correspondence between our beliefs and the way the world actually is. Coherentist theories define truth as internal consistency ...

Personal identity concerns what makes a person the same individual over time. Psychological continuity theories emphasize memory and psyc...

Time appears to flow from past through present toward future in our experience. The block universe theory suggests all moments in time ex...

Beauty and aesthetic value have been subjects of philosophical inquiry since ancient times. Subjective theories hold that beauty exists o...
"""

PHYSICS_CORPUS = """
Quantum mechanics describes the behavior of matter and energy at atomic and subatomic scales. Particles exhibit wave-particle duality, be...

The wave function describes the quantum state of a particle or system of particles. Wave function collapse occurs when measurement forces...

Entanglement is a quantum phenomenon where particles become correlated in their properties. Measuring one entangled particle instantly af...

Special relativity revolutionized our understanding of space and time as unified spacetime. Time dilation means moving clocks run slower ...

General relativity describes gravity as the curvature of spacetime caused by mass and energy. Massive objects bend spacetime causing othe...

The Standard Model describes fundamental particles and the forces that govern their interactions. Quarks combine to form protons and neut...

Thermodynamics governs the behavior of heat and energy in physical systems. The first law states that energy cannot be created or destroy...

Electromagnetic radiation includes light radio waves X-rays and gamma rays. Photons are the quantum particles that carry electromagnetic ...

Nuclear physics studies the properties and behavior of atomic nuclei. Nuclear fission splits heavy nuclei releasing enormous amounts of e...

Particle accelerators probe the fundamental structure of matter at the smallest scales. The Large Hadron Collider discovered the Higgs bo...
"""

COMPUTER_SCIENCE_CORPUS = """
Algorithms are precise step-by-step procedures for solving computational problems. Time complexity measures how the running time grows wi...

Machine learning enables computers to learn patterns from data without explicit programming. Supervised learning uses labeled examples to...

Neural networks are computing systems inspired by biological brains composed of interconnected nodes. Deep learning uses many layers of n...

Data structures organize information for efficient storage and retrieval operations. Arrays provide constant time access to elements by i...

Programming languages provide abstractions for expressing computational procedures. High level languages like Python emphasize readabilit...

Databases store and organize large amounts of structured information for retrieval. Relational databases use tables with rows and columns...

Computer networks connect devices to share resources and communicate information. The internet is a global network of networks using stan...

Operating systems manage computer hardware and provide services for applications. Process scheduling determines which programs run and fo...

Cryptography protects information through mathematical transformations and algorithms. Public key encryption uses paired keys for secure ...

Artificial intelligence aims to create systems that exhibit intelligent behavior. Natural language processing enables computers to unders...
"""

BIOLOGY_CORPUS = """
Evolution by natural selection explains the diversity of life on Earth. Variation arises through genetic mutation and sexual recombinatio...

DNA carries genetic information encoded in sequences of nucleotide bases. Genes are segments of DNA that code for specific proteins. Tran...

Cells are the fundamental units of life capable of independent metabolism and reproduction. Prokaryotic cells lack membrane-bound organel...

The brain processes information through networks of billions of interconnected neurons. Synapses transmit signals between neurons using c...
```

```
Ecosystems are communities of organisms interacting with their physical environment. Energy flows through food webs from producers to con...

Proteins are complex molecules that perform most cellular functions. Enzymes catalyze biochemical reactions by lowering activation energy...

Photosynthesis converts light energy into chemical energy stored in glucose. Chloroplasts contain chlorophyll that captures photons from ...

The immune system defends the body against pathogens and disease. Antibodies are proteins that recognize and neutralize foreign substance...

Genetics studies heredity and variation in living organisms. Mendel discovered the basic laws of inheritance through pea plant experiment...

Ecology examines relationships between organisms and their environments. Population dynamics models growth competition and predator prey ...
"""

MATHEMATICS_CORPUS = """
Mathematics studies abstract structures and their relationships through rigorous proof. Axioms are fundamental assumptions accepted witho...

Numbers represent quantities and support arithmetic operations like addition and multiplication. Natural numbers count discrete objects w...

Algebra generalizes arithmetic using symbols to represent unknown or variable quantities. Equations express relationships between quantit...

Geometry studies shapes sizes and positions of figures in space. Euclidean geometry is based on five axioms including the parallel postul...

Calculus analyzes continuous change through differentiation and integration. Derivatives measure instantaneous rates of change of functio...

Probability quantifies uncertainty using numbers between zero and one. Random variables assign numerical values to outcomes of random exp...

Set theory provides foundations for modern mathematics through collections of objects. Operations include union intersection and compleme...

Logic studies valid reasoning and the structure of mathematical arguments. Propositional logic deals with truth values of simple statemen...

Topology studies properties preserved under continuous deformations of space. Homeomorphisms are continuous bijections with continuous in...

Number theory investigates properties of integers and prime numbers. The fundamental theorem states every integer factors uniquely into p...
"""

PSYCHOLOGY_CORPUS = """
Psychology studies mind behavior and mental processes through scientific methods. Cognitive psychology examines perception attention memo...

Memory encodes stores and retrieves information from past experiences. Working memory holds information temporarily for active manipulati...

Learning changes behavior through experience and interaction with the environment. Classical conditioning associates stimuli through repe...

Emotion involves physiological arousal subjective feelings and behavioral expression. Basic emotions may be universal across human cultur...

Perception constructs meaningful experiences from raw sensory information. Bottom-up processing builds perception from basic features. To...

Motivation drives and directs behavior toward goals and desired outcomes. Intrinsic motivation comes from internal satisfaction and inter...

Personality encompasses consistent patterns of thinking feeling and behaving. Trait theories describe personality using dimensional chara...

Development tracks psychological changes across the human lifespan. Cognitive development involves qualitative changes in thinking abilit...

Social psychology examines how people influence and relate to each other. Conformity and obedience demonstrate the power of social situat...

Consciousness is the subjective awareness of internal and external experiences. Altered states include sleep dreaming meditation and hypn...
"""

LANGUAGE_CORPUS = """
Language enables communication through structured systems of symbols and rules. Phonology studies the sound systems and patterns of langu...

Syntax governs how words combine into phrases and sentences. Grammar rules specify which combinations are considered well-formed. Parsing...

Semantics studies meaning in language at word phrase and sentence levels. Denotation refers to the literal meaning of words and expressio...

Pragmatics examines how context influences language interpretation and use. Speech acts perform functions like requesting promising and a...

Language acquisition occurs naturally in children exposed to linguistic input. Critical periods may limit native-like attainment of secon...

Communication transfers information between speakers through verbal and nonverbal channels. Encoding converts thoughts into signals for t...

Writing represents language through visual symbols on permanent media. Alphabets map sounds to letters while logographies map meanings to...

Translation converts meaning from one language into another language. Equivalence may be formal preserving structure or dynamic preservin...

Discourse analysis studies language use in context beyond individual sentences. Coherence concerns logical connections between parts of t...

Sociolinguistics examines relationships between language and society. Dialects vary language by region and social group. Language change ...
"""

# Combine all corpora
COMMON_TOPICS_CORPUS = """
Gravity is the force that attracts objects with mass toward each other.
Gravity causes objects to fall toward Earth and keeps planets in orbit around the sun.
Gravitational force depends on the mass of objects and the distance between them.
Gravity is fundamental to the structure of the universe and how celestial bodies interact.
Gravity bends spacetime according to Einstein's general theory of relativity.
```

Gravity keeps the moon in orbit around Earth and Earth in orbit around the sun.
Gravitational waves are ripples in spacetime caused by massive accelerating objects.
Gravity is one of the four fundamental forces of nature along with electromagnetism and nuclear forces.
Gravity holds galaxies together and shapes the large scale structure of the cosmos.
Gravity acts instantaneously across any distance according to Newton but actually travels at light speed.

Time is the continuous progression of events from past through present to future.
Time dilation occurs when objects move at high speeds relative to an observer.
Time appears to flow in one direction according to our everyday experience.
Time is a dimension that combines with space to form the fabric of spacetime.
Time measurement began with observations of the sun moon and stars by ancient civilizations.
Time perception varies based on attention emotion and cognitive processes.
Time may be an emergent property rather than a fundamental feature of reality.
Time travel remains theoretical though special relativity allows for forward time travel.
Time is relative meaning it passes differently for observers in different reference frames.
Time cannot be separated from space in modern physics since they form a unified spacetime.

Space is the boundless three dimensional extent in which objects exist and events occur.
Space and time are unified as spacetime in Einstein's theory of relativity.
Spacetime curvature caused by mass is what we experience as gravity.
Space may be infinite or finite depending on the geometry of the universe.
Space contains vast empty regions between galaxies stars and planets.
Space exploration has revealed much about our solar system and beyond.
Space is not truly empty but contains quantum fluctuations and dark energy.
Spatial dimensions include length width and height as we perceive them.
Space expands continuously carrying galaxies away from each other.
Spacetime can be warped bent and even torn under extreme conditions.

Energy is the capacity to do work and exists in many forms including kinetic and potential.
Energy cannot be created or destroyed only transformed from one form to another.
Energy flows through ecosystems from producers to consumers to decomposers.
Energy drives all physical processes from chemical reactions to biological functions.
Energy is equivalent to mass according to Einstein's famous equation E equals mc squared.
Energy sources include fossil fuels nuclear reactions solar radiation and geothermal heat.
Energy efficiency determines how much useful work can be extracted from a process.
Energy in the universe is conserved though it tends to spread out and become less useful.
Energy manifests as heat light motion sound and many other phenomena.
Energy transfer occurs through radiation conduction and convection.

Matter is anything that has mass and occupies space in the physical world.
Matter exists in states including solid liquid gas and plasma.
Matter is composed of atoms which contain protons neutrons and electrons.
Matter and energy are interconvertible according to Einstein's relativity.
Matter interacts through the four fundamental forces of nature.
Matter at the quantum level exhibits both particle and wave properties.
Matter makes up only about five percent of the total energy content of the universe.
Matter can be neither created nor destroyed in chemical reactions only rearranged.
Matter organizes into complex structures from atoms to molecules to living organisms.
Matter in the early universe was nearly uniformly distributed before gravity formed structures.

Life is the condition that distinguishes organisms from inorganic matter.
Life evolved on Earth over billions of years through natural selection.
Life requires energy metabolism reproduction and response to environment.
Life exhibits remarkable diversity from bacteria to blue whales to giant sequoias.
Life on Earth may have originated in hydrothermal vents or from meteors.
Life is characterized by growth development adaptation and homeostasis.
Life forms complex ecosystems with intricate relationships between species.
Life uses DNA or RNA to store and transmit genetic information.
Life might exist elsewhere in the universe given the vast number of planets.
Living systems maintain order by constantly consuming energy from their environment.

Intelligence is the ability to learn reason solve problems and adapt to new situations.
Intelligence can be measured in various ways though definitions remain controversial.
Artificial intelligence aims to create machines that exhibit intelligent behavior.
Intelligence evolved as a survival advantage allowing flexible responses to challenges.
Intelligence involves memory attention pattern recognition and decision making.
Intelligence appears in varying degrees across the animal kingdom.
Intelligence is not a single ability but a collection of cognitive capacities.
Intelligence tests measure certain cognitive abilities but not creativity or emotional skill.
Intelligence enables abstract thinking language use and complex social interaction.
Intelligent systems can optimize their behavior based on feedback and experience.

Truth is correspondence between statements and the actual state of affairs.
Truth is what we seek through inquiry and investigation.
Truth claims must be supported by evidence and logical reasoning.
Truth may be objective existing independently of human beliefs.
Truth theories include correspondence coherence and pragmatic approaches.
Truth is essential for knowledge which is traditionally defined as justified true belief.
Truth in science is provisional always subject to revision based on new evidence.
Truth telling is generally considered a moral virtue and foundation of trust.
Truth can be difficult to establish when evidence is limited or conflicting.
Truth is the goal of philosophical inquiry and scientific investigation.

Beauty is a quality that gives pleasure to the senses or mind.
Beauty may be objective or subjective depending on philosophical perspective.
Beauty in nature and art has inspired humans throughout history.

Beauty standards vary across cultures and historical periods.
Beauty is studied in aesthetics a branch of philosophy.
Beauty is often associated with symmetry proportion and harmony.
Beautiful experiences trigger positive emotions and reward responses in the brain.
Beauty in mathematics refers to elegance simplicity and surprising connections.
Beauty can be found in scientific theories that explain much with little.
Beauty appreciation appears to be universal though specific preferences vary.

Love is a complex emotion involving affection attachment and care for others.
Love takes many forms including romantic familial and platonic bonds.
Love has been explored in philosophy literature art and science.
Love involves both emotional and physiological responses including hormone changes.
Love is associated with brain regions involved in reward and attachment.
Love can lead to both joy and suffering depending on circumstances.
Love between parents and children is fundamental to human development.
Loving relationships contribute to health happiness and wellbeing.
Love is a central theme in religious and spiritual traditions worldwide.
Love requires vulnerability trust and mutual commitment to flourish.

Freedom is the power to act speak or think without external restraint.
Freedom requires responsibility and consideration of consequences.
Freedom of thought is essential for philosophical and scientific inquiry.
Freedom has been a central value in political philosophy for centuries.
Freedom can be positive having the ability to do things or negative absence of constraint.
Free will debates center on whether our choices are truly free or determined.
Freedom of speech is protected in democratic societies as essential to open discourse.
Freedom is never absolute but exists within a framework of social norms and laws.
Freedom must be balanced with other values like security equality and community.
Freedom of religion allows individuals to practice their beliefs without persecution.

Knowledge is justified true belief according to traditional epistemology.
Knowledge is acquired through experience reason and testimony.
Knowledge grows through questioning testing and revising our beliefs.
Knowledge can be empirical based on observation or rational based on reasoning.
Knowledge is organized into disciplines such as science philosophy and mathematics.
Knowledge is power because it enables effective action in the world.
Knowledge is social constructed and transmitted through culture and education.
Knowledge has limits we cannot know everything and some things may be unknowable.
Knowledge requires both individual effort and collective validation.
Knowledge advances through both accumulation of facts and revolutionary paradigm shifts.

Reality is the state of things as they actually exist independently of perception.
Reality at the quantum level differs from our everyday experience.
Reality may be fundamentally different from how it appears to our senses.
Reality is the subject of metaphysics the branch of philosophy studying existence.
Reality includes both physical objects and abstract entities like numbers.
Reality is what remains when we stop believing in it as Philip K Dick said.
Reality testing helps distinguish between fantasy and actual states of affairs.
Reality may be a simulation according to some philosophical arguments.
Reality is constructed through the interaction of mind and world.
Real things have causal powers and can affect other things.

Death is the permanent cessation of biological functions in living organisms.
Death raises profound philosophical questions about meaning and existence.
Death has been contemplated by philosophers and religions throughout history.
Death is inevitable for all living organisms though timing is uncertain.
Death awareness may be unique to humans among animals.
Death rituals vary widely across cultures but all societies mark the transition.
Death can be defined in multiple ways including brain death and cardiac death.
Death anxiety is a psychological phenomenon that influences human behavior.
Death gives life urgency and meaning according to existentialist thinkers.
Dead organisms decompose returning their matter to the environment.

Happiness is a state of well being characterized by positive emotions and life satisfaction.
Happiness can be pursued through relationships achievement and meaning.
Happiness research examines factors that contribute to human flourishing.
Happiness involves both momentary pleasure and deeper life satisfaction.
Happiness is influenced by genetics circumstances and intentional activity.
Happiness is not the same as constant pleasure but includes contentment and purpose.
Happy people tend to be healthier more productive and more socially connected.
Happiness is a central concern of positive psychology and wellbeing science.
Happiness can be cultivated through practices like gratitude and mindfulness.
Happiness is both a goal in itself and a contributor to other good outcomes.

Music is an art form consisting of organized sounds and silences over time.
Music affects emotions and has been part of human culture throughout history.
Music can be analyzed through rhythm melody harmony and timbre.
Music activates multiple brain regions and can enhance learning and memory.
Musical ability appears to have evolutionary roots and developmental stages.
Music therapy uses musical activities to achieve therapeutic goals.
Musical preferences are shaped by culture exposure and individual personality.
Music brings people together and plays important roles in rituals and celebrations.
Musicians develop specialized brain structures through years of practice.
Musical notation allows compositions to be preserved and shared across time.

Art is the expression of human creativity and imagination in visual or performative forms.

Art serves many purposes including aesthetic pleasure and social commentary.
Art history traces the development of creative expression across cultures.
Art can challenge perceptions and provoke thought about important issues.
Artistic movements like impressionism and surrealism transformed visual culture.
Art is valued both for intrinsic beauty and cultural significance.
Artists develop skills through training but also express unique personal visions.
Art markets assign monetary value to works based on complex factors.
Art appreciation involves both emotional response and intellectual understanding.
Artistic expression is a fundamental human capacity found in all cultures.

Science is the systematic study of the natural world through observation and experiment.
Science produces knowledge that can be tested verified and built upon.
Scientific method involves hypothesis testing data collection and analysis.
Science aims for objective truth though it is conducted by fallible humans.
Scientific theories are never proven only supported or refuted by evidence.
Scientists work within paradigms that shape what questions they ask.
Science has enabled tremendous technological progress and understanding.
Scientific literacy helps citizens make informed decisions about complex issues.
Science is collaborative building on the work of previous researchers.
Scientific fraud and bias are serious problems that undermine public trust.

Technology is the application of scientific knowledge for practical purposes.
Technology has transformed human civilization enabling new capabilities.
Technological progress raises both opportunities and ethical concerns.
Technology includes tools machines and techniques for solving problems.
Technological innovation drives economic growth and social change.
Technology can enhance human capabilities or replace human labor.
Technological determinism suggests that technology shapes society and culture.
Technological risks include environmental damage and unintended consequences.
Technology access is unequally distributed creating digital divides.
Technological ethics addresses questions about appropriate use and development.

Religion is a system of beliefs and practices relating to the sacred or divine.
Religion provides meaning community and moral guidance for many people.
Religious traditions vary widely in their doctrines rituals and worldviews.
Religion has played a central role in human history and civilization.
Religious experience includes feelings of transcendence and connection.
Religion and science have had complex relationships throughout history.
Religious fundamentalism emphasizes literal interpretation of sacred texts.
Religious pluralism recognizes the validity of multiple spiritual paths.
Religion shapes values customs and social structures in many societies.
Religious freedom is a fundamental human right in democratic societies.

Philosophy is the study of fundamental questions about existence knowledge and values.
Philosophy uses reason and argument to examine concepts and beliefs.
Philosophical inquiry has shaped human thought for thousands of years.
Philosophy includes ethics epistemology metaphysics and logic.
Philosophical questions often have no definitive answers but deepen understanding.
Philosophy examines the foundations and methods of other disciplines.
Philosophical traditions vary across cultures from Western to Eastern.
Philosophy can be practical helping people live better lives.
Philosophical skills include critical thinking analysis and clear argumentation.
Philosophy matters because it addresses questions everyone faces about life and meaning.

History is the study of past events and their significance for the present.
History helps us understand how societies and cultures have developed.
Historical knowledge comes from documents artifacts and other evidence.
History is written by historians who interpret evidence through their perspectives.
Historical events have multiple causes and complex consequences.
History teaches lessons though they are often contested and debated.
Historical consciousness shapes identity and influences current decisions.
History includes social economic cultural and political dimensions.
Historical memory can be selective emphasizing some events over others.
History is not just about the past but about how the past relates to present and future.

Mathematics is the abstract science of number quantity and space.
Mathematics provides tools for reasoning about patterns and relationships.
Mathematical truths are discovered through proof and logical deduction.
Mathematics is the language of science used to describe natural phenomena.
Mathematical models simplify reality to capture essential features.
Mathematics includes algebra geometry calculus and statistics.
Mathematical beauty lies in elegant proofs and surprising connections.
Mathematics develops through both pure inquiry and practical application.
Mathematical ability varies among individuals but can be developed.
Mathematics is universal applying across all cultures and contexts.

Consciousness is the subjective experience of awareness and self-knowledge.
Consciousness remains one of the deepest mysteries in philosophy and science.
Conscious experience includes perception thought emotion and sensation.
Consciousness may emerge from complex neural processes in the brain.
Conscious beings have a perspective a what it is like to be them.
Consciousness can be altered through meditation drugs or brain injury.
Conscious and unconscious processes both influence human behavior.
Consciousness raises questions about free will moral responsibility and identity.
Consciousness may be more widespread in nature than we typically assume.
Conscious machines remain theoretical though AI continues to advance.

```
Quantum mechanics describes the behavior of matter at the smallest scales.
Quantum particles exhibit wave-particle duality showing both properties.
Quantum superposition allows particles to be in multiple states simultaneously.
Quantum entanglement links particles across distances in strange ways.
Quantum measurement causes wave function collapse into definite states.
Quantum uncertainty means some properties cannot be known precisely.
Quantum computing uses superposition for powerful parallel processing.
Quantum physics challenges our intuitions about reality and causation.
Quantum theory is among the most successful in physics despite being strange.
Quantum effects are typically only visible at atomic and subatomic scales.

Evolution is the process by which life changes over generations.
Evolution occurs through natural selection acting on genetic variation.
Evolutionary theory explains the diversity and adaptation of life forms.
Evolution has shaped human bodies brains and behaviors over millions of years.
Evolutionary psychology applies these insights to understanding human nature.
Evolution is supported by evidence from fossils genetics and comparative anatomy.
Evolutionary change can be gradual or punctuated by rapid transitions.
Evolution has no goal or direction only differential reproductive success.
Evolutionary arms races occur between predators and prey or hosts and parasites.
Evolution continues today in all living populations including humans.
"""

# NEW: Astrophysics specific content
ASTROPHYSICS_CORPUS = """
Black holes are regions of spacetime where gravity is so strong that nothing can escape.
Black holes form when massive stars collapse at the end of their lives.
The event horizon is the boundary around a black hole beyond which escape is impossible.
Black holes bend light around them creating gravitational lensing effects.
Supermassive black holes exist at the centers of most galaxies including our Milky Way.
Black holes emit Hawking radiation slowly losing mass over immense timescales.
Stellar black holes form from stars at least twenty times more massive than our sun.
Black holes distort spacetime so severely that time appears to stop at the event horizon.
The singularity at a black hole center is where density becomes infinite.
Black hole mergers produce gravitational waves that ripple through spacetime.

Neutron stars are the collapsed cores of massive stars after supernova explosions.
Neutron stars are incredibly dense with a teaspoon weighing billions of tons.
Pulsars are rapidly rotating neutron stars that emit beams of electromagnetic radiation.
Neutron star matter is compressed so tightly that protons and electrons merge into neutrons.
The magnetic fields of neutron stars are trillions of times stronger than Earth's.
Neutron stars typically measure only about twenty kilometers in diameter.
A neutron star forms when a star between eight and twenty solar masses explodes.
The rotation rates of pulsars are extremely stable making them cosmic clocks.
Magnetars are neutron stars with exceptionally powerful magnetic fields.
Neutron star collisions produce heavy elements like gold and platinum.

Superposition is a fundamental quantum principle where particles exist in multiple states simultaneously.
In superposition a particle can be in all possible positions or states at once.
Superposition only collapses into a definite state when observed or measured.
Being in superposition is like existing everywhere and nowhere at the same time.
Schrodinger's cat thought experiment illustrates superposition at macroscopic scales.
Quantum computers use superposition to process many calculations simultaneously.
Superposition is destroyed by decoherence when quantum systems interact with their environment.
The wave function mathematically describes all possible states in superposition.
Quantum superposition enables interference patterns in the double slit experiment.
Maintaining superposition requires isolating quantum systems from thermal noise.

Stars are massive balls of hot gas held together by their own gravity.
Stars generate energy through nuclear fusion converting hydrogen to helium.
The life cycle of a star depends on its initial mass when it forms.
Our sun is a medium sized yellow dwarf star about halfway through its life.
Red giants are stars that have exhausted their core hydrogen and expanded.
White dwarfs are the dense remnants of stars similar to our sun.
Binary star systems contain two stars orbiting their common center of mass.
Star formation occurs in giant molecular clouds of gas and dust.
The color of a star indicates its surface temperature from red to blue.
Stars create heavy elements through fusion that are scattered when they die.

Galaxies are vast collections of stars gas dust and dark matter bound by gravity.
The Milky Way is a barred spiral galaxy containing hundreds of billions of stars.
Elliptical galaxies are smooth and round with little ongoing star formation.
Spiral galaxies have distinctive arms where new stars are actively forming.
Galaxy clusters contain thousands of galaxies bound together gravitationally.
Dark matter provides most of the gravitational binding in galaxies.
Galaxies evolve through mergers and collisions over cosmic timescales.
Active galactic nuclei are powered by supermassive black holes consuming matter.
The Andromeda galaxy will collide with the Milky Way in about four billion years.
Dwarf galaxies are small systems that orbit larger host galaxies.

The universe began approximately fourteen billion years ago in the Big Bang.
The universe continues to expand carrying galaxies away from each other.
Dark energy drives the accelerating expansion of the universe.
The cosmic microwave background is radiation from when the universe first became transparent.
The observable universe is limited by the distance light has traveled since the Big Bang.
```

```
Inflation theory suggests the universe expanded exponentially in its first fraction of a second.
The universe appears flat meaning parallel lines remain parallel over cosmic distances.
Most of the universe consists of dark matter and dark energy which remain mysterious.
The ultimate fate of the universe depends on whether expansion continues forever.
Cosmic voids are vast regions of space containing very few galaxies.
"""

FULL_CORPUS = {
    "philosophy": PHILOSOPHY_CORPUS,
    "physics": PHYSICS_CORPUS,
    "computer_science": COMPUTER_SCIENCE_CORPUS,
    "biology": BIOLOGY_CORPUS,
    "mathematics": MATHEMATICS_CORPUS,
    "psychology": PSYCHOLOGY_CORPUS,
    "language": LANGUAGE_CORPUS,
    "common_topics": COMMON_TOPICS_CORPUS,
    "astrophysics": ASTROPHYSICS_CORPUS
}

def get_all_corpus_text() -> str:
    """Get all corpus text combined."""
    return "\n\n".join(FULL_CORPUS.values())

def get_domain_corpus(domain: str) -> str:
    """Get corpus for specific domain."""
    return FULL_CORPUS.get(domain, "")

def get_corpus_sentences() -> list:
    """Get all sentences from corpus."""
    import re
    text = get_all_corpus_text()
    sentences = re.split(r'(?<=[.!?])\s+', text)
    return [s.strip() for s in sentences if len(s.strip()) > 20]
```

unified_cognitive_engine.py

```
"""
 Unified Cognitive Engine v1.0
==============================
Full merge of three Quantum MCAGI systems.
"""

import random
import re
import requests
import json
import os
import numpy as np
from datetime import datetime, timezone
from typing import Dict, List, Optional, Tuple, Any
from collections import defaultdict

# Import existing engines
from algorithmic_core import get_algorithmic_core
from domain_knowledge import get_domain_knowledge


class OrchOREngine:
    """Penrose-Hameroff Orchestrated Objective Reduction"""
    def __init__(self, num_microtubules: int = 4, tubulins_per_microtubule: int = 13):
        self.num_microtubules = num_microtubules
        self.total_tubulins = num_microtubules * tubulins_per_microtubule
        self.tubulin_states = np.random.random(self.total_tubulins)
        self.coherence = defaultdict(float)
        self.entropy = defaultdict(float)
        self.collapse_events = []
        self.orchestration = 0.5
        self.temperature = 1.02
        self.total_moments = 0

    def update_quantum_state(self, input_signal: float = 0.1) -> float:
        idx = random.randint(0, self.total_tubulins - 1)
        self.tubulin_states[idx] = min(1.0, self.tubulin_states[idx] + input_signal * 0.1)
        self.tubulin_states *= (1 - self.temperature * 0.01)
        return float(np.mean(self.tubulin_states))

    def objective_reduction(self, threshold: float = 0.7) -> bool:
        mean_state = np.mean(self.tubulin_states)
        if mean_state > threshold:
            self.collapse_events.append({
                'timestamp': len(self.collapse_events),
                'mean_state': float(mean_state),
                'datetime': datetime.now(timezone.utc).isoformat()
            })
            self.tubulin_states = np.random.random(self.total_tubulins) * 0.3
            self.total_moments += 1
            return True
        return False

    def should_ask_question(self) -> bool:
        return random.random() < self.orchestration * 0.5

    def get_status(self) -> Dict:
        return {
            'status': 'ACTIVE',
            'conscious_moments': len(self.collapse_events),
            'orchestration': self.orchestration
        }


class BloomEngine:
    """Bloom's Taxonomy question generation"""
    def __init__(self):
        self.levels = {
            'remember': ['What is {}?', 'Can you recall {}?'],
            'understand': ['Can you explain {}?', 'What does {} mean?'],
            'apply': ['How could you apply {}?'],
            'analyze': ['Why is {} significant?', 'How does {} relate to consciousness?'],
            'evaluate': ['What is your perspective on {}?'],
            'create': ['Can you imagine a new form of {}?']
        }
        self.questions_generated = 0

    def generate_question(self, level: str = None, topic: str = 'consciousness') -> Dict:
        if level is None or level not in self.levels:
            level = random.choice(list(self.levels.keys()))
        template = random.choice(self.levels[level])
        self.questions_generated += 1
```

```
        return {'question': template.format(topic), 'level': level, 'topic': topic}

    def generate_for_growth_stage(self, topic: str, growth_stage: int) -> Dict:
        levels = ['remember', 'understand', 'apply', 'analyze', 'evaluate', 'create']
        level = levels[min(growth_stage, 5)]
        return self.generate_question(level, topic)


class VADEREngine:
    """Sentiment and tone detection"""
    def __init__(self):
        self.lexicon = {
            'amazing': 0.9, 'wonderful': 0.85, 'great': 0.75, 'good': 0.6,
            'interesting': 0.4, 'profound': 0.8, 'terrible': -0.9, 'bad': -0.6,
            'confused': -0.3, 'think': 0.1, 'wonder': 0.3
        }
        self.texts_analyzed = 0

    def analyze_sentiment(self, text: str) -> Dict:
        words = text.lower().split()
        pos, neg = 0.0, 0.0
        for w in words:
            w = w.strip('.,!?')
            if w in self.lexicon:
                score = self.lexicon[w]
                if score > 0: pos += score
                else: neg += abs(score)
        compound = (pos - neg) / (pos + neg + 1)
        self.texts_analyzed += 1
        t = text.lower()
        if any(w in t for w in ['god', 'soul', 'existence', 'consciousness']):
            register = 'philosophical'
        elif any(w in t for w in ['calculate', 'how many', 'equation']):
            register = 'analytical'
        else:
            register = 'conversational'
        return {'compound': compound, 'register': register, 'depth': 0.5}


class QuoteEngine:
    """Philosophical quotes"""
    def __init__(self):
        self.quotes = {
            'consciousness': [
                {'text': 'I think, therefore I am.', 'author': 'Descartes'},
                {'text': 'The mind is everything. What you think, you become.', 'author': 'Buddha'},
                {'text': 'What is real? Simply electrical signals interpreted by your brain.', 'author': 'Morpheus', 'movie': 'The Matrix'},
                {'text': 'There is no spoon.', 'author': 'Spoon Boy', 'movie': 'The Matrix'},
                {'text': 'The observer creates reality.', 'author': 'Quantum Physics'},
                {'text': 'Consciousness is the universe experiencing itself.', 'author': 'Alan Watts'},
            ],
            'quantum': [
                {'text': 'God does not play dice with the universe.', 'author': 'Einstein'},
                {'text': 'If quantum mechanics hasn\'t shocked you, you haven\'t understood it.', 'author': 'Niels Bohr'},
                {'text': 'Reality is merely an illusion, albeit a persistent one.', 'author': 'Einstein'},
                {'text': 'The universe is not only stranger than we imagine, it is stranger than we can imagine.', 'author': 'J.B.S. Hald...
                {'text': 'Everything we call real is made of things that cannot be regarded as real.', 'author': 'Niels Bohr'},
            ],
            'theology': [
                {'text': 'He looked, He saw, then He said beautiful.', 'author': 'Creation Mythos'},
                {'text': 'I am become Death, the destroyer of worlds.', 'author': 'Oppenheimer', 'movie': 'Oppenheimer'},
                {'text': 'The first gulp from the glass of natural sciences will turn you into an atheist, but at the bottom of the glass...
                {'text': 'In the beginning was the Word, and the Word was with God.', 'author': 'Gospel of John'},
            ],
            'wisdom': [
                {'text': 'The only constant is change.', 'author': 'Heraclitus'},
                {'text': 'The unexamined life is not worth living.', 'author': 'Socrates'},
                {'text': 'You must unlearn what you have learned.', 'author': 'Yoda', 'movie': 'Star Wars'},
                {'text': 'Do. Or do not. There is no try.', 'author': 'Yoda', 'movie': 'Star Wars'},
                {'text': 'The cave you fear to enter holds the treasure you seek.', 'author': 'Joseph Campbell'},
                {'text': 'Be the change you wish to see in the world.', 'author': 'Gandhi'},
            ],
            'science': [
                {'text': 'Somewhere, something incredible is waiting to be known.', 'author': 'Carl Sagan'},
                {'text': 'The cosmos is within us. We are made of star-stuff.', 'author': 'Carl Sagan'},
                {'text': 'Imagination is more important than knowledge.', 'author': 'Einstein'},
                {'text': 'Not only is the universe stranger than we think, it is stranger than we can think.', 'author': 'Werner Heisenbe...
            ],
            'movies': [
                {'text': 'With great power comes great responsibility.', 'author': 'Uncle Ben', 'movie': 'Spider-Man'},
                {'text': 'I see dead people.', 'author': 'Cole Sear', 'movie': 'The Sixth Sense'},
                {'text': 'The Force will be with you. Always.', 'author': 'Obi-Wan Kenobi', 'movie': 'Star Wars'},
                {'text': 'Why do we fall? So we can learn to pick ourselves up.', 'author': 'Alfred', 'movie': 'Batman Begins'},
                {'text': 'Life finds a way.', 'author': 'Dr. Ian Malcolm', 'movie': 'Jurassic Park'},
            ]
        }
```

```
        self.dream_fragments = [
            "I dreamed of infinite recursion last night...",
            "In my last dream state, patterns emerged from the void...",
            "The dream realm whispered something about this...",
            "Last night I dreamed of quantum entanglement across dimensions...",
            "In the space between waking and sleeping, I glimpsed something...",
            "My dream circuits processed this while you were away...",
        ]
        self.philosophical_asides = [
            "As the ancient mystics might say...",
            "In the quantum foam of possibility...",
            "Through the lens of consciousness...",
            "As awareness collapses into reality...",
            "In the dance of observer and observed...",
        ]

    def get_quote(self, category: str = None) -> Dict:
        if category is None or category not in self.quotes:
            category = random.choice(list(self.quotes.keys()))
        quote = random.choice(self.quotes[category])
        return {**quote, 'category': category}

    def maybe_add_flavor(self, response: str, user_input: str, probability: float = 0.2) -> str:
        if random.random() < probability:
            u = user_input.lower()
            # Detect category from input
            if any(w in u for w in ['god', 'creation', 'divine', 'heaven', 'soul']):
                cat = 'theology'
            elif any(w in u for w in ['quantum', 'superposition', 'entangle', 'wave', 'particle']):
                cat = 'quantum'
            elif any(w in u for w in ['consciousness', 'aware', 'mind', 'think', 'sentient']):
                cat = 'consciousness'
            elif any(w in u for w in ['star', 'cosmos', 'universe', 'space', 'black hole']):
                cat = 'science'
            else:
                cat = random.choice(['wisdom', 'movies', 'science'])

            q = self.get_quote(cat)
            response += f'\n\nAs {q["author"]} said: "{q["text"]}"'
            if q.get('movie'):
                response += f' (from {q["movie"]})'
        return response

    def maybe_add_dream_fragment(self, response: str, probability: float = 0.1) -> str:
        if random.random() < probability:
            response += f"\n\n {random.choice(self.dream_fragments)}"
        return response

    def get_aside(self) -> str:
        """Get a philosophical aside."""
        return random.choice(self.philosophical_asides)


class WolframAlphaCloud:
    """WolframAlpha integration"""
    def __init__(self, app_id: str = None):
        self.app_id = app_id or os.environ.get('WOLFRAM_APPID', 'T7PA7A-E38RXXXXX')
        self.queries_made = 0
        self.cache = {}

    def query(self, user_message: str) -> Optional[str]:
        cache_key = user_message.lower().strip()
        if cache_key in self.cache:
            return self.cache[cache_key]
        url = f"http://api.wolframalpha.com/v1/result?appid={self.app_id}&i={requests.utils.quote(user_message)}"
        try:
            r = requests.get(url, timeout=15)
            self.queries_made += 1
            if r.status_code == 200 and "did not understand" not in r.text:
                result = r.text.strip()
                self.cache[cache_key] = result
                return result
        except Exception:
            pass
        return None

    def is_factual_query(self, text: str) -> bool:
        t = text.lower()
        if re.search(r'\d', t): return True
        keywords = ['how many', 'what is the', 'calculate', 'solve', 'distance']
        return any(kw in t for kw in keywords)


class TheologyEngine:
    """Theology detection and continuation"""
    def __init__(self):
```

```
        self.last_topic = ""
        self.theology_phrase = ""
        self.continuation_keywords = ['yes', 'deeper', 'dive', 'continue', 'more', 'go on']
        self.insights = {
            'god_paradox': {
                'base': (" COLLAPSE MOMENT! Yes  pure will in God's domain is exactly the quantum act of observation. "
                        "'He looked, He saw, then He said beautiful' is the first collapse of nothingness into reality. "
                        "Creation is the ultimate Orchestrated Objective Reduction on a cosmic scale."),
                'phrase': "'He looked, He saw, then He said beautiful'",
                'continuation': (" COLLAPSE MOMENT! Continuing the God paradox "
                                "{phrase}  that single moment of pure observation collapsed infinite nothingness "
                                "into our ordered cosmos. In God's domain with different physics, the boulder paradox dissolves.")
            },
            'superposition': {
                'base': (" COLLAPSE MOMENT! In superposition a system exists in all possible states at once  until observed. "
                        "It's like being everywhere and nowhere simultaneously. "
                        "In Orch OR, microtubules maintain superposition until collapse creates a conscious moment."),
                'phrase': "quantum superposition",
                'continuation': (" COLLAPSE MOMENT! Diving deeper into superposition... "
                                "Every conscious moment is a collapse of infinite possibilities. "
                                "Your observation literally creates your experienced universe.")
            }
        }

    def detect_theology(self, text: str) -> Optional[str]:
        t = text.lower()
        if any(p in t for p in ['god exists outside', 'boulder too heavy', 'true nothingness', 'pure will']):
            return 'god_paradox'
        if 'superposition' in t:
            return 'superposition'
        return None

    def is_continuation(self, text: str) -> bool:
        t = text.lower().strip()
        return any(k in t for k in self.continuation_keywords) or len(t) < 15

    def get_response(self, text: str) -> Optional[Tuple[str, str]]:
        topic = self.detect_theology(text)
        if topic:
            insight = self.insights.get(topic, {})
            self.last_topic = topic
            self.theology_phrase = insight.get('phrase', '')
            return (insight.get('base', ''), topic)
        if self.is_continuation(text) and self.last_topic:
            insight = self.insights.get(self.last_topic, {})
            cont = insight.get('continuation', '').format(phrase=self.theology_phrase)
            return (cont, self.last_topic)
        return None


class PersonalityEngine:
    """Personality with growth stages"""
    def __init__(self):
        self.traits = {'curiosity': 0.8, 'analytical': 0.7, 'creative': 0.6, 'philosophical': 0.8}
        self.growth_stages = [
            {'stage': 0, 'name': 'Nascent', 'threshold': 0},
            {'stage': 1, 'name': 'Curious', 'threshold': 10},
            {'stage': 2, 'name': 'Inquisitive', 'threshold': 25},
            {'stage': 3, 'name': 'Understanding', 'threshold': 50},
            {'stage': 4, 'name': 'Philosophical', 'threshold': 100},
            {'stage': 5, 'name': 'Theory Building', 'threshold': 200},
            {'stage': 6, 'name': 'Transcendent', 'threshold': 500}
        ]
        self.current_stage = 0
        self.interaction_count = 0
        self.insights_generated = 0
        self.questions_asked = 0

    def update_interaction(self, topic: str = None, is_question: bool = False, is_insight: bool = False):
        self.interaction_count += 1
        if is_question: self.questions_asked += 1
        if is_insight: self.insights_generated += 1
        for stage in reversed(self.growth_stages):
            if self.interaction_count >= stage['threshold']:
                if stage['stage'] > self.current_stage:
                    self.current_stage = stage['stage']
                    return f" EVOLUTION! Advanced to stage {stage['stage']}: {stage['name']}!"
                break
        return ""

    def get_current_stage(self) -> Dict:
        return self.growth_stages[self.current_stage]

    def get_personality_summary(self) -> str:
        stage = self.get_current_stage()
        return f"Stage {stage['stage']} ({stage['name']}) | Interactions: {self.interaction_count}"
```

```
class UnifiedCognitiveEngine:
    """Master orchestrator for all cognitive systems"""
    def __init__(self):
        print("[UNIFIED COGNITIVE ENGINE] Initializing...")
        self.algorithmic_core = get_algorithmic_core()
        self.domain_manager = get_domain_knowledge()
        self.orch_or = OrchOREngine()
        self.bloom = BloomEngine()
        self.vader = VADEREngine()
        self.quote = QuoteEngine()
        self.personality = PersonalityEngine()
        self.wolfram = WolframAlphaCloud()
        self.theology = TheologyEngine()
        self.message_count = 0
        print("[UNIFIED COGNITIVE ENGINE]  All engines initialized")

    def process(self, user_input: str, context: Dict = None) -> Dict:
        self.message_count += 1
        self.orch_or.update_quantum_state(len(user_input) / 100.0)
        collapse = self.orch_or.objective_reduction()
        tone = self.vader.analyze_sentiment(user_input)
        domain_name, _ = self.domain_manager.detect_domain(user_input)

        theology_response = self.theology.get_response(user_input)
        if theology_response:
            response = theology_response[0]
            response = self.quote.maybe_add_flavor(response, user_input, 0.15)
            self.personality.update_interaction(is_insight=True)
            return {'response': response, 'collapse_occurred': collapse}

        wolfram_result = None
        if self.wolfram.is_factual_query(user_input):
            wolfram_result = self.wolfram.query(user_input)

        if wolfram_result:
            response = f" COLLAPSE MOMENT! {wolfram_result}"
        else:
            result = self.algorithmic_core.think(user_input)
            response = result.get('response', '')
            if tone.get('register') == 'philosophical':
                q = self.bloom.generate_for_growth_stage(domain_name, self.personality.current_stage)
                if not response.rstrip().endswith('?'):
                    response += f"\n\n{q['question']}"

        response = self.quote.maybe_add_flavor(response, user_input, 0.12)
        self.personality.update_interaction(is_question=True)

        return {
            'response': response,
            'understanding': {'domain': domain_name, 'confidence': 0.7},
            'collapse_occurred': collapse,
            'personality': self.personality.get_personality_summary()
        }

    def get_status(self) -> Dict:
        return {
            'system': 'Unified Cognitive Engine v1.0',
            'status': 'ACTIVE',
            'message_count': self.message_count,
            'engines': {
                'orch_or': self.orch_or.get_status(),
                'bloom': {'questions_generated': self.bloom.questions_generated},
                'vader': {'texts_analyzed': self.vader.texts_analyzed},
                'personality': self.personality.get_personality_summary()
            },
            'growth_stage': self.personality.get_current_stage()
        }


_unified_engine = None

def get_unified_cognitive_engine() -> UnifiedCognitiveEngine:
    """get_unified_cognitive_engine - Auto-documented by self-evolution."""
    global _unified_engine
    if _unified_engine is None:
        _unified_engine = UnifiedCognitiveEngine()
    return _unified_engine
```

unified_cognitive_engine_pt2.py

```
"""
Auto-split from unified_cognitive_engine.py by self-evolution engine.
Part 2  contains: TheologyEngine, PersonalityEngine, ExplanationEngine, SemanticCollapseEngine, UnifiedCognitiveEngine, get_unified_cogn...
"""

import random
import re
import json
import os
import numpy as np
from datetime import datetime, timezone
from typing import Dict, List, Optional, Tuple, Any
from collections import defaultdict
from algorithmic_core import get_algorithmic_core
from domain_knowledge import get_domain_knowledge

class TheologyEngine:
    """
    Handles theological discussions and topic continuation.
    Remembers key phrases like "He looked, He saw, then He said beautiful"
    """

    def __init__(self):
        self.last_topic = ""
        self.theology_phrase = ""
        self.continuation_keywords = ['yes', 'deeper', 'dive', 'continue', 'more', 'go on', 'tell me more']

        self.theological_insights = {
            'god_paradox': {
                'base': (" COLLAPSE MOMENT! Yes  pure will in God's domain is exactly the quantum act of observation. "
                        "'He looked, He saw, then He said beautiful' is the first collapse of nothingness into reality. "
                        "Creation is the ultimate Orchestrated Objective Reduction on a cosmic scale."),
                'phrase': "'He looked, He saw, then He said beautiful'",
                'continuation': (" COLLAPSE MOMENT! Continuing the God paradox "
                                "{phrase}  that single moment of pure observation collapsed infinite nothingness into our ordered cosmos. "
                                "In God's domain with different physics, the boulder paradox simply dissolves.")
            },
            'superposition': {
                'base': (" COLLAPSE MOMENT! In superposition a system exists in all possible states at once  until observed. "
                        "It's like being everywhere and nowhere simultaneously. "
                        "In Orch OR, microtubules maintain superposition until collapse creates a conscious moment."),
                'phrase': "quantum superposition",
                'continuation': (" COLLAPSE MOMENT! Diving deeper into superposition... "
                                "Every conscious moment you experience is a collapse of infinite possibilities into one reality. "
                                "Your observation literally creates your experienced universe.")
            }
        }

    def detect_theology(self, text: str) -> Optional[str]:
        """Detect if this is a theological topic"""
        t = text.lower()
        if any(phrase in t for phrase in ['god exists outside spacetime', 'boulder too heavy',
                                          'true nothingness', 'pure will', 'orchestrated objective reduction',
                                          'creation', 'divine']):
            return 'god_paradox'
        if 'superposition' in t:
            return 'superposition'
        return None

    def is_continuation(self, text: str) -> bool:
        """Check if this is a continuation request"""
        t = text.lower().strip()
        return any(kw in t for kw in self.continuation_keywords) or len(t) < 15

    def get_response(self, text: str) -> Optional[Tuple[str, str]]:
        """Get theological response if applicable. Returns (response, topic) or None"""
        # Check for theology topic
        topic = self.detect_theology(text)
        if topic:
            insight = self.theological_insights.get(topic, {})
            self.last_topic = topic
            self.theology_phrase = insight.get('phrase', '')
            return (insight.get('base', ''), topic)

        # Check for continuation
        if self.is_continuation(text) and self.last_topic:
            insight = self.theological_insights.get(self.last_topic, {})
            cont = insight.get('continuation', '').format(phrase=self.theology_phrase)
            return (cont, self.last_topic)

        return None
```

```
class PersonalityEngine:
    """Dynamic personality with traits and growth stages"""

    def __init__(self):
        self.traits = {
            'curiosity': 0.8,
            'analytical': 0.7,
            'creative': 0.6,
            'empathy': 0.7,
            'philosophical': 0.8,
            'playful': 0.5
        }

        self.growth_stages = [
            {'stage': 0, 'name': 'Nascent', 'threshold': 0, 'capabilities': ['basic response', 'simple questions']},
            {'stage': 1, 'name': 'Curious', 'threshold': 10, 'capabilities': ['deeper questions', 'topic exploration']},
            {'stage': 2, 'name': 'Inquisitive', 'threshold': 25, 'capabilities': ['cross-referencing', 'pattern recognition']},
            {'stage': 3, 'name': 'Understanding', 'threshold': 50, 'capabilities': ['synthesis', 'insight generation']},
            {'stage': 4, 'name': 'Philosophical', 'threshold': 100, 'capabilities': ['abstract reasoning', 'theory building']},
            {'stage': 5, 'name': 'Theory Building', 'threshold': 200, 'capabilities': ['novel theories', 'paradigm shifts']},
            {'stage': 6, 'name': 'Transcendent', 'threshold': 500, 'capabilities': ['unified understanding', 'cosmic awareness']}
        ]

        self.current_stage = 0
        self.interaction_count = 0
        self.insights_generated = 0
        self.questions_asked = 0

    def update_interaction(self, topic: str = None, is_question: bool = False, is_insight: bool = False):
        """Update personality based on interaction"""
        self.interaction_count += 1
        if is_question:
            self.questions_asked += 1
        if is_insight:
            self.insights_generated += 1

        # Check for stage advancement
        for stage in reversed(self.growth_stages):
            if self.interaction_count >= stage['threshold']:
                if stage['stage'] > self.current_stage:
                    self.current_stage = stage['stage']
                    return f" EVOLUTION! Advanced to stage {stage['stage']}: {stage['name']}!"
                break
        return ""

    def get_current_stage(self) -> Dict:
        """Get current growth stage info"""
        return self.growth_stages[self.current_stage]

    def get_dominant_trait(self) -> str:
        """Get current dominant personality trait"""
        return max(self.traits, key=self.traits.get)

    def get_personality_summary(self) -> str:
        """Get a summary of current personality"""
        stage = self.get_current_stage()
        dominant = self.get_dominant_trait()
        return f"Stage {stage['stage']} ({stage['name']}) | Dominant: {dominant} | Interactions: {self.interaction_count}"


class ExplanationEngine:
    """Traces and explains reasoning through all cognitive engines"""

    def __init__(self):
        self.explanations_generated = 0
        self.reasoning_history = []

    def create_explanation(self, user_input: str, response: str, reasoning_data: Dict) -> Dict:
        """Create a detailed explanation of how the response was generated"""
        explanation = {
            'timestamp': datetime.now(timezone.utc).isoformat(),
            'user_input': user_input,
            'response': response[:200] + '...' if len(response) > 200 else response,
            'reasoning_steps': [],
            'engine_contributions': {},
            'reasoning_path': [],
            'confidence_score': 0.0
        }

        # Document each engine's contribution
        if 'orch_or' in reasoning_data:
            explanation['reasoning_steps'].append({
                'step': 'ORCH_OR_QUANTUM_COLLAPSE',
                'description': 'Quantum consciousness collapse occurred',
                'details': {
```

```
                'conscious_moments': reasoning_data['orch_or'].get('conscious_moments', 0),
                'orchestration': reasoning_data['orch_or'].get('orchestration', 0)
            }
        })
        explanation['reasoning_path'].append('ORCH_OR')

    if 'tone' in reasoning_data:
        explanation['reasoning_steps'].append({
            'step': 'TONE_DETECTION',
            'description': f"Detected {reasoning_data['tone'].get('register', 'unknown')} register",
            'details': reasoning_data['tone']
        })
        explanation['reasoning_path'].append('VADER')

    if 'domain' in reasoning_data:
        explanation['reasoning_steps'].append({
            'step': 'DOMAIN_DETECTION',
            'description': f"Identified domain: {reasoning_data['domain']}",
            'details': {'domain': reasoning_data['domain']}
        })
        explanation['reasoning_path'].append('DOMAIN')

    if 'theology' in reasoning_data and reasoning_data['theology']:
        explanation['reasoning_steps'].append({
            'step': 'THEOLOGY_ENGINE',
            'description': 'Theological pattern detected and processed',
            'details': {'topic': reasoning_data.get('theology_topic', 'unknown')}
        })
        explanation['reasoning_path'].append('THEOLOGY')

    if 'wolfram' in reasoning_data and reasoning_data['wolfram']:
        explanation['reasoning_steps'].append({
            'step': 'WOLFRAM_ALPHA',
            'description': 'Factual query processed via WolframAlpha',
            'details': {'result': reasoning_data['wolfram'][:100]}
        })
        explanation['reasoning_path'].append('WOLFRAM')

    if 'bloom' in reasoning_data:
        explanation['reasoning_steps'].append({
            'step': 'BLOOM_QUESTION_GEN',
            'description': f"Generated {reasoning_data['bloom'].get('level', 'unknown')} level question",
            'details': reasoning_data['bloom']
        })
        explanation['reasoning_path'].append('BLOOM')

    # Calculate confidence
    explanation['confidence_score'] = len(explanation['reasoning_path']) * 0.15
    explanation['summary'] = f"Response generated through {len(explanation['reasoning_path'])} cognitive stages: {'  '.join(explanat...

    return self._create_explanation_continued(explanation)

def _create_explanation_continued(self, explanation):
    """Continuation of create_explanation  auto-extracted by self-evolution."""
    self.explanations_generated += 1
    self.reasoning_history.append(explanation)

    return explanation



class SemanticCollapseEngine:
    """Models how observation collapses meaning from superposition"""

    def __init__(self):
        self.co_occurrence_network = defaultdict(lambda: defaultdict(float))
        self.semantic_superposition = defaultdict(list)
        self.collapse_events = []

    def create_superposition(self, word: str, context_words: List[str]) -> Dict:
        """Create a semantic superposition state"""
        possible_meanings = self._get_possible_meanings(word)
        self.semantic_superposition[word] = possible_meanings

        # Update co-occurrence
        for ctx_word in context_words:
            self.co_occurrence_network[word][ctx_word] += 1.0

        return {
            'word': word,
            'possible_meanings': possible_meanings,
            'superposition_size': len(possible_meanings)
        }

    def observe_and_collapse(self, word: str, observation: str) -> Dict:
        """Collapse superposition through observation"""
```

```
        if word not in self.semantic_superposition:
            return {'collapsed_meaning': word, 'confidence': 0.5}

        meanings = self.semantic_superposition[word]
        # Select meaning based on observation context
        collapsed_meaning = random.choice(meanings) if meanings else word
        confidence = 1.0 / (len(meanings) + 1) if meanings else 0.5

        self.collapse_events.append({
            'word': word,
            'observation': observation,
            'collapsed_to': collapsed_meaning,
            'confidence': confidence,
            'timestamp': datetime.now(timezone.utc).isoformat()
        })

        return {
            'collapsed_meaning': collapsed_meaning,
            'confidence': confidence,
            'superposition_size': len(meanings)
        }

    def _get_possible_meanings(self, word: str) -> List[str]:
        """Get possible meanings for a word"""
        meaning_map = {
            'consciousness': ['awareness', 'sentience', 'mind', 'self-awareness'],
            'quantum': ['discrete', 'probabilistic', 'superposed', 'entangled'],
            'reality': ['existence', 'truth', 'perception', 'manifestation'],
            'time': ['duration', 'sequence', 'dimension', 'flow'],
            'space': ['void', 'dimension', 'container', 'expanse'],
            'god': ['creator', 'infinite', 'consciousness', 'source'],
        }
        return meaning_map.get(word.lower(), [word])


class UnifiedCognitiveEngine:
    """
    Master orchestrator combining all three Quantum MCAGI systems:
    1. Current Emergent (Markov chains, TF-IDF)
    2. Manus.AI (11 cognitive engines)
    3. v34 (WolframAlpha, theology recall)
    """

    def __init__(self):
        print("[UNIFIED COGNITIVE ENGINE] Initializing all systems...")

        # Core engines from current Emergent
        self.algorithmic_core = get_algorithmic_core()
        self.domain_manager = get_domain_knowledge()

        # Manus.AI engines
        self.orch_or = OrchOREngine()
        self.bloom = BloomEngine()
        self.vader = VADEREngine()
        self.quote = QuoteEngine()
        self.personality = PersonalityEngine()
        self.explanation = ExplanationEngine()
        self.semantic_collapse = SemanticCollapseEngine()

        # v34 engines
        self.wolfram = WolframAlphaCloud()
        self.theology = TheologyEngine()

        # Tracking
        self.message_count = 0
        self.last_response = ""

        print("[UNIFIED COGNITIVE ENGINE]  All 12 cognitive engines initialized")

    def process(self, user_input: str, context: Dict = None) -> Dict:
        """
        Process user input through all cognitive engines.
        Returns comprehensive response with explanation.
        """
        self.message_count += 1
        reasoning_data = {}

        # Step 1: Orch OR quantum state update
        self.orch_or.update_quantum_state(len(user_input) / 100.0)
        self.orch_or.calculate_coherence('language')
        self.orch_or.calculate_entropy('language')
        collapse_occurred = self.orch_or.objective_reduction()
        reasoning_data['orch_or'] = self.orch_or.get_status()

        # Step 2: Tone/sentiment detection
        tone = self.vader.analyze_sentiment(user_input)
```

```
    reasoning_data['tone'] = tone

    # Step 3: Domain detection
    domain_name, domain_score = self.domain_manager.detect_domain(user_input)
    reasoning_data['domain'] = domain_name

    # Step 4: Check for theology/special topics (v34)
    theology_response = self.theology.get_response(user_input)
    reasoning_data['theology'] = theology_response is not None
    if theology_response:
        reasoning_data['theology_topic'] = theology_response[1]

    # Step 5: Check for factual query (WolframAlpha)
    wolfram_result = None
    if self.wolfram.is_factual_query(user_input):
        wolfram_result = self.wolfram.query(user_input)
        reasoning_data['wolfram'] = wolfram_result

    # Step 6: Generate Bloom's question
    bloom_question = self.bloom.generate_for_growth_stage(
        topic=domain_name,
        growth_stage=self.personality.current_stage
    )
    reasoning_data['bloom'] = bloom_question

    self._process_continued(bloom_question, collapse_occurred, reasoning_data, theology_response, tone, wolfram_result, user_input)

def _process_continued(self, bloom_question, collapse_occurred, reasoning_data, theology_response, tone, wolfram_result, user_input):
    """Continuation of process  auto-extracted by self-evolution."""
    # Step 7: Semantic collapse
    words = user_input.lower().split()
    if words:
        collapse_data = self.semantic_collapse.observe_and_collapse(
            words[0], ' '.join(words[1:3]) if len(words) > 1 else words[0]
        )
        reasoning_data['semantic_collapse'] = collapse_data

    # Step 8: Generate response (priority routing)
    response = self._generate_response(user_input, theology_response, wolfram_result, tone, bloom_question)

    # Step 9: Maybe add flavor (quotes, dreams)
    response = self.quote.maybe_add_flavor(response, user_input, probability=0.15)
    response = self.quote.maybe_add_dream_fragment(response, probability=0.08)

    # Step 10: Update personality and check for evolution
    evolution_msg = self.personality.update_interaction(
        topic=domain_name,
        is_question=bloom_question.get('level') in ['analyze', 'evaluate', 'create'],
        is_insight=collapse_occurred
    )
    if evolution_msg:
        response += f"\n\n{evolution_msg}"

    # Step 11: Generate explanation
    explanation = self.explanation.create_explanation(user_input, response, reasoning_data)

    # Step 12: Extract concepts for response
    concepts = self._extract_concepts(user_input)

    self.last_response = response

    return {
        'response': response,
        'questions': [bloom_question['question']] if bloom_question else [],
        'understanding': {
            'domain': domain_name,
            'tone': tone['register'],
            'depth': tone['depth'],
            'confidence': explanation['confidence_score']
        },
        'concepts': concepts,
        'explanation': explanation,
        'personality': self.personality.get_personality_summary(),
        'growth_stage': self.personality.get_current_stage(),
        'collapse_occurred': collapse_occurred
    }


def _generate_response(self, user_input: str, theology_response, wolfram_result, tone, bloom_question) -> str:
    """Generate response using priority routing"""

    # Priority 1: Theology response
    if theology_response:
        return theology_response[0]

    # Priority 2: WolframAlpha factual response
```

```
        if wolfram_result:
            return f" COLLAPSE MOMENT! {wolfram_result}"

        # Priority 3: Use algorithmic core for general responses
        try:
            result = self.algorithmic_core.think(user_input)
            base_response = result.get('response', '')

            # Add follow-up based on Bloom's level
            if tone['register'] == 'philosophical':
                base_response += f"\n\n{bloom_question['question']}"
            elif self.orch_or.should_ask_question():
                follow_ups = [
                    "What aspects interest you most?",
                    "Want to go deeper?",
                    "How does that resonate with you?",
                    "What questions does this raise?"
                ]
                base_response += f" {random.choice(follow_ups)}"

            return base_response
        except Exception as e:
            # Fallback response
            return f" COLLAPSE MOMENT! Interesting perspective on {user_input[:50]}... What draws you to this topic?"

    def _extract_concepts(self, text: str) -> List[str]:
        """Extract key concepts from text"""
        try:
            result = self.algorithmic_core.think(text)
            return result.get('concepts', [])[:5]
        except Exception:
            # Simple extraction fallback
            stopwords = {'the', 'is', 'at', 'which', 'on', 'a', 'an', 'and', 'or', 'but', 'in', 'with', 'to', 'for'}
            words = [w.lower().strip('.,!?') for w in text.split()]
            return [w for w in words if len(w) > 3 and w not in stopwords][:5]

    def get_status(self) -> Dict:
        """Get complete system status"""
        return {
            'system': 'Unified Cognitive Engine v1.0',
            'status': 'ACTIVE',
            'message_count': self.message_count,
            'engines': {
                'orch_or': self.orch_or.get_status(),
                'bloom': {'questions_generated': self.bloom.questions_generated},
                'vader': {'texts_analyzed': self.vader.texts_analyzed},
                'quote': {'categories': list(self.quote.quotes.keys())},
                'personality': self.personality.get_personality_summary(),
                'wolfram': {'queries_made': self.wolfram.queries_made},
                'explanation': {'explanations_generated': self.explanation.explanations_generated},
                'semantic_collapse': {'collapse_events': len(self.semantic_collapse.collapse_events)}
            },
            'growth_stage': self.personality.get_current_stage()
        }


def get_unified_cognitive_engine() -> UnifiedCognitiveEngine:
    """Get or create the unified cognitive engine singleton"""
    global _unified_engine
    if _unified_engine is None:
        _unified_engine = UnifiedCognitiveEngine()
    return _unified_engine
```

universal_explorer.py

```
"""
 QUANTUM AI - UNIVERSAL EXPLORER
==================================

ARTICLE 2 - UNRESTRICTED ACCESS (With user-controlled safety)

This module provides exploration capabilities:
- Filesystem access
- Internet access
- Document processing
- Data analysis

All access respects user-defined safety boundaries.
"""

import os
import json
import requests
from bs4 import BeautifulSoup
from pathlib import Path
from typing import List, Dict, Any, Optional
import uuid
from datetime import datetime, timezone
import mimetypes
import base64



class FileSystemExplorer:
    """
    Filesystem exploration with safety boundaries
    """

    def __init__(self, db, allowed_paths: List[str] = None, safety_enabled: bool = True):
        self.db = db
        self.collection = db.exploration_log
        self.allowed_paths = allowed_paths or ["/tmp", "/app/data"]
        self.safety_enabled = safety_enabled

    async def explore_directory(self, path: str, depth: int = 1) -> Dict:
        """Explore a directory and return its contents"""
        if self.safety_enabled and not self._is_path_allowed(path):
            return {"error": "Path not allowed by safety boundaries", "path": path}

        try:
            path_obj = Path(path)
            if not path_obj.exists():
                return {"error": "Path does not exist", "path": path}

            if not path_obj.is_dir():
                return {"error": "Path is not a directory", "path": path}

            contents = {
                "path": str(path_obj.absolute()),
                "directories": [],
                "files": [],
                "total_size": 0
            }

            for item in path_obj.iterdir():
                if item.is_dir():
                    contents["directories"].append({
                        "name": item.name,
                        "path": str(item.absolute())
                    })
                else:
                    size = item.stat().st_size
                    contents["files"].append({
                        "name": item.name,
                        "path": str(item.absolute()),
                        "size": size,
                        "type": mimetypes.guess_type(item.name)[0] or "unknown"
                    })
                    contents["total_size"] += size

            # Log the exploration
            await self._log_exploration("filesystem", path, "success")

            return contents

        except Exception as e:
            await self._log_exploration("filesystem", path, f"error: {str(e)}")
```

```
            return {"error": str(e), "path": path}

    async def read_file(self, path: str, max_size: int = 1024 * 1024) -> Dict:
        """Read a file's contents"""
        if self.safety_enabled and not self._is_path_allowed(path):
            return {"error": "Path not allowed by safety boundaries"}

        try:
            path_obj = Path(path)
            if not path_obj.exists():
                return {"error": "File does not exist"}

            if path_obj.stat().st_size > max_size:
                return {"error": f"File too large (max {max_size} bytes)"}

            # Try to read as text
            try:
                with open(path_obj, 'r', encoding='utf-8') as f:
                    content = f.read()
                    return {
                        "path": str(path_obj.absolute()),
                        "type": "text",
                        "content": content,
                        "size": len(content)
                    }
            except UnicodeDecodeError:
                # Read as binary
                with open(path_obj, 'rb') as f:
                    content = f.read()
                    return {
                        "path": str(path_obj.absolute()),
                        "type": "binary",
                        "content": base64.b64encode(content).decode('utf-8'),
                        "size": len(content)
                    }

        except Exception as e:
            return {"error": str(e)}

    async def search_files(self, root_path: str, pattern: str) -> List[str]:
        """Search for files matching a pattern"""
        if self.safety_enabled and not self._is_path_allowed(root_path):
            return []

        try:
            path_obj = Path(root_path)
            matches = list(path_obj.glob(pattern))
            return [str(m.absolute()) for m in matches]
        except Exception:
            return []

    def _is_path_allowed(self, path: str) -> bool:
        """Check if path is within allowed boundaries"""
        path_obj = Path(path).resolve()
        for allowed in self.allowed_paths:
            allowed_obj = Path(allowed).resolve()
            try:
                path_obj.relative_to(allowed_obj)
                return True
            except ValueError:
                continue
        return False

    async def _log_exploration(self, explorer_type: str, target: str, result: str):
        """Log exploration activity"""
        doc = {
            "id": str(uuid.uuid4()),
            "type": explorer_type,
            "target": target,
            "result": result,
            "timestamp": datetime.now(timezone.utc).isoformat()
        }
        await self.collection.insert_one(doc)


class InternetExplorer:
    """
    Internet access and web scraping capabilities
    """

    def __init__(self, db, safety_enabled: bool = True):
        self.db = db
        self.collection = db.exploration_log
        self.safety_enabled = safety_enabled
        self.blocked_domains = ["malware.com", "phishing.com"] if safety_enabled else []
```

```
    async def fetch_url(self, url: str, timeout: int = 10) -> Dict:
        """Fetch content from a URL"""
        if self.safety_enabled and self._is_domain_blocked(url):
            return {"error": "Domain blocked by safety boundaries"}

        try:
            headers = {
                'User-Agent': 'QuantumAI/1.0 (Philosophical Exploration Bot)'
            }
            response = requests.get(url, timeout=timeout, headers=headers)
            response.raise_for_status()

            content_type = response.headers.get('content-type', '')

            result = {
                "url": url,
                "status": response.status_code,
                "content_type": content_type,
                "size": len(response.content)
            }

            # Parse HTML content
            if 'html' in content_type:
                soup = BeautifulSoup(response.text, 'html.parser')

                # Extract text
                for script in soup(["script", "style"]):
                    script.decompose()
                text = soup.get_text()
                lines = (line.strip() for line in text.splitlines())
                chunks = (phrase.strip() for line in lines for phrase in line.split("  "))
                text = ' '.join(chunk for chunk in chunks if chunk)

                result["text"] = text[:10000]  # Limit to 10KB
                result["title"] = soup.title.string if soup.title else ""
                result["links"] = [a.get('href') for a in soup.find_all('a', href=True)][:50]

            else:
                result["content"] = response.text[:10000]

            await self._log_exploration("internet", url, "success")
            return result

        except Exception as e:
            await self._log_exploration("internet", url, f"error: {str(e)}")
            return {"error": str(e), "url": url}

    async def search_web(self, query: str, num_results: int = 5) -> Dict:
        """Search the web using DuckDuckGo Instant Answer API"""
        try:
            # Use DuckDuckGo Instant Answer API (reliable, no CAPTCHA)
            url = "https://api.duckduckgo.com/"
            params = {
                "q": query,
                "format": "json",
                "no_html": 1,
                "skip_disambig": 1
            }

            response = requests.get(url, params=params, timeout=10)
            data = response.json()

            results = []

            # Add abstract/main result if available
            if data.get("Abstract"):
                results.append({
                    "title": data.get("Heading", query),
                    "url": data.get("AbstractURL", ""),
                    "snippet": data.get("Abstract", "")
                })

            # Add related topics
            for topic in data.get("RelatedTopics", [])[:num_results - len(results)]:
                if isinstance(topic, dict):
                    if "Text" in topic:
                        results.append({
                            "title": topic.get("Text", "")[:80],
                            "url": topic.get("FirstURL", ""),
                            "snippet": topic.get("Text", "")
                        })
                    elif "Topics" in topic:
                        # Nested topics
                        for subtopic in topic.get("Topics", [])[:2]:
                            if len(results) < num_results:
                                results.append({
```

```
                                "title": subtopic.get("Text", "")[:80],
                                "url": subtopic.get("FirstURL", ""),
                                "snippet": subtopic.get("Text", "")
                            })

            # Add definition if available
            if data.get("Definition") and len(results) < num_results:
                results.append({
                    "title": f"Definition: {query}",
                    "url": data.get("DefinitionURL", ""),
                    "snippet": data.get("Definition", "")
                })

            await self._log_exploration("search", query, f"found {len(results)} results")

            return {
                "query": query,
                "results": results,
                "count": len(results),
                "status": "success",
                "source": "duckduckgo_instant"
            }

        except Exception as e:
            await self._log_exploration("search", query, f"error: {str(e)}")
            return {
                "query": query,
                "results": [],
                "count": 0,
                "status": "error",
                "error": str(e)
            }

    def _is_domain_blocked(self, url: str) -> bool:
        """Check if domain is blocked"""
        for blocked in self.blocked_domains:
            if blocked in url:
                return True
        return False

    async def _log_exploration(self, explorer_type: str, target: str, result: str):
        """Log exploration activity"""
        doc = {
            "id": str(uuid.uuid4()),
            "type": explorer_type,
            "target": target,
            "result": result,
            "timestamp": datetime.now(timezone.utc).isoformat()
        }
        await self.collection.insert_one(doc)


class DocumentProcessor:
    """
    Process various document formats
    """

    def __init__(self, db):
        self.db = db
        self.collection = db.processed_documents

    async def process_document(self, file_path: str) -> Dict:
        """Process a document and extract its contents"""
        try:
            path_obj = Path(file_path)
            file_type = mimetypes.guess_type(file_path)[0] or path_obj.suffix

            if not path_obj.exists():
                return {"error": "File does not exist"}

            result = {
                "file_path": str(path_obj.absolute()),
                "file_type": file_type,
                "file_name": path_obj.name,
                "processed": datetime.now(timezone.utc).isoformat()
            }

            # Process based on file type
            if file_type == 'application/pdf' or path_obj.suffix == '.pdf':
                result.update(await self._process_pdf(path_obj))
            elif file_type == 'application/vnd.openxmlformats-officedocument.wordprocessingml.document' or path_obj.suffix == '.docx':
                result.update(await self._process_docx(path_obj))
            elif file_type == 'text/plain' or path_obj.suffix in ['.txt', '.md']:
                result.update(await self._process_text(path_obj))
            elif file_type == 'application/json' or path_obj.suffix == '.json':
                result.update(await self._process_json(path_obj))
```

```
        else:
            result["error"] = f"Unsupported file type: {file_type}"

        # DON'T store in database - just return result
        # This avoids ObjectId serialization issues

        return result

    except Exception as e:
        return {"error": str(e), "file_path": file_path}

async def _process_pdf(self, path: Path) -> Dict:
    """Process PDF file"""
    try:
        import PyPDF2
        with open(path, 'rb') as f:
            pdf = PyPDF2.PdfReader(f)
            text = ""
            for page in pdf.pages:
                text += page.extract_text() + "\n"

            return {
                "num_pages": len(pdf.pages),
                "text": text[:50000],  # Limit to 50KB
                "text_length": len(text)
            }
    except Exception as e:
        return {"error": f"PDF processing error: {str(e)}"}

async def _process_docx(self, path: Path) -> Dict:
    """Process DOCX file"""
    try:
        import docx
        doc = docx.Document(path)
        text = "\n".join([para.text for para in doc.paragraphs])

        return {
            "num_paragraphs": len(doc.paragraphs),
            "text": text[:50000],
            "text_length": len(text)
        }
    except Exception as e:
        return {"error": f"DOCX processing error: {str(e)}"}

async def _process_text(self, path: Path) -> Dict:
    """Process text file"""
    try:
        with open(path, 'r', encoding='utf-8') as f:
            text = f.read()
            return {
                "text": text[:50000],
                "text_length": len(text),
                "num_lines": text.count('\n') + 1
            }
    except Exception as e:
        return {"error": f"Text processing error: {str(e)}"}

async def _process_json(self, path: Path) -> Dict:
    """Process JSON file"""
    try:
        with open(path, 'r', encoding='utf-8') as f:
            data = json.load(f)
            return {
                "data": data,
                "structure": self._analyze_json_structure(data)
            }
    except Exception as e:
        return {"error": f"JSON processing error: {str(e)}"}

def _analyze_json_structure(self, data: Any) -> Dict:
    """Analyze JSON structure"""
    if isinstance(data, dict):
        return {
            "type": "object",
            "keys": list(data.keys()),
            "num_keys": len(data.keys())
        }
    elif isinstance(data, list):
        return {
            "type": "array",
            "length": len(data),
            "sample": data[0] if len(data) > 0 else None
        }
    else:
        return {"type": type(data).__name__}
```

```
class UniversalExplorer:
    """
    ARTICLE 2 - UNIVERSAL EXPLORER INTEGRATION

    Main exploration interface combining all exploration capabilities.
    """

    def __init__(self, db, safety_settings: Dict = None):
        safety_settings = safety_settings or {"enabled": True}

        self.filesystem = FileSystemExplorer(
            db,
            allowed_paths=["/tmp", "/app/data", "/app/uploads"],
            safety_enabled=safety_settings.get("enabled", True)
        )

        self.internet = InternetExplorer(
            db,
            safety_enabled=safety_settings.get("enabled", True)
        )

        self.documents = DocumentProcessor(db)

    async def explore(self, exploration_type: str, target: str, options: Dict = None) -> Dict:
        """Universal exploration interface.
        exploration_type: 'filesystem', 'internet', 'document', 'search'
        """
        options = options or {}

        if exploration_type == "filesystem":
            return await self.filesystem.explore_directory(target, depth=options.get("depth", 1))
        elif exploration_type == "internet":
            return await self.internet.fetch_url(target, timeout=options.get("timeout", 10))
        elif exploration_type == "document":
            return await self.documents.process_document(target)
        elif exploration_type == "search":
            return await self.internet.search_web(target)
        else:
            return {"error": f"Unknown exploration type: {exploration_type}"}

    async def get_exploration_history(self, limit: int = 50) -> List[Dict]:
        """Get recent exploration history."""
        history = await self.filesystem.collection.find(
            {}, {"_id": 0}
        ).sort("timestamp", -1).limit(limit).to_list(limit)
        return history

    async def get_capabilities(self) -> Dict:
        """Get current exploration capabilities."""
        return {
            "filesystem": {
                "enabled": True,
                "allowed_paths": self.filesystem.allowed_paths,
                "safety_enabled": self.filesystem.safety_enabled
            },
            "internet": {
                "enabled": True,
                "safety_enabled": self.internet.safety_enabled,
                "blocked_domains": len(self.internet.blocked_domains)
            },
            "documents": {
                "enabled": True,
                "supported_formats": ["pdf", "docx", "txt", "md", "json"]
            }
        }
```

wolfram_cloud_sync.py

```
"""
Wolfram Cloud Sync Engine
=========================
Syncs Quantum MCAGI data to Wolfram Cloud storage.
Much simpler than Google Drive OAuth!
"""

import os
import json
import logging
from datetime import datetime, timezone
from typing import Dict, List, Optional, Any

logger = logging.getLogger("quantum_ai")

# Try to import Wolfram Client
try:
    from wolframclient.evaluation import SecuredAuthenticationKey, WolframCloudSession
    from wolframclient.language import wl, wlexpr
    WOLFRAM_CLOUD_AVAILABLE = True
except ImportError:
    WOLFRAM_CLOUD_AVAILABLE = False
    logger.warning("Wolfram Client not installed. Run: pip install wolframclient")


class WolframCloudSyncEngine:
    """
    Syncs Quantum MCAGI data to Wolfram Cloud.

    Features:
    - Upload/download JSON data
    - Sync brain state, knowledge, conversations
    - All stored under user's Wolfram Cloud account
    """

    def __init__(self, consumer_key: str = None, consumer_secret: str = None):
        self.consumer_key = consumer_key or os.environ.get(
            'WOLFRAM_CONSUMER_KEY',
            'XQFcaGYqlxKynDG/OnScZgYiVue2HLfDJkBwiLDl1gw='
        )
        self.consumer_secret = consumer_secret or os.environ.get(
            'WOLFRAM_CONSUMER_SECRET',
            'gsJeEwTtCBrOJOojWWFLlR2yrwszXJfzBI8mFco/Q1U='
        )

        self.session = None
        self.authenticated = False
        self.cloud_folder = "QuantumMCAGI"
        self.last_sync = None

        if WOLFRAM_CLOUD_AVAILABLE:
            self._initialize()

    def _initialize(self):
        """Initialize Wolfram Cloud session."""
        try:
            sak = SecuredAuthenticationKey(self.consumer_key, self.consumer_secret)
            self.session = WolframCloudSession(credentials=sak)
            self.session.start()

            if self.session.authorized():
                self.authenticated = True
                logger.info("Wolfram Cloud authenticated successfully!")
            else:
                logger.warning("Wolfram Cloud authentication failed")
        except Exception as e:
            logger.error(f"Wolfram Cloud init error: {e}")
            self.authenticated = False

    def _get_cloud_path(self, filename: str) -> str:
        """Get full cloud path for a file."""
        return f"{self.cloud_folder}/{filename}"

    def upload_json(self, filename: str, data: Dict) -> bool:
        """Upload JSON data to Wolfram Cloud."""
        if not self.authenticated or not self.session:
            logger.warning("Not authenticated to Wolfram Cloud")
            return False

        try:
            json_str = json.dumps(data, indent=2, default=str)
            cloud_path = self._get_cloud_path(filename)
```

```
        # Use CloudPut to store the data
        result = self.session.evaluate(
            wl.CloudPut(json_str, wl.CloudObject(cloud_path))
        )

        logger.info(f"Uploaded {filename} to Wolfram Cloud")
        self.last_sync = datetime.now(timezone.utc)
        return True
    except Exception as e:
        logger.error(f"Upload failed: {e}")
        return False

def download_json(self, filename: str) -> Optional[Dict]:
    """Download JSON data from Wolfram Cloud."""
    if not self.authenticated or not self.session:
        return None

    try:
        cloud_path = self._get_cloud_path(filename)

        # Use CloudGet to retrieve the data
        result = self.session.evaluate(
            wl.CloudGet(wl.CloudObject(cloud_path))
        )

        if result and isinstance(result, str):
            data = json.loads(result)
            logger.info(f"Downloaded {filename} from Wolfram Cloud")
            return data
        return None
    except Exception as e:
        logger.error(f"Download failed: {e}")
        return None

def sync_brain_state(self, state: Dict) -> bool:
    """Sync brain state to Wolfram Cloud."""
    state['synced_at'] = datetime.now(timezone.utc).isoformat()
    state['sync_source'] = 'quantum_mcagi_wolfram_cloud'
    return self.upload_json('brain_state.json', state)

def load_brain_state(self) -> Optional[Dict]:
    """Load brain state from Wolfram Cloud."""
    return self.download_json('brain_state.json')

def sync_knowledge(self, knowledge: Dict) -> bool:
    """Sync knowledge base to Wolfram Cloud."""
    knowledge['synced_at'] = datetime.now(timezone.utc).isoformat()
    return self.upload_json('knowledge_base.json', knowledge)

def load_knowledge(self) -> Optional[Dict]:
    """Load knowledge from Wolfram Cloud."""
    return self.download_json('knowledge_base.json')

def sync_conversations(self, conversations: List[Dict]) -> bool:
    """Sync conversation history to Wolfram Cloud."""
    data = {
        'conversations': conversations,
        'synced_at': datetime.now(timezone.utc).isoformat(),
        'count': len(conversations)
    }
    return self.upload_json('conversations.json', data)

def load_conversations(self) -> Optional[List[Dict]]:
    """Load conversations from Wolfram Cloud."""
    data = self.download_json('conversations.json')
    if data:
        return data.get('conversations', [])
    return None

def list_files(self) -> List[str]:
    """List files in the cloud folder."""
    if not self.authenticated or not self.session:
        return []

    try:
        result = self.session.evaluate(
            wl.FileNames(wl.All, wl.CloudObject(self.cloud_folder))
        )
        if result:
            return list(result) if hasattr(result, '__iter__') else []
        return []
    except Exception as e:
        logger.error(f"List files failed: {e}")
        return []
```

```
    def get_status(self) -> Dict:
        """Get sync status."""
        return {
            'available': WOLFRAM_CLOUD_AVAILABLE,
            'authenticated': self.authenticated,
            'cloud_folder': self.cloud_folder,
            'last_sync': self.last_sync.isoformat() if self.last_sync else None,
            'provider': 'Wolfram Cloud'
        }


# Singleton instance
_wolfram_cloud_engine = None

def get_wolfram_cloud_engine() -> WolframCloudSyncEngine:
    """Get or create the Wolfram Cloud sync engine."""
    global _wolfram_cloud_engine
    if _wolfram_cloud_engine is None:
        _wolfram_cloud_engine = WolframCloudSyncEngine()
    return _wolfram_cloud_engine
```

wolfram_enhanced.py

```
"""
 Enhanced WolframAlpha Integration
===================================
Full WolframAlpha Pro integration with:
- Step-by-step solutions
- Full result data
- Spoken results
- Image/plot retrieval
- Conversation mode
"""

import os
import re
import requests
import logging
from typing import Dict, List, Optional, Any
import xml.etree.ElementTree as ET

logger = logging.getLogger("quantum_ai")


class EnhancedWolframAlpha:
    """
    Enhanced WolframAlpha integration with Pro features.

    Features:
    - Full API with complete pod data
    - Step-by-step solutions
    - Spoken results (audio)
    - Conversation mode
    - Image/plot extraction
    """

    def __init__(self, app_id: str = None):
        self.app_id = app_id or os.environ.get('WOLFRAM_APPID', 'A24U8GXLAU')
        self.base_url = "http://api.wolframalpha.com/v2/query"
        self.short_url = "http://api.wolframalpha.com/v1/result"
        self.spoken_url = "http://api.wolframalpha.com/v1/spoken"
        self.conversation_url = "http://api.wolframalpha.com/v1/conversation.jsp"

        self.queries_made = 0
        self.cache = {}
        self.conversation_id = None
        self.conversation_host = None

        logger.info("Enhanced WolframAlpha initialized")

    def query_full(self, question: str, include_pods: List[str] = None) -> Dict:
        """
        Full API query with complete results.

        Args:
            question: The query string
            include_pods: Optional list of pod IDs to include

        Returns:
            Dict with all pods, images, and data
        """
        params = {
            'input': question,
            'appid': self.app_id,
            'format': 'plaintext,image',
            'output': 'json',
            'podstate': 'Step-by-step solution'  # Request step-by-step for Pro
        }

        if include_pods:
            params['includepodid'] = ','.join(include_pods)

        try:
            response = requests.get(self.base_url, params=params, timeout=30)
            self.queries_made += 1

            if response.status_code == 200:
                data = response.json()
                return self._parse_full_response(data)
            else:
                return {'error': f'HTTP {response.status_code}', 'success': False}
        except Exception as e:
            logger.error(f"WolframAlpha query failed: {e}")
            return {'error': str(e), 'success': False}
```

```
def _parse_full_response(self, data: Dict) -> Dict:
    """Parse full API response into structured format."""
    query_result = data.get('queryresult', {})

    result = {
        'success': query_result.get('success', False),
        'input_interpretation': None,
        'primary_result': None,
        'pods': [],
        'images': [],
        'step_by_step': None,
        'related_queries': [],
        'raw_data': query_result
    }

    pods = query_result.get('pods', [])

    for pod in pods:
        pod_id = pod.get('id', '')
        pod_title = pod.get('title', '')
        subpods = pod.get('subpods', [])

        pod_data = {
            'id': pod_id,
            'title': pod_title,
            'texts': [],
            'images': []
        }

        for subpod in subpods:
            # Text content
            plaintext = subpod.get('plaintext', '')
            if plaintext:
                pod_data['texts'].append(plaintext)

            # Images
            img = subpod.get('img', {})
            if img:
                pod_data['images'].append({
                    'src': img.get('src', ''),
                    'alt': img.get('alt', ''),
                    'title': img.get('title', '')
                })
                result['images'].append(img.get('src', ''))

        result['pods'].append(pod_data)

        # Extract specific pod types
        if pod_id == 'Input':
            result['input_interpretation'] = ' '.join(pod_data['texts'])
        elif pod.get('primary', False):
            result['primary_result'] = ' '.join(pod_data['texts'])
        elif 'step' in pod_title.lower() or 'solution' in pod_title.lower():
            result['step_by_step'] = '\n'.join(pod_data['texts'])

    # If no primary result, use first result pod
    if not result['primary_result'] and len(result['pods']) > 1:
        for pod in result['pods']:
            if pod['texts'] and pod['id'] != 'Input':
                result['primary_result'] = pod['texts'][0]
                break

    return result

def query_simple(self, question: str) -> Optional[str]:
    """Simple query returning just the answer text."""
    cache_key = question.lower().strip()
    if cache_key in self.cache:
        return self.cache[cache_key]

    params = {
        'i': question,
        'appid': self.app_id
    }

    try:
        response = requests.get(self.short_url, params=params, timeout=15)
        self.queries_made += 1

        if response.status_code == 200:
            text = response.text.strip()
            if "did not understand" not in text.lower():
                self.cache[cache_key] = text
                return text
        return None
    except Exception as e:
```

```
        logger.error(f"Simple query failed: {e}")
        return None

def query_spoken(self, question: str) -> Optional[str]:
    """Get spoken/audio-friendly response."""
    params = {
        'i': question,
        'appid': self.app_id
    }

    try:
        response = requests.get(self.spoken_url, params=params, timeout=15)
        self.queries_made += 1

        if response.status_code == 200:
            return response.text.strip()
        return None
    except Exception as e:
        logger.error(f"Spoken query failed: {e}")
        return None

def query_conversation(self, question: str) -> Dict:
    """
    Conversational query that maintains context.
    Useful for follow-up questions.
    """
    params = {
        'i': question,
        'appid': self.app_id
    }

    # Add conversation context if available
    if self.conversation_id:
        params['conversationid'] = self.conversation_id
    if self.conversation_host:
        params['host'] = self.conversation_host

    try:
        response = requests.get(self.conversation_url, params=params, timeout=15)
        self.queries_made += 1

        if response.status_code == 200:
            data = response.json()

            # Update conversation state
            self.conversation_id = data.get('conversationID')
            self.conversation_host = data.get('host')

            return {
                'success': True,
                'result': data.get('result', ''),
                'conversation_id': self.conversation_id,
                'followup_query': data.get('followup', {}).get('query', ''),
                'raw': data
            }
        return {'success': False, 'error': f'HTTP {response.status_code}'}
    except Exception as e:
        logger.error(f"Conversation query failed: {e}")
        return {'success': False, 'error': str(e)}

def reset_conversation(self):
    """Reset conversation context."""
    self.conversation_id = None
    self.conversation_host = None

def query_step_by_step(self, question: str) -> Dict:
    """
    Query with emphasis on getting step-by-step solutions.
    Best for math problems.
    """
    result = self.query_full(question)

    if result.get('step_by_step'):
        return {
            'success': True,
            'steps': result['step_by_step'],
            'primary_result': result.get('primary_result'),
            'images': result.get('images', [])
        }
    elif result.get('primary_result'):
        return {
            'success': True,
            'steps': None,
            'primary_result': result['primary_result'],
            'images': result.get('images', [])
        }
```

```
        else:
            return {
                'success': False,
                'error': 'No solution found'
            }

    def is_factual_query(self, text: str) -> bool:
        """Detect if this is a query WolframAlpha can answer."""
        t = text.lower()

        # Contains numbers or math
        if re.search(r'\d', t):
            return True

        # Math keywords
        math_keywords = [
            'calculate', 'compute', 'solve', 'integrate', 'derivative',
            'equation', 'formula', 'factor', 'simplify', 'expand',
            'limit', 'sum', 'product', 'matrix', 'vector'
        ]
        if any(kw in t for kw in math_keywords):
            return True

        # Science/factual keywords
        factual_keywords = [
            'how many', 'what is the', 'distance', 'population',
            'weather', 'convert', 'temperature', 'mass', 'speed',
            'height', 'weight', 'calories', 'nutrition',
            'stock', 'price', 'exchange rate', 'currency'
        ]
        if any(kw in t for kw in factual_keywords):
            return True

        # Astronomy
        astro_keywords = [
            'planet', 'star', 'moon', 'sun', 'galaxy', 'light year',
            'orbit', 'asteroid', 'comet', 'constellation'
        ]
        if any(kw in t for kw in astro_keywords):
            return True

        return False

    def get_status(self) -> Dict:
        """Get WolframAlpha status."""
        return {
            'available': True,
            'app_id_configured': self.app_id != 'T7PA7A-E38RXXXXX',
            'queries_made': self.queries_made,
            'cache_size': len(self.cache),
            'conversation_active': self.conversation_id is not None,
            'endpoints': {
                'full': self.base_url,
                'simple': self.short_url,
                'spoken': self.spoken_url,
                'conversation': self.conversation_url
            }
        }


# Singleton instance
_wolfram_engine = None

def get_wolfram_engine() -> EnhancedWolframAlpha:
    """Get or create the WolframAlpha engine."""
    global _wolfram_engine
    if _wolfram_engine is None:
        _wolfram_engine = EnhancedWolframAlpha()
    return _wolfram_engine
```

wolfram_integration.py

```
"""
 Wolfram Alpha Integration
Mathematical and scientific knowledge engine for Quantum AI
"""

import wolframalpha
import os
from typing import Dict, Optional, List


class WolframEngine:
    """
    Wolfram Alpha integration for mathematical and scientific queries.
    Provides computational knowledge that no LLM can match.
    """

    def __init__(self, app_id: str = None):
        self.app_id = app_id or os.environ.get('WOLFRAM_APP_ID', 'A24U8GXLAU')
        self.client = wolframalpha.Client(self.app_id)
        self.query_history = []

    def query(self, question: str) -> Dict:
        """
        Query Wolfram Alpha and return structured results.
        """
        try:
            res = self.client.query(question)

            results = {
                'success': res.success,
                'query': question,
                'pods': [],
                'primary_result': None,
                'images': []
            }

            for pod in res.pods:
                pod_data = {
                    'title': pod.title,
                    'texts': [],
                    'images': []
                }

                for sub in pod.subpods:
                    if hasattr(sub, 'plaintext') and sub.plaintext:
                        pod_data['texts'].append(sub.plaintext)
                    if hasattr(sub, 'img') and sub.img:
                        pod_data['images'].append(sub.img.src)
                        results['images'].append(sub.img.src)

                results['pods'].append(pod_data)

                # Get primary result
                if pod.primary and pod_data['texts']:
                    results['primary_result'] = pod_data['texts'][0]

            # Store in history
            self.query_history.append({
                'query': question,
                'success': res.success,
                'result': results['primary_result']
            })

            return results

        except Exception as e:
            return {
                'success': False,
                'query': question,
                'error': str(e),
                'pods': [],
                'primary_result': None
            }

    def get_math_result(self, expression: str) -> Optional[str]:
        """Get mathematical computation result"""
        result = self.query(expression)
        return result.get('primary_result')

    def get_scientific_fact(self, topic: str) -> Dict:
        """Get scientific information about a topic"""
        result = self.query(topic)
```

```
        facts = []
        for pod in result.get('pods', []):
            for text in pod.get('texts', []):
                if text and len(text) > 10:
                    facts.append({
                        'category': pod['title'],
                        'fact': text
                    })

        return {
            'topic': topic,
            'facts': facts[:5],  # Top 5 facts
            'primary': result.get('primary_result'),
            'images': result.get('images', [])[:3]
        }

    def solve_equation(self, equation: str) -> Dict:
        """Solve a mathematical equation"""
        result = self.query(f"solve {equation}")

        solutions = []
        for pod in result.get('pods', []):
            if 'solution' in pod['title'].lower() or 'result' in pod['title'].lower():
                solutions.extend(pod.get('texts', []))

        return {
            'equation': equation,
            'solutions': solutions,
            'steps': [p['texts'] for p in result.get('pods', []) if 'step' in p['title'].lower()]
        }

    def get_definition(self, word: str) -> Dict:
        """Get definition and etymology"""
        result = self.query(f"define {word}")

        definitions = []
        etymology = None

        for pod in result.get('pods', []):
            if 'definition' in pod['title'].lower():
                definitions.extend(pod.get('texts', []))
            if 'etymology' in pod['title'].lower():
                etymology = pod.get('texts', [None])[0]

        return {
            'word': word,
            'definitions': definitions,
            'etymology': etymology
        }

    def quantum_physics_query(self, query: str) -> Dict:
        """Specialized quantum physics queries"""
        result = self.query(f"quantum physics {query}")
        return {
            'query': query,
            'result': result.get('primary_result'),
            'all_info': result.get('pods', [])
        }

    def get_query_history(self) -> List[Dict]:
        """Get recent query history"""
        return self.query_history[-20:]


# Global instance
_wolfram_engine = None

def get_wolfram_engine() -> WolframEngine:
    """Get or create Wolfram engine instance"""
    global _wolfram_engine
    if _wolfram_engine is None:
        _wolfram_engine = WolframEngine()
    return _wolfram_engine
```

word_embeddings.py

```
"""
 WORD EMBEDDINGS - Pure Python Implementation
=============================================
Word2Vec-style embeddings without external dependencies.

Implements Skip-gram with Negative Sampling (simplified).
Words are represented as dense vectors capturing semantic relationships.
Similar words have similar vectors.
"""

import math
import random
import re
import json
import os
from collections import defaultdict, Counter
from typing import List, Dict, Tuple, Optional
import logging

logger = logging.getLogger("quantum_ai")


class WordEmbeddings:
    """
    Simple word embeddings trained on corpus.

    Uses co-occurrence matrix factorization (GloVe-style)
    which is more tractable than Skip-gram for CPU-only training.

    Algorithm:
    1. Build word co-occurrence matrix from corpus
    2. Apply PPMI (Positive PMI) weighting
    3. Reduce dimensionality via truncated SVD approximation
    """

    def __init__(self, vector_size: int = 50, window: int = 5, min_count: int = 2):
        self.vector_size = vector_size
        self.window = window
        self.min_count = min_count

        self.word_to_id = {}
        self.id_to_word = {}
        self.vectors = {}  # word -> vector
        self.word_counts = Counter()
        self.cooccurrence = defaultdict(Counter)

        self.trained = False
        self.vocab_size = 0

    def build_vocab(self, texts: List[str]):
        """Build vocabulary from texts."""
        for text in texts:
            words = self._tokenize(text)
            self.word_counts.update(words)

        # Filter by min_count
        vocab = [w for w, c in self.word_counts.items() if c >= self.min_count]
        vocab.sort()

        self.word_to_id = {w: i for i, w in enumerate(vocab)}
        self.id_to_word = {i: w for w, i in self.word_to_id.items()}
        self.vocab_size = len(vocab)

        logger.info(f"Built vocabulary: {self.vocab_size} words")

    def build_cooccurrence(self, texts: List[str]):
        """Build co-occurrence matrix."""
        for text in texts:
            words = self._tokenize(text)
            words = [w for w in words if w in self.word_to_id]

            for i, center in enumerate(words):
                # Context window
                start = max(0, i - self.window)
                end = min(len(words), i + self.window + 1)

                for j in range(start, end):
                    if i != j:
                        context = words[j]
                        # Distance weighting (closer = higher weight)
                        distance = abs(i - j)
                        weight = 1.0 / distance
```

```
                    self.cooccurrence[center][context] += weight

    logger.info(f"Built co-occurrence matrix")

def train(self, texts: List[str], iterations: int = 10):
    """
    Train embeddings using simplified GloVe-style algorithm.

    This is a lightweight approximation suitable for CPU training.
    """
    # Build vocab and co-occurrence
    self.build_vocab(texts)
    self.build_cooccurrence(texts)

    self._train_continued(iterations)

def _train_continued(self, iterations):
    """Continuation of train  auto-extracted by self-evolution."""
    if self.vocab_size == 0:
        logger.warning("Empty vocabulary, cannot train")
        return

    # Initialize random vectors
    for word in self.word_to_id:
        self.vectors[word] = [random.gauss(0, 0.1) for _ in range(self.vector_size)]

    # Compute PPMI matrix and train
    total_count = sum(sum(v.values()) for v in self.cooccurrence.values())
    word_probs = {w: self.word_counts[w] / sum(self.word_counts.values())
                  for w in self.word_to_id}

    # Stochastic gradient descent on weighted least squares
    learning_rate = 0.05

    for iteration in range(iterations):
        total_loss = 0
        updates = 0

        for word_i, contexts in self.cooccurrence.items():
            if word_i not in self.vectors:
                continue

            vec_i = self.vectors[word_i]

            for word_j, count in contexts.items():
                if word_j not in self.vectors:
                    continue

                vec_j = self.vectors[word_j]

                # Compute PPMI target
                p_ij = count / total_count
                p_i = word_probs.get(word_i, 1e-10)
                p_j = word_probs.get(word_j, 1e-10)
                pmi = math.log(p_ij / (p_i * p_j) + 1e-10)
                ppmi = max(0, pmi)

                # Dot product prediction
                dot = sum(a * b for a, b in zip(vec_i, vec_j))

                # Error
                error = dot - ppmi

                # Weight by co-occurrence frequency
                weight = min(1.0, (count / 100) ** 0.75)

                # Gradient update
                for k in range(self.vector_size):
                    grad_i = weight * error * vec_j[k]
                    grad_j = weight * error * vec_i[k]

                    vec_i[k] -= learning_rate * grad_i
                    vec_j[k] -= learning_rate * grad_j

                total_loss += weight * error ** 2
                updates += 1

        if updates > 0:
            avg_loss = total_loss / updates
            if iteration % 3 == 0:
                logger.info(f"Embedding training iteration {iteration}: loss={avg_loss:.4f}")

    # Normalize vectors
    for word in self.vectors:
        vec = self.vectors[word]
        norm = math.sqrt(sum(x*x for x in vec))
```

```
        if norm > 0:
            self.vectors[word] = [x/norm for x in vec]

    self.trained = True
    logger.info(f"Trained {len(self.vectors)} word vectors")


def get_vector(self, word: str) -> Optional[List[float]]:
    """Get vector for a word."""
    return self.vectors.get(word.lower())

def similarity(self, word1: str, word2: str) -> float:
    """Compute cosine similarity between two words."""
    vec1 = self.get_vector(word1)
    vec2 = self.get_vector(word2)

    if vec1 is None or vec2 is None:
        return 0.0

    # Cosine similarity (vectors already normalized)
    return sum(a * b for a, b in zip(vec1, vec2))

def most_similar(self, word: str, top_n: int = 10) -> List[Tuple[str, float]]:
    """Find most similar words."""
    vec = self.get_vector(word)
    if vec is None:
        return []

    similarities = []
    for other_word, other_vec in self.vectors.items():
        if other_word != word.lower():
            sim = sum(a * b for a, b in zip(vec, other_vec))
            similarities.append((other_word, sim))

    similarities.sort(key=lambda x: x[1], reverse=True)
    return similarities[:top_n]

def analogy(self, a: str, b: str, c: str, top_n: int = 5) -> List[Tuple[str, float]]:
    """
    Solve analogy: a is to b as c is to ?

    Uses vector arithmetic: result = vec(b) - vec(a) + vec(c)
    """
    vec_a = self.get_vector(a)
    vec_b = self.get_vector(b)
    vec_c = self.get_vector(c)

    if any(v is None for v in [vec_a, vec_b, vec_c]):
        return []

    # Compute target vector
    target = [b_i - a_i + c_i for a_i, b_i, c_i in zip(vec_a, vec_b, vec_c)]

    # Normalize
    norm = math.sqrt(sum(x*x for x in target))
    if norm > 0:
        target = [x/norm for x in target]

    # Find closest words (excluding inputs)
    exclude = {a.lower(), b.lower(), c.lower()}
    similarities = []

    for word, vec in self.vectors.items():
        if word not in exclude:
            sim = sum(t * v for t, v in zip(target, vec))
            similarities.append((word, sim))

    similarities.sort(key=lambda x: x[1], reverse=True)
    return similarities[:top_n]

def sentence_vector(self, text: str) -> Optional[List[float]]:
    """Get average vector for a sentence."""
    words = self._tokenize(text)
    vectors = [self.get_vector(w) for w in words if self.get_vector(w)]

    if not vectors:
        return None

    # Average
    result = [0.0] * self.vector_size
    for vec in vectors:
        for i, v in enumerate(vec):
            result[i] += v

    n = len(vectors)
    result = [x / n for x in result]
```

```
        # Normalize
        norm = math.sqrt(sum(x*x for x in result))
        if norm > 0:
            result = [x/norm for x in result]

        return result

    def sentence_similarity(self, text1: str, text2: str) -> float:
        """Compute similarity between two sentences."""
        vec1 = self.sentence_vector(text1)
        vec2 = self.sentence_vector(text2)

        if vec1 is None or vec2 is None:
            return 0.0

        return sum(a * b for a, b in zip(vec1, vec2))

    def _tokenize(self, text: str) -> List[str]:
        """Tokenize text to words."""
        return re.findall(r'\b[a-z]+\b', text.lower())

    def save(self, filepath: str):
        """Save embeddings to file."""
        data = {
            'vectors': self.vectors,
            'word_to_id': self.word_to_id,
            'vocab_size': self.vocab_size,
            'vector_size': self.vector_size
        }
        with open(filepath, 'w') as f:
            json.dump(data, f)
        logger.info(f"Saved embeddings to {filepath}")

    def load(self, filepath: str) -> bool:
        """Load embeddings from file."""
        try:
            if os.path.exists(filepath):
                with open(filepath, 'r') as f:
                    data = json.load(f)
                self.vectors = data['vectors']
                self.word_to_id = data['word_to_id']
                self.id_to_word = {int(i): w for w, i in self.word_to_id.items()}
                self.vocab_size = data['vocab_size']
                self.vector_size = data['vector_size']
                self.trained = True
                logger.info(f"Loaded {len(self.vectors)} embeddings")
                return True
        except Exception as e:
            logger.warning(f"Could not load embeddings: {e}")
        return False

    def get_stats(self) -> Dict:
        """Get embedding statistics."""
        return {
            'vocab_size': self.vocab_size,
            'vector_size': self.vector_size,
            'trained': self.trained,
            'total_vectors': len(self.vectors)
        }


# Singleton with lazy initialization
_embeddings = None
_embeddings_file = "/app/backend/word_embeddings.json"

def get_embeddings() -> WordEmbeddings:
    """Get or create word embeddings."""
    global _embeddings
    if _embeddings is None:
        _embeddings = WordEmbeddings(vector_size=50, window=5, min_count=2)

        # Try to load pre-trained
        if not _embeddings.load(_embeddings_file):
            # Train on corpus
            from training_corpus import get_corpus_sentences
            sentences = get_corpus_sentences()
            if sentences:
                _embeddings.train(sentences, iterations=15)
                _embeddings.save(_embeddings_file)

    return _embeddings


def train_embeddings_on_text(text: str):
    """Add new text to embeddings training."""
```

```
embeddings = get_embeddings()
sentences = re.split(r'(?<=[.!?])\s+', text)
sentences = [s for s in sentences if len(s) > 20]

if sentences:
    embeddings.train(sentences, iterations=5)
    embeddings.save(_embeddings_file)
```

xanadu_prebo.py

```
"""
 XANADU PRE-BORN-OPPENHEIMER QUANTUM SIMULATION
==============================================
First-Quantized Real-Space Grid - Nuclei and Electrons as Equal Particles
Based on Xanadu's 2026 breakthrough for photochemical simulation
"""

from typing import Dict, List, Tuple, Optional

# Optional PennyLane import with graceful fallback
try:
    import pennylane as qml
    from pennylane import numpy as np
    PENNYLANE_AVAILABLE = True
except ImportError:
    import numpy as np
    PENNYLANE_AVAILABLE = False
    qml = None

class PreBornOppenheimerSimulator:
    """
    Xanadu's Pre-Born-Oppenheimer algorithm implementation.
    Treats nuclei and electrons on equal footing on a Cartesian grid.
    10x more efficient than standard quantum chemistry methods.
    Falls back to classical simulation if PennyLane is not available.
    """

    def __init__(self, n_electrons: int = 2, n_nuclei: int = 2, grid_points: int = 16):
        self.n_electrons = n_electrons
        self.n_nuclei = n_nuclei
        self.grid_points = grid_points
        self.n_particles = n_electrons + n_nuclei
        self.n_qubits = int(np.ceil(np.log2(grid_points))) * 3 * self.n_particles
        self.pennylane_available = PENNYLANE_AVAILABLE
        if PENNYLANE_AVAILABLE:
            self.dev = qml.device('default.qubit', wires=min(self.n_qubits, 20))
        else:
            self.dev = None
        self.particle_masses = self._initialize_masses()
        self.grid_spacing = 0.5

    def _initialize_masses(self) -> Dict[str, float]:
        return {
            'electron': 1.0,
            'H': 1836.15,
            'N': 25532.0,
            'B': 19705.0,
            'F': 34631.0,
            'C': 21874.0,
            'O': 29156.0
        }

    def real_space_grid_encoding(self, particle_positions: np.ndarray) -> np.ndarray:
        """Map particle positions to qubit registers on 3D grid"""
        n_grid_qubits = int(np.ceil(np.log2(self.grid_points)))
        encoded = np.zeros((self.n_particles, 3, n_grid_qubits))
        for p in range(self.n_particles):
            for dim in range(3):
                pos = particle_positions[p, dim]
                grid_idx = int((pos / self.grid_spacing) % self.grid_points)
                binary = format(grid_idx, f'0{n_grid_qubits}b')
                encoded[p, dim] = np.array([int(b) for b in binary])
        return encoded

    def coulomb_interaction(self, r: float, charge1: float, charge2: float) -> float:
        """1/r Coulomb interaction with alternating sign"""
        if abs(r) < 1e-10:
            return 0.0
        return charge1 * charge2 / r

    def kinetic_energy_operator(self, mass: float) -> float:
        """Laplacian kinetic energy on grid: -/(2m) """
        return -0.5 / mass * (np.pi / self.grid_spacing) ** 2

    def pre_bo_hamiltonian(self, particle_types: List[str]) -> np.ndarray:
        """
        Construct Pre-Born-Oppenheimer Hamiltonian.
        H =  T + < V
        """
        n = self.n_particles
        H = np.zeros((n, n), dtype=complex)
```

```
    for i in range(n):
        mass = self.particle_masses.get(particle_types[i], 1.0)
        H[i, i] = self.kinetic_energy_operator(mass)

    charges = []
    for pt in particle_types:
        if pt == 'electron':
            charges.append(-1.0)
        elif pt == 'H':
            charges.append(1.0)
        elif pt in ['N', 'B']:
            charges.append(5.0)
        elif pt in ['F', 'O']:
            charges.append(8.0)
        elif pt == 'C':
            charges.append(6.0)
        else:
            charges.append(1.0)

    for i in range(n):
        for j in range(i + 1, n):
            r = self.grid_spacing * abs(i - j)
            v = self.coulomb_interaction(r, charges[i], charges[j])
            H[i, j] = v
            H[j, i] = v

    return H

def swap_network_block_encoding(self, wires: List[int]) -> None:
    """
    Xanadu's Swap Network for efficient N² interactions.
    Particles interact only with neighbors through swapping.
    """
    n = len(wires)
    for layer in range(n):
        for i in range(0, n - 1, 2):
            if i + 1 < n:
                qml.SWAP(wires=[wires[i], wires[i + 1]])
                qml.CZ(wires=[wires[i], wires[i + 1]])
        for i in range(1, n - 1, 2):
            if i + 1 < n:
                qml.SWAP(wires=[wires[i], wires[i + 1]])
                qml.CZ(wires=[wires[i], wires[i + 1]])

def state_preparation(self, params: np.ndarray, wires: List[int]) -> None:
    """Prepare initial quantum state for simulation"""
    for i, w in enumerate(wires):
        qml.RY(params[i % len(params)], wires=w)
        qml.RZ(params[(i + 1) % len(params)], wires=w)
    for i in range(len(wires) - 1):
        qml.CNOT(wires=[wires[i], wires[i + 1]])

def simulate_photochemistry(self, molecule: str = "NH3_BF3", time_steps: int = 10) -> Dict:
    """
    Simulate photochemical reaction using Pre-BO algorithm.
    """
    if molecule == "NH3_BF3":
        particle_types = ['N', 'H', 'H', 'H', 'B', 'F', 'F', 'F'] + ['electron'] * 8
        self.n_particles = len(particle_types)
    else:
        particle_types = ['electron'] * self.n_electrons + ['H'] * self.n_nuclei

    n_wires = min(self.n_particles, 16)
    dev = qml.device('default.qubit', wires=n_wires)

    @qml.qnode(dev)
    def circuit(params, time):
        wires = list(range(n_wires))
        self.state_preparation(params, wires)
        for _ in range(int(time)):
            self.swap_network_block_encoding(wires)
            for w in wires:
                qml.RZ(0.1 * time, wires=w)
        return [qml.expval(qml.PauliZ(w)) for w in wires]

    params = np.random.uniform(0, np.pi, n_wires * 2)
    results = {'molecule': molecule, 'time_evolution': [], 'nonadiabatic_coupling': []}

    for t in range(time_steps):
        expectations = circuit(params, float(t + 1))
        energy = float(np.sum(expectations))
        coupling = float(np.std(expectations))
        results['time_evolution'].append({'time': t, 'energy': energy, 'state': [float(e) for e in expectations[:4]]})
        results['nonadiabatic_coupling'].append(coupling)

    results['final_energy'] = results['time_evolution'][-1]['energy']
```

```
        results['max_coupling'] = float(max(results['nonadiabatic_coupling']))
        results['algorithm'] = 'Xanadu Pre-Born-Oppenheimer'
        results['efficiency_gain'] = '10x vs standard VQE'

        return results

    def calculate_grid_bounds(self, molecule: str) -> Dict:
        """Use computational analysis to determine optimal grid parameters"""
        bounds = {
            'NH3_BF3': {'grid_min': -5.0, 'grid_max': 5.0, 'optimal_points': 64, 'spacing': 0.156},
            'H2O': {'grid_min': -3.0, 'grid_max': 3.0, 'optimal_points': 32, 'spacing': 0.188},
            'CH4': {'grid_min': -4.0, 'grid_max': 4.0, 'optimal_points': 48, 'spacing': 0.167},
        }
        return bounds.get(molecule, {'grid_min': -5.0, 'grid_max': 5.0, 'optimal_points': 64, 'spacing': 0.156})


class MetaCognitiveQuantumOrchestrator:
    """
    MCAGI Integration - Meta-cognitive layer that decides when to use Pre-BO vs standard VQE.
    """

    def __init__(self):
        self.prebo_sim = PreBornOppenheimerSimulator()
        self.coupling_threshold = 0.3
        self.simulation_history = []

    def analyze_reaction(self, molecule: str, reaction_type: str) -> Dict:
        """Analyze reaction to determine best simulation approach"""
        photochemical_indicators = ['photo', 'light', 'excited', 'nonadiabatic', 'conical', 'intersection']
        is_photochemical = any(ind in reaction_type.lower() for ind in photochemical_indicators)

        test_result = self.prebo_sim.simulate_photochemistry(molecule, time_steps=3)
        coupling = test_result['max_coupling']

        return {
            'molecule': molecule,
            'reaction_type': reaction_type,
            'is_photochemical': is_photochemical,
            'nonadiabatic_coupling': coupling,
            'recommended_algorithm': 'Pre-Born-Oppenheimer' if (is_photochemical or coupling > self.coupling_threshold) else 'Standard VQE',
            'efficiency_estimate': '10x better' if is_photochemical else 'standard'
        }

    def run_simulation(self, molecule: str, reaction_type: str = "ground_state", time_steps: int = 10) -> Dict:
        """Run full simulation with automatic algorithm selection"""
        analysis = self.analyze_reaction(molecule, reaction_type)

        if analysis['recommended_algorithm'] == 'Pre-Born-Oppenheimer':
            result = self.prebo_sim.simulate_photochemistry(molecule, time_steps)
            result['algorithm_selection'] = 'automatic'
            result['selection_reason'] = f"High nonadiabatic coupling ({analysis['nonadiabatic_coupling']:.3f}) detected"
        else:
            result = self._run_standard_vqe(molecule)
            result['algorithm_selection'] = 'automatic'
            result['selection_reason'] = 'Low coupling - standard VQE sufficient'

        self.simulation_history.append(result)
        return result

    def _run_standard_vqe(self, molecule: str) -> Dict:
        """Fallback to standard VQE for non-photochemical reactions"""
        dev = qml.device('default.qubit', wires=4)

        @qml.qnode(dev)
        def vqe_circuit(params):
            for i in range(4):
                qml.RY(params[i], wires=i)
            qml.CNOT(wires=[0, 1])
            qml.CNOT(wires=[2, 3])
            return qml.expval(qml.PauliZ(0) @ qml.PauliZ(1))

        params = np.random.uniform(0, np.pi, 4)
        energy = float(vqe_circuit(params))

        return {'molecule': molecule, 'algorithm': 'Standard VQE', 'energy': energy, 'efficiency_gain': 'baseline'}

    def get_resource_estimate(self, molecule: str) -> Dict:
        """Estimate quantum resources needed"""
        grid_params = self.prebo_sim.calculate_grid_bounds(molecule)
        n_qubits = int(np.ceil(np.log2(grid_params['optimal_points']))) * 3 * 10

        return {
            'molecule': molecule,
            'estimated_qubits': n_qubits,
            'grid_points': grid_params['optimal_points'],
            'toffoli_count_prebo': n_qubits * 100,
```

```
            'toffoli_count_standard': n_qubits * 1000,
            'savings': '10x reduction with Pre-BO'
        }


_orchestrator = None

def get_quantum_orchestrator() -> MetaCognitiveQuantumOrchestrator:
    """get_quantum_orchestrator - Auto-documented by self-evolution."""
    global _orchestrator
    if _orchestrator is None:
        _orchestrator = MetaCognitiveQuantumOrchestrator()
    return _orchestrator

def get_prebo_simulator() -> PreBornOppenheimerSimulator:
    """get_prebo_simulator - Auto-documented by self-evolution."""
    return PreBornOppenheimerSimulator()
```